

2025前端Vue面试题

第1题：Vue 有了数据响应式，为何还要 diff？

Vue 中的数据响应式和虚拟 DOM 的 diff 算法是两个不同的概念，它们分别解决了不同的问题，相互协作以提高页面渲染的效率和性能。

数据响应式

Vue 的数据响应式系统通过 `Object.defineProperty` 或者 ES6 的 `Proxy` 来实现，主要解决了以下问题：

1. 数据绑定：保证了视图与数据的同步更新，当数据发生变化时，视图会自动更新，避免了手动操作 DOM 的繁琐和易出错性。
2. 依赖追踪：Vue 能够追踪每个数据的依赖关系，即哪些组件或者计算属性依赖于某个数据。当数据变化时，自动更新依赖的组件或者计算属性。

虚拟 DOM 和 Diff 算法

虚拟 DOM 是一种内存中的表示结构，它是对真实 DOM 的抽象。Diff 算法是一种高效更新 DOM 的策略，它通过比较新旧虚拟 DOM 树的差异，最小化了更新操作，提高了页面的渲染效率。

为什么还需要 Diff 算法？

1. 性能优化：直接操作真实 DOM 是非常昂贵的，而虚拟 DOM 可以在内存中快速进行比较和计算差异。Diff 算法帮助减少了更新操作的次数和范围，从而提升了页面渲染的性能。
2. 批量更新：Diff 算法能够将多次 DOM 更新操作合并为一次，避免了频繁的 DOM 操作，减少了浏览器的重排和重绘。
3. 跨平台兼容：虚拟 DOM 和 Diff 算法使得 Vue 可以运行在不同的平台上，例如浏览器、Weex 等，统一了渲染逻辑和数据响应式的实现。
4. 更新效率：即使是响应式系统可以自动更新视图，但是如果每次数据变化都直接操作真实 DOM，可能会带来性能问题。Diff 算法可以智能地比较新旧 DOM 树的变化，只更新必要的部分，从而提高了更新效率。

综合作用

Vue 的数据响应式系统和虚拟 DOM + Diff 算法是紧密协作的：

- 数据响应式：保证了数据和视图的同步更新，提供了便捷的开发方式。
- 虚拟 DOM + Diff 算法：提高了页面渲染的效率和性能，减少了不必要的 DOM 操作，确保了页面的流畅性和响应性。

总体来说，数据响应式和 Diff 算法是为了解决不同层面的问题，结合起来使得 Vue 能够提供高效、流畅的用户体验。

第2题：vue3 为什么不需要时间分片？

Vue 3 不需要时间分片（time slicing）主要是因为它的核心渲染机制和性能优化策略已经足够高效，能够在大多数情况下提供流畅的用户体验。以下是详细的原因：

1. 编译器优化

Vue 3 引入了一个全新的编译器，能够生成更高效的渲染函数。这个编译器在编译过程中进行了一系列优化，例如：

- 静态提升：将不变的节点提升为常量，只在初次渲染时计算一次。
- 预字符串化：将静态内容直接转化为字符串，减少了运行时的开销。
- 缓存事件处理程序：避免了不必要的重新绑定。

这些优化措施大大减少了 Vue 3 在更新 DOM 时的计算量，使得渲染过程更加高效。

2. 响应式系统的改进

Vue 3 使用了基于代理的响应式系统，替代了 Vue 2 中基于 `Object.defineProperty` 的实现。新的响应式系统更加高效，具备以下优点：

- 精细的依赖追踪：只追踪实际使用的属性，避免了不必要的依赖收集。
- 懒惰计算：仅在需要时才计算依赖，减少了计算量。

这些改进使得 Vue 3 能够更快速地响应数据变化，从而减少了渲染开销。

3. 虚拟 DOM 和 Diff 算法的优化

Vue 3 对虚拟 DOM 及其 diff 算法进行了优化，使得差异计算更加高效：

- 静态标记：编译期间标记静态节点，跳过不变的部分。
- 块级优化：将动态节点分块，只对发生变化的块进行更新。

这些优化措施减少了 DOM 更新的频率和范围，提高了整体渲染性能。

4. 单次异步队列

Vue 3 的更新机制基于单次异步队列（single asynchronous queue），它确保在同一事件循环中只进行一次批量更新。这种方式减少了不必要的重复计算和 DOM 操作，使得更新过程更加高效。

5. 自动批处理

Vue 3 实现了自动批处理机制，在同一个事件循环中对多次数据更新进行合并，从而减少了渲染次数。这种机制在避免频繁重绘的同时，保证了界面的流畅性。

6. 现代浏览器的性能

现代浏览器的性能已经得到了极大的提升，尤其是在处理 JavaScript 和 DOM 操作方面。Vue 3 的优化能够充分利用这些性能改进，从而在绝大多数情况下不需要时间分片。

总结

Vue 3 通过编译器优化、响应式系统改进、虚拟 DOM 和 Diff 算法优化、单次异步队列、自动批处理等技术手段，大幅提升了渲染效率和性能。再加上现代浏览器的性能提升，使得 Vue 3 能够在大多数情况下提供流畅的用户体验，而无需借助时间分片等复杂的技术。

第3题：vue3 为什么要引入 Composition API？

Vue 3 引入 Composition API 的原因主要是为了更好地解决以下几个问题：

1. 更好的代码组织和重用

在 Vue 2 中，使用选项式 API（Options API）来定义组件的逻辑，通常将数据、方法、计算属性和生命周期钩子分开写在不同的配置对象中。当组件变得复杂时，不同功能的代码可能会散落在各个部分，难以维护和重用。

示例：

```
1  // Vue 2 中使用 Options API
2  export default {
3    data() {
4      return {
5        count: 0,
6        message: 'Hello World'
7      };
8    },
9    computed: {
10     doubleCount() {
11       return this.count * 2;
12     }
13   },
14   methods: {
15     increment() {
16       this.count++;
17     }
18   },
19   created() {
20     console.log(this.message);
21   }
22 };
```

通过 Composition API，可以将相关功能的逻辑组织在一起，从而提高代码的可读性和可维护性。

示例：

```
1  // Vue 3 中使用 Composition API
2  import { ref, computed, onMounted } from 'vue';
3
4  export default {
5    setup() {
6      const count = ref(0);
7      const message = ref('Hello World');
8
9      const doubleCount = computed(() => count.value * 2);
10
11     const increment = () => {
12       count.value++;
13     };
14
15     onMounted(() => {
16       console.log(message.value);
17     });
18
19     return {
20       count,
21       doubleCount,
22       increment,
23       message
24     };
25   }
26   };
```

2. 更好的逻辑复用

在 Vue 2 中，逻辑复用主要通过 mixins 和 scoped slots 实现，但它们都有一些缺点，比如命名冲突和代码可读性差。

Composition API 通过组合函数（composable functions）来实现逻辑复用，这些函数可以在多个组件之间共享和复用逻辑，避免了 mixins 的缺点。

示例：

```
1  // 一个组合函数，可以在多个组件中复用
2  import { ref, onMounted } from 'vue';
3
4  export function useMessage() {
```

```

5     const message = ref('Hello World');
6
7     onMounted(() => {
8         console.log(message.value);
9     });
10
11    return {
12        message
13    };
14 }

```

在组件中使用：

```

1  import { ref } from 'vue';
2  import { useMessage } from './useMessage';
3
4  export default {
5      setup() {
6          const count = ref(0);
7          const { message } = useMessage();
8
9          const increment = () => {
10              count.value++;
11          };
12
13          return {
14              count,
15              message,
16              increment
17          };
18      }
19  };

```

3. 更好的 TypeScript 支持

Composition API 天然地支持 TypeScript，使得类型推断和类型检查更为自然和方便。相比于 Options API，通过 Composition API 定义的逻辑更容易进行类型声明和类型推断，提升了开发体验。

示例：

```

1  import { ref, computed } from 'vue';
2
3  export default {

```

```
4   setup() {
5       const count = ref<number>(0);
6       const doubleCount = computed<number>(() => count.value * 2);
7
8       const increment = (): void => {
9           count.value++;
10      };
11
12      return {
13          count,
14          doubleCount,
15          increment
16      };
17  }
18  };
```

4. 适应函数式编程趋势

Composition API 借鉴了函数式编程的思想，将逻辑封装成函数，使得代码更加简洁、模块化、可测试，同时也更符合现代 JavaScript 开发趋势。

总结

Vue 3 引入 Composition API 主要是为了提升代码组织和复用性、提供更好的 TypeScript 支持、适应函数式编程趋势，并且解决 Vue 2 中存在的一些问题。通过 Composition API，可以让组件逻辑更加清晰、灵活和易于维护。

第4题：谈谈 Vue 事件机制，并手写\$on、\$off、\$emit、\$once

Vue 的事件机制允许组件之间进行通信，通过 `$on`、`$off`、`$emit` 和 `$once` 等方法进行事件的订阅、取消订阅、触发和一次性订阅。我们可以通过手写这些方法来理解其工作原理。

Vue 事件机制

1. `$on(event, callback)`：监听特定事件。
2. `$off(event, callback)`：停止监听特定事件。
3. `$emit(event, ...args)`：触发特定事件。
4. `$once(event, callback)`：只监听一次特定事件。

手写实现

下面是手写这些方法的简单实现：

```
1  class EventEmitter {
2      constructor() {
3          this.events = {};
4      }
5
6      // 监听事件
7      $on(event, callback) {
8          if (!this.events[event]) {
9              this.events[event] = [];
10         }
11         this.events[event].push(callback);
12     }
13
14     // 停止监听事件
15     $off(event, callback) {
16         if (!this.events[event]) return;
17
18         if (!callback) {
19             // 如果没有传递 callback, 移除所有事件监听
20             this.events[event] = [];
21         } else {
22             // 移除特定的事件监听
23             this.events[event] = this.events[event].filter(cb => cb !== callback);
24         }
25     }
26
27     // 触发事件
28     $emit(event, ...args) {
29         if (this.events[event]) {
30             this.events[event].forEach(callback => callback.apply(this, args));
31         }
32     }
33
34     // 只监听一次事件
35     $once(event, callback) {
36         const wrapper = (...args) => {
37             callback.apply(this, args);
38             this.$off(event, wrapper);
39         };
40         this.$on(event, wrapper);
41     }
42 }
43
44 // 示例
45 const eventBus = new EventEmitter();
46
47 // 监听事件
```

```
48   EventBus.$on('test', (msg) => console.log('test event:', msg));
49
50   // 触发事件
51   EventBus.$emit('test', 'Hello, World!');
52
53   // 监听一次事件
54   EventBus.$once('once', (msg) => console.log('once event:', msg));
55
56   // 触发一次性事件
57   EventBus.$emit('once', 'This should appear once');
58   EventBus.$emit('once', 'This should not appear');
59
60   // 停止监听事件
61   EventBus.$off('test');
62
63   // 触发事件 (已经移除监听)
64   EventBus.$emit('test', 'This should not appear');
```

解释

1. `$on`：将事件和回调函数添加到 `events` 对象中。
2. `$off`：如果没有传递回调函数，则移除所有监听。如果传递了回调函数，则只移除特定的回调。
3. `$emit`：触发事件，调用所有注册的回调函数并传递参数。
4. `$once`：使用一个包装函数 (`wrapper`) 包装原始回调函数，确保回调只执行一次，然后移除事件监听。

第5题：computed 计算值为什么还可以依赖另外一个 computed 计算值？

参考答案：

在 Vue 中，`computed` 计算属性可以依赖其他 `computed` 计算属性，因为 Vue 的响应式系统能够正确地追踪依赖关系。

以下是具体的原理和机制：

响应式系统的依赖追踪

Vue 的响应式系统通过依赖收集和依赖追踪来实现。当一个响应式属性被读取时，Vue 会记录下当前的依赖（即哪个组件或者计算属性正在读取这个值）。当这个属性发生变化时，Vue 会通知这些依赖进行更新。

`computed` 计算属性的实现

`computed` 计算属性本质上是具有缓存功能的特殊方法。它们只有在其依赖的响应式属性发生变化时才会重新计算，否则返回缓存的值。

依赖其他计算属性

当一个 `computed` 计算属性依赖于另一个 `computed` 计算属性时，Vue 能够正确地追踪这些依赖关系。这是因为：

1. 依赖收集：当第一个计算属性被访问时，它的 `getter` 会被调用，并且 Vue 会记录下这个依赖关系。这个过程包括记录依赖的其他计算属性。
2. 缓存机制：如果另一个计算属性的值没有改变，Vue 不会重复计算它，而是直接使用缓存值。这保证了性能优化。
3. 更新机制：当基础的响应式属性发生变化时，所有依赖它的计算属性都会被重新计算，并且最终更新到组件的渲染中。

示例

下面是一个具体的例子来说明这个过程：

```
1  new Vue({
2    data() {
3      return {
4        a: 1,
5        b: 2
6      };
7    },
8    computed: {
9      sum() {
10       return this.a + this.b;
11     },
12     doubleSum() {
13       return this.sum * 2;
14     }
15   }
16 });
```

在这个例子中：

1. 依赖收集：
 - 当 `doubleSum` 被访问时，Vue 会调用其 `getter`，发现它依赖于 `sum`。
 - 然后 Vue 会访问 `sum`，记录下 `sum` 依赖于 `a` 和 `b`。
2. 缓存机制：

- 如果 `a` 和 `b` 没有改变，再次访问 `doubleSum` 时，Vue 直接返回缓存的值，不会重复计算 `sum`。

3. 更新机制：

- 如果 `a` 或 `b` 发生改变，Vue 会重新计算 `sum`，然后更新 `doubleSum`。

总结

Vue 的响应式系统能够正确地追踪 `computed` 计算属性之间的依赖关系，因为它使用了依赖收集和缓存机制。当基础的响应式属性发生变化时，所有依赖关系都会被重新计算并更新，从而保持应用状态的一致性。

第6题：说一下 `vm.$set` 原理

`vm.$set` 是 Vue 中用于在对象上设置属性并确保新属性是响应式的方法。其实现原理可以简化为以下几个步骤：

1. 处理数组情况：如果目标是数组，并且键是有效的数组索引，使用 `splice` 方法添加新元素以保持响应性。
2. 处理已有属性：如果属性已经存在于对象中，直接赋值。
3. 处理新属性：如果目标对象不是响应式对象，直接赋值新属性。
4. 添加响应式新属性：如果目标对象是响应式的，通过 `defineReactive` 方法将新属性定义为响应式。这包括定义 `getter` 和 `setter`。
5. 通知依赖更新：调用 `ob.dep.notify()` 通知所有依赖于该对象的 `watchers` 执行更新。

`defineReactive` 简要实现

`defineReactive` 方法定义对象属性为响应式，主要步骤：

- 依赖管理：创建一个 `Dep` 实例管理依赖。
- 递归观察：使用 `observe` 递归地将属性值转化为响应式。
- 定义 `getter` 和 `setter`：使用 `Object.defineProperty` 定义属性的 `getter` 和 `setter`。在 `getter` 中收集依赖，在 `setter` 中通知依赖更新。

总结

`vm.$set` 使得在运行时动态添加的新属性能够响应数据变化，从而保持 Vue 的响应式特性。

第7题：怎么在 Vue 中定义全局方法？

在 Vue.js 中定义全局方法，可以通过多种方式实现，包括直接在 Vue 的原型对象上添加方法、使用 Vue 3 的全局 API (`app.config.globalProperties`)、以及通过混入 (`mixin`) 等方法。

以下是几种常见的方法：

方法一：在 Vue 2 中通过 Vue.prototype 定义全局方法

```
1  // main.js
2  import Vue from 'vue';
3  import App from './App.vue';
4
5  Vue.config.productionTip = false;
6
7  // 定义全局方法
8  Vue.prototype.$myGlobalMethod = function () {
9    console.log('这是一个全局方法');
10 };
11
12 new Vue({
13   render: h => h(App),
14 }).$mount('#app');
```

在组件中使用

```
1  <template>
2    <div>
3      <button @click="useGlobalMethod">调用全局方法</button>
4    </div>
5  </template>
6
7  <script>
8  export default {
9    methods: {
10      useGlobalMethod() {
11        this.$myGlobalMethod();
12      }
13    }
14  }
15 </script>
```

方法二：在 Vue 3 中通过 `app.config.globalProperties` 定义全局方法

```
1  // main.js
2  import { createApp } from 'vue';
3  import App from './App.vue';
```

```

4
5  const app = createApp(App);
6
7  // 定义全局方法
8  app.config.globalProperties.$myGlobalMethod = function () {
9    console.log('这是一个全局方法');
10 };
11
12 app.mount('#app');

```

在组件中使用

```

1  <template>
2    <div>
3      <button @click="useGlobalMethod">调用全局方法</button>
4    </div>
5  </template>
6
7  <script>
8  export default {
9    methods: {
10      useGlobalMethod() {
11        this.$myGlobalMethod();
12      }
13    }
14  }
15 </script>

```

方法三：使用混入（Mixin）

你可以创建一个混入对象并将其全局注册，从而在所有组件中使用这个混入对象定义的方法。

```

1  // globalMixin.js
2  export const globalMixin = {
3    methods: {
4      $myGlobalMethod() {
5        console.log('这是一个全局方法');
6      }
7    }
8  };

```

```

1  // main.js
2  import Vue from 'vue';
3  import App from './App.vue';
4  import { globalMixin } from './globalMixin';
5
6  Vue.config.productionTip = false;
7
8  // 全局混入
9  Vue.mixin(globalMixin);
10
11 new Vue({
12   render: h => h(App),
13 }).$mount('#app');
```

在组件中使用

```

1  <template>
2    <div>
3      <button @click="useGlobalMethod">调用全局方法</button>
4    </div>
5  </template>
6
7  <script>
8    export default {
9      methods: {
10        useGlobalMethod() {
11          this.$myGlobalMethod();
12        }
13      }
14    }
15  </script>
```

方法四：创建插件

你可以创建一个 Vue 插件来封装全局方法，并在 `main.js` 中安装插件。

```

1  // myPlugin.js
2  export default {
3    install(Vue) {
4      Vue.prototype.$myGlobalMethod = function () {
5        console.log('这是一个全局方法');
6      }
7    }
8  }
```

```
8   };
```

```
1  // main.js
2  import Vue from 'vue';
3  import App from './App.vue';
4  import myPlugin from './myPlugin';
5
6  Vue.config.productionTip = false;
7
8  // 安装插件
9  Vue.use(myPlugin);
10
11 new Vue({
12   render: h => h(App),
13 }).$mount('#app');
```

在组件中使用

```
1  <template>
2    <div>
3      <button @click="useGlobalMethod">调用全局方法</button>
4    </div>
5  </template>
6
7  <script>
8    export default {
9      methods: {
10        useGlobalMethod() {
11          this.$myGlobalMethod();
12        }
13      }
14    }
15  </script>
```

第8题：Vue 中父组件怎么监听到子组件的生命周期？

在 Vue.js 中，父组件不能直接监听子组件的生命周期钩子。然而，有几种方法可以实现父组件对子组件生命周期的间接监听或执行特定操作。

方法一：通过事件通信

子组件在生命周期钩子中触发自定义事件，父组件监听这些事件。

1. 子组件

```
1  <template>
2    <div>子组件</div>
3  </template>
4
5  <script>
6    export default {
7      name: 'ChildComponent',
8      mounted() {
9        this.$emit('childMounted');
10     }
11   }
12 </script>
```

2. 父组件

```
1  <template>
2    <div>
3      <ChildComponent @childMounted="handleChildMounted" />
4    </div>
5  </template>
6
7  <script>
8    import ChildComponent from './ChildComponent.vue';
9
10   export default {
11     components: {
12       ChildComponent
13     },
14     methods: {
15       handleChildMounted() {
16         console.log('子组件已挂载');
17       }
18     }
19   }
20 </script>
```

方法二：使用 `ref`

父组件通过 `ref` 直接访问子组件实例，并在父组件的生命周期钩子中调用子组件的方法。

3. 子组件

```
1 <template>
2   <div>子组件</div>
3 </template>
4
5 <script>
6 export default {
7   name: 'ChildComponent',
8   mounted() {
9     console.log('子组件已挂载');
10  }
11 }
12 </script>
```

4. 父组件

```
1 <template>
2   <div>
3     <ChildComponent ref="child" />
4   </div>
5 </template>
6
7 <script>
8 import ChildComponent from './ChildComponent.vue';
9
10 export default {
11   components: {
12     ChildComponent
13   },
14   mounted() {
15     this.$refs.child.$on('hook:mounted', this.handleChildMounted);
16   },
17   methods: {
18     handleChildMounted() {
19       console.log('通过 ref 获取子组件的挂载');
20     }
21   }
22 }
23 </script>
```

方法三：使用 `provide` 和 `inject`

通过 `provide` 和 `inject` 实现跨层级组件通信。

5. 子组件

```
1 <template>
2   <div>子组件</div>
3 </template>
4
5 <script>
6 export default {
7   name: 'ChildComponent',
8   inject: ['notifyParent'],
9   mounted() {
10     this.notifyParent('mounted');
11   }
12 }
13 </script>
```

6. 父组件

```
1 <template>
2   <div>
3     <ChildComponent />
4   </div>
5 </template>
6
7 <script>
8 import ChildComponent from './ChildComponent.vue';
9
10 export default {
11   components: {
12     ChildComponent
13   },
14   provide() {
15     return {
16       notifyParent: this.handleNotification
17     }
18   },
19   methods: {
20     handleNotification(hook) {
21       if (hook === 'mounted') {
22         console.log('子组件已挂载');
23       }
24     }
25   }
26 }
```

```
26   }
27   </script>
```

第9题：vue组件里写的原生addEventListener监听事件，要手动去销毁吗？为什么？

在 Vue 组件中，如果你使用 `addEventListener` 添加了原生的 DOM 事件监听器，通常需要在组件销毁时手动移除这些监听器。

原因如下：

1. 内存泄漏：如果不手动移除事件监听器，监听器会继续存在于内存中，即使对应的 DOM 元素已经被移除。这会导致内存泄漏，因为监听器持有对 DOM 元素的引用，导致垃圾回收机制无法回收这些元素。
2. 意外行为：如果监听器没有被移除，在组件销毁后这些监听器可能会继续响应事件，这可能导致应用程序的意外行为或错误。
3. 性能问题：随着时间的推移，未移除的事件监听器会堆积，导致性能下降，尤其是在频繁创建和销毁组件的情况下。

在 Vue 组件中，可以利用生命周期钩子来添加和移除事件监听器：

```
1   <template>
2     <div ref="myElement">点击我</div>
3   </template>
4
5   <script>
6     export default {
7       mounted() {
8         // 在组件挂载时添加事件监听器
9         this.$refs.myElement.addEventListener('click', this.handleClick);
10      },
11      beforeDestroy() {
12        // 在组件销毁前移除事件监听器
13        this.$refs.myElement.removeEventListener('click', this.handleClick);
14      },
15      methods: {
16        handleClick(event) {
17          console.log('元素被点击了');
18        }
19      }
20    };
21  </script>
```

第10题：说说 vue3 中的响应式设计原理

Vue 3 中的响应式原理可谓是非常之重要，通过学习 Vue3 的响应式原理，不仅能让我们学习到 Vue.js 的一些设计模式和思想，还能帮助我们提高项目开发效率和代码调试能力。

一、Vue 3 响应式使用

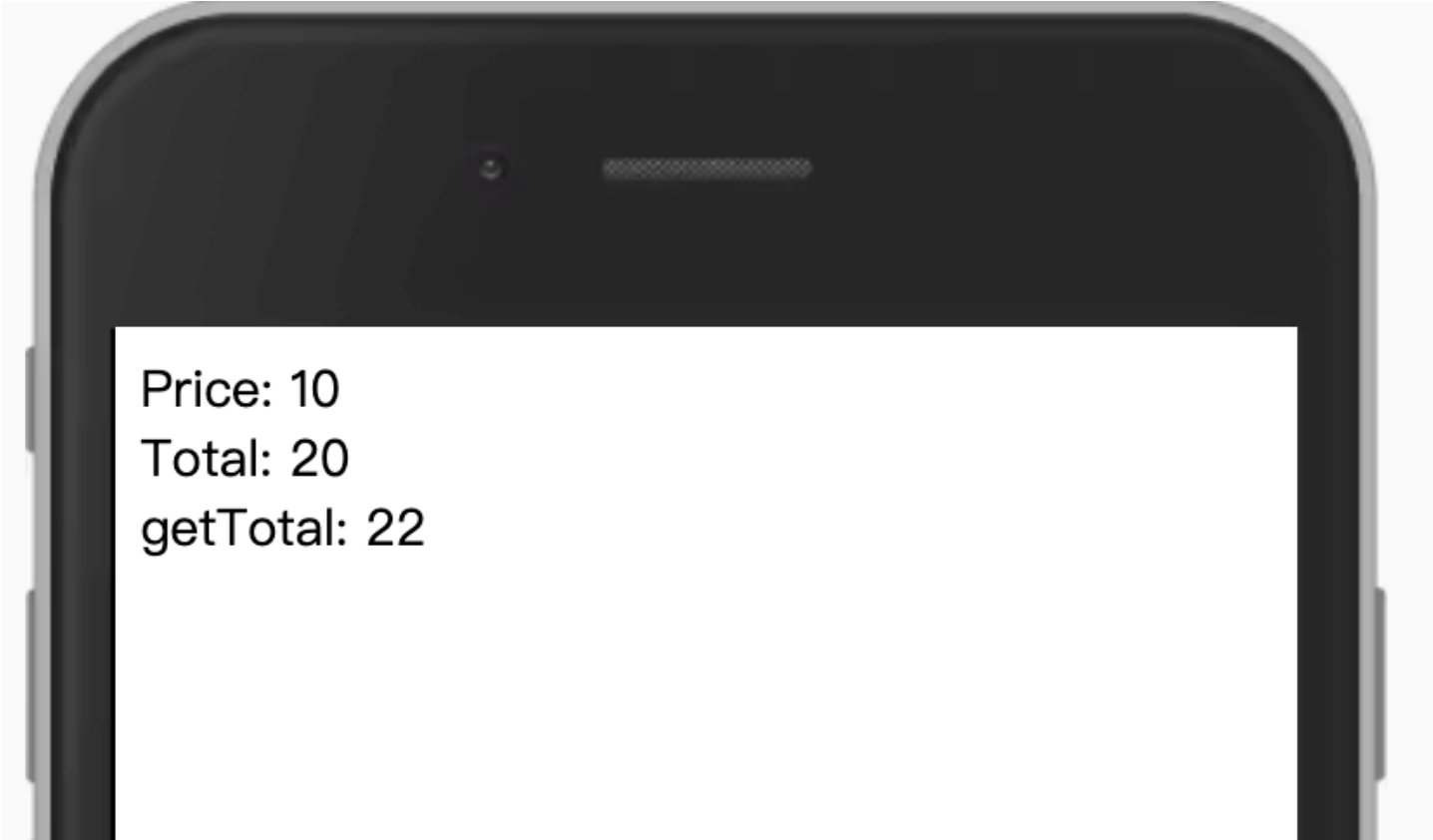
1. Vue 3 中的使用

当我们在学习 Vue 3 的时候，可以通过一个简单示例，看看什么是 Vue 3 中的响应式：

```
1  <!-- HTML 内容 -->
2  <div id="app">
3      <div>Price: {{price}}</div>
4      <div>Total: {{price * quantity}}</div>
5      <div>getTotal: {{getTotal}}</div>
6  </div>
```

```
1  const app = Vue.createApp({ // ① 创建 APP 实例
2      data() {
3          return {
4              price: 10,
5              quantity: 2
6          }
7      },
8      computed: {
9          getTotal() {
10             return this.price * this.quantity * 1.1
11          }
12      }
13 })
14 app.mount('#app') // ② 挂载 APP 实例
```

通过创建 APP 实例和挂载 APP 实例即可，这时可以看到页面中分别显示对应数值：



Price: 10
Total: 20
getTotal: 22

当我们修改 `price` 或 `quantity` 值的时候，页面上引用它们的地方，内容也能正常展示变化后的结果。这时，我们会好奇为何数据发生变化后，相关的数据也会跟着变化，那么我们接着往下看。

2. 实现单个值的响应式

在普通 JS 代码执行中，并不会响应式变化，比如在控制台执行下面代码：

```
1  let price = 10, quantity = 2;  
2  const total = price * quantity;  
3  console.log(`total: ${total}`); // total: 20  
4  price = 20;  
5  console.log(`total: ${total}`); // total: 20
```

从这可以看出，在修改 `price` 变量的值后，`total` 的值并没有发生改变。

那么如何修改上面代码，让 `total` 能够自动更新呢？我们其实可以将修改 `total` 值的方法保存起来，等到与 `total` 值相关的变量（如 `price` 或 `quantity` 变量的值）发生变化时，触发该方法，更新 `total` 即可。我们可以这么实现：

```
1  let price = 10, quantity = 2, total = 0;  
2  const dep = new Set(); // ①  
3  const effect = () => { total = price * quantity };  
4  const track = () => { dep.add(effect) }; // ②  
5  const trigger = () => { dep.forEach( effect => effect() )}; // ③
```

```

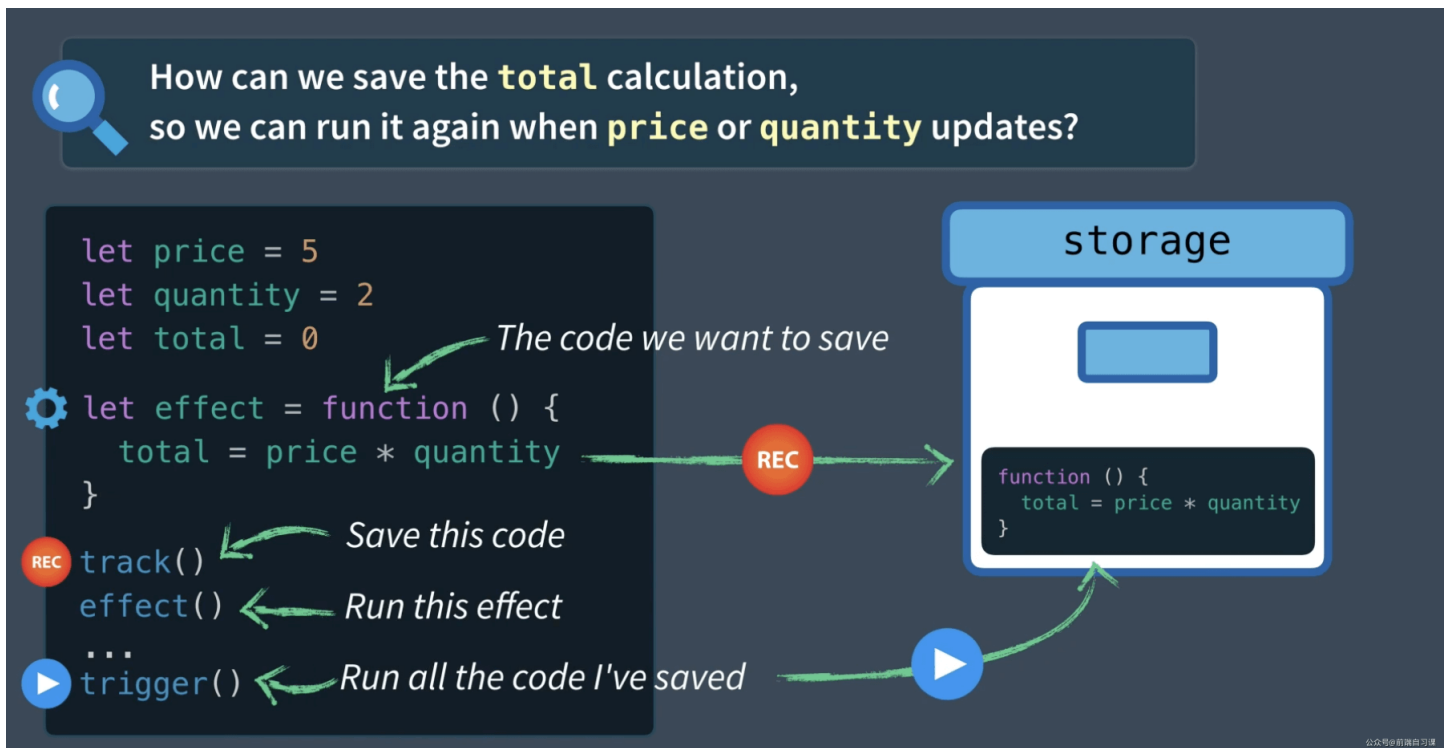
6
7  track();
8  console.log(`total: ${total}`); // total: 0
9  trigger();
10 console.log(`total: ${total}`); // total: 20
11 price = 20;
12 trigger();
13 console.log(`total: ${total}`); // total: 40

```

上面代码通过 3 个步骤，实现对 `total` 数据进行响应式变化：

- ① 初始化一个 `Set` 类型的 `dep` 变量，用来存放需要执行的副作用（`effect` 函数），这边是修改 `total` 值的方法；
- ② 创建 `track()` 函数，用来将需要执行的副作用保存到 `dep` 变量中（也称收集副作用）；
- ③ 创建 `trigger()` 函数，用来执行 `dep` 变量中的所有副作用；

在每次修改 `price` 或 `quantity` 后，调用 `trigger()` 函数执行所有副作用后，`total` 值将自动更新为最新值。



(图片来源：Vue Mastery)

3. 实现单个对象的响应式

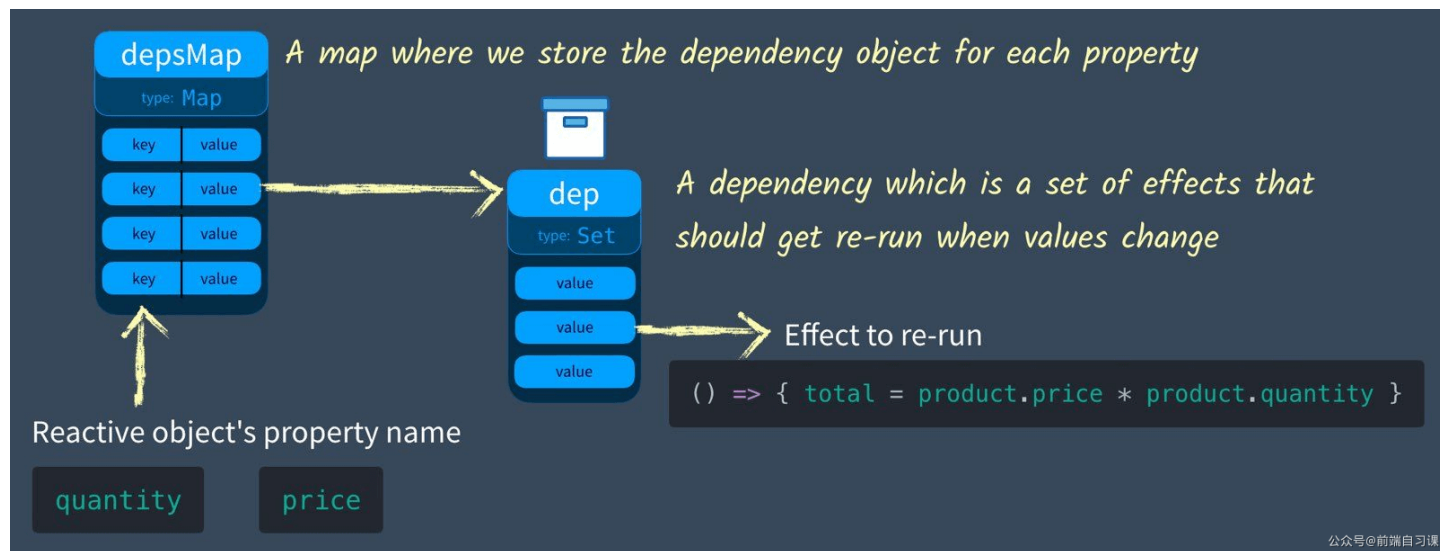
通常，我们的对象具有多个属性，并且每个属性都需要自己的 `dep`。我们如何存储这些？比如：

```

1  let product = { price: 10, quantity: 2 };

```

从前面介绍我们知道，我们将所有副作用保存在一个 `Set` 集合中，而该集合不会有重复项，这里我们引入一个 `Map` 类型集合（即 `depsMap` ），其 `key` 为对象的属性（如： `price` 属性），`value` 为前面保存副作用的 `Set` 集合（如： `dep` 对象），大致结构如下图：



（图片来源：Vue Mastery）

实现代码：

```
1  let product = { price: 10, quantity: 2 }, total = 0;
2  const depsMap = new Map(); // ①
3  const effect = () => { total = product.price * product.quantity };
4  const track = key => { // ②
5    let dep = depsMap.get(key);
6    if(!dep) {
7      depsMap.set(key, (dep = new Set()));
8    }
9    dep.add(effect);
10 }
11
12 const trigger = key => { // ③
13   let dep = depsMap.get(key);
14   if(dep) {
15     dep.forEach( effect => effect() );
16   }
17 };
18
19 track('price');
20 console.log(`total: ${total}`); // total: 0
21 effect();
22 console.log(`total: ${total}`); // total: 20
```

```
23 product.price = 20;
24 trigger('price');
25 console.log(`total: ${total}`); // total: 40
```

上面代码通过 3 个步骤，实现对 `total` 数据进行响应式变化：

- ① 初始化一个 `Map` 类型的 `depsMap` 变量，用来保存每个需要响应式变化的对象属性（`key` 为对象的属性，`value` 为前面 `Set` 集合）；
- ② 创建 `track()` 函数，用来将需要执行的副作用保存到 `depsMap` 变量中对应的对象属性下（也称收集副作用）；
- ③ 创建 `trigger()` 函数，用来执行 `dep` 变量中指定对象属性的所有副作用；

这样就实现监听对象的响应式变化，在 `product` 对象中的属性值发生变化，`total` 值也会跟着更新。

4. 实现多个对象的响应式

如果我们有多个响应式数据，比如同时需要观察对象 `a` 和对象 `b` 的数据，那么又要如何跟踪每个响应变化的对象？

这里我们引入一个 `WeakMap` 类型的对象，将需要观察的对象作为 `key`，值为前面用来保存对象属性的 `Map` 变量。代码如下：

```
1  let product = { price: 10, quantity: 2 }, total = 0;
2  const targetMap = new WeakMap(); // ① 初始化 targetMap, 保存观察对象
3  const effect = () => { total = product.price * product.quantity };
4  const track = (target, key) => { // ② 收集依赖
5    let depsMap = targetMap.get(target);
6    if(!depsMap){
7      targetMap.set(target, (depsMap = new Map()));
8    }
9    let dep = depsMap.get(key);
10   if(!dep) {
11     depsMap.set(key, (dep = new Set()));
12   }
13   dep.add(effect);
14 }
15
16 const trigger = (target, key) => { // ③ 执行指定对象的指定属性的所有副作用
17   const depsMap = targetMap.get(target);
18   if(!depsMap) return;
19   let dep = depsMap.get(key);
20   if(dep) {
21     dep.forEach( effect => effect() );
22   }
```

```

23  };
24
25  track(product, 'price');
26  console.log(`total: ${total}`); // total: 0
27  effect();
28  console.log(`total: ${total}`); // total: 20
29  product.price = 20;
30  trigger(product, 'price');
31  console.log(`total: ${total}`); // total: 40

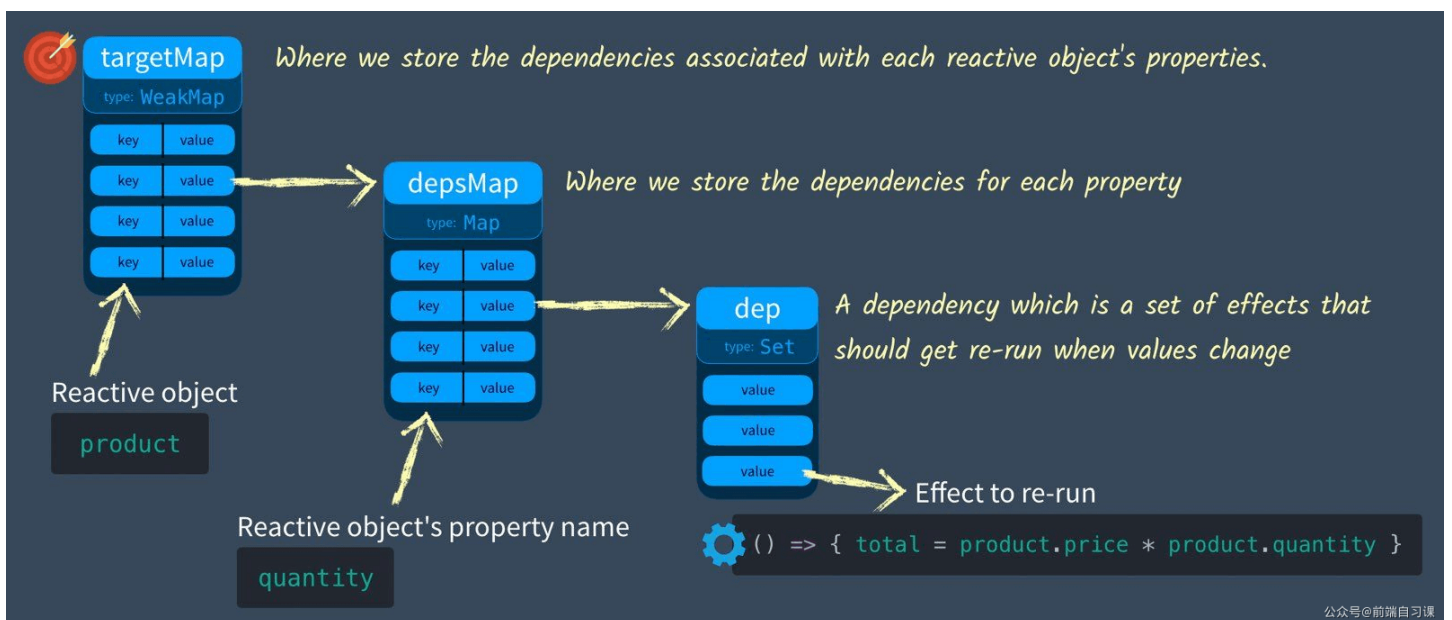
```

上面代码通过 3 个步骤，实现对 `total` 数据进行响应式变化：

- ① 初始化一个 `WeakMap` 类型的 `targetMap` 变量，用来要观察每个响应式对象；
- ② 创建 `track()` 函数，用来将需要执行的副作用保存到指定对象（`target`）的依赖中（也称收集副作用）；
- ③ 创建 `trigger()` 函数，用来执行指定对象（`target`）中指定属性（`key`）的所有副作用；

这样就实现监听对象的响应式变化，在 `product` 对象中的属性值发生变化，`total` 值也会跟着更新。

大致流程如下图：



(图片来源：Vue Mastery)

二、Proxy 和 Reflect

在上一节内容中，介绍了如何在数据发生变化后，自动更新数据，但存在的问题是，每次需要手动通过触发 `track()` 函数搜集依赖，通过 `trigger()` 函数执行所有副作用，达到数据更新目的。

这一节将来解决这个问题，实现这两个函数自动调用。

5. 如何实现自动操作

这里我们引入 JS 对象访问器的概念，解决办法如下：

- 在读取（GET 操作）数据时，自动执行 `track()` 函数自动收集依赖；
- 在修改（SET 操作）数据时，自动执行 `trigger()` 函数执行所有副作用；

那么如何拦截 GET 和 SET 操作？接下来看看 Vue2 和 Vue3 是如何实现的：

- 在 Vue2 中，使用 ES5 的 `Object.defineProperty()` 函数实现；
- 在 Vue3 中，使用 ES6 的 `Proxy` 和 `Reflect` API 实现；

需要注意的是：Vue3 使用的 `Proxy` 和 `Reflect` API 并不支持 IE。

`Object.defineProperty()` 函数这边就不多做介绍，可以阅读文档，下文将主要介绍 `Proxy` 和 `Reflect` API。

6. 如何使用 Reflect

通常我们有三种方法读取一个对象的属性：

1. 使用 `.` 操作符： `leo.name` ；
2. 使用 `[]` ： `leo['name']` ；
3. 使用 `Reflect` API： `Reflect.get(leo, 'name')` 。

这三种方式输出结果相同。

4. 如何使用 Proxy

`Proxy` 对象用于创建一个对象的代理，从而实现基本操作的拦截和自定义（如属性查找、赋值、枚举、函数调用等）。语法如下：

```
1  const p = new Proxy(target, handler)
```

参数如下：

- `target`：要使用 `Proxy` 包装的目标对象（可以是任何类型的对象，包括原生数组，函数，甚至另一个代理）。
- `handler`：一个通常以函数作为属性的对象，各属性中的函数分别定义了在执行各种操作时代理 `p` 的行为。

我们通过官方文档，体验一下 `Proxy` API：

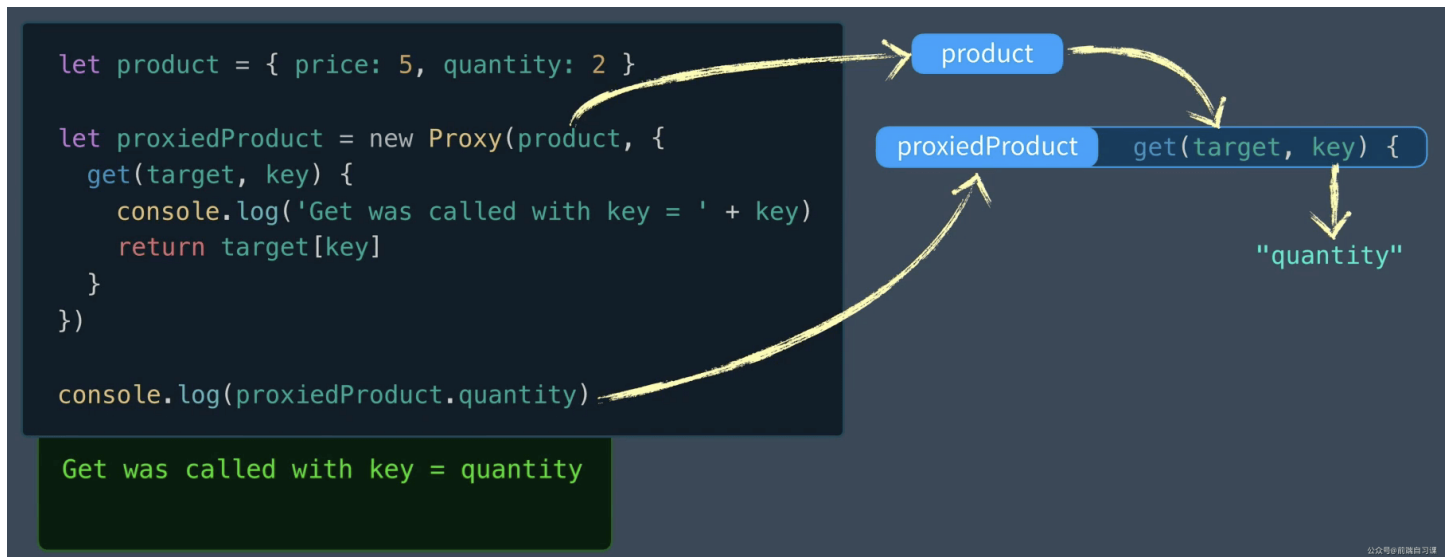
```
1  let product = { price: 10, quantity: 2 };
2  let proxiedProduct = new Proxy(product, {
3    get(target, key){
4      console.log('正在读取的数据: ',key);
```

```

5     return target[key];
6   }
7 })
8 console.log(proxyedProduct.price);
9 // 正在读取的数据: price
10 // 10

```

这样就保证我们每次在读取 `proxiedProduct.price` 都会执行到其中代理的 `get` 处理函数。其过程如下：



(图片来源: Vue Mastery)

然后结合 `Reflect` 使用，只需修改 `get` 函数：

```

1   get(target, key, receiver){
2     console.log('正在读取的数据: ',key);
3     return Reflect.get(target, key, receiver);
4   }

```

输出结果还是一样。

接下来增加 `set` 函数，来拦截对象的修改操作：

```

1   let product = { price: 10, quantity: 2 };
2   let proxiedProduct = new Proxy(product, {
3     get(target, key, receiver){
4       console.log('正在读取的数据: ',key);
5       return Reflect.get(target, key, receiver);
6     },

```

```

7     set(target, key, value, receiver){
8         console.log('正在修改的数据: ', key, ',值为: ', value);
9         return Reflect.set(target, key, value, receiver);
10    }
11 })
12 proxiedProduct.price = 20;
13 console.log(proxiedProduct.price);
14 // 正在修改的数据:  price ,值为:  20
15 // 正在读取的数据:  price
16 // 20

```

这样便完成 get 和 set 函数来拦截对象的读取和修改的操作。为了方便对比 Vue 3 源码，我们将上面代码抽象一层，使它看起来更像 Vue3 源码：

```

1  function reactive(target){
2      const handler = { // ① 封装统一处理函数对象
3          get(target, key, receiver){
4              console.log('正在读取的数据: ',key);
5              return Reflect.get(target, key, receiver);
6          },
7          set(target, key, value, receiver){
8              console.log('正在修改的数据: ', key, ',值为: ', value);
9              return Reflect.set(target, key, value, receiver);
10         }
11     }
12
13     return new Proxy(target, handler); // ② 统一调用 Proxy API
14 }
15
16 let product = reactive({price: 10, quantity: 2}); // ③ 将对象转换为响应式对象
17 product.price = 20;
18 console.log(product.price);
19 // 正在修改的数据:  price ,值为:  20
20 // 正在读取的数据:  price
21 // 20

```

这样输出结果仍然不变。

5. 修改 track 和 trigger 函数

通过上面代码，我们已经实现一个简单 `reactive()` 函数，用来将普通对象转换为响应式对象。但是还缺少自动执行 `track()` 函数和 `trigger()` 函数，接下来修改上面代码：

```

1  const targetMap = new WeakMap();

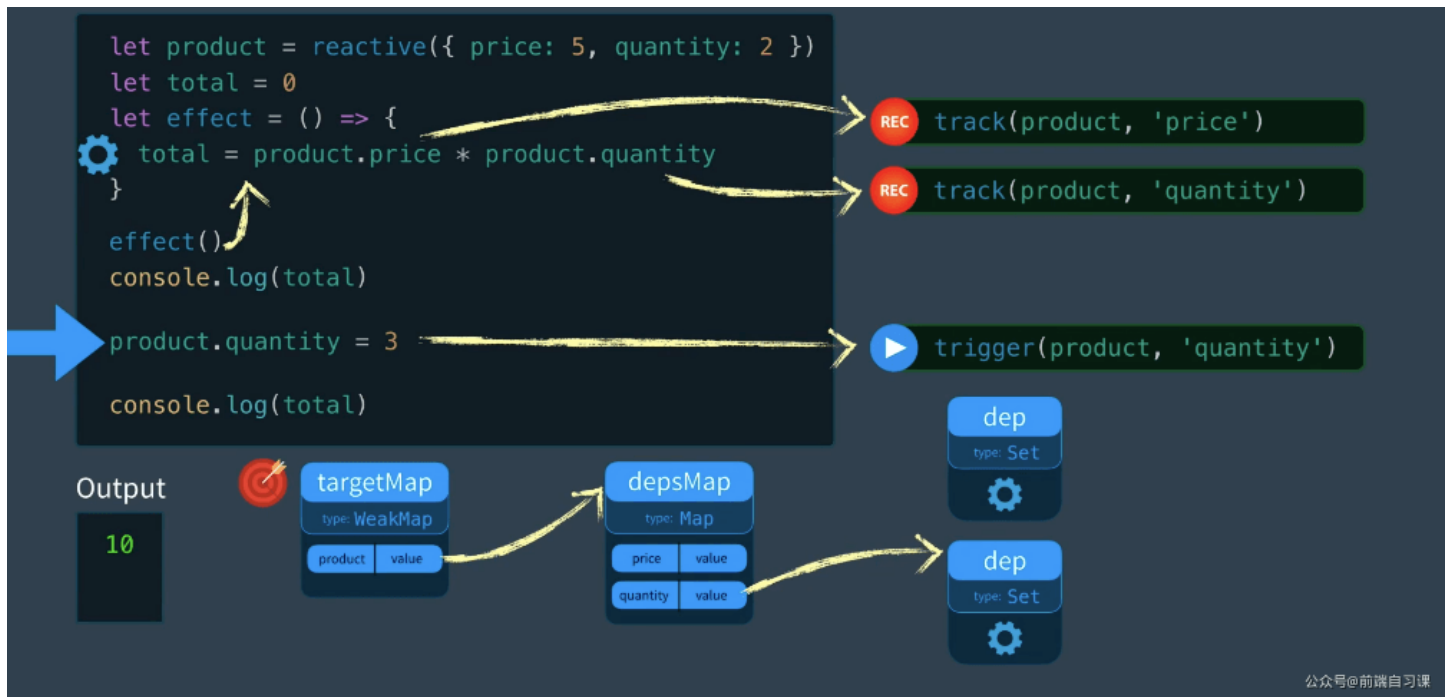
```

```
2  let total = 0;
3  const effect = () => { total = product.price * product.quantity };
4  const track = (target, key) => {
5    let depsMap = targetMap.get(target);
6    if(!depsMap){
7      targetMap.set(target, (depsMap = new Map()));
8    }
9    let dep = depsMap.get(key);
10   if(!dep) {
11     depsMap.set(key, (dep = new Set()));
12   }
13   dep.add(effect);
14 }
15
16 const trigger = (target, key) => {
17   const depsMap = targetMap.get(target);
18   if(!depsMap) return;
19   let dep = depsMap.get(key);
20   if(dep) {
21     dep.forEach( effect => effect() );
22   }
23 };
24
25 const reactive = (target) => {
26   const handler = {
27     get(target, key, receiver){
28       console.log('正在读取的数据: ',key);
29       const result = Reflect.get(target, key, receiver);
30       track(target, key); // 自动调用 track 方法收集依赖
31       return result;
32     },
33     set(target, key, value, receiver){
34       console.log('正在修改的数据: ', key, ',值为: ', value);
35       const oldValue = target[key];
36       const result = Reflect.set(target, key, value, receiver);
37       if(oldValue !== result){
38         trigger(target, key); // 自动调用 trigger 方法执行依赖
39       }
40       return result;
41     }
42   }
43
44   return new Proxy(target, handler);
45 }
46
47 let product = reactive({price: 10, quantity: 2});
48 effect();
```

```

49 console.log(total);
50 product.price = 20;
51 console.log(total);
52 // 正在读取的数据: price
53 // 正在读取的数据: quantity
54 // 20
55 // 正在修改的数据: price ,值为: 20
56 // 正在读取的数据: price
57 // 正在读取的数据: quantity
58 // 40

```



(图片来源: Vue Mastery)

三、activeEffect 和 ref

在上一节代码中, 还存在一个问题: `track` 函数中的依赖 (`effect` 函数) 是外部定义的, 当依赖发生变化, `track` 函数收集依赖时都要手动修改其依赖的方法名。

比如现在的依赖为 `foo` 函数, 就要修改 `track` 函数的逻辑, 可能是这样:

```

1 const foo = () => { /**/ };
2 const track = (target, key) => { // ②
3   // ...
4   dep.add(foo);
5 }

```

那么如何解决这个问题呢?

6. 引入 activeEffect 变量

接下来引入 `activeEffect` 变量，来保存当前运行的 effect 函数。

```
1  let activeEffect = null;
2  const effect = eff => {
3    activeEffect = eff; // 1. 将 eff 函数赋值给 activeEffect
4    activeEffect();      // 2. 执行 activeEffect
5    activeEffect = null; // 3. 重置 activeEffect
6  }
```

然后在 `track` 函数中将 `activeEffect` 变量作为依赖：

```
1  const track = (target, key) => {
2    if (activeEffect) { // 1. 判断当前是否有 activeEffect
3      let depsMap = targetMap.get(target);
4      if (!depsMap) {
5        targetMap.set(target, (depsMap = new Map()));
6      }
7      let dep = depsMap.get(key);
8      if (!dep) {
9        depsMap.set(key, (dep = new Set()));
10     }
11     dep.add(activeEffect); // 2. 添加 activeEffect 依赖
12   }
13 }
```

使用方式修改为：

```
1  effect(() => {
2    total = product.price * product.quantity
3  });
```

这样就可以解决手动修改依赖的问题，这也是 Vue3 解决该方法。完善一下测试代码后，如下：

```
1  const targetMap = new WeakMap();
2  let activeEffect = null; // 引入 activeEffect 变量
3
4  const effect = eff => {
5    activeEffect = eff; // 1. 将副作用赋值给 activeEffect
6    activeEffect();      // 2. 执行 activeEffect
```

```
7     activeEffect = null; // 3. 重置 activeEffect
8 }
9
10 const track = (target, key) => {
11     if (activeEffect) { // 1. 判断当前是否有 activeEffect
12         let depsMap = targetMap.get(target);
13         if (!depsMap) {
14             targetMap.set(target, (depsMap = new Map()));
15         }
16         let dep = depsMap.get(key);
17         if (!dep) {
18             depsMap.set(key, (dep = new Set()));
19         }
20         dep.add(activeEffect); // 2. 添加 activeEffect 依赖
21     }
22 }
23
24 const trigger = (target, key) => {
25     const depsMap = targetMap.get(target);
26     if (!depsMap) return;
27     let dep = depsMap.get(key);
28     if (dep) {
29         dep.forEach(effect => effect());
30     }
31 };
32
33 const reactive = (target) => {
34     const handler = {
35         get(target, key, receiver) {
36             const result = Reflect.get(target, key, receiver);
37             track(target, key);
38             return result;
39         },
40         set(target, key, value, receiver) {
41             const oldValue = target[key];
42             const result = Reflect.set(target, key, value, receiver);
43             if (oldValue !== result) {
44                 trigger(target, key);
45             }
46             return result;
47         }
48     };
49
50     return new Proxy(target, handler);
51 }
52
53 let product = reactive({ price: 10, quantity: 2 });
```

```

54 let total = 0, salePrice = 0;
55 // 修改 effect 使用方式，将副作用作为参数传给 effect 方法
56 effect(() => {
57     total = product.price * product.quantity
58 });
59 effect(() => {
60     salePrice = product.price * 0.9
61 });
62 console.log(total, salePrice); // 20 9
63 product.quantity = 5;
64 console.log(total, salePrice); // 50 9
65 product.price = 20;
66 console.log(total, salePrice); // 100 18

```

思考一下，如果把第一个 `effect` 函数中 `product.price` 换成 `salePrice` 会如何：

```

1  effect(() => {
2      total = salePrice * product.quantity
3  });
4  effect(() => {
5      salePrice = product.price * 0.9
6  });
7  console.log(total, salePrice); // 0 9
8  product.quantity = 5;
9  console.log(total, salePrice); // 45 9
10 product.price = 20;
11 console.log(total, salePrice); // 45 18

```

得到的结果完全不同，因为 `salePrice` 并不是响应式变化，而是需要调用第二个 `effect` 函数才会变化，也就是 `product.price` 变量值发生变化。

7. 引入 ref 方法

熟悉 Vue3 Composition API 的朋友可能会想到 Ref，它接收一个值，并返回一个响应式可变的 [Ref 对象](#)，其值可以通过 `value` 属性获取。

ref：接受一个内部值并返回一个响应式且可变的 ref 对象。ref 对象具有指向内部值的单个 property `.value`。

官网的使用示例如下：

```

1  const count = ref(0)
2  console.log(count.value) // 0
3

```

```
4 count.value++
5 console.log(count.value) // 1
```

我们有 2 种方法实现 ref 函数：

1. 使用 `reactive` 函数

```
1 const ref = initialValue => reactive({value: initialValue});
```

这样是可以的，虽然 Vue3 不是这么实现。

1. 使用对象的属性访问器（计算属性）

属性方式包括：`getter` 和 `setter`。

```
1 const ref = raw => {
2   const r = {
3     get value(){
4       track(r, 'value');
5       return raw;
6     },
7
8     set value(newVal){
9       raw = newVal;
10      trigger(r, 'value');
11    }
12  }
13  return r;
14 }
```

使用方式如下：

```
1 let product = reactive({ price: 10, quantity: 2 });
2 let total = 0, salePrice = ref(0);
3 effect(() => {
4   salePrice.value = product.price * 0.9
5 });
6 effect(() => {
7   total = salePrice.value * product.quantity
8 });
9 console.log(total, salePrice.value); // 18 9
10 product.quantity = 5;
```

```
11 console.log(total, salePrice.value); // 45 9
12 product.price = 20;
13 console.log(total, salePrice.value); // 90 18
```

在 Vue3 中 ref 实现的核心也是如此。

四、实现简易 Computed 方法

用过 Vue 的同学可能会好奇，上面的 `salePrice` 和 `total` 变量为什么不使用 `computed` 方法呢？

没错，这个可以的，接下来一起实现个简单的 `computed` 方法。

```
1  const computed = getter => {
2    let result = ref();
3    effect(() => result.value = getter());
4    return result;
5  }
6
7  let product = reactive({ price: 10, quantity: 2 });
8  let salePrice = computed(() => {
9    return product.price * 0.9;
10 })
11 let total = computed(() => {
12   return salePrice.value * product.quantity;
13 })
14
15 console.log(total.value, salePrice.value);
16 product.quantity = 5;
17 console.log(total.value, salePrice.value);
18 product.price = 20;
19 console.log(total.value, salePrice.value);
```

这里我们将一个函数作为参数传入 `computed` 方法，`computed` 方法内通过 `ref` 方法构建一个 ref 对象，然后通过 `effect` 方法，将 `getter` 方法返回值作为 `computed` 方法的返回值。

这样我们实现了个简单的 `computed` 方法，执行效果和前面一样。

五、源码学习建议

2. 构建 reactivity.cjs.js

这一节介绍如何去从 [Vue 3 仓库](#) 打包一个 Reactivity 包来学习和使用。

准备流程如下：

1. 从 [Vue 3 仓库](#) 下载最新 Vue3 源码；

```
1 git clone https://github.com/vuejs/vue-next.git
```

1. 安装依赖：

```
1 yarn install
```

1. 构建 Reactivity 代码：

```
1 yarn build reactivity
```

1. 复制 reactivity.cjs.js 到你的学习 demo 目录：

上一步构建完的内容，会保存在 `packages/reactivity/dist` 目录下，我们只要在自己的学习 demo 中引入该目录的 `reactivity.cjs.js` 文件即可。

1. 学习 demo 中引入：

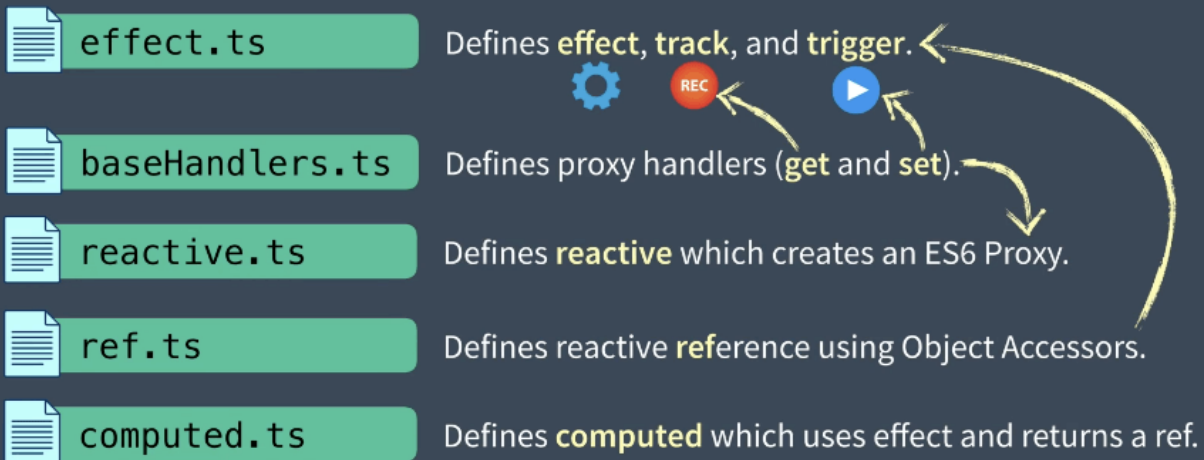
```
1 const { reactive, computed, effect } = require("./reactivity.cjs.js");
```

2. Vue3 Reactivity 文件目录

在源码的 `packages/reactivity/src` 目录下，有以下几个主要文件：

1. `effect.ts`：用来定义 `effect` / `track` / `trigger` ；
2. `baseHandlers.ts`：定义 Proxy 处理器（`get` 和 `set`）；
3. `reactive.ts`：定义 `reactive` 方法并创建 ES6 Proxy；
4. `ref.ts`：定义 `reactive` 的 `ref` 使用的对象访问器；
5. `computed.ts`：定义计算属性的方法；

Inside the `/packages/reactivity/src/` directory we have a few `.ts` (TypeScript files):



公众号@前端自习课

第11题：Vue中，created和mounted两个钩子之间调用时间差值受什么影响？

`created` 和 `mounted` 这两个生命周期钩子，分别在实例创建和挂载的不同阶段被调用。

它们之间的时间差值主要受以下几个因素的影响：

1. 模板编译时间：

- 当实例被创建时，Vue 会编译模板（或将模板转换为渲染函数），这个过程在 `created` 钩子之前完成。如果模板非常复杂或包含大量指令、组件，这个过程会更耗时，从而延长 `created` 和 `mounted` 之间的时间差。

2. 虚拟 DOM 渲染时间：

- 在 `mounted` 钩子调用之前，Vue 会将虚拟 DOM 渲染为实际的 DOM 元素。渲染复杂的组件树或处理大量数据绑定会增加这段时间。

3. 异步操作：

- 如果在 `created` 钩子中发起了异步操作（如 API 请求），这些操作本身不会直接影响 `created` 和 `mounted` 的时间差，但如果这些操作涉及数据更新，可能会间接增加挂载时间。

4. 浏览器性能：

- 浏览器的性能和设备的硬件配置也会影响模板编译和 DOM 渲染的速度，从而影响这两个钩子之间的时间差。

5. 其他钩子执行时间：

- 在 `beforeCreate`、`created`、`beforeMount` 等钩子中执行的代码也会影响到 `mounted` 钩子的触发时间。如果这些钩子中有大量计算或耗时操作，也会增加时间差。

总结起来，`created` 和 `mounted` 之间的时间差主要受到模板编译、虚拟 DOM 渲染的复杂性、异步操作、浏览器性能及其他生命周期钩子中执行代码的影响。在编写 Vue 应用时，优化这些方面可以

减少 `created` 和 `mounted` 之间的时间差，提高应用性能。

第12题：vue中，推荐在哪个生命周期发起请求？

推荐在 `mounted` 生命周期钩子中发起请求。这样做有几个重要的理由：

1. 确保 DOM 已经被渲染：

- `mounted` 钩子在组件的 DOM 已经被插入文档之后调用。这意味着你可以确保所有的 DOM 元素都已经存在，如果你的请求结果需要直接操作或依赖这些 DOM 元素，那么在 `mounted` 中发起请求是安全的。

2. 避免不必要的请求：

- 在 `created` 钩子中发起请求有时会导致在组件还没有挂载时请求数据。如果组件在请求完成之前被销毁，可能会引发内存泄漏或不必要的资源浪费。因此，等待组件挂载完成再发起请求可以减少这些潜在问题。

3. 处理组件状态：

- 在 `mounted` 钩子中发起请求，能够确保你有机会在请求开始前处理组件的状态（例如设置加载状态），并且在请求完成后更新组件的状态（例如显示数据或处理错误）。

尽管 `mounted` 是推荐的生命周期钩子，但也有一些特定场景可能需要在 `created` 钩子中发起请求，例如：

- SSR（服务器端渲染）：在服务器端渲染中，Vue 实例的 `mounted` 钩子不会被调用，因为 DOM 并不会被真正挂载。在这种情况下，你可能需要在 `created` 钩子中发起请求。
- 依赖数据初始化：如果组件在挂载之前就需要某些数据来初始化，可以在 `created` 钩子中发起请求，以确保数据在组件挂载时已经可用。

代码示例

```
1  export default {
2    data() {
3      return {
4        items: [],
5        loading: false,
6        error: null
7      };
8    },
9    mounted() {
10     this.fetchData();
11   },
12   methods: {
13     async fetchData() {
14       this.loading = true;
```

```
15     try {
16         const response = await axios.get('/api/items');
17         this.items = response.data;
18     } catch (error) {
19         this.error = error;
20     } finally {
21         this.loading = false;
22     }
23 }
24 }
25 };
```

第13题：为什么 react 需要 fiber 架构，而 Vue 却不需要？

React引入Fiber架构的主要原因是为了实现更好的异步渲染和更高效的任务调度。Fiber架构使得React能够更细粒度地控制和中断渲染过程，以便更好地响应用户交互、实现懒加载等功能。Vue在设计上采用了不同的策略，因此并不需要类似于Fiber的架构。

以下是一些原因解释为什么React选择了Fiber架构，而Vue没有类似的架构：

1. 异步渲染和任务优先级： React的Fiber架构使得实现异步渲染和任务优先级变得更加容易。这对于复杂的用户界面和大规模应用中的性能优化非常重要。React可以通过中断和恢复渲染过程，根据任务的优先级调度渲染工作，从而更好地响应用户输入和满足实时性要求。
2. 更好的中断和恢复机制： Fiber架构提供了一种更灵活的中断和恢复机制，允许React在渲染过程中暂停、中断，然后根据优先级恢复。这使得React能够更好地处理复杂的渲染逻辑，并在需要时放弃低优先级的工作。
3. 增量更新： Fiber允许React实现增量更新，即只更新变化的部分而不必重新渲染整个组件树。这对于提高渲染性能和减少不必要的工作非常有帮助。

Vue在设计上采用了一种不同的响应式系统和渲染机制，不需要像React那样进行复杂的中断和任务调度。Vue的设计目标可能更注重简洁性和开发体验，而React的目标之一是提供更灵活和强大的性能优化工具。每个框架在设计上都有权衡和取舍，选择适合其目标和使用场景的策略。

第14题：SPA（单页应用）首屏加载速度慢怎么解决？

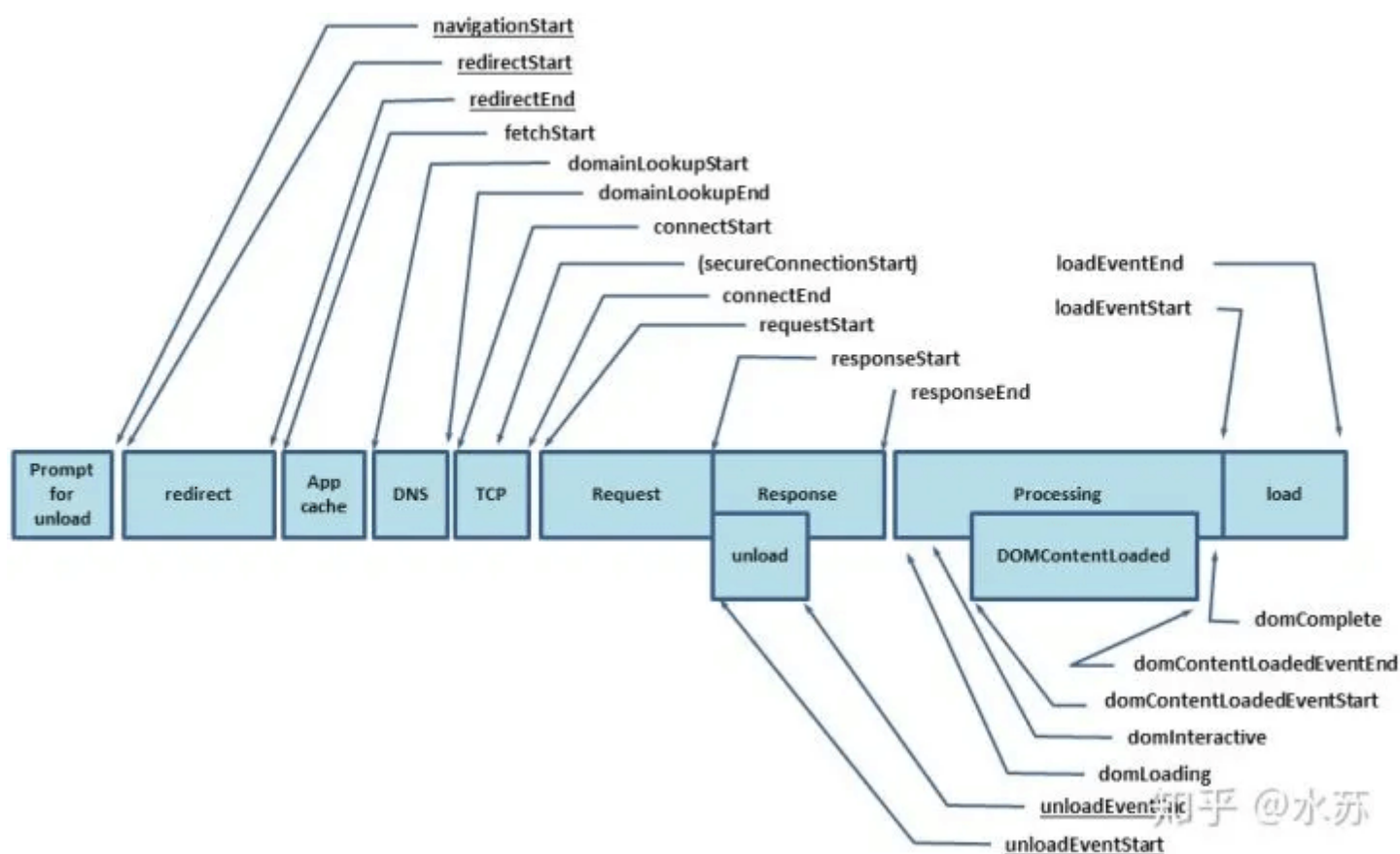
一、什么是首屏加载

首屏时间（First Contentful Paint），指的是浏览器从响应用户输入网址地址，到首屏内容渲染完成的时间，此时整个网页不一定要全部渲染完成，但需要展示当前视窗需要的内容

首屏加载可以说是用户体验中最重要的一环

关于计算首屏时间

利用 `performance.timing` 提供的的数据：



通过 `DOMContentLoaded` 或者 `performance` 来计算出首屏时间

```
1 // 方案一：
2 document.addEventListener('DOMContentLoaded', (event) => {
3     console.log('first contentful painting');
4 });
5 // 方案二：
6 performance.getEntriesByName("first-contentful-paint")[0].startTime
7
8 // performance.getEntriesByName("first-contentful-paint")[0]
9 // 会返回一个 PerformancePaintTiming的实例，结构如下：
10 {
11     name: "first-contentful-paint",
12     entryType: "paint",
13     startTime: 507.800000002123415,
14     duration: 0,
15 };
```

二、加载慢的原因

在页面渲染的过程，导致加载速度慢的因素可能如下：

- 网络延时问题
- 资源文件体积是否过大
- 资源是否重复发送请求去加载了
- 加载脚本的时候，渲染内容堵塞了

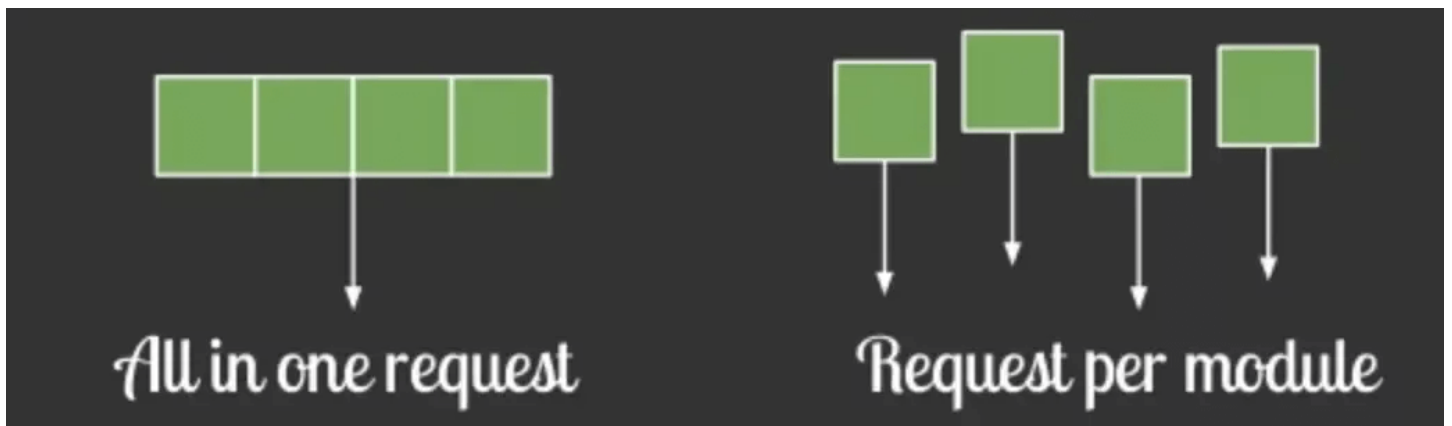
三、解决方案

常见的几种SPA首屏优化方式

- 减小入口文件体积
- 静态资源本地缓存
- UI框架按需加载
- 图片资源的压缩
- 组件重复打包
- 开启GZip压缩
- 使用SSR

减小入口文件体积

常用的手段是路由懒加载，把不同路由对应的组件分割成不同的代码块，待路由被请求的时候会单独打包路由，使得入口文件变小，加载速度大大增加



在 `vue-router` 配置路由的时候，采用动态加载路由的形式

```
1 routes:[
2   path: 'Blogs',
3   name: 'ShowBlogs',
4   component: () => import('./components/ShowBlogs.vue')
5 ]
```

以函数的形式加载路由，这样就可以把各自的路由文件分别打包，只有在解析给定的路由时，才会加载路由组件

静态资源本地缓存

后端返回资源问题：

- 采用 HTTP 缓存，设置 Cache-Control，Last-Modified，Etag 等响应头
- 采用 Service Worker 离线缓存

前端合理利用 localStorage

UI框架按需加载

在日常使用 UI 框架，例如 element-ui、或者 antd，我们通常会直接引用整个 UI 库

```
1 import ElementUI from 'element-ui'
2 Vue.use(ElementUI)
```

但实际上我用到的组件只有按钮，分页，表格，输入与警告 所以我们要按需引用

```
1 import { Button, Input, Pagination, Table, TableColumn, MessageBox } from
  'element-ui';
2 Vue.use(Button)
3 Vue.use(Input)
4 Vue.use(Pagination)
```

组件重复打包

假设 A.js 文件是一个常用的库，现在有多个路由使用了 A.js 文件，这就造成了重复下载

解决方案：在 webpack 的 config 文件中，修改 CommonsChunkPlugin 的配置

```
1 minChunks: 3
```

minChunks 为3表示会把使用3次及以上的包抽离出来，放进公共依赖文件，避免了重复加载组件

图片资源的压缩

图片资源虽然不在编码过程中，但它却是对页面性能影响最大的因素

对于所有的图片资源，我们可以进行适当的压缩

对页面上使用到的 `icon`，可以使用在线字体图标，或者雪碧图，将众多小图标合并到同一张图上，用以减轻 `http` 请求压力。

开启GZip压缩

拆完包之后，我们再用 `gzip` 做一下压缩 安装 `compression-webpack-plugin`

```
1  cnmp i compression-webpack-plugin -D
```

在 `vue.config.js` 中引入并修改 `webpack` 配置

```
1  const CompressionPlugin = require('compression-webpack-plugin')
2
3  configureWebpack: (config) => {
4      if (process.env.NODE_ENV === 'production') {
5          // 为生产环境修改配置...
6          config.mode = 'production'
7          return {
8              plugins: [new CompressionPlugin({
9                  test: /\.js$|\.html$|\.css/, //匹配文件名
10                 threshold: 10240, //对超过10k的数据进行压缩
11                 deleteOriginalAssets: false //是否删除原文件
12             })]
13         }
14     }
```

在服务器我们也要做相应的配置 如果发送请求的浏览器支持 `gzip`，就发送给它 `gzip` 格式的文件 我的服务器是用 `express` 框架搭建的 只要安装一下 `compression` 就能使用

```
1  const compression = require('compression')
2  app.use(compression()) // 在其他中间件使用之前调用
```

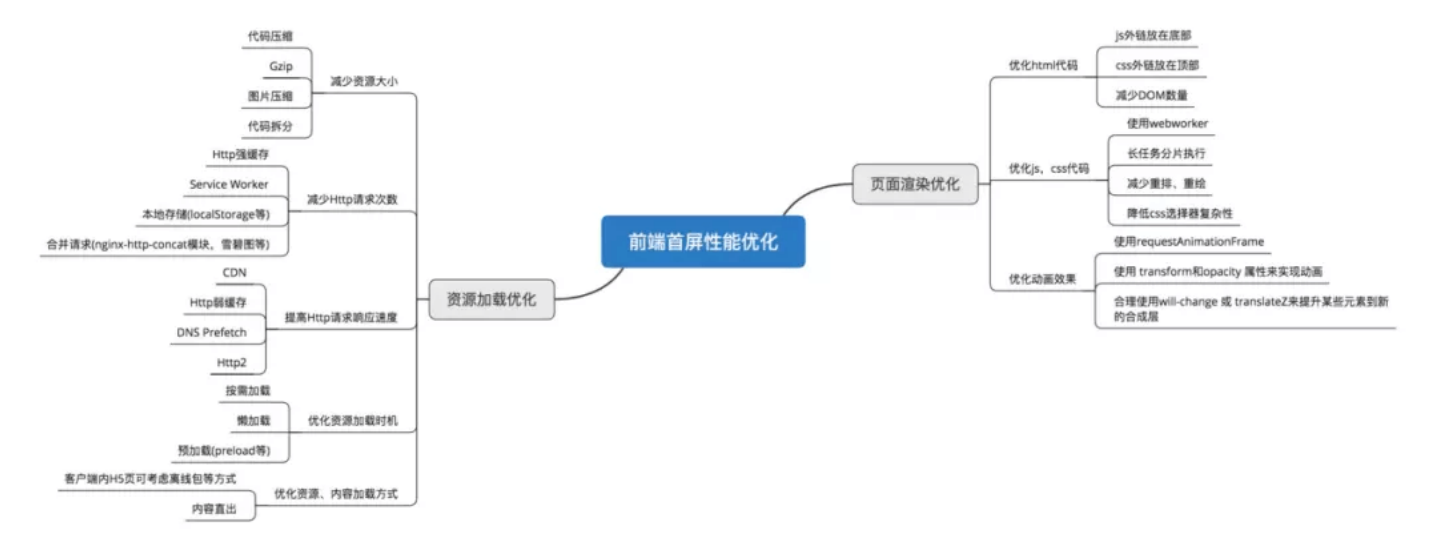
使用SSR

SSR（Server side），也就是服务端渲染，组件或页面通过服务器生成html字符串，再发送到浏览器 从头搭建一个服务端渲染是很复杂的，`vue` 应用建议使用 `Nuxt.js` 实现服务端渲染

小结：

减少首屏渲染时间的方法有很多，总的来讲可以分成两大部分：资源加载优化 和 页面渲染优化

下图是更为全面的首屏优化的方案



大家可以根据自己项目的情况选择各种方式进行首屏渲染的优化

第15题：说下Vite的原理

背景

这里的背景介绍会从与 `Vite` 紧密相关的两个概念的发展史说起，一个是 `JavaScript` 的模块化标准，另一个是前端构建工具。

共存的模块化标准

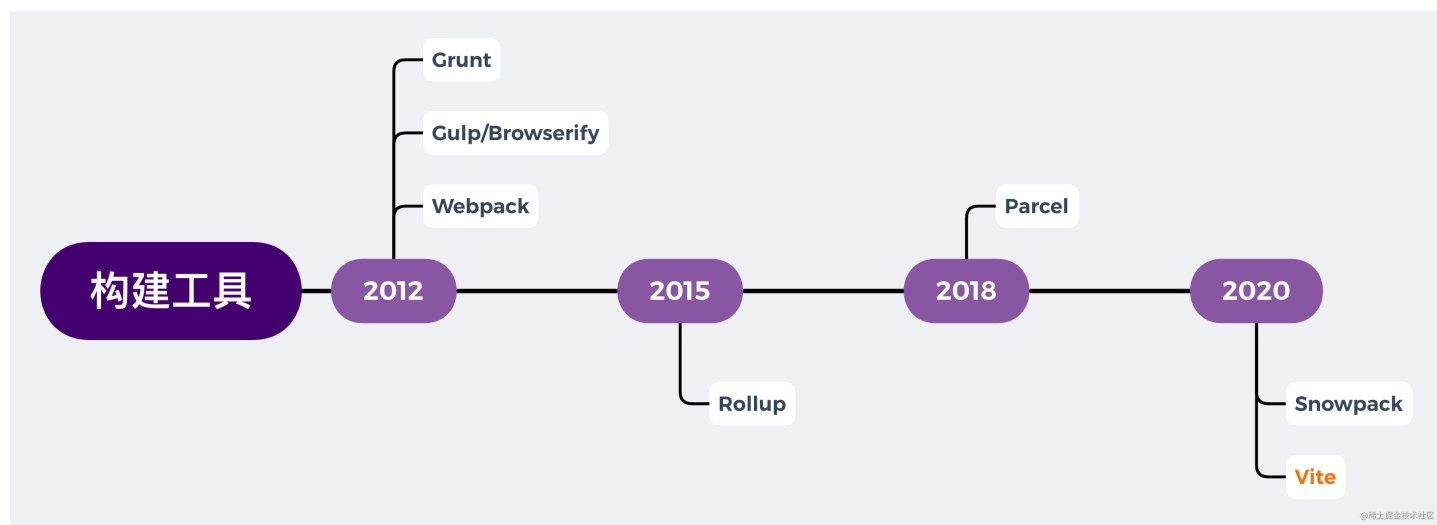
为什么 `JavaScript` 会有多种共存的模块化标准？因为js在设计之初并没有模块化的概念，随着前端业务复杂度不断提高，模块化越来越受到开发者的重视，社区开始涌现多种模块化解决方案，它们相互借鉴，也争议不断，形成多个派系，从 `CommonJS` 开始，到 `ES6` 正式推出 `ES Modules` 规范结束，所有争论，终成历史，`ES Modules` 也成为前端重要的基础设施。

- `CommonJS`：现主要用于Node.js（Node@13.2.0开始支持直接使用ES Module）
- `AMD`：`require.js` 依赖前置，市场存量不建议使用
- `CMD`：`sea.js` 就近执行，市场存量不建议使用
- `ES Module`：ES语言规范，标准，趋势，未来

对模块化发展史感兴趣的可以看下《前端模块化开发那点历史》@玉伯，而 `Vite` 的核心正是依靠浏览器对ES Module规范的实现。

发展中的构建工具

近些年前端工程化发展迅速，各种构建工具层出不穷，目前 `Webpack` 仍然占据统治地位，npm 每周下载量达到两千多万次。下面是我按 npm 发版时间线列出的开发者比较熟知的一些构建工具。



当前工程化痛点

现在常用的构建工具如 `Webpack`，主要是通过抓取-编译-构建整个应用的代码（也就是常说的打包过程），生成一份编译、优化后能良好兼容各个浏览器的生产环境代码。在开发环境流程也基本相同，需要先将整个应用构建打包后，再把打包后的代码交给 `dev server`（开发服务器）。

`Webpack` 等构建工具的诞生给前端开发带来了极大的便利，但随着前端业务的复杂化，js代码量呈指数增长，打包构建时间越来越久，`dev server`（开发服务器）性能遇到瓶颈：

- 缓慢的服务启动：大型项目中 `dev server` 启动时间达到几十秒甚至几分钟。
- 缓慢的HMR热更新：即使采用了 HMR 模式，其热更新速度也会随着应用规模的增长而显著下降，已达到性能瓶颈，无多少优化空间。

缓慢的开发环境，大大降低了开发者的幸福感，在以上背景下 `Vite` 应运而生。

什么是Vite?

基于esbuild与Rollup，依靠浏览器自身ESM编译功能，实现极致开发体验的新一代构建工具！

概念

先介绍以下文中会经常提到的一些基础概念：

- 依赖：指开发不会变动的部分(npm包、UI组件库)，esbuild进行预构建。
- 源码：浏览器不能直接执行的非js代码(.jsx、.css、.vue等)，vite只在浏览器请求相关源码的时候进行转换，以提供ESM源码。

开发环境

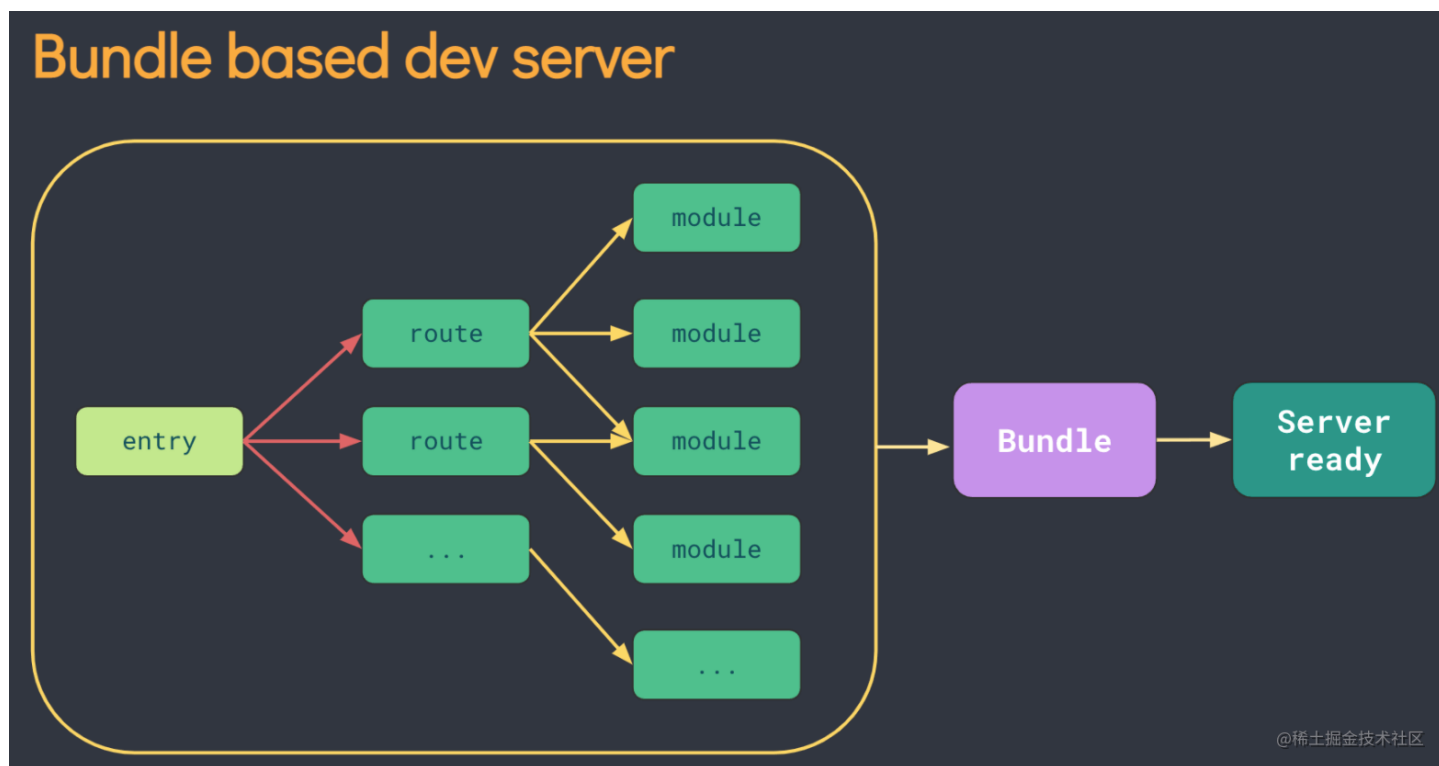
- 利用浏览器原生的 `ES Module` 编译能力，省略费时的编译环节，直给浏览器开发环境源码，`dev server` 只提供轻量服务。
- 浏览器执行ESM的 `import` 时，会向 `dev server` 发起该模块的 `ajax` 请求，服务器对源码做简单处理后返回给浏览器。
- `Vite` 中HMR是在原生 ESM 上执行的。当编辑一个文件时，Vite 只需要精确地使已编辑的模块失活，使得无论应用大小如何，HMR 始终能保持快速更新。
- 使用 `esbuild` 处理项目依赖，`esbuild` 使用go编写，比一般 `node.js` 编写的编译器快几个数量级。

生产环境

- 集成 `Rollup` 打包生产环境代码，依赖其成熟稳定的生态与更简洁的插件机制。

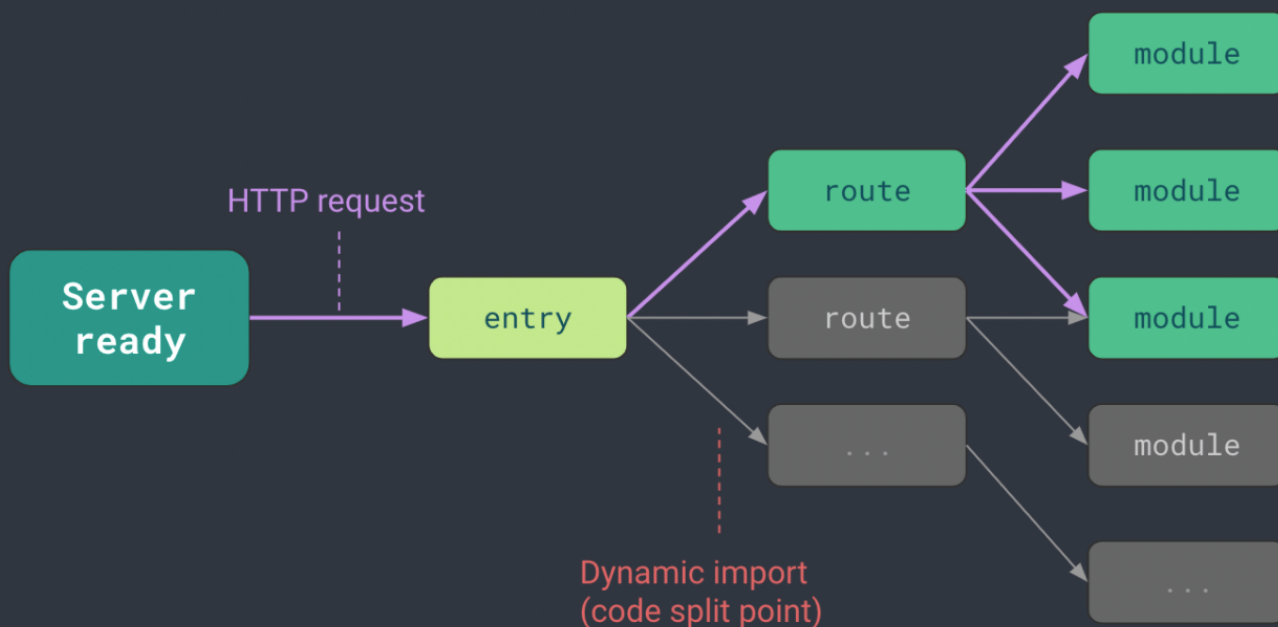
处理流程对比

`Webpack` 通过先将整个应用打包，再将打包后代码提供给 `dev server`，开发者才能开始开发。



`Vite` 直接将源码交给浏览器，实现 `dev server` 秒开，浏览器显示页面需要相关模块时，再向 `dev server` 发起请求，服务器简单处理后，将该模块返回给浏览器，实现真正意义的按需加载。

Native ESM based dev server



@稀土掘金技术社区

基本用法

创建vite项目

```
1 $ npm create vite@latest
```

选取模板

Vite 内置6种常用模板与对应的TS版本，可满足前端大部分开发场景，可以点击下列表格中模板直接在 [StackBlitz](#) 中在线试用，还有其他更多的 [社区维护模板](#) 可以使用。

JavaScript	TypeScript
vanilla	vanilla-ts
vue	vue-ts
react	react-ts
preact	preact-ts
lit	lit-ts
svelte	svelte-ts

启动

```
1  {
2    "scripts": {
3      "dev": "vite", // 启动开发服务器, 别名: `vite dev`, `vite serve`
4      "build": "vite build", // 为生产环境构建产物
5      "preview": "vite preview" // 本地预览生产构建产物
6    }
7  }
```

实现原理

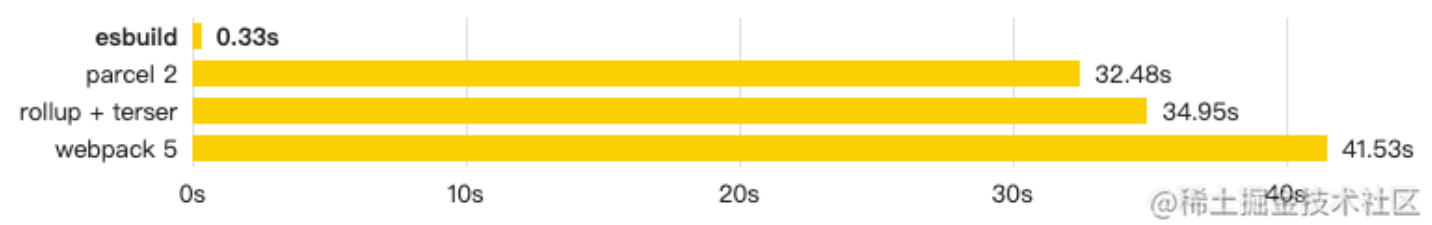
ESbuild 编译

`esbuild` 使用go编写, cpu密集下更具性能优势, 编译速度更快, 以下摘自官网的构建速度对比:

浏览器: “开始了吗?”

服务器: “已经结束了。”

开发者: “好快, 好喜欢!!”



依赖预构建

- 模块化兼容：如开头背景所写，现仍共存多种模块化标准代码，Vite 在预构建阶段将依赖中各种其他模块化规范(CommonJS、UMD)转换成ESM，以提供给浏览器。
- 性能优化：npm包中大量的ESM代码，大量的import 请求，会造成网络拥塞。Vite 使用 esbuild ，将有大量内部模块的ESM关系转换成单个模块，以减少 import 模块请求次数。

按需加载

- 服务器只在接受到import请求的时候，才会编译对应的文件，将ESM源码返回给浏览器，实现真正的按需加载。

缓存

- HTTP缓存：充分利用 http 缓存做优化，依赖（不会变动的代码）部分用max-age,immutable 强缓存，源码部分用304协商缓存，提升页面打开速度。
- 文件系统缓存：Vite 在预构建阶段，将构建后的依赖缓存到 node_modules/.vite ，相关配置更改时，或手动控制时才会重新构建，以提升预构建速度。

重写模块路径

浏览器 import 只能引入相对/绝对路径，而开发代码经常使用 npm 包名直接引入 node_module 中的模块，需要做路径转换后交给浏览器。

- es-module-lexer 扫描 import 语法
- magic-string 重写模块的引入路径

```
1  // 开发代码
2  import { createApp } from 'vue'
3
4  // 转换后
5  import { createApp } from '/node_modules/vue/dist/vue.js'
```

源码分析

与 Webpack-dev-server 类似 Vite 同样使用 WebSocket 与客户端建立连接，实现热更新，源码实现基本可分为两部分，源码位置在：

- vite/packages/vite/src/client client（用于客户端）
- vite/packages/vite/src/node server（用于开发服务器）

client 代码会在启动服务时注入到客户端，用于客户端对于 WebSocket 消息的处理（如更新页面某个模块、刷新页面）；server 代码是服务端逻辑，用于处理代码的构建与页面模块的请求。

简单看了下源码（vite@2.7.2），核心功能主要是以下几个方法（以下为源码截取，部分逻辑做了删减）：

1. 命令行启动服务 `npm run dev` 后，源码执行 `cli.ts`，调用 `createServer` 方法，创建http服务，监听开发服务器端口。

```
1 // 源码位置 vite/packages/vite/src/node/cli.ts
2 const { createServer } = await import('./server')
3 try {
4   const server = await createServer({
5     root,
6     base: options.base,
7     ...
8   })
9   if (!server.httpServer) {
10     throw new Error('HTTP server not available')
11   }
12   await server.listen()
13 }
```

1. `createServer` 方法的执行做了很多工作，如整合配置项、创建http服务（早期通过koa创建）、创建 `WebSocket` 服务、创建源码的文件监听、插件执行、optimize优化等。下面注释中标出。

```
1 // 源码位置 vite/packages/vite/src/node/server/index.ts
2 export async function createServer(
3   inlineConfig: InlineConfig = {}
4 ): Promise<ViteDevServer> {
5   // Vite 配置整合
6   const config = await resolveConfig(inlineConfig, 'serve', 'development')
7   const root = config.root
8   const serverConfig = config.server
9
10  // 创建http服务
11  const httpServer = await resolveHttpServer(serverConfig, middlewares,
12    httpsOptions)
13
14  // 创建ws服务
15  const ws = createWebSocketServer(httpServer, config, httpsOptions)
16
17  // 创建watcher，设置代码文件监听
18  const watcher = chokidar.watch(path.resolve(root), {
19    ignored: [
20      '**/node_modules/**',
21      '**/.git/**',
22      ...(Array.isArray(ignored) ? ignored : [ignored])
23    ]
24  })
```

```

22     ],
23     ...watchOptions
24   }) as FSWatcher
25
26   // 创建server对象
27   const server: ViteDevServer = {
28     config,
29     middlewares,
30     httpServer,
31     watcher,
32     ws,
33     moduleGraph,
34     listen,
35     ...
36   }
37
38   // 文件监听变动, websocket向前端通信
39   watcher.on('change', async (file) => {
40     ...
41     handleHMRUpdate()
42   })
43
44   // 非常多的 middleware
45   middlewares.use(...)
46
47   // optimize
48   const runOptimize = async () => {...}
49
50   return server
51 }

```

1. 使用chokidar监听文件变化，绑定监听事件。

```

1  // 源码位置 vite/packages/vite/src/node/server/index.ts
2  const watcher = chokidar.watch(path.resolve(root), {
3    ignored: [
4      '**/node_modules/**',
5      '**/.git/**',
6      ...(Array.isArray(ignored) ? ignored : [ignored])
7    ],
8    ignoreInitial: true,
9    ignorePermissionErrors: true,
10   disableGlobbing: true,
11   ...watchOptions
12 }) as FSWatcher

```

1. 通过 ws 来创建 WebSocket 服务，用于监听到文件变化时触发热更新，向客户端发送消息。

```
1  // 源码位置 vite/packages/vite/src/node/server/ws.ts
2  export function createWebSocketServer(...){
3      let wss: WebSocket
4      const hmr = isObject(config.server.hmr) && config.server.hmr
5      const wsServer = (hmr && hmr.server) || server
6
7      if (wsServer) {
8          wss = new WebSocket({ noServer: true })
9          wsServer.on('upgrade', (req, socket, head) => {
10              // 服务就绪
11              if (req.headers['sec-websocket-protocol'] === HMR_HEADER) {
12                  wss.handleUpgrade(req, socket as Socket, head, (ws) => {
13                      wss.emit('connection', ws, req)
14                  })
15              }
16          })
17      } else {
18          ...
19      }
20      // 服务准备就绪，就能在浏览器控制台看到熟悉的打印 [vite] connected.
21      wss.on('connection', (socket) => {
22          socket.send(JSON.stringify({ type: 'connected' }))
23          ...
24      })
25      // 失败
26      wss.on('error', (e: Error & { code: string }) => {
27          ...
28      })
29      // 返回ws对象
30      return {
31          on: wss.on.bind(wss),
32          off: wss.off.bind(wss),
33          // 向客户端发送信息
34          // 多个客户端同时触发
35          send(payload: HMRRPayload) {
36              const stringified = JSON.stringify(payload)
37              wss.clients.forEach((client) => {
38                  // readyState 1 means the connection is open
39                  client.send(stringified)
40              })
41          }
42      }
43  }
```

1. 在服务启动时会向浏览器注入代码，用于处理客户端接收到的 `WebSocket` 消息，如重新发起模块请求、刷新页面。

```
1  //源码位置 vite/packages/vite/src/client/client.ts
2  async function handleMessage(payload: HMREPayload) {
3      switch (payload.type) {
4          case 'connected':
5              console.log(`[vite] connected.`)
6              break
7          case 'update':
8              notifyListeners('vite:beforeUpdate', payload)
9              ...
10             break
11         case 'custom': {
12             notifyListeners(payload.event as CustomEventName<any>, payload.data)
13             ...
14             break
15         }
16         case 'full-reload':
17             notifyListeners('vite:beforeFullReload', payload)
18             ...
19             break
20         case 'prune':
21             notifyListeners('vite:beforePrune', payload)
22             ...
23             break
24         case 'error': {
25             notifyListeners('vite:error', payload)
26             ...
27             break
28         }
29         default: {
30             const check: never = payload
31             return check
32         }
33     }
34 }
```

优势

- 快！快！非常快！！

- 高度集成，开箱即用。
- 基于ESM急速热更新，无需打包编译。
- 基于 `esbuild` 的依赖预处理，比 `Webpack` 等node编写的编译器快几个数量级。
- 兼容 `Rollup` 庞大的插件机制，插件开发更简洁。
- 不与 `Vue` 绑定，支持 `React` 等其他框架，独立的构建工具。
- 内置SSR支持。
- 天然支持TS。

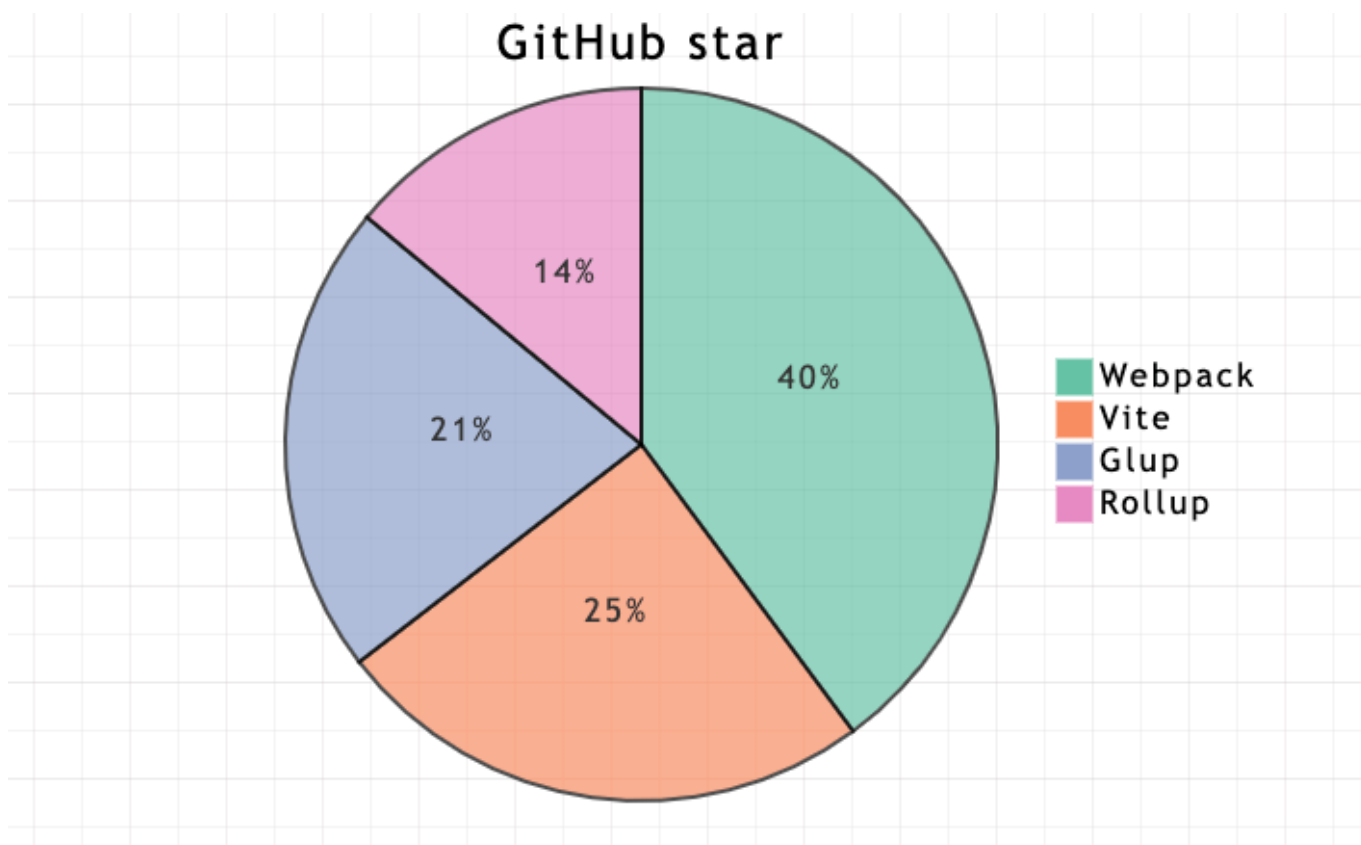
不足

- `Vue` 仍为第一优先支持，量身定做的编译插件，对 `React` 的支持不如 `Vue` 强大。
- 虽然已经推出2.0正式版，已经可以用于正式线上生产，但目前市场上实践少。
- 生产环境集成 `Rollup` 打包，与开发环境最终执行的代码不一致。

与 webpack 对比

由于 `Vite` 主打的是开发环境的极致体验，生产环境集成 `Rollup`，这里的对比主要是 `Webpack-dev-server` 与 `Vite-dev-server` 的对比：

- 到目前很长时间以来 `Webpack` 在前端工程领域占统治地位，`Vite` 推出以来备受关注，社区活跃，GitHub star 数量激增，目前达到37.4K



- `Webpack` 配置丰富使用极为灵活但上手成本高，`Vite` 开箱即用配置高度集成
 - `Webpack` 启动服务需打包构建，速度慢，`Vite` 免编译可秒开
 - `Webpack` 热更新需打包构建，速度慢，`Vite` 毫秒响应
 - `Webpack` 成熟稳定、资源丰富、大量实践案例，`Vite` 实践较少
 - `Vite` 使用 `esbuild` 编译，构建速度比 `webpack` 快几个数量级
-

兼容性

- 默认目标浏览器是在 `script` 标签上支持原生 ESM 和 原生 ESM 动态导入
 - 可使用官方插件 `@vitejs/plugin-legacy`，转义成传统版本和相对应的 `polyfill`
-

未来探索

- 传统构建工具性能已到瓶颈，主打开发体验的 `Vite`，可能会受到欢迎。
 - 主流浏览器基本支持ESM，ESM将成为主流。
 - `Vite` 在 `Vue3.0` 代替 `vue-cli`，作为官方脚手架，会大大提高使用量。
 - `Vite2.0` 推出后，已可以在实际项目中使用 `Vite`。
 - 如果觉得直接使用 `Vite` 太冒险，又确实有 `dev server` 速度慢的问题需要解决，可以尝试用 `Vite` 单独搭建一套 `dev server`
-

相关资源

官方插件

除了支持现有的 `Rollup` 插件系统外，官方提供了四个最关键的插件

- `@vitejs/plugin-vue` 提供 Vue3 单文件组件支持
- `@vitejs/plugin-vue-jsx` 提供 Vue3 JSX 支持（专用的 Babel 转换插件）
- `@vitejs/plugin-react` 提供完整的 React 支持
- `@vitejs/plugin-legacy` 为打包后的文件提供传统浏览器兼容性支持

第16题：Vue2.0为什么不能检查数组的变化，该怎么解决？

前言

我们都知道，Vue2.0对于响应式数据的实现有一些不足：

- 无法检测数组/对象的新增
- 无法检测通过索引改变数组的操作。

分析

- 无法检测数组/对象的新增？

Vue检测数据的变动是通过Object.defineProperty实现的，所以无法监听数组的添加操作是可以理解的，因为是在构造函数中就已经为所有属性做了这个检测绑定操作。

- 无法检测通过索引改变数组的操作。即vm.items[indexOfItem] = newValue?

[官方文档](#)中对于这两点都是简要的概括为“由于JavaScript的限制”无法实现，而Object.defineProperty是实现检测数据改变的方案，这个限制是指Object.defineProperty

思考

vm.items[indexOfItem] = newValue真的不能被监听么？

Vue对数组的7个变异方法（push、pop、shift、unshift、splice、sort、reverse）实现了响应式。这里就不做测试了。我们测试一下通过索引改变数组的操作，能不能被监听到。

遍历数组，用Object.defineProperty对每一项进行监测

```
1  function defineReactive(data, key, value) {
2      Object.defineProperty(data, key, {
3          enumerable: true,
4          configurable: true,
5          get: function defineGet() {
6              console.log(`get key: ${key} value: ${value}`)
7              return value
8          },
9          set: function defineSet(newVal) {
10             console.log(`set key: ${key} value: ${newVal}`)
11             value = newVal
12         }
13     })
14 }
15
16 function observe(data) {
17     Object.keys(data).forEach(function(key) {
18         defineReactive(data, key, data[key])
19     })
20 }
21
```

```
22 let arr = [1, 2, 3]
23 observe(arr)
```

```
> arr[1]=2
  set key: 1 value: 2
< 2
> |
```

测试说明

通过索引改变arr[1]，我们发现触发了set，也就是Object.defineProperty是可以检测到通过索引改变数组的操作的，那Vue2.0为什么没有实现呢？是尤大能力不行？这肯定毋庸置疑。那他为什么不实现呢？

hfhan commented on 27 Jul 2018

+

😊

...

What problem does this feature solve?

在vue中，通过Object.defineProperty实现了对对象属性的监听，但是Object.defineProperty对数组已有元素也是实现监听的，那么为什么vue中没有提供这一功能，使得 arr[已有元素下标]=val 变成响应式？

具体分析：https://segmentfault.com/a/1190000015783546?_ea=4074035

What does the proposed API look like?

yyx990803 commented on 27 Jul 2018 • edited ▾

Member

+

😊

...

要求写得很清楚，不要用 issue 问问题。

看你的文章研究得挺有热情，破例回答一下：就是因为性能问题。

🚫

yyx990803 closed this on 27 Jul 2018

hfhan commented on 27 Jul 2018

Author

+

😊

...

尤大，我想问下究竟是什么样的性能问题限制了这一功能呢？因为现代浏览器以及电脑配置都还不错，监测几万乃至几十万个属性应该还是可以的，如果数组长度被限制，是不是就可以使用这一功能了？

yyx990803 commented on 27 Jul 2018

Member

+

😊

...

性能代价和获得的用户体验收益不成正比。

hfhan commented on 27 Jul 2018

Author

+

😊

...

谢谢尤大，明白了。(◦˘▽˘◦)

小结：是出于对性能原因的考虑，没有去实现它。而不是不能实现。

对于对象而言，每一次的数据变更都会对对象的属性进行一次枚举，一般对象本身的属性数量有限，所以对于遍历枚举等方式产生的性能损耗可以忽略不计，但是对于数组而言呢？数组包含的元素量是

可能达到成千上万，假设对于每一次数组元素的更新都触发了枚举/遍历，其带来的性能损耗将与获得的用户体验不成正比，故vue无法检测数组的变动。

不过Vue3.0用proxy代替了defineProperty之后就解决了这个问题。

解决方案

数组

1. this.\$set(array, index, data)

```
1  //这是个深度的修改，某些情况下可能导致你不希望的结果，因此最好还是慎用
2  this.dataArr = this.originArr
3  this.$set(this.dataArr, 0, {data: '修改第一个元素'})
4  console.log(this.dataArr)
5  console.log(this.originArr)  //同样的 源数组也会被修改 在某些情况下会导致你不希望的结果
```

1. splice

```
1  //因为splice会被监听有响应式，而splice又可以做到增删改。
```

1. 利用临时变量进行中转

```
1  let tempArr = [...this.targetArr]
2  tempArr[0] = {data: 'test'}
3  this.targetArr = tempArr
```

对象

1. this.\$set(obj, key, value) - 可实现增、改

2. watch时添加 `deep: true` 深度监听，只能监听到属性值的变化，新增、删除属性无法监听

```
1  this.$watch('blog', this.getCatalog, {
2    deep: true
3    // immediate: true // 是否第一次触发
4  });
```

1. watch时直接监听某个key

```
1  watch: {
2    'obj.name' (curVal, oldVal) {
3      // TODO
4    }
5  }
```

第17题：说说Vue 页面渲染流程

前言

在 `Vue` 核心中除了响应式原理外，视图渲染也是重中之重。我们都知道每次更新数据，都会走视图渲染的逻辑，而这当中牵扯的逻辑也是十分繁琐。

本文主要解析的是初始化视图渲染流程，你将会了解到从挂载组件开始，`Vue` 是如何构建 `VNode`，又是如何将 `VNode` 转为真实节点并挂载到页面。

挂载组件(\$mount)

`Vue` 是一个构造函数，通过 `new` 关键字进行实例化。

```
1  // src/core/instance/index.js
2  function Vue (options) {
3    if (process.env.NODE_ENV !== 'production' &&
4      !(this instanceof Vue)
5    ) {
6      warn('Vue is a constructor and should be called with the `new` keyword')
7    }
8    this._init(options)
9  }
```

在实例化时，会调用 `_init` 进行初始化。

```
1  // src/core/instance/init.js
2  Vue.prototype._init = function (options?: Object) {
3    const vm: Component = this
4    // ...
5    if (vm.$options.el) {
6      vm.$mount(vm.$options.el)
7    }
8  }
```

```
8 }
```

`_init` 内会调用 `$mount` 来挂载组件，而 `$mount` 方法实际调用的是 `mountComponent`。

```
1  // src/core/instance/lifecycle.js
2  export function mountComponent (
3    vm: Component,
4    el: ?Element,
5    hydrating?: boolean
6  ): Component {
7    vm.$el = el
8    // ...
9    callHook(vm, 'beforeMount')
10
11    let updateComponent
12    /* istanbul ignore if */
13    if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
14      // ...
15    } else {
16      updateComponent = () => {
17        vm._update(vm._render(), hydrating) // 渲染页面函数
18      }
19    }
20
21    // we set this to vm._watcher inside the watcher's constructor
22    // since the watcher's initial patch may call $forceUpdate (e.g. inside
23    // component's mounted hook), which relies on vm._watcher being already
24    // defined
25    new Watcher(vm, updateComponent, noop, { // 渲染watcher
26      before () {
27        if (vm._isMounted && !vm._isDestroyed) {
28          callHook(vm, 'beforeUpdate')
29        }
30      }, true /* isRenderWatcher */)
31    hydrating = false
32
33    // manually mounted instance, call mounted on self
34    // mounted is called for render-created child components in its inserted
35    // hook
36    if (vm.$vnode == null) {
37      vm._isMounted = true
38      callHook(vm, 'mounted')
39    }
```

```
39     return vm
40 }
```

`mountComponent` 除了调用一些生命周期的钩子函数外，最主要是 `updateComponent`，它就是负责渲染视图的核心方法，其只有一行核心代码：

```
1  vm._update(vm._render(), hydrating)
```

`vm._render` 创建并返回 `VNode`，`vm._update` 接受 `VNode` 将其转为真实节点。

`updateComponent` 会被传入 `渲染Watcher`，每当数据变化触发 `Watcher` 更新就会执行该函数，重新渲染视图。`updateComponent` 在传入 `渲染Watcher` 后会被执行一次进行初始化页面渲染。

所以我们着重分析的是 `vm._render` 和 `vm._update` 两个方法，这也是本文主要了解的原理——`Vue` 视图渲染流程。

构建VNode(_render)

首先是 `_render` 方法，它用来构建组件的 `VNode`。

```
1  // src/core/instance/render.js
2  Vue.prototype._render = function () {
3      const { render, _parentVnode } = vm.$options
4      vnode = render.call(vm._renderProxy, vm.$createElement)
5      return vnode
6  }
```

`_render` 内部会执行 `render` 方法并返回构建好的 `VNode`。`render` 一般是模板编译后生成的方法，也有可能是用户自定义。

```
1  // src/core/instance/render.js
2  export function initRender (vm) {
3      vm._c = (a, b, c, d) => createElement(vm, a, b, c, d, false)
4      vm.$createElement = (a, b, c, d) => createElement(vm, a, b, c, d, true)
5  }
```

`initRender` 在初始化就会执行为实例上绑定两个方法，分别是 `vm._c` 和 `vm.$createElement`。它们两者都是调用 `createElement` 方法，它是创建 `VNode` 的核心

方法，最后一个参数用于区别是否为用户自定义。

`vm._c` 应用场景是在编译生成的 `render` 函数中调用，`vm.$createElement` 则用于用户自定义 `render` 函数的场景。就像上面 `render` 在调用时会传入参数 `vm.$createElement`，我们在自定义 `render` 函数接收到的参数就是它。

createElement

```
1  // src/core/vdom/create-element.js
2  export function createElement (
3    context: Component,
4    tag: any,
5    data: any,
6    children: any,
7    normalizationType: any,
8    alwaysNormalize: boolean
9  ): VNode | Array<VNode> {
10    if (Array.isArray(data) || isPrimitive(data)) {
11      normalizationType = children
12      children = data
13      data = undefined
14    }
15    if (isTrue(alwaysNormalize)) {
16      normalizationType = ALWAYS_NORMALIZE
17    }
18    return _createElement(context, tag, data, children, normalizationType)
19  }
```

`createElement` 方法实际上是对 `_createElement` 方法的封装，它允许传入的参数更加灵活。

```
1  export function _createElement (
2    context: Component,
3    tag?: string | Class<Component> | Function | Object,
4    data?: VNodeData,
5    children?: any,
6    normalizationType?: number
7  ): VNode | Array<VNode> {
8    if (isDef(data) && isDef(data.is)) {
9      tag = data.is
10    }
11    if (!tag) {
12      // in case of component :is set to falsy value
13      return createEmptyVNode()
```

```

14     }
15     // support single function children as default scoped slot
16     if (Array.isArray(children) &&
17         typeof children[0] === 'function'
18     ) {
19         data = data || {}
20         data.scopedSlots = { default: children[0] }
21         children.length = 0
22     }
23     if (normalizationType === ALWAYS_NORMALIZE) {
24         children = normalizeChildren(children)
25     } else if (normalizationType === SIMPLE_NORMALIZE) {
26         children = simpleNormalizeChildren(children)
27     }
28     let vnode, ns
29     if (typeof tag === 'string') {
30         let Ctor
31         ns = (context.$vnode && context.$vnode.ns) || config.getTagNamespace(tag)
32         if (config.isReservedTag(tag)) {
33             // platform built-in elements
34             vnode = new VNode(
35                 config.parsePlatformTagName(tag), data, children,
36                 undefined, undefined, context
37             )
38         } else if ((isDef(Ctor = resolveAsset(context.$options, 'components',
tag)))) {
39             // component
40             vnode = createComponent(Ctor, data, context, children, tag)
41         } else {
42             // unknown or unlisted namespaced elements
43             // check at runtime because it may get assigned a namespace when its
44             // parent normalizes children
45             vnode = new VNode(
46                 tag, data, children,
47                 undefined, undefined, context
48             )
49         }
50     } else {
51         // direct component options / constructor
52         vnode = createComponent(tag, data, context, children)
53     }
54     if (Array.isArray(vnode)) {
55         return vnode
56     } else if (isDef(vnode)) {
57         if (isDef(ns)) applyNS(vnode, ns)
58         if (isDef(data)) registerDeepBindings(data)
59         return vnode

```

```

60     } else {
61         return createEmptyVNode()
62     }
63 }

```

`_createElement` 参数中会接收 `children`，它表示当前 `VNode` 的子节点，因为它是任意类型的，所以接下来需要将其规范为标准的 `VNode` 数组；

```

1  // 这里规范化 children
2  if (normalizationType === ALWAYS_NORMALIZE) {
3      children = normalizeChildren(children)
4  } else if (normalizationType === SIMPLE_NORMALIZE) {
5      children = simpleNormalizeChildren(children)
6  }

```

`simpleNormalizeChildren` 和 `normalizeChildren` 均用于规范化 `children`。由 `normalizationType` 判断 `render` 函数是编译生成的还是用户自定义的。

```

1  // 1. When the children contains components - because a functional component
2  // may return an Array instead of a single root. In this case, just a simple
3  // normalization is needed - if any child is an Array, we flatten the whole
4  // thing with Array.prototype.concat. It is guaranteed to be only 1-level deep
5  // because functional components already normalize their own children.
6  export function simpleNormalizeChildren (children: any) {
7      for (let i = 0; i < children.length; i++) {
8          if (Array.isArray(children[i])) {
9              return Array.prototype.concat.apply([], children)
10         }
11     }
12     return children
13 }
14
15 // 2. When the children contains constructs that always generated nested
16 // Arrays,
17 // e.g. <template>, <slot>, v-for, or when the children is provided by user
18 // with hand-written render functions / JSX. In such cases a full
19 // normalization
20 // is needed to cater to all possible types of children values.
21 export function normalizeChildren (children: any): ?Array<VNode> {
22     return isPrimitive(children)
23         ? [createTextVNode(children)]
24         : Array.isArray(children)
25             ? normalizeArrayChildren(children)

```

```
24      : undefined
25    }
```

`simpleNormalizeChildren` 方法调用场景是 render 函数当函数是编译生成的。

`normalizeChildren` 方法的调用场景主要是 render 函数是用户手写的。

经过对 `children` 的规范化, `children` 变成了一个类型为 `VNode` 的数组。之后就是创建 `VNode` 的逻辑。

```
1  // src/core/vdom/patch.js
2  let vnode, ns
3  if (typeof tag === 'string') {
4    let Ctor
5    ns = (context.$vnode && context.$vnode.ns) || config.getTagNamespace(tag)
6    if (config.isReservedTag(tag)) {
7      // platform built-in elements
8      vnode = new VNode(
9        config.parsePlatformTagName(tag), data, children,
10       undefined, undefined, context
11     )
12   } else if (isDef(Ctor = resolveAsset(context.$options, 'components', tag)))
13   {
14     // component
15     vnode = createComponent(Ctor, data, context, children, tag)
16   } else {
17     // unknown or unlisted namespaced elements
18     // check at runtime because it may get assigned a namespace when its
19     // parent normalizes children
20     vnode = new VNode(
21       tag, data, children,
22       undefined, undefined, context
23     )
24   } else {
25     // direct component options / constructor
26     vnode = createComponent(tag, data, context, children)
27   }
```

如果 `tag` 是 `string` 类型, 则接着判断如果是内置的一些节点, 创建一个普通 `VNode`; 如果是为已注册的组件名, 则通过 `createComponent` 创建一个组件类型的 `VNode`; 否则创建一个未知的标签的 `VNode`。

如果 `tag` 不是 `string` 类型, 那就是 `Component` 类型, 则直接调用 `createComponent` 创建一个组件类型的 `VNode` 节点。

最后 `_createElement` 会返回一个 `VNode`，也就是调用 `vm._render` 时创建得到的 `VNode`。之后 `VNode` 会传递给 `vm._update` 函数，用于生成真实dom。

生成真实dom(_update)

```
1  // src/core/instance/lifecycle.js
2  Vue.prototype._update = function (vnode: VNode, hydrating?: boolean) {
3    const vm: Component = this
4    const prevEl = vm.$el
5    const prevVnode = vm._vnode
6    const prevActiveInstance = activeInstance
7    activeInstance = vm
8    vm._vnode = vnode
9    // Vue.prototype.__patch__ is injected in entry points
10   // based on the rendering backend used.
11   if (!prevVnode) {
12     // initial render
13     vm.$el = vm.__patch__(vm.$el, vnode, hydrating, false /* removeOnly */)
14   } else {
15     // updates
16     vm.$el = vm.__patch__(prevVnode, vnode)
17   }
18   activeInstance = prevActiveInstance
19   // update __vue__ reference
20   if (prevEl) {
21     prevEl.__vue__ = null
22   }
23   if (vm.$el) {
24     vm.$el.__vue__ = vm
25   }
26   // if parent is an HOC, update its $el as well
27   if (vm.$vnode && vm.$parent && vm.$vnode === vm.$parent._vnode) {
28     vm.$parent.$el = vm.$el
29   }
30   // updated hook is called by the scheduler to ensure that children are
31   // updated in a parent's updated hook.
32 }
```

`_update` 里最核心的方法就是 `vm.__patch__` 方法，不同平台的 `patch` 方法的定义会稍有不同，在 web 平台中它是这样定义的：

```
1  // src/platforms/web/runtime/index.js
2  import { patch } from './patch'
```

```
3 // install platform patch function
4 Vue.prototype.__patch__ = inBrowser ? patch : noop
```

可以看到 `patch` 实际调用的是 `patch` 方法。

```
1 // src/platforms/web/runtime/patch.js
2 import * as nodeOps from 'web/runtime/node-ops'
3 import { createPatchFunction } from 'core/vdom/patch'
4 import baseModules from 'core/vdom/modules/index'
5 import platformModules from 'web/runtime/modules/index'
6
7 // the directive module should be applied last, after all
8 // built-in modules have been applied.
9 const modules = platformModules.concat(baseModules)
10
11 export const patch: Function = createPatchFunction({ nodeOps, modules })
```

而 `patch` 方法是由 `createPatchFunction` 方法创建返回出来的函数。

```
1 // src/core/vdom/patch.js
2 const hooks = ['create', 'activate', 'update', 'remove', 'destroy']
3
4 export function createPatchFunction (backend) {
5   let i, j
6   const cbs = {}
7   const { modules, nodeOps } = backend
8
9   for (i = 0; i < hooks.length; ++i) {
10     cbs[hooks[i]] = []
11     for (j = 0; j < modules.length; ++j) {
12       if (isDef(modules[j][hooks[i]])) {
13         cbs[hooks[i]].push(modules[j][hooks[i]])
14       }
15     }
16   }
17
18   // ...
19   return function patch (oldVnode, vnode, hydrating, removeOnly){}
20 }
```

这里有两个比较重要的对象 `nodeOps` 和 `modules`。`nodeOps` 是封装的原生dom操作方法，在生成真实节点树的过程中，dom相关操作都是调用 `nodeOps` 内的方法。

`modules` 是待执行的钩子函数。在进入函数时，会将不同模块的钩子函数分类放置到 `cbs` 中，其中包括自定义指令钩子函数，`ref` 钩子函数。在 `patch` 阶段，会根据操作节点的行为取出对应类型进行调用。

patch

```
1 // initial render
2 vm.$el = vm.__patch__(vm.$el, vnode, hydrating, false /* removeOnly */)
```

在首次渲染时，`vm.$el` 对应的是根节点 `dom` 对象，也就是我们熟知的 `id` 为 `app` 的 `div`。它作为 `oldVNode` 参数传入 `patch`：

```
1 return function patch (oldVnode, vnode, hydrating, removeOnly) {
2   if (isUndef(vnode)) {
3     if (isDef(oldVnode)) invokeDestroyHook(oldVnode)
4     return
5   }
6
7   let isInitialPatch = false
8   const insertedVnodeQueue = []
9
10  if (isUndef(oldVnode)) {
11    // empty mount (likely as component), create new root element
12    isInitialPatch = true
13    createElm(vnode, insertedVnodeQueue)
14  } else {
15    const isRealElement = isDef(oldVnode.nodeType)
16    if (!isRealElement && sameVnode(oldVnode, vnode)) {
17      // patch existing root node
18      patchVnode(oldVnode, vnode, insertedVnodeQueue, null, null, removeOnly)
19    } else {
20      if (isRealElement) {
21        // mounting to a real element
22        // check if this is server-rendered content and if we can perform
23        // a successful hydration.
24        if (oldVnode.nodeType === 1 && oldVnode.hasAttribute(SSR_ATTR)) {
25          oldVnode.removeAttribute(SSR_ATTR)
26          hydrating = true
27        }
28        if (isTrue(hydrating)) {
29          if (hydrate(oldVnode, vnode, insertedVnodeQueue)) {
30            invokeInsertHook(vnode, insertedVnodeQueue, true)
31            return oldVnode
```

```

32     } else if (process.env.NODE_ENV !== 'production') {
33         warn(
34             'The client-side rendered virtual DOM tree is not matching ' +
35             'server-rendered content. This is likely caused by incorrect ' +
36             'HTML markup, for example nesting block-level elements inside '
+
37             '<p>, or missing <tbody>. Bailing hydration and performing ' +
38             'full client-side render.'
39         )
40     }
41 }
42 // either not server-rendered, or hydration failed.
43 // create an empty node and replace it
44 oldVnode = emptyNodeAt(oldVnode)
45 }
46
47 // replacing existing element
48 const oldElm = oldVnode.elm
49 const parentElm = nodeOps.parentNode(oldElm)
50
51 // create new node
52 createElm(
53     vnode,
54     insertedVnodeQueue,
55     // extremely rare edge case: do not insert if old element is in a
56     // leaving transition. Only happens when combining transition +
57     // keep-alive + HOCs. (#4590)
58     oldElm._leaveCb ? null : parentElm,
59     nodeOps.nextSibling(oldElm)
60 )
61
62 // update parent placeholder node element, recursively
63 if (isDef(vnode.parent)) {
64     let ancestor = vnode.parent
65     const patchable = isPatchable(vnode)
66     while (ancestor) {
67         for (let i = 0; i < cbs.destroy.length; ++i) {
68             cbs.destroy[i](ancestor)
69         }
70         ancestor.elm = vnode.elm
71         if (patchable) {
72             for (let i = 0; i < cbs.create.length; ++i) {
73                 cbs.create[i](emptyNode, ancestor)
74             }
75             // #6513
76             // invoke insert hooks that may have been merged by create hooks.
77             // e.g. for directives that uses the "inserted" hook.

```

```

78         const insert = ancestor.data.hook.insert
79         if (insert.merged) {
80             // start at index 1 to avoid re-invoking component mounted hook
81             for (let i = 1; i < insert.fns.length; i++) {
82                 insert.fns[i]()
83             }
84         }
85     } else {
86         registerRef(ancestor)
87     }
88     ancestor = ancestor.parent
89 }
90 }
91
92 // destroy old node
93 if (isDef(parentElm)) {
94     removeVnodes([oldVnode], 0, 0)
95 } else if (isDef(oldVnode.tag)) {
96     invokeDestroyHook(oldVnode)
97 }
98 }
99 }
100
101 invokeInsertHook(vnode, insertedVnodeQueue, isInitialPatch)
102 return vnode.elm
103 }

```

通过检查属性 `nodeType`（真实节点才有的属性），判断 `oldVnode` 是否为真实节点。

```

1  const isRealElement = isDef(oldVnode.nodeType)
2  if (isRealElement) {
3      // ...
4      oldVnode = emptyNodeAt(oldVnode)
5  }

```

很明显第一次的 `isRealElement` 是为 `true`，因此会调用 `emptyNodeAt` 将其转为 `VNode`：

```

1  function emptyNodeAt (elm) {
2      return new VNode(nodeOps.tagName(elm).toLowerCase(), {}, [], undefined, elm)
3  }

```

接着会调用 `createElm` 方法，它就是将 `VNode` 转为真实dom 的核心方法：

```
1  function createElm (  
2    vnode,  
3    insertedVnodeQueue,  
4    parentElm,  
5    refElm,  
6    nested,  
7    ownerArray,  
8    index  
9  ) {  
10   if (isDef(vnode.elm) && isDef(ownerArray)) {  
11     // This vnode was used in a previous render!  
12     // now it's used as a new node, overwriting its elm would cause  
13     // potential patch errors down the road when it's used as an insertion  
14     // reference node. Instead, we clone the node on-demand before creating  
15     // associated DOM element for it.  
16     vnode = ownerArray[index] = cloneVNode(vnode)  
17   }  
18  
19   vnode.isRootInsert = !nested // for transition enter check  
20   if (createComponent(vnode, insertedVnodeQueue, parentElm, refElm)) {  
21     return  
22   }  
23  
24   const data = vnode.data  
25   const children = vnode.children  
26   const tag = vnode.tag  
27   if (isDef(tag)) {  
28     vnode.elm = vnode.ns  
29       ? nodeOps.createElementNS(vnode.ns, tag)  
30       : nodeOps.createElement(tag, vnode)  
31     setScope(vnode)  
32  
33     /* istanbul ignore if */  
34     if (__WEEX__) {  
35       // ...  
36     } else {  
37       createChildren(vnode, children, insertedVnodeQueue)  
38       if (isDef(data)) {  
39         invokeCreateHooks(vnode, insertedVnodeQueue)  
40       }  
41       insert(parentElm, vnode.elm, refElm)  
42     }  
43  
44     if (process.env.NODE_ENV !== 'production' && data && data.pre) {
```

```

45     creatingElmInVPre--
46   }
47 } else if (isTrue(vnode.isComment)) {
48   vnode.elm = nodeOps.createComment(vnode.text)
49   insert(parentElm, vnode.elm, refElm)
50 } else {
51   vnode.elm = nodeOps.createTextNode(vnode.text)
52   insert(parentElm, vnode.elm, refElm)
53 }
54 }

```

一开始会调用 `createComponent` 尝试创建组件类型的节点，如果成功会返回 `true`。在创建过程中也会调用 `$mount` 进行组件范围内的挂载，所以走的还是 `patch` 这套流程。

```

1  if (createComponent(vnode, insertedVnodeQueue, parentElm, refElm)) {
2    return
3  }

```

如果没有完成创建，代表该 `VNode` 对应的是真实节点，往下继续创建真实节点的逻辑。

```

1  vnode.elm = vnode.ns
2    ? nodeOps.createElementNS(vnode.ns, tag)
3    : nodeOps.createElement(tag, vnode)

```

根据 `tag` 创建对应类型真实节点，赋值给 `vnode.elm`，它作为父节点容器，创建的子节点会被放到里面。

然后调用 `createChildren` 创建子节点：

```

1  function createChildren (vnode, children, insertedVnodeQueue) {
2    if (Array.isArray(children)) {
3      if (process.env.NODE_ENV !== 'production') {
4        checkDuplicateKeys(children)
5      }
6      for (let i = 0; i < children.length; ++i) {
7        createElm(children[i], insertedVnodeQueue, vnode.elm, null, true,
children, i)
8      }
9    } else if (isPrimitive(vnode.text)) {
10      nodeOps.appendChild(vnode.elm, nodeOps.createTextNode(String(vnode.text)))
11    }

```

```
12 }
```

内部进行遍历子节点数组，再次调用 `createElm` 创建节点，而上面创建的 `vnode.elm` 作为父节点传入。如此循环，直到没有子节点，就会创建文本节点插入到 `vnode.elm` 中。

执行完成后出来，会调用 `invokeCreateHooks`，它负责执行 dom 操作时的 `create` 钩子函数，同时将 `VNode` 加入到 `insertedVnodeQueue` 中：

```
1 function invokeCreateHooks (vnode, insertedVnodeQueue) {
2   for (let i = 0; i < cbs.create.length; ++i) {
3     cbs.create[i](emptyNode, vnode)
4   }
5   i = vnode.data.hook // Reuse variable
6   if (isDef(i)) {
7     if (isDef(i.create)) i.create(emptyNode, vnode)
8     if (isDef(i.insert)) insertedVnodeQueue.push(vnode)
9   }
10 }
```

最后一步就是调用 `insert` 方法将节点插入到父节点：

```
1 function insert (parent, elm, ref) {
2   if (isDef(parent)) {
3     if (isDef(ref)) {
4       if (nodeOps.parentNode(ref) === parent) {
5         nodeOps.insertBefore(parent, elm, ref)
6       }
7     } else {
8       nodeOps.appendChild(parent, elm)
9     }
10  }
11 }
```

可以看到 `Vue` 是通过递归调用 `createElm` 来创建节点树的。同时也说明最深的子节点会先调用 `insert` 插入节点。所以整个节点树的插入顺序是“先子后父”。插入节点方法就是原生 dom 的方法 `insertBefore` 和 `appendChild`。

```
1 if (isDef(parentElm)) {
2   removeVnodes([oldVnode], 0, 0)
3 }
```

`createElm` 流程走完后，构建完成的节点树已经插入到页面上了。其实 `Vue` 在初始化渲染页面时，并不是把原来的根节点 `app` 给真正替换掉，而是在其后面插入一个新的节点，接着再把旧节点给移除掉。

所以在 `createElm` 之后会调用 `removeVnodes` 来移除旧节点，它里面同样是调用的原生dom方法 `removeChild`。

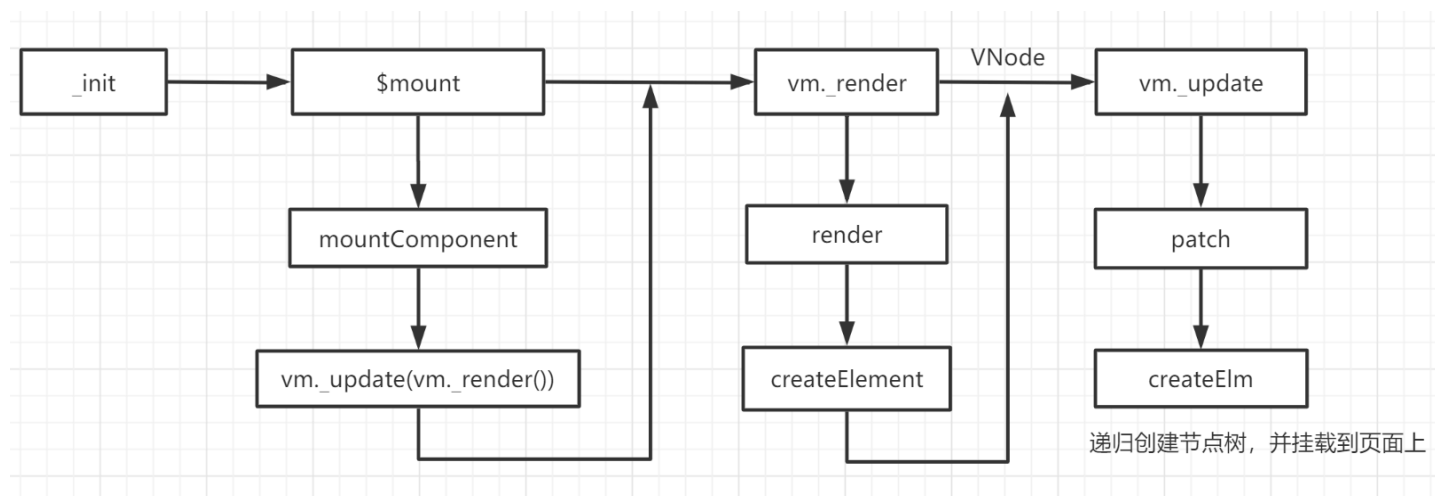
```
1 invokeInsertHook(vnode, insertedVnodeQueue, isInitialPatch)
```

```
1 function invokeInsertHook (vnode, queue, initial) {
2   // delay insert hooks for component root nodes, invoke them after the
3   // element is really inserted
4   if (isTrue(initial) && isDef(vnode.parent)) {
5     vnode.parent.data.pendingInsert = queue
6   } else {
7     for (let i = 0; i < queue.length; ++i) {
8       queue[i].data.hook.insert(queue[i])
9     }
10  }
11 }
```

在 `patch` 的最后就是调用 `invokeInsertHook` 方法，触发节点插入的钩子函数。

至此整个页面渲染的流程完毕~

总结



初始化调用 `$mount` 挂载组件。

`_render` 开始构建 `VNode`，核心方法为 `createElement`，一般会创建普通的 `VNode`，遇到组件就创建组件类型的 `VNode`，否则就是未知标签的 `VNode`，构建完成传递给 `_update`。

`patch` 阶段根据 `VNode` 创建真实节点树，核心方法为 `createElm`，首先遇到组件类型的 `VNode`，内部会执行 `$mount`，再走一遍相同的流程。普通节点类型则创建一个真实节点，如果它有子节点开始递归调用 `createElm`，使用 `insert` 插入子节点，直到没有子节点就填充内容节点。最后递归完成后，同样也是使用 `insert` 将整个节点树插入到页面中，再将旧的根节点移除。

第18题：vue中computed和watch区别

computed 和 watch看似都能实现对数据的监听，但还是有区别。

以下通过一个小栗子来理解一下这两者的区别。

computed 计算属性

计算属性基于 data 中声明过或者父组件传递的 props 中的数据通过计算得到的一个新值，这个新值只会根据已知值的变化而变化，简言之：这个属性依赖其他属性，由其他属性计算而来的。

```
1  <p>姓名: {{ fullName }}</p>
2  ...
3  data: {
4      firstName: 'David',
5      lastName: 'Beckham'
6  },
7  computed: {
8      fullName: function() { //方法的返回值作为属性值
9          return this.firstName + ' ' + this.lastName
10     }
11 }
```

在 computed 属性对象中定义计算属性的方法，和取data对象里的数据属性一样以属性访问的形式调用，即在页面中使用 `{{ 方法名 }}` 来显示计算的结果。

注：计算属性 `fullName` 不能在 data 中定义，而计算属性值的相关已知值在data中；

如果 `fullName` 在 data 中定义了会报错如下图：

```
✖ [Vue warn]: The computed property "fullName" is already defined in data.  
  
found in  
  
---> <Home> at src/views/Home.vue  
      <App> at src/App.vue  
      <Root>
```

因为如果 computed 属性值是一个函数，那么默认会走 get 方法，必须要有一个返回值，函数的返回值就是属性的属性值。计算属性定义了 fullName 并返回对应的结果给这个变量，变量不可被重复定义和赋值。

在官方文档中，还强调了 computed 一个重要的特点，就是 computed 带有缓存功能。比如我在页面中多次显示 fullName：

```
1  <p>姓名: {{ fullName }}</p>  
2  <p>姓名: {{ fullName }}</p>  
3  <p>姓名: {{ fullName }}</p>  
4  <p>姓名: {{ fullName }}</p>  
5  <p>姓名: {{ fullName }}</p>  
6  ...  
7  
8  computed: {  
9    fullName: function () {  
10      console.log('computed') // 在控制台只打印了一次  
11      return this.firstName + ' ' + this.lastName  
12    }  
13  }
```

我们知道 computed 内定义的 function 只执行一次，仅当初始化显示或者相关的 data、props 等属性数据发生变化的时候调用；

而 computed 属性值默认会缓存计算结果，计算属性是基于它们的响应式依赖进行缓存的；

只有当 computed 属性被使用后，才会执行 computed 的代码，在重复的调用中，只要依赖数据不变，直接取缓存中的计算结果。只有依赖型数据发生改变，computed 才会重新计算。

计算属性的高级：

在 computed 中的属性都有一个 get 和一个 set 方法，当数据变化时，调用 set 方法。下面我们通过计算属性的 getter/setter 方法来实现对属性数据的显示和监视，即双向绑定。

```

1  computed: {
2      fullName: {
3          get() { //读取当前属性值的回调，根据相关的数据计算并返回当前属性的值
4              return this.firstName + ' ' + this.lastName
5          },
6          set(val) { // 当属性值发生改变时回调，更新相关的属性数据，val就是fullName的
            最新属性值
7              const names = val ? val.split(' ') : [];
8              this.firstName = names[0]
9              this.lastName = names[1]
10         }
11     }
12 }

```

watch 监听属性

通过 vm 对象的 \$watch() 或 watch 配置来监听 Vue 实例上的属性变化，或某些特定数据的变化，然后执行某些具体的业务逻辑操作。当属性变化时，回调函数自动调用，在函数内部进行计算。其可以监听的数据来源：data，props，computed 内的数据。

以上示例通过 watch 来实现：

```

1  watch: {
2      // 监听 data 中的 firstName，如果发生了变化，就把变化的值给 data 中的
        fullName，val 就是 firstName 的最新值
3      firstName: function(val) {
4          this.fullName = val + ' ' + this.lastName
5      },
6      lastName: function(val) {
7          this.fullName = this.firstName + ' ' + val
8      }
9  }
10 // 由上可以看出 watch 要监听两个数据，而且代码是同类型的重复的，所以相比用 computed 更
    简洁

```

注：监听函数有两个参数，第一个参数是最新的值，第二个参数是输入之前的值，顺序一定是新值，旧值，如果只写一个参数，那就是最新属性值。

在使用时选择 watch 还是 computed，还有一个参考点就是官网说的：当需要在数据变化时执行异步或开销较大的操作时，watch 方式是最有用的。所以 watch 一定是支持异步的。

上面仅限监听简单数据类型，监听复杂数据类型就需要用到深度监听 deep。

****deep:** **为了发现对象内部值的变化，可以在选项参数中指定 deep: true。注意监听数组的变更不需要这么做。

```

1  data: {
2    fullName: {
3      firstName: 'David',
4      lastName: 'Beckham'
5    }
6  },
7  watch: {
8    fullName: {
9      handler(newVal, oldVal) {
10         console.log(newVal);
11         console.log(oldVal);
12       },
13       deep: true
14     }
15   }

```

以上打印结果：

```

newVal ▼ {__ob__: Observer} ⓘ
  firstName: "Brooklyn"
  lastName: "Beckham"
  ▶ __ob__: Observer {value: {...}, dep: Dep, vmCount: 0}
  ▶ get firstName: f reactiveGetter()
  ▶ set firstName: f reactiveSetter(newVal)
  ▶ get lastName: f reactiveGetter()
  ▶ set lastName: f reactiveSetter(newVal)
  ▶ __proto__: Object

```

```

oldVal ▼ {__ob__: Observer} ⓘ
  firstName: "Brooklyn"
  lastName: "Beckham"
  ▶ __ob__: Observer {value: {...}, dep: Dep, vmCount: 0}
  ▶ get firstName: f reactiveGetter()
  ▶ set firstName: f reactiveSetter(newVal)
  ▶ get lastName: f reactiveGetter()
  ▶ set lastName: f reactiveSetter(newVal)
  ▶ __proto__: Object

```

打印出来的 newVal 和 oldVal 值是一样的，所以深度监听虽然可以监听到对象的变化，但是无法监听到对象里面哪个具体属性的变化。这是因为它们的引用指向同一个对象/数组。Vue 不会保留变更之前值的副本。

若果要监听对象的单个属性的变化，有两种方法：

1.直接监听对象的属性

```
1  watch:{
2      fullName.firstName: function(newVal,oldVal){
3          console.log(newVal,oldVal);
4      }
5  }
```

2.与 computed 属性配合使用，computed 返回想要监听的属性值，watch 用来监听

```
1  computed: {
2      firstNameChange() {
3          return this.fullName.firstName
4      }
5  },
6  watch: {
7      firstNameChange() {
8          console.log(this.fullName)
9      }
10 }
```

总结：

watch和computed都是以Vue的依赖追踪机制为基础的，当某一个依赖型数据（依赖型数据：简单理解即放在 data 等对象下的实例数据）发生变化的时候，所有依赖这个数据的相关数据会自动发生变化，即自动调用相关的函数，来实现数据的变动。

当依赖的值变化时，在watch中，是可以做一些复杂的操作的，而computed中的依赖，仅仅是一个值依赖于另一个值，是值上的依赖。

应用场景：

computed：用于处理复杂的逻辑运算；一个数据受一个或多个数据影响；用来处理watch和methods无法处理的，或处理起来不方便的情况。例如处理模板中的复杂表达式、购物车里面的商品数量和总金额之间的变化关系等。

watch：用来处理当一个属性发生变化时，需要执行某些具体的业务逻辑操作，或要在数据变化时执行异步或开销较大的操作；一个数据改变影响多个数据。例如用来监控路由、input 输入框值的特殊处理等。

区别：

computed

- 初始化显示或者相关的 data、props 等属性数据发生变化的时候调用；
- 计算属性不在 data 中，它是基于 data 或 props 中的数据通过计算得到的一个新值，这个新值根据已知值的变化而变化；
- 在 computed 属性对象中定义计算属性的方法，和取 data 对象里的数据属性一样，以属性访问的形式调用；
- 如果 computed 属性值是函数，那么默认会走 get 方法，必须要有一个返回值，函数的返回值就是属性的属性值；
- computed 属性值默认会缓存计算结果，在重复的调用中，只要依赖数据不变，直接取缓存中的计算结果，只有依赖型数据发生改变，computed 才会重新计算；
- 在 computed 中的，属性都有一个 get 和一个 set 方法，当数据变化时，调用 set 方法。

watch

- 主要用来监听某些特定数据的变化，从而进行某些具体的业务逻辑操作，可以看作是 computed 和 methods 的结合体；
- 可以监听的数据来源：data，props，computed 内的数据；
- watch 支持异步；
- 不支持缓存，监听的数据改变，直接会触发相应的操作；
- 监听函数有两个参数，第一个参数是最新的值，第二个参数是输入之前的值，顺序一定是新值，旧值。

第19题：vuex中的辅助函数怎么使用？

参考答案：

在实际开发中，我们经常会用到 vuex 来对数据进行管理，随着数据越来越多，我们逐渐开始使用一些语法糖来帮助我们快速开发。即 vuex 中的 mapState、mapGetters、mapMutations、mapActions 等辅助函数是我们经常使用到的。

辅助函数

通过辅助函数 `mapState`、`mapActions`、`mapMutations`，把 `vuex.store` 中的属性映射到 `vue` 实例身上，这样在 `vue` 实例中就能访问 `vuex.store` 中的属性了，对于操作 `vuex.store` 就很方便了。

`state` 辅助函数为 `mapState`，`actions` 辅助函数为 `mapActions`，`mutations` 辅助函数为 `mapMutations`。（`Vuex` 实例身上有 `mapState`、`mapActions`、`mapMutations` 属性，属性值都是函数）

如何使用辅助函数

首先，需要在当前组件中引入 `Vuex`。

然后，通过Vuex来调用辅助函数。

辅助函数如何去映射vuex.store中的属性

1、mapState:把state属性映射到computed身上

```
1  computed:{
2    ...Vuex.mapState({
3      input:state=>state.inputVal,
4      n:state=>state.n
5    })
6  }
```

`state`：用来存储公共的状态 在 `state` 中的数据都是响应式的。

响应式原因：`state` 里面有一个 `getters`、`setters` 方法；`data` 中的数据也是响应式的，因为里面也有 `getters` 和 `setters` 方法

在 `computed` 属性中来接收 `state` 中的数据,接收方式有2种（数组和对象，推荐对象）。

优点：

- 本身key值是别名，要的是val的值，key的值a 和 `val="a"`一样就行，随意写。减少state里面长的属性名。
- 可以在函数内部查看state中的数据，数组方式的话，必须按照state中的属性名。

```
1  computed:Vuex.mapState({
2    key:state=>state.属性
3  })
```

如果自身组件也需要使用computed的话，通过解构赋值去解构出来

```
1  computed:{
2    ...Vuex.mapState({
3      key:state=>state.属性
4    })
5  }
```

2、mapActions：把actions里面的方法映射到methods中

```

1  methods:{
2      ...Vuex.mapActions({
3          add:"handleTodoAdd",    //val为actions里面的方法名称
4          change:"handleInput"
5      })
6  }

```

add、change为action方法别名，直接代用add和change方法就行，不过要记得在actions里面做完数据业务逻辑的操作。

等价于如下的函数调用，

```

1  methods: {
2      handleInput(e){
3          let val = e.target.value;
4          this.$store.dispatch("handleInput",val )
5      },
6      handleAdd(){
7          this.$store.dispatch("handleTodoAdd")
8      }
9  }

```

`actions` 里面的函数主要用来处理异步的函数以及一些业务逻辑,每一个函数里面都有一个形参，这个形参是一个对象，里面有一个 `commit` 方法，这个方法用来触发mutations里面的方法

3、mapMutations：把mutations里面的方法映射到methods中

只是做简单的数据修改（例如n++），它没有涉及到数据的处理，没有用到业务逻辑或者异步函数，可以直接调用mutations里的方法修改数据。

```

1  methods:{
2      ...Vuex.mapMutations({
3          handleAdd:"handleMutationseAdd"
4      })
5  }
6

```

`mutations` 里面的函数主要用来修改 `state` 中的数据。`mutations` 里面的所有方法都会有2个参数，一个是 `store` 中的 `state`，另外一个是需要传递的参数。

理解 `state`、`actions`、`mutations`，可以参考 `MVC` 框架。

- `state` 看成一个数据库，只是它是响应式的，刷新页面数据就会改变；

- `actions` 看成controller层，做数据的业务逻辑；
- `mutations` 看成model层，做数据的增删改查操作。

4、mapGetters:把getters属性映射到computed身上

```
1  computed:{
2      ...Vuex.mapGetters({
3          NumN:"NumN"
4      })
5  }
```

`getters` 类似于组件里面 `computed`，同时也监听属性的变化，当 `state` 中的属性发生改变的时候就会触发 `getters` 里面的方法。`getters` 里面的每一个方法中都会有一个参数 `state`。

5、modules属性: 模块

把公共的状态按照模块进行划分

- 每个模块都相当于一个小型的Vuex
- 每个模块里面都会有 `state` `getters` `actions` `mutations`
- 切记在导出模块的时候加一个 `namespaced:true` 主要的作用是将每个模块都有独立命名空间
- `namespace: true` 在多人协作开发的时候，可能子模块和主模块中的函数名字会相同，这样在调用函数的时候，相同名字的函数都会被调用，就会发生问题。为了解决这个问题，导出模块的时候要加 `namespace: true`。

那么怎么调用子模块中的函数呢？假如我的子模块名字为todo.js。函数名字就需要改成todo/函数名字。输出模块后的store实例如下图所示：


```
1  mapXXS('命名空间名称', ['属性名1', '属性名2'])
2
3  mapXXS('命名空间名称', {
4
5      '组件中的新名称1': 'Vuex中的原名称1',
6
7      '组件中的新名称2': 'Vuex中的原名称2',
8
9  })
```

第20题：如果使用Vue3.0实现一个 Modal，你会怎么进行设计？

一、组件设计

组件就是把图形、非图形的各种逻辑均抽象为一个统一的概念（组件）来实现开发的模式

现在有一个场景，点击新增与编辑都弹框出来进行填写，功能上大同小异，可能只是标题内容或者是显示的主体内容稍微不同

这时候就没必要写两个组件，只需要根据传入的参数不同，组件显示不同内容即可

这样，下次开发相同界面程序时就可以写更少的代码，意味着更高的开发效率，更少的 `Bug` 和更少的程序体积

二、需求分析

实现一个 `Modal` 组件，首先确定需要完成的内容：

- 遮罩层
- 标题内容
- 主体内容
- 确定和取消按钮

主体内容需要灵活，所以可以是字符串，也可以是一段 `html` 代码

特点是它们在当前 `vue` 实例之外独立存在，通常挂载于 `body` 之上

除了通过引入 `import` 的形式，我们还可通过 `API` 的形式进行组件的调用

还可以包括配置全局样式、国际化、与 `typeScript` 结合

三、实现流程

首先看看大致流程：

- 目录结构

- 组件内容
- 实现 API 形式
- 事件处理
- 其他完善

目录结构

Modal 组件相关的目录结构

```
1  |— plugins
2  |   └─ modal
3  |       └─ Content.tsx // 维护 Modal 的内容，用于 h 函数和 jsx 语法
4  |       └─ Modal.vue // 基础组件
5  |       └─ config.ts // 全局默认配置
6  |       └─ index.ts // 入口
7  |       └─ locale // 国际化相关
8  |       |   └─ index.ts
9  |       |   └─ lang
10 |       |       └─ en-US.ts
11 |       |       └─ zh-CN.ts
12 |       |       └─ zh-TW.ts
13 |       └─ modal.type.ts // ts类型声明相关
```

因为 Modal 会被 `app.use(Modal)` 调用作为一个插件，所以都放在 `plugins` 目录下

组件内容

首先实现 `modal.vue` 的主体显示内容大致如下

```
1  <Teleport to="body" :disabled="!isTeleport">
2    <div v-if="modelValue" class="modal">
3      <div
4        class="mask"
5        :style="style"
6        @click="maskClose && !loading && handleCancel()"
7      ></div>
8      <div class="modal__main">
9        <div class="modal__title line line--b">
10          <span>{{ title || t('r.title') }}</span>
11          <span
12            v-if="close"
13            :title="t('r.close')"
14            class="close"
```

```

15         @click="!loading && handleCancel()"
16     > </span>
17 >
18 </div>
19 <div class="modal__content">
20     <Content v-if="typeof content === 'function'"
:render="content" />
21     <slot v-else>
22         {{ content }}
23     </slot>
24 </div>
25 <div class="modal__btns line line--t">
26     <button :disabled="loading" @click="handleConfirm">
27         <span class="loading" v-if="loading"> </span>{{
t("r.confirm") }}
28     </button>
29     <button @click="!loading && handleCancel()">
30         {{ t("r.cancel") }}
31     </button>
32 </div>
33 </div>
34 </div>
35 </Teleport>

```

最外层上通过Vue3 `Teleport` 内置组件进行包裹，其相当于传送门，将里面的内容传送到 `body` 之上

并且从 `DOM` 结构上来看，把 `modal` 该有的内容（遮罩层、标题、内容、底部按钮）都实现了关于主体内容

```

1 <div class="modal__content">
2     <Content v-if="typeof content==='function'"
3         :render="content" />
4     <slot v-else>
5         {{content}}
6     </slot>
7 </div>

```

可以看到根据传入 `content` 的类型不同，对应显示不同得到内容

最常见的则是通过调用字符串和默认插槽的形式

```
1 // 默认插槽
```

```

2   <Modal v-model="show"
3       title="演示 slot">
4       <div>hello world~</div>
5   </Modal>
6
7   // 字符串
8   <Modal v-model="show"
9       title="演示 content"
10      content="hello world~" />

```

通过 API 形式调用 `Modal` 组件的时候，`content` 可以使用下面两种

- h 函数

```

1   $modal.show({
2     title: '演示 h 函数',
3     content(h) {
4       return h(
5         'div',
6         {
7           style: 'color:red;',
8           onClick: ($event: Event) => console.log('clicked', $event.target)
9         },
10        'hello world ~'
11      );
12    }
13  });

```

- JSX

```

1   $modal.show({
2     title: '演示 jsx 语法',
3     content() {
4       return (
5         <div
6           onClick={$event => console.log('clicked', $event.target)}
7         >
8           hello world ~
9         </div>
10      );
11    }
12  });

```

实现 API 形式

那么组件如何实现 API 形式调用 Modal 组件呢？

在 Vue2 中，我们可以借助 Vue 实例以及 Vue.extend 的方式获得组件实例，然后挂载到 body 上

```
1 import Modal from './Modal.vue';
2 const ComponentClass = Vue.extend(Modal);
3 const instance = new ComponentClass({ el: document.createElement("div") });
4 document.body.appendChild(instance.$el);
```

虽然 Vue3 移除了 Vue.extend 方法，但可以通过 createVNode 实现

```
1 import Modal from './Modal.vue';
2 const container = document.createElement('div');
3 const vnode = createVNode(Modal);
4 render(vnode, container);
5 const instance = vnode.component;
6 document.body.appendChild(container);
```

在 Vue2 中，可以通过 this 的形式调用全局 API

```
1 export default {
2   install(vue) {
3     vue.prototype.$create = create
4   }
5 }
```

而在 Vue3 的 setup 中已经没有 this 概念了，需要调用 app.config.globalProperties 挂载到全局

```
1 export default {
2   install(app) {
3     app.config.globalProperties.$create = create
4   }
5 }
```

事件处理

下面再来看看 `Modal` 组件内部是如何处理「确定」「取消」事件的，既然是 `Vue3`，当然采用 `Compositon API` 形式

```
1  // Modal.vue
2  setup(props, ctx) {
3    let instance = getCurrentInstance(); // 获得当前组件实例
4    onBeforeMount(() => {
5      instance._hub = {
6        'on-cancel': () => {},
7        'on-confirm': () => {}
8      };
9    });
10
11    const handleConfirm = () => {
12      ctx.emit('on-confirm');
13      instance._hub['on-confirm']();
14    };
15    const handleCancel = () => {
16      ctx.emit('on-cancel');
17      ctx.emit('update:modelValue', false);
18      instance._hub['on-cancel']();
19    };
20
21    return {
22      handleConfirm,
23      handleCancel
24    };
25  }
```

在上面代码中，可以看得到除了使用传统 `emit` 的形式使父组件监听，还可通过 `_hub` 属性中添加 `on-cancel`，`on-confirm` 方法实现在 `API` 中进行监听

```
1  app.config.globalProperties.$modal = {
2    show({}) {
3      /* 监听 确定、取消 事件 */
4    }
5  }
```

下面再来目睹下 `_hub` 是如何实现

```
1  // index.ts
2  app.config.globalProperties.$modal = {
```

```
3     show({
4         /* 其他选项 */
5         onConfirm,
6         onCancel
7     }) {
8         /* ... */
9
10        const { props, _hub } = instance;
11
12        const _closeModal = () => {
13            props.modelValue = false;
14            container.parentNode!.removeChild(container);
15        };
16        // 往 _hub 新增事件的具体实现
17        Object.assign(_hub, {
18            async 'on-confirm'() {
19                if (onConfirm) {
20                    const fn = onConfirm();
21                    // 当方法返回为 Promise
22                    if (fn && fn.then) {
23                        try {
24                            props.loading = true;
25                            await fn;
26                            props.loading = false;
27                            _closeModal();
28                        } catch (err) {
29                            // 发生错误时, 不关闭弹框
30                            console.error(err);
31                            props.loading = false;
32                        }
33                    } else {
34                        _closeModal();
35                    }
36                } else {
37                    _closeModal();
38                }
39            },
40            'on-cancel'() {
41                onCancel && onCancel();
42                _closeModal();
43            }
44        });
45    }
46};
```

第21题：Vue 3.0中Treeshaking特性是什么，并举例进行说明？

一、是什么

Tree shaking 是一种通过清除多余代码方式来优化项目打包体积的技术，专业术语叫 Dead code elimination

简单来讲，就是在保持代码运行结果不变的前提下，去除无用的代码

如果把代码打包比作制作蛋糕，传统的方式是把鸡蛋（带壳）全部丢进去搅拌，然后放入烤箱，最后把（没有用的）蛋壳全部挑选并剔除出去

而 treeshaking 则是一开始就把有用的蛋白蛋黄（import）放入搅拌，最后直接作出蛋糕

也就是说，tree shaking 其实是找出使用的代码

在 Vue2 中，无论我们使用什么功能，它们最终都会出现在生产代码中。主要原因是 Vue 实例在项目是单例的，捆绑程序无法检测到该对象的哪些属性在代码中被使用到

```
1 import Vue from 'vue'
2
3 Vue.nextTick(() => {})
```

而 Vue3 源码引入 tree shaking 特性，将全局 API 进行分块。如果您不使用其某些功能，它们将不会包含在您的基础包中

```
1 import { nextTick, observable } from 'vue'
2
3 nextTick(() => {})
```

二、如何做

Tree shaking 是基于 ES6 模板语法（import 与 export），主要是借助 ES6 模块的静态编译思想，在编译时就能确定模块的依赖关系，以及输入和输出的变量

Tree shaking 无非就是做了两件事：

- 编译阶段利用 ES6 Module 判断哪些模块已经加载
- 判断那些模块和变量未被使用或者引用，进而删除对应代码

下面就来举个例子：

通过脚手架 `vue-cli` 安装 `Vue2` 与 `Vue3` 项目

```
1 vue create vue-demo
```

Vue2 项目

组件中使用 `data` 属性

```
1 <script>
2   export default {
3     data: () => ({
4       count: 1,
5     }),
6   };
7 </script>
```

对项目进行打包，体积如下图

DONE Compiled successfully in 18393ms

File	Size	Gzipped
dist\js\chunk-vendors.28d0d835.js	89.59 KiB	32.11 KiB
dist\js\app.3f482fc1.js	2.01 KiB	1.01 KiB

Images and other types of assets omitted.

为组件设置其他属性（`computed`、`watch`）

```
1 export default {
2   data: () => ({
3     question:"",
4     count: 1,
5   }),
6   computed: {
7     double: function () {
8       return this.count * 2;
9     },
10  },
11  watch: {
12    question: function (newQuestion, oldQuestion) {
13      this.answer = 'xxxx'
14    }
15  }
16 }
```

```
15   };
```

再一次打包，发现打包出来的体积并没有变化

File	Size	Gzipped
dist\js\chunk-vendors.28d0d835.js	89.59 KiB	32.11 KiB
dist\js\app.94092e3d.js	2.07 KiB	1.04 KiB

Images and other types of assets omitted.

Vue3 项目

组件中简单使用

```
1  import { reactive, defineComponent } from "vue";
2  export default defineComponent({
3    setup() {
4      const state = reactive({
5        count: 1,
6      });
7      return {
8        state,
9      };
10   },
11 });
```

将项目进行打包

DONE Compiled successfully in 12015ms

File	Size	Gzipped
dist\js\chunk-vendors.de2030ce.js	78.91 KiB	29.62 KiB
dist\js\app.c06bf53e.js	1.92 KiB	0.93 KiB

在组件中引入 `computed` 和 `watch`

```
1  import { reactive, defineComponent, computed, watch } from "vue";
2  export default defineComponent({
3    setup() {
4      const state = reactive({
5        count: 1,
6      });
```

```

7      const double = computed(() => {
8        return state.count * 2;
9      });
10
11     watch(
12       () => state.count,
13       (count, preCount) => {
14         console.log(count);
15         console.log(preCount);
16       }
17     );
18     return {
19       state,
20       double,
21     };
22   },
23 });

```

再次对项目进行打包，可以看到在引入 `computer` 和 `watch` 之后，项目整体体积变大了

DONE Compiled successfully in 8011ms

File	Size	Gzipped
dist\js\chunk-vendors.19e22567.js	79.05 KiB	29.64 KiB
dist\js\app.00e83bfe.js	2.15 KiB	1.00 KiB

三、作用

通过 `Tree shaking`，`Vue3` 给我们带来的好处是：

- 减少程序体积（更小）
- 减少程序执行时间（更快）
- 便于将来对程序架构进行优化（更友好）

第22题：Vue3.0 所采用的 Composition Api 与 Vue2.x 使用的 Options Api 有什么不同？

开始之前

`Composition API` 可以说是 `Vue3` 的最大特点，那么为什么要推出 `Composition Api`，解决了什么问题？

通常使用 `Vue2` 开发的项目，普遍会存在以下问题：

- 代码的可读性随着组件变大而变差
- 每一种代码复用的方式，都存在缺点
- TypeScript支持有限

以上通过使用 `Composition Api` 都能迎刃而解

正文

一、Options Api

`Options API`，即大家常说的选项API，即以 `vue` 为后缀的文件，通过定义 `methods`，`computed`，`watch`，`data` 等属性与方法，共同处理页面逻辑

如下图：

Options API

```
export default {  
  data() {  
    return {  
      功能 A  
      功能 B  
    };  
  },  
  methods: {  
    功能 A  
    功能 B  
  },  
  computed: {  
    功能 A  
  },  
  watch: {  
    功能 B  
  }  
}
```

可以看到 Options 代码编写方式，如果是组件状态，则写在 data 属性上，如果是方法，则写在 methods 属性上...

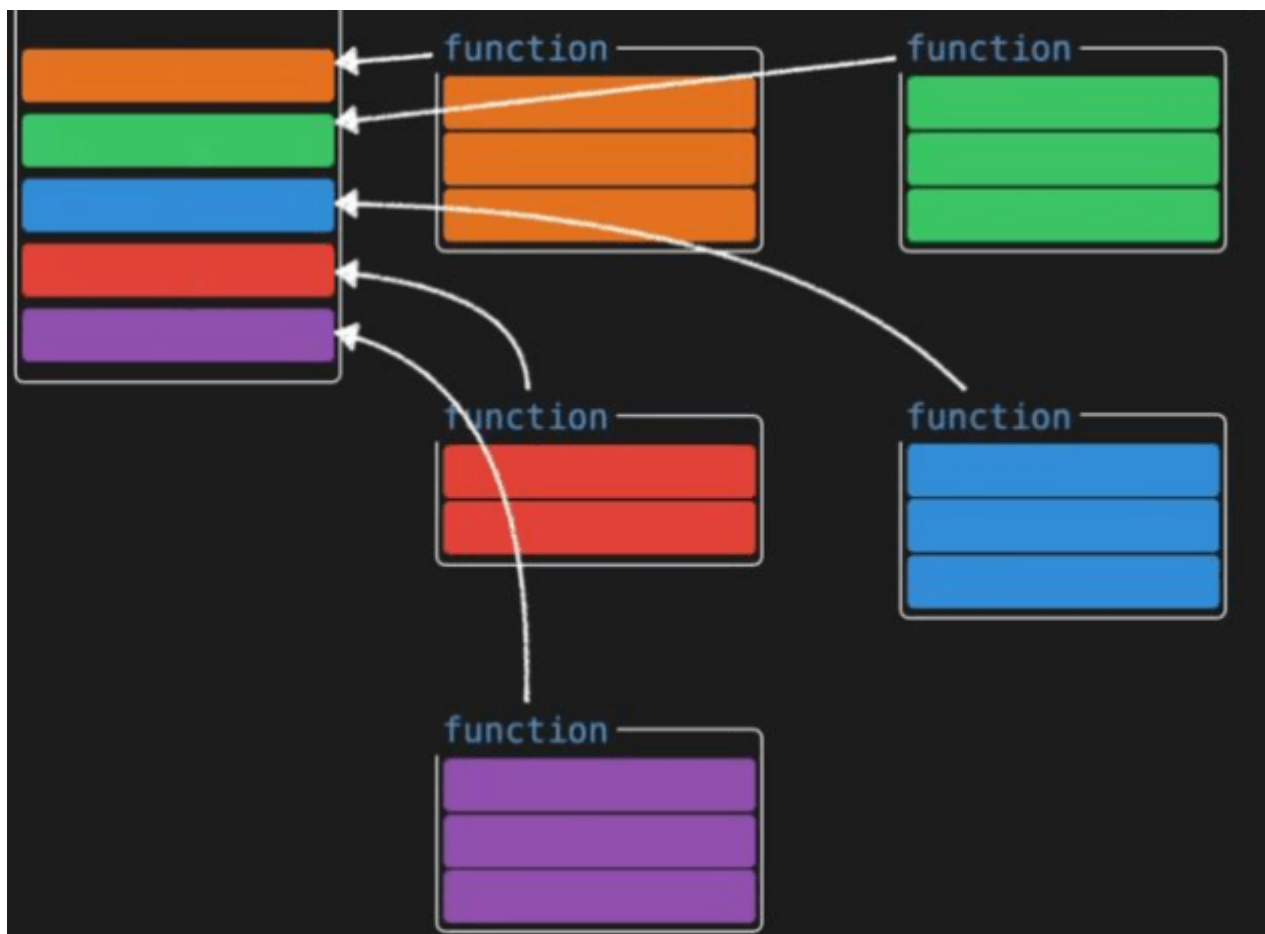
用组件的选项 (data 、 computed 、 methods 、 watch) 组织逻辑在大多数情况下都有效

然而，当组件变得复杂，导致对应属性的列表也会增长，这可能会导致组件难以阅读和理解

二、Composition Api

在 Vue3 Composition API 中，组件根据逻辑功能来组织的，一个功能所定义的所有 API 会放在一起（更加的高内聚，低耦合）

即使项目很大，功能很多，我们都能快速的定位到这个功能所用到的所有 API



三、对比

下面对 Composition Api 与 Options Api 进行两大方面的比较

- 逻辑组织
- 逻辑复用

逻辑组织

Options API

假设一个组件是一个大型组件，其内部有很多处理逻辑关注点（对应下图不用颜色）

Compostion API

而 `Compositon API` 正是解决上述问题，将某个逻辑关注点相关的代码全都放在一个函数里，这样当需要修改一个功能时，就不再需要在文件中跳来跳去

下面举个简单例子，将处理 `count` 属性相关的代码放在同一个函数了

```
1  function useCount() {
2      let count = ref(10);
3      let double = computed(() => {
4          return count.value * 2;
5      });
6
7      const handleConut = () => {
8          count.value = count.value * 2;
9      };
10
11     console.log(count);
12
13     return {
14         count,
15         double,
16         handleConut,
17     };
18 }
```

组件上中使用 `count`

```
1  export default defineComponent({
2      setup() {
3          const { count, double, handleConut } = useCount();
4          return {
5              count,
6              double,
7              handleConut
8          }
9      },
10  });
```

再来一张图进行对比，可以很直观地感受到 `Composition API` 在逻辑组织方面的优势，以后修改一个属性功能的时候，只需要跳到控制该属性的方法中即可

Options API

```
import { createApp } from 'vue'
import './style.css'
import App from './App.vue'

createApp(App).mount('#app')
```

Composition API

```
import { createApp } from 'vue'
import './style.css'
import App from './App.vue'

createApp(App).mount('#app')
```

逻辑复用

在 `Vue2` 中，我们是用过 `mixin` 去复用相同的逻辑

下面举个例子，我们会另起一个 `mixin.js` 文件

```
1 export const MoveMixin = {
2   data() {
3     return {
4       x: 0,
5       y: 0,
6     };
7   },
8
9   methods: {
```

```

10     handleKeyup(e) {
11         console.log(e.code);
12         // 上下左右 x y
13         switch (e.code) {
14             case "ArrowUp":
15                 this.y--;
16                 break;
17             case "ArrowDown":
18                 this.y++;
19                 break;
20             case "ArrowLeft":
21                 this.x--;
22                 break;
23             case "ArrowRight":
24                 this.x++;
25                 break;
26         }
27     },
28 },
29
30 mounted() {
31     window.addEventListener("keyup", this.handleKeyup);
32 },
33
34 unmounted() {
35     window.removeEventListener("keyup", this.handleKeyup);
36 },
37 };

```

然后在组件中使用

```

1  <template>
2    <div>
3      Mouse position: x {{ x }} / y {{ y }}
4    </div>
5  </template>
6  <script>
7    import mousePositionMixin from './mouse'
8    export default {
9      mixins: [mousePositionMixin]
10    }
11  </script>

```

使用单个 `mixin` 似乎问题不大，但是当我们一个组件混入大量不同的 `mixins` 的时候

```
1 mixins: [mousePositionMixin, fooMixin, barMixin, otherMixin]
```

会存在两个非常明显的问题：

- 命名冲突
- 数据来源不清晰

现在通过 `Composition API` 这种方式改写上面的代码

```
1 import { onMounted, onUnmounted, reactive } from "vue";
2 export function useMove() {
3   const position = reactive({
4     x: 0,
5     y: 0,
6   });
7
8   const handleKeyup = (e) => {
9     console.log(e.code);
10    // 上下左右 x y
11    switch (e.code) {
12      case "ArrowUp":
13        // y.value--;
14        position.y--;
15        break;
16      case "ArrowDown":
17        // y.value++;
18        position.y++;
19        break;
20      case "ArrowLeft":
21        // x.value--;
22        position.x--;
23        break;
24      case "ArrowRight":
25        // x.value++;
26        position.x++;
27        break;
28    }
29  };
30
31  onMounted(() => {
32    window.addEventListener("keyup", handleKeyup);
33  });
34
35  onUnmounted(() => {
```

```
36     window.removeEventListener("keyup", handleKeyup);
37   });
38
39   return { position };
40 }
```

在组件中使用

```
1  <template>
2    <div>
3      Mouse position: x {{ x }} / y {{ y }}
4    </div>
5  </template>
6
7  <script>
8    import { useMove } from "../useMove";
9    import { toRefs } from "vue";
10   export default {
11     setup() {
12       const { position } = useMove();
13       const { x, y } = toRefs(position);
14       return {
15         x,
16         y,
17       };
18     },
19   },
20 };
21 </script>
```

可以看到，整个数据来源清晰了，即使去编写更多的 hook 函数，也不会出现命名冲突的问题

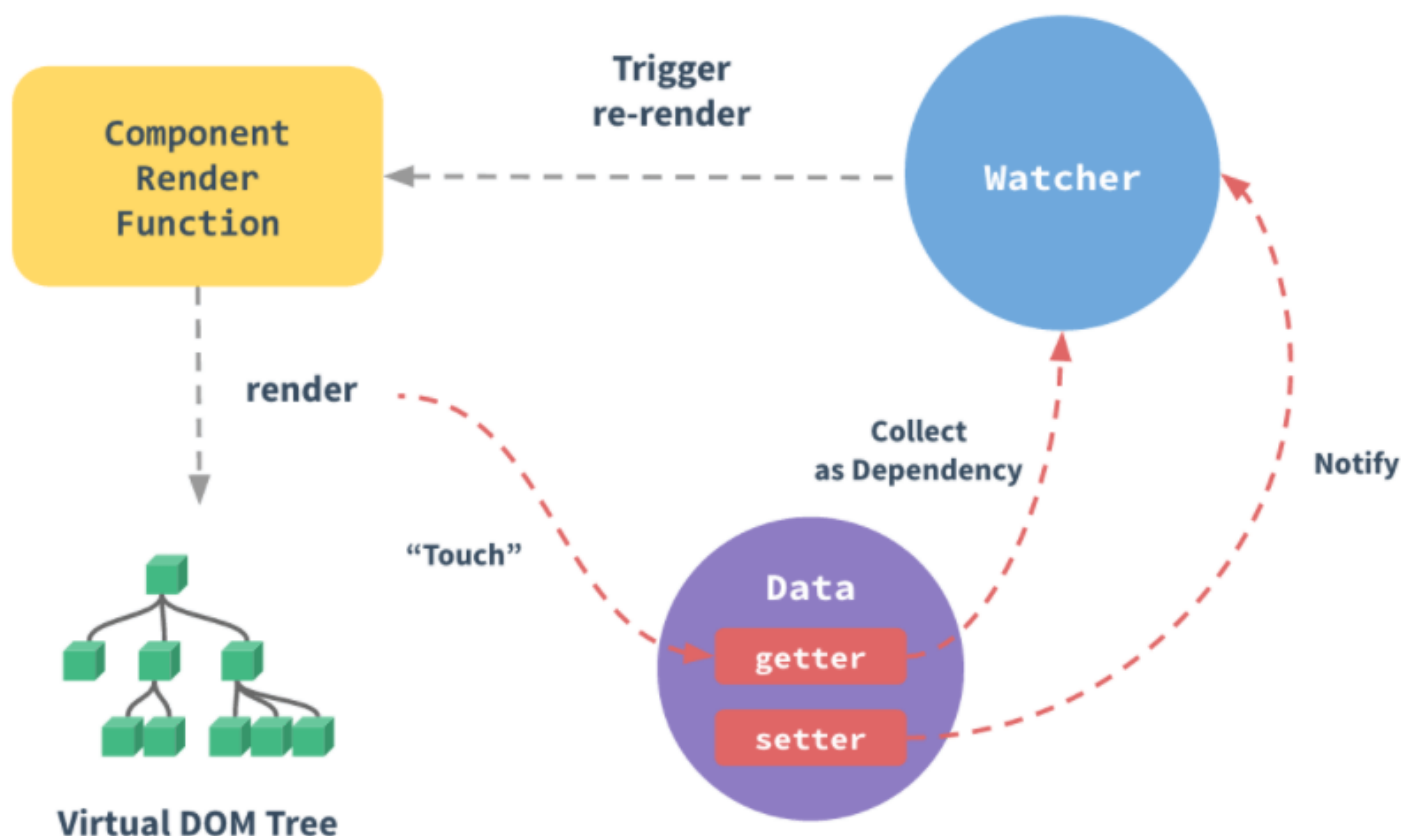
小结

- 在逻辑组织和逻辑复用方面，Composition API 是优于 Options API
- 因为 Composition API 几乎是函数，会有更好的类型推断。
- Composition API 对 tree-shaking 友好，代码也更容易压缩
- Composition API 中见不到 this 的使用，减少了 this 指向不明的情况
- 如果是小型组件，可以继续使用 Options API，也是十分友好的

第23题：Vue3.0 性能提升主要是通过哪几方面体现的？

一、编译阶段

回顾 Vue2，我们知道每个组件实例都对应一个 `watcher` 实例，它会在组件渲染的过程中把用到的数据 `property` 记录为依赖，当依赖发生改变，触发 `setter`，则会通知 `watcher`，从而使关联的组件重新渲染



试想一下，一个组件结构如下图

```
1  <template>
2    <div id="content">
3      <p class="text">静态文本</p>
4      <p class="text">静态文本</p>
5      <p class="text">{{ message }}</p>
6      <p class="text">静态文本</p>
7      ...
8      <p class="text">静态文本</p>
9    </div>
10 </template>
```

可以看到，组件内部只有一个动态节点，剩余一堆都是静态节点，所以这里很多 `diff` 和遍历其实都是不需要的，造成性能浪费

因此，Vue3 在编译阶段，做了进一步优化。主要有如下：

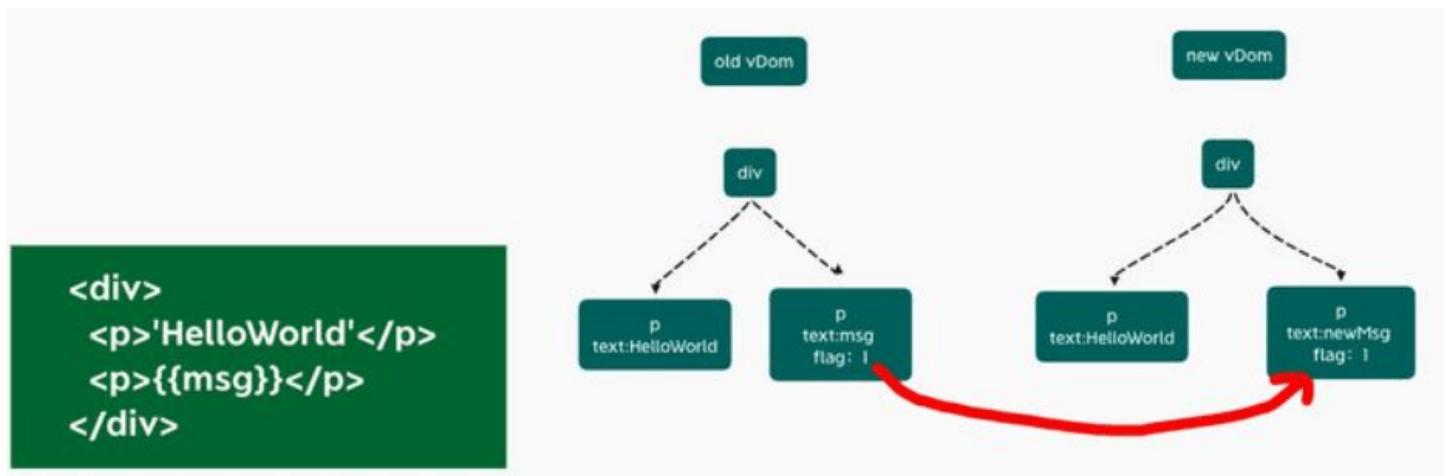
- diff算法优化
- 静态提升
- 事件监听缓存
- SSR优化

diff算法优化

vue3 在 diff 算法中相比 vue2 增加了静态标记

关于这个静态标记，其作用是为了会发生变化的地方添加一个 flag 标记，下次发生变化的时候直接找该地方进行比较

下图这里，已经标记静态节点的 p 标签在 diff 过程中则不会比较，把性能进一步提高



关于静态类型枚举如下

```
1 export const enum PatchFlags {
2   TEXT = 1, // 动态的文本节点
3   CLASS = 1 << 1, // 2 动态的 class
4   STYLE = 1 << 2, // 4 动态的 style
5   PROPS = 1 << 3, // 8 动态属性，不包括类名和样式
6   FULL_PROPS = 1 << 4, // 16 动态 key，当 key 变化时需要完整的 diff 算法做比较
7   HYDRATE_EVENTS = 1 << 5, // 32 表示带有事件监听器的节点
8   STABLE_FRAGMENT = 1 << 6, // 64 一个不会改变子节点顺序的 Fragment
9   KEYED_FRAGMENT = 1 << 7, // 128 带有 key 属性的 Fragment
10  UNKEYED_FRAGMENT = 1 << 8, // 256 子节点没有 key 的 Fragment
11  NEED_PATCH = 1 << 9, // 512
12  DYNAMIC_SLOTS = 1 << 10, // 动态 slot
13  HOISTED = -1, // 特殊标志是负整数表示永远不会用作 diff
14  BAIL = -2 // 一个特殊的标志，指代差异算法
15 }
```

静态提升

Vue3 中对不参与更新的元素，会做静态提升，只会被创建一次，在渲染时直接复用

这样就免去了重复的创建节点，大型应用会受益于这个改动，免去了重复的创建操作，优化了运行时候的内存占用

```
1 <span>你好</span>
2
3 <div>{{ message }}</div>
```

没有做静态提升之前

```
1 export function render(_ctx, _cache, $props, $setup, $data, $options) {
2   return (_openBlock(), _createBlock(_Fragment, null, [
3     _createVNode("span", null, "你好"),
4     _createVNode("div", null, _toDisplayString(_ctx.message), 1 /* TEXT */)
5   ], 64 /* STABLE_FRAGMENT */))
6 }
```

做了静态提升之后

```
1 const _hoisted_1 = /*#__PURE__*/_createVNode("span", null, "你好", -1 /*
  HOISTED */)
2
3 export function render(_ctx, _cache, $props, $setup, $data, $options) {
4   return (_openBlock(), _createBlock(_Fragment, null, [
5     _hoisted_1,
6     _createVNode("div", null, _toDisplayString(_ctx.message), 1 /* TEXT */)
7   ], 64 /* STABLE_FRAGMENT */))
8 }
9
10 // Check the console for the AST
```

静态内容 `_hoisted_1` 被放置在 `render` 函数外，每次渲染的时候只要取 `_hoisted_1` 即可

同时 `_hoisted_1` 被打上了 `PatchFlag`，静态标记值为 -1，特殊标志是负整数表示永远不会用于 Diff

事件监听缓存

默认情况下绑定事件行为会被视为动态绑定，所以每次都会去追踪它的变化

```
1 <div>
2   <button @click = 'onClick'>点我</button>
3 </div>
```

没开启事件监听器缓存

```
1 export const render = /*#__PURE__*/_withId(function render(_ctx, _cache,
  $props, $setup, $data, $options) {
2   return (_openBlock(), _createBlock("div", null, [
3     _createVNode("button", { onClick: _ctx.onClick }, "点我", 8 /* PROPS */,
      ["onClick"]))
4                                     // PROPS=1<<3, // 8 //动态属性，但不
      包含类名和样式
5   ]))
6 })
```

开启事件侦听器缓存后

```
1 export function render(_ctx, _cache, $props, $setup, $data, $options) {
2   return (_openBlock(), _createBlock("div", null, [
3     _createVNode("button", {
4       onClick: _cache[1] || (_cache[1] = (...args) => (_ctx.onClick(...args)))
5     }, "点我")
6   ]))
7 }
```

上述发现开启了缓存后，没有了静态标记。也就是说下次 `diff` 算法的时候直接使用

SSR优化

当静态内容大到一定量级时候，会用 `createStaticVNode` 方法在客户端去生成一个static node，这些静态 `node`，会被直接 `innerHTML`，就不需要创建对象，然后根据对象渲染

```
1 div>
2   <div>
3     <span>你好</span>
4   </div>
5   ... // 很多个静态属性
```

```

6      <div>
7          <span>{{ message }}</span>
8      </div>
9  </div>

```

编译后

```

1  import { mergeProps as _mergeProps } from "vue"
2  import { ssrRenderAttrs as _ssrRenderAttrs, ssrInterpolate as _ssrInterpolate
   } from "@vue/server-renderer"
3
4  export function ssrRender(_ctx, _push, _parent, _attrs, $props, $setup,
   $data, $options) {
5      const _cssVars = { style: { color: _ctx.color }}
6      _push(`<div${
7          _ssrRenderAttrs(_mergeProps(_attrs, _cssVars))
8      }><div><span>你好</span>...<div><span>你好</span><div><span>${
9          _ssrInterpolate(_ctx.message)
10     }</span></div></div>`)
11 }

```

二、源码体积

相比 `Vue2`，`Vue3` 整体体积变小了，除了移出一些不常用的API，再重要的是 `Tree shanking` 任何一个函数，如 `ref`、`reactived`、`computed` 等，仅仅在用到的时候才打包，没用到的模块都被摇掉，打包的整体体积变小

```

1  import { computed, defineComponent, ref } from 'vue';
2  export default defineComponent({
3      setup(props, context) {
4          const age = ref(18)
5
6          let state = reactive({
7              name: 'test'
8          })
9
10         const readOnlyAge = computed(() => age.value++) // 19
11
12         return {
13             age,
14             state,
15             readOnlyAge

```

```
16      }  
17      }  
18  });
```

三、响应式系统

vue2 中采用 `defineProperty` 来劫持整个对象，然后进行深度遍历所有属性，给每个属性添加 `getter` 和 `setter`，实现响应式

vue3 采用 `proxy` 重写了响应式系统，因为 `proxy` 可以对整个对象进行监听，所以不需要深度遍历

- 可以监听动态属性的添加
- 可以监听到数组的索引和数组 `length` 属性
- 可以监听删除属性

关于这两个 API 具体的不同，我们下篇文章会进行一个更加详细的介绍

第24题：Vue3.0的设计目标是什么？做了哪些优化？

一、设计目标

不以解决实际业务痛点的更新都是耍流氓，下面我们来列举一下 `Vue3` 之前我们或许会面临的问题

- 随着功能的增长，复杂组件的代码变得越来越难以维护
- 缺少一种比较「干净」的在多个组件之间提取和复用逻辑的机制
- 类型推断不够友好
- `bundle` 的时间太久了

而 `Vue3` 经过长达两三年时间的筹备，做了哪些事情？

我们从结果反推

- 更小
- 更快
- TypeScript支持
- API设计一致性
- 提高自身可维护性
- 开放更多底层功能

一句话概述，就是更小更快更友好了

更小

Vue3 移除一些不常用的 API

引入 tree-shaking，可以将无用模块“剪辑”，仅打包需要的，使打包的整体体积变小了

更快

主要体现在编译方面：

- diff算法优化
- 静态提升
- 事件监听缓存
- SSR优化

下篇文章我们会进一步介绍

更友好

vue3 在兼顾 vue2 的 options API 的同时还推出了 composition API，大大增加了代码的逻辑组织和代码复用能力

这里代码简单演示下：

存在一个获取鼠标位置的函数

```
1  import { toRefs, reactive } from 'vue';
2  function useMouse(){
3      const state = reactive({x:0,y:0});
4      const update = e=>{
5          state.x = e.pageX;
6          state.y = e.pageY;
7      }
8      onMounted(()=>{
9          window.addEventListener('mousemove',update);
10     })
11     onUnmounted(()=>{
12         window.removeEventListener('mousemove',update);
13     })
14
15     return toRefs(state);
16 }
```

我们只需要调用这个函数，即可获取 x、y 的坐标，完全不用关注实现过程

试想一下，如果很多类似的第三方库，我们只需要调用即可，不必关注实现过程，开发效率大大提高

同时，VUE3 是基于 typescript 编写的，可以享受到自动的类型定义提示

三、优化方案

`vue3` 从很多层面都做了优化，可以分成三个方面：

- 源码
- 性能
- 语法 API

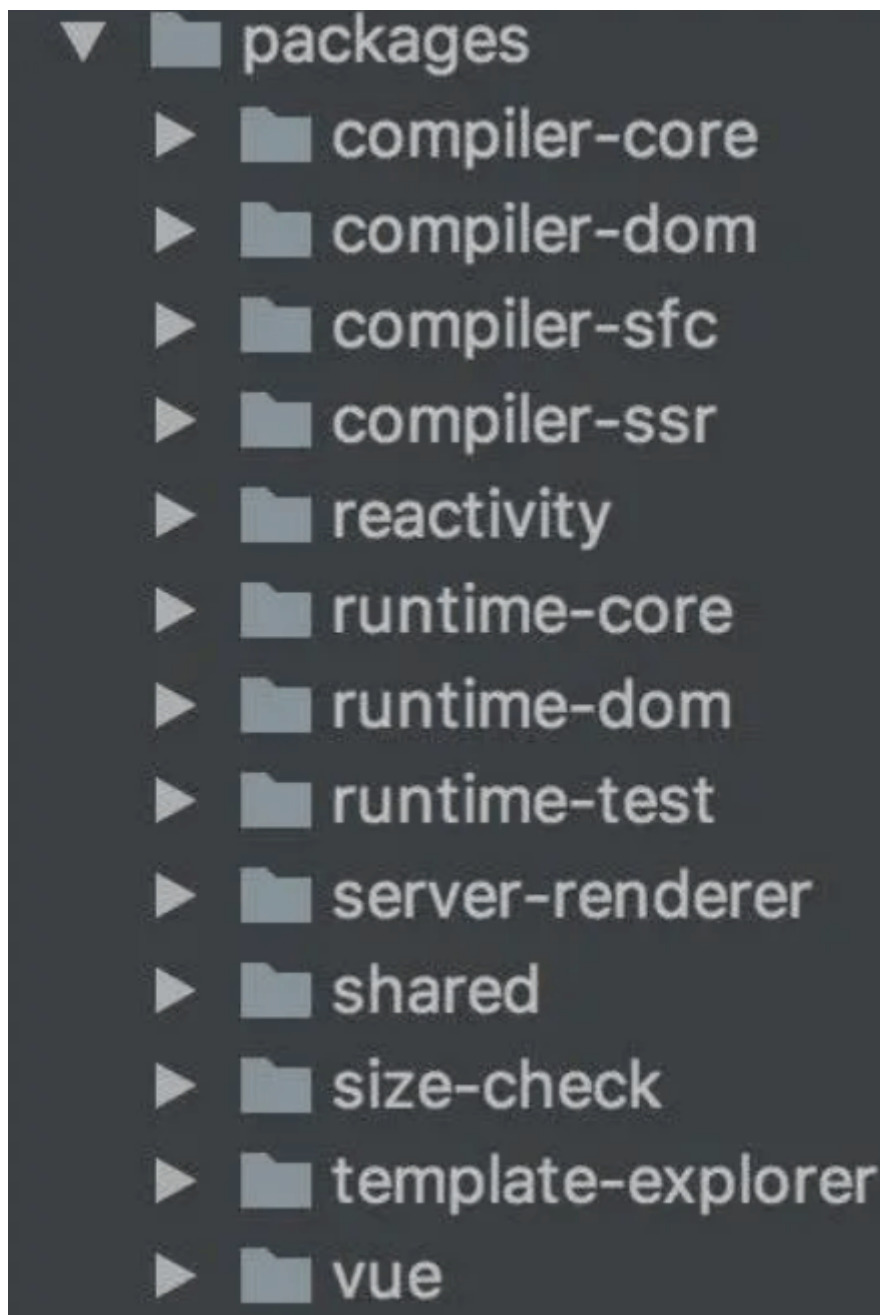
源码

源码可以从两个层面展开：

- 源码管理
- TypeScript

源码管理

`vue3` 整个源码是通过 `monorepo` 的方式维护的，根据功能将不同的模块拆分到 `packages` 目录下面不同的子目录中



这样使得模块拆分更细化，职责划分更明确，模块之间的依赖关系也更加明确，开发人员也更容易阅读、理解和更改所有模块源码，提高代码的可维护性

另外一些 `package`（比如 `reactivity` 响应式库）是可以独立于 `Vue` 使用的，这样用户如果只想使用 `Vue3` 的响应式能力，可以单独依赖这个响应式库而不用去依赖整个 `Vue`

TypeScript

`Vue3` 是基于 `typeScript` 编写的，提供了更好的类型检查，能支持复杂的类型推导

性能

`vue3` 是从哪些方面对性能进行进一步优化呢？

- 体积优化
- 编译优化

- 数据劫持优化

这里讲述数据劫持：

在 `vue2` 中，数据劫持是通过 `Object.defineProperty`，这个 API 有一些缺陷，并不能检测对象属性的添加和删除

```
1  Object.defineProperty(data, 'a',{
2    get(){
3      // track
4    },
5    set(){
6      // trigger
7    }
8  })
```

尽管 `Vue` 为了解决这个问题提供了 `set` 和 `delete` 实例方法，但是对于用户来说，还是增加了一定的心智负担

同时在面对嵌套层级比较深的情况下，就存在性能问题

```
1  default {
2    data: {
3      a: {
4        b: {
5          c: {
6            d: 1
7          }
8        }
9      }
10   }
11 }
```

相比之下，`vue3` 是通过 `proxy` 监听整个对象，那么对于删除还是监听当然也能监听到

同时 `Proxy` 并不能监听到内部深层次的对象变化，而 `Vue3` 的处理方式是在 `getter` 中去递归响应式，这样的好处是真正访问到的内部对象才会变成响应式，而不是无脑递归

语法 API

这里当然说的就是 `composition API`，其两大显著的优化：

- 优化逻辑组织
- 优化逻辑复用

逻辑组织

一张图，我们可以很直观地感受到 Composition API 在逻辑组织方面的优势

Options API



Composition API



相同功能的代码编写在一块，而不像 options API 那样，各个功能的代码混成一块

逻辑复用

在 vue2 中，我们是通过 mixin 实现功能混合，如果多个 mixin 混合，会存在两个非常明显的问题：命名冲突和数据来源不清晰

而通过 composition 这种形式，可以将一些复用的代码抽离出来作为一个函数，只要使用的地方直接进行调用即可

同样是上文的获取鼠标位置的例子

```

1  import { toRefs, reactive, onUnmounted, onMounted } from 'vue';
2  function useMouse(){
3      const state = reactive({x:0,y:0});
4      const update = e=>{
5          state.x = e.pageX;
6          state.y = e.pageY;
7      }
8      onMounted(()=>{
9          window.addEventListener('mousemove',update);
10     })
11     onUnmounted(()=>{
12         window.removeEventListener('mousemove',update);
13     })
14
15     return toRefs(state);
16 }

```

组件使用

```

1  import useMousePosition from './mouse'
2  export default {
3      setup() {
4          const { x, y } = useMousePosition()
5          return { x, y }
6      }
7  }

```

可以看到，整个数据来源清晰了，即使去编写更多的 `hook` 函数，也不会出现命名冲突的问题

第25题：你是怎么处理vue项目中的错误的？

一、错误类型

任何一个框架，对于错误的处理都是一种必备的能力

在 `Vue` 中，则是定义了一套对应的错误处理规则给到使用者，且在源代码级别，对部分必要的过程做了一定的错误处理。

主要的错误来源包括：

- 后端接口错误
- 代码中本身逻辑错误

二、如何处理

后端接口错误

通过 `axios` 的 `interceptor` 实现网络请求的 `response` 先进行一层拦截

```
1  apiClient.interceptors.response.use(  
2    response => {  
3      return response;  
4    },  
5    error => {  
6      if (error.response.status === 401) {  
7        router.push({ name: "Login" });  
8      } else {  
9        message.error("出错了");  
10       return Promise.reject(error);  
11     }  
12   }  
13 );
```

代码逻辑问题

全局设置错误处理

设置全局错误处理函数

```
1  Vue.config.errorHandler = function (err, vm, info) {  
2    // handle error  
3    // `info` 是 Vue 特定的错误信息，比如错误所在的生命周期钩子  
4    // 只在 2.2.0+ 可用  
5  }
```

`errorHandler` 指定组件的渲染和观察期间未捕获错误的处理函数。这个处理函数被调用时，可获取错误信息和 `Vue` 实例

不过值得注意的是，在不同 `Vue` 版本中，该全局 `API` 作用的范围会有所不同：

从 2.2.0 起，这个钩子也会捕获组件生命周期钩子里的错误。同样的，当这个钩子是 `undefined` 时，被捕获的错误会通过 `console.error` 输出而避免应用崩

从 2.4.0 起，这个钩子也会捕获 `Vue` 自定义事件处理函数内部的错误了

从 2.6.0 起，这个钩子也会捕获 `v-on` DOM 监听器内部抛出的错误。另外，如果任何被覆盖的钩子或处理函数返回一个 `Promise` 链 (例如 `async` 函数)，则来自其 `Promise` 链的错误也会被处理

生命周期钩子

`errorCaptured` 是 2.5.0 新增的一个生命钩子函数，当捕获到一个来自子孙组件的错误时被调用

基本类型

```
1 (err: Error, vm: Component, info: string) => ?boolean
```

此钩子会收到三个参数：错误对象、发生错误的组件实例以及一个包含错误来源信息的字符串。此钩子可以返回 `false` 以阻止该错误继续向上传播

参考官网，错误传播规则如下：

- 默认情况下，如果全局的 `config.errorHandler` 被定义，所有的错误仍会发送它，因此这些错误仍然会向单一的分析服务的地方进行汇报
- 如果一个组件的继承或父级从属链路中存在多个 `errorCaptured` 钩子，则它们将会被相同的错误逐个唤起。
- 如果此 `errorCaptured` 钩子自身抛出了一个错误，则这个新错误和原本被捕获的错误都会发送给全局的 `config.errorHandler`
- 一个 `errorCaptured` 钩子能够返回 `false` 以阻止错误继续向上传播。本质上是说“这个错误已经被搞定了且应该被忽略”。它会阻止其它任何会被这个错误唤起的 `errorCaptured` 钩子和全局的 `config.errorHandler`

下面来看个例子

定义一个父组件 `cat`

```
1 Vue.component('cat', {
2   template:`
3     <div>
4       <h1>Cat: </h1>
5       <slot></slot>
6     </div>`,
7   props:{
8     name:{
9       required:true,
10      type:String
11    }
12  },
13  errorCaptured(err,vm,info) {
14    console.log(`cat EC: ${err.toString()}\ninfo: ${info}`);
15    return false;
16  }
17 }
```

```
18   });
```

定义一个子组件 `kitten`，其中 `dontexist()` 并没有定义，存在错误

```
1  Vue.component('kitten', {
2    template: '<div><h1>Kitten: {{ dontexist() }}</h1></div>',
3    props: {
4      name: {
5        required: true,
6        type: String
7      }
8    }
9  });
```

页面中使用组件

```
1  <div id="app" v-cloak>
2    <cat name="my cat">
3      <kitten></kitten>
4    </cat>
5  </div>
```

在父组件的 `errorCaptured` 则能够捕获到信息

```
1  cat EC: TypeError: dontexist is not a function
2  info: render
```

三、源码分析

异常处理源码

源码位置： `/src/core/util/error.js`

```
1  // Vue 全局配置,也就是上面的Vue.config
2  import config from '../config'
3  import { warn } from './debug'
4  // 判断环境
5  import { inBrowser, inWeex } from './env'
6  // 判断是否是Promise,通过val.then === 'function' && val.catch === 'function',
   val !== null && val !== undefined
```

```

7   import { isPromise } from 'shared/util'
8   // 当错误函数处理错误时，停用deps跟踪以避免可能出现的infinite rendering
9   // 解决以下出现的问题https://github.com/vuejs/vuex/issues/1505的问题
10  import { pushTarget, popTarget } from '../observer/dep'
11
12  export function handleError (err: Error, vm: any, info: string) {
13    // Deactivate deps tracking while processing error handler to avoid
14    // possible infinite rendering.
15    pushTarget()
16    try {
17      // vm指当前报错的组件实例
18      if (vm) {
19        let cur = vm
20        // 首先获取到报错的组件，之后递归查找当前组件的父组件，依次调用
21        // errorCaptured 方法。
22        // 在遍历调用完所有 errorCaptured 方法、或 errorCaptured 方法有报错时，
23        // 调用 globalHandleError 方法
24        while ((cur = cur.$parent)) {
25          const hooks = cur.$options.errorCaptured
26          // 判断是否存在errorCaptured钩子函数
27          if (hooks) {
28            // 选项合并的策略，钩子函数会被保存在一个数组中
29            for (let i = 0; i < hooks.length; i++) {
30              // 如果errorCaptured 钩子执行自身抛出了错误，
31              // 则用try{}catch{}捕获错误，将这个新错误和原本被捕获的错误
32              // 都会发送给全局的config.errorHandler
33              // 调用globalHandleError方法
34              try {
35                // 当前errorCaptured执行，根据返回是否是false值
36                // 是false, capture = true, 阻止其它任何会被这个错误唤
37                // 起的 errorCaptured 钩子和全局的 config.errorHandler
38                // 是true capture = false, 组件的继承或父级从属链路中存
39                // 在的多个 errorCaptured 钩子，会被相同的错误逐个唤起
40                // 调用对应的钩子函数，处理错误
41                const capture = hooks[i].call(cur, err, vm, info)
42                === false
43                if (capture) return
44              } catch (e) {
45                globalHandleError(e, cur, 'errorCaptured hook')
46              }
47            }
48          }
49        }
50        // 除非禁止错误向上传播，否则都会调用全局的错误处理函数
51        globalHandleError(err, vm, info)
52      } finally {

```

```

47         popTarget()
48     }
49 }
50 // 异步错误处理函数
51 export function invokeWithErrorHandling (
52   handler: Function,
53   context: any,
54   args: null | any[],
55   vm: any,
56   info: string
57 ) {
58     let res
59     try {
60         // 根据参数选择不同的handle执行方式
61         res = args ? handler.apply(context, args) :
62         handler.call(context)
63         // handle返回结果存在
64         // res._isVue an flag to avoid this being observed, 如果传入值的
65         // _isVue为ture时(即传入的值是Vue实例本身)不会新建observer实例
66         // isPromise(res) 判断val.then === 'function' && val.catch
67         === 'function', val !== null && val !== undefined
68         // !res._handled _handle是Promise 实例的内部变量之一, 默认是
69         false, 代表onFulfilled,onRejected是否被处理
70         if (res && !res._isVue && isPromise(res) && !res._handled) {
71             res.catch(e => handleError(e, vm, info + `
72             (Promise/async)`))
73             // avoid catch triggering multiple times when nested calls
74             // 避免嵌套调用时catch多次的触发
75             res._handled = true
76         }
77     } catch (e) {
78         // 处理执行错误
79         handleError(e, vm, info)
80     }
81     return res
82 }
83 //全局错误处理
84 function globalHandleError (err, vm, info) {
85     // 获取全局配置, 判断是否设置处理函数, 默认undefined
86     // 已配置
87     if (config.errorHandler) {
88         // try{}catch{} 住全局错误处理函数
89         try {
90             // 执行设置的全局错误处理函数, handle error 想干啥就干啥💖
91             return config.errorHandler.call(null, err, vm, info)
92         } catch (e) {

```

```

89         // 如果开发者在 errorHandler 函数中手动抛出同样错误信息 throw err
90         // 判断 err 信息是否相等，避免 log 两次
91         // 如果抛出新的错误信息 throw err Error('你好毒')，将会一起 log 输出
92         if (e !== err) {
93             logError(e, null, 'config.errorHandler')
94         }
95     }
96 }
97 // 未配置常规 log 输出
98 logError(err, vm, info)
99 }
100
101 // 错误输出函数
102 function logError (err, vm, info) {
103     if (process.env.NODE_ENV !== 'production') {
104         warn(`Error in ${info}: "${err.toString()}"`, vm)
105     }
106     /* istanbul ignore else */
107     if ((inBrowser || inWeex) && typeof console !== 'undefined') {
108         console.error(err)
109     } else {
110         throw err
111     }
112 }

```

小结

- `handleError` 在需要捕获异常的地方调用，首先获取到报错的组件，之后递归查找当前组件的父组件，依次调用 `errorCaptured` 方法，在遍历调用完所有 `errorCaptured` 方法或 `errorCaptured` 方法有报错时，调用 `globalHandleError` 方法
- `globalHandleError` 调用全局的 `errorHandler` 方法，再通过 `logError` 判断环境输出错误信息
- `invokeWithErrorHandling` 更好的处理异步错误信息
- `logError` 判断环境，选择不同的抛错方式。非生产环境下，调用 `warn` 方法处理错误

第26题：Vue项目如何进行部署？是否有遇到部署服务器后刷新404问题？

一、如何部署

前后端分离开发模式下，前后端是独立部署的，前端只需要将最后的构建物上传至目标服务器的 `web` 容器指定的静态目录下即可

我们知道 `vue` 项目在构建后，是生成一系列的静态文件

常规布署我们只需要将这个目录上传至目标服务器即可

```
1 // scp 上传 user为主机登录用户, host为主机外网ip, xx为web容器静态资源路径
2 scp dist.zip user@host:/xx/xx/xx
```

让 `web` 容器跑起来，以 `nginx` 为例

```
1 server {
2     listen 80;
3     server_name www.xxx.com;
4
5     location / {
6         index /data/dist/index.html;
7     }
8 }
```

配置完成记得重启 `nginx`

```
1 // 检查配置是否正确
2 nginx -t
3
4 // 平滑重启
5 nginx -s reload
```

操作完后就可以在浏览器输入域名进行访问了

当然上面只是提到最简单也是最直接的一种布署方式

什么自动化，镜像，容器，流水线布署，本质也是将这套逻辑抽象，隔离，用程序来代替重复性的劳动，本文不展开

二、404问题

这是一个经典的问题，相信很多同学都有遇到过，那么你知道其真正的原因吗？

我们先还原一下场景：

- `vue` 项目在本地时运行正常，但部署到服务器中，刷新页面，出现了404错误

先定位一下，HTTP 404 错误意味着链接指向的资源不存在

问题在于为什么不存在？且为什么只有 `history` 模式下会出现这个问题？

为什么history模式下有问题

Vue 是属于单页应用（single-page application）

而 SPA 是一种网络应用程序或网站的模型，所有用户交互是通过动态重写当前页面，前面我们也看到了，不管我们应用有多少页面，构建物都只会产出一个 `index.html`

现在，我们回头来看一下我们的 `nginx` 配置

```
1  server {
2      listen 80;
3      server_name www.xxx.com;
4
5      location / {
6          index /data/dist/index.html;
7      }
8  }
```

可以根据 `nginx` 配置得出，当我们在地址栏输入 `www.xxx.com` 时，这时会打开我们 `dist` 目录下的 `index.html` 文件，然后我们在跳转路由进入到 `www.xxx.com/login`

关键在这里，当我们在 `website.com/login` 页执行刷新操作，`nginx location` 是没有相关配置的，所以就会出现 404 的情况

为什么hash模式下没有问题

`router hash` 模式我们都知道是用符号#表示的，如 `website.com/#/login`，`hash` 的值为 `#/login`

它的特点在于：`hash` 虽然出现在 `URL` 中，但不会被包括在 `HTTP` 请求中，对服务端完全没有影响，因此改变 `hash` 不会重新加载页面

`hash` 模式下，仅 `hash` 符号之前的内容会被包含在请求中，如 `website.com/#/login` 只有 `website.com` 会被包含在请求中，因此对于服务端来说，即使没有配置 `location`，也不会返回 404 错误

解决方案

看到这里我相信大部分同学都能想到怎么解决问题了，

产生问题的本质是因为我们的路由是通过JS来执行视图切换的，

当我们进入到子路由时刷新页面，`web` 容器没有相对应的页面此时会出现 404

所以我们只需要配置将任意页面都重定向到 `index.html`，把路由交由前端处理

对 `nginx` 配置文件 `.conf` 修改，添加 `try_files $uri $uri/ /index.html;`

```
1  server {
2      listen 80;
3      server_name www.xxx.com;
4
5      location / {
6          index /data/dist/index.html;
7          try_files $uri $uri/ /index.html;
8      }
9  }
```

修改完配置文件后记得配置的更新

```
1  nginx -s reload
```

这么做以后，你的服务器就不再返回 404 错误页面，因为对于所有路径都会返回 `index.html` 文件
为了避免这种情况，你应该在 `Vue` 应用里面覆盖所有的路由情况，然后在给出一个 404 页面

```
1  const router = new VueRouter({
2      mode: 'history',
3      routes: [
4          { path: '*', component: NotFoundComponent }
5      ]
6  })
```

关于后端配置方案还有：`Apache`、`nodejs` 等，思想是一致的，这里就不展开述说了

第27题：Vue项目中如何解决跨域问题？

解决跨域的方法有很多，下面列举了三种：

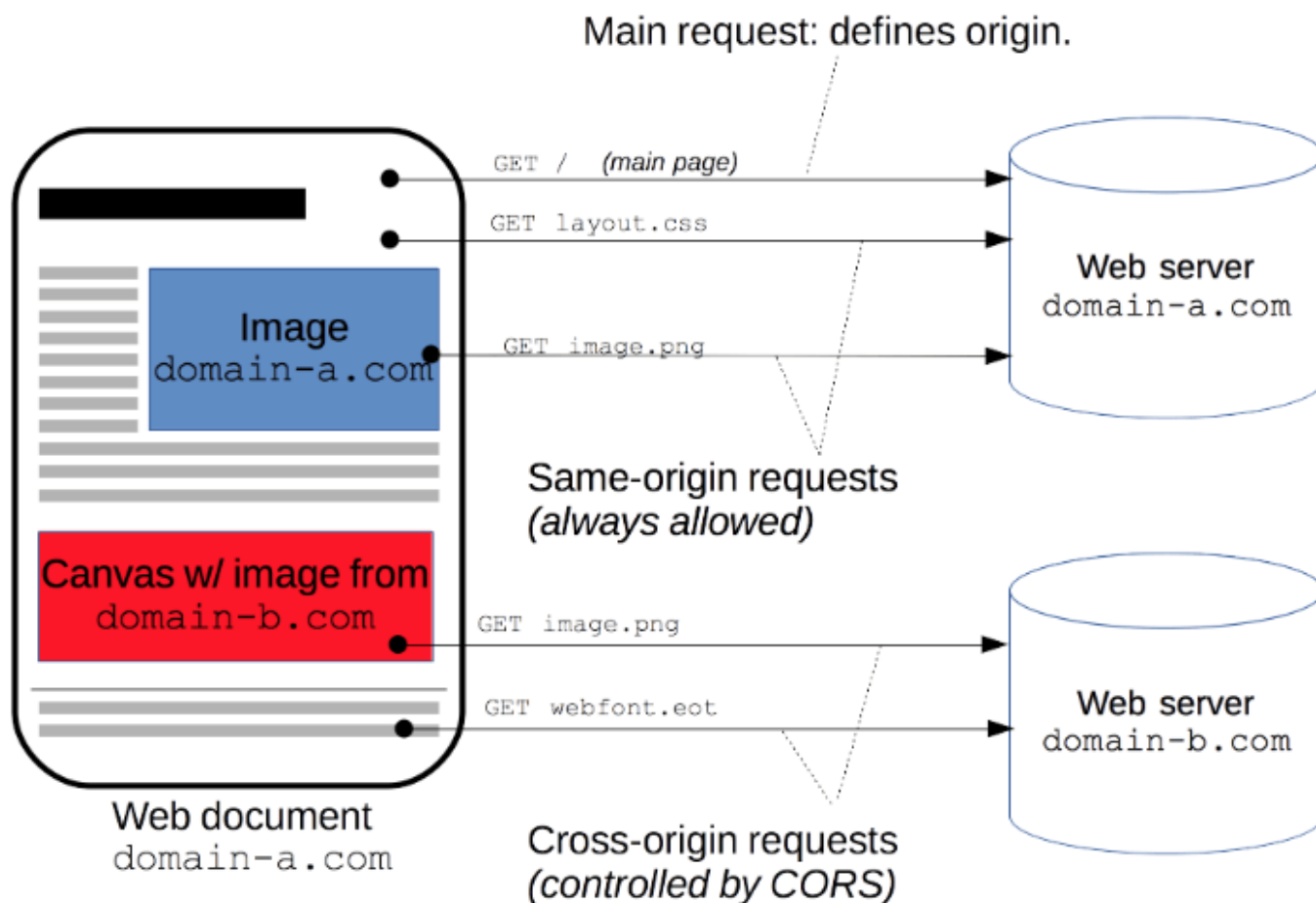
- JSONP
- CORS
- Proxy

而在 `vue` 项目中，我们主要针对 `CORS` 或 `Proxy` 这两种方案进行展开

CORS

CORS（Cross-Origin Resource Sharing，跨域资源共享）是一个系统，它由一系列传输的HTTP头组成，这些HTTP头决定浏览器是否阻止前端 JavaScript 代码获取跨域请求的响应

CORS 实现起来非常方便，只需要增加一些 HTTP 头，让服务器能声明允许的访问来源
只要后端实现了 CORS，就实现了跨域



以 koa 框架举例

添加中间件，直接设置 `Access-Control-Allow-Origin` 请求头

```
1 app.use(async (ctx, next)=> {
2   ctx.set('Access-Control-Allow-Origin', '*');
3   ctx.set('Access-Control-Allow-Headers', 'Content-Type, Content-Length,
  Authorization, Accept, X-Requested-With , yourHeaderFeild');
4   ctx.set('Access-Control-Allow-Methods', 'PUT, POST, GET, DELETE, OPTIONS');
5   if (ctx.method == 'OPTIONS') {
6     ctx.body = 200;
7   } else {
8     await next();
9   }
10 })
```

ps: `Access-Control-Allow-Origin` 设置为*其实意义不大，可以说是形同虚设，实际应用中，上线前我们会将 `Access-Control-Allow-Origin` 值设为我们目标 host

Proxy

代理（Proxy）也称网络代理，是一种特殊的网络服务，允许一个（一般为客户端）通过这个服务与另一个网络终端（一般为服务器）进行非直接连接。一些网关、路由器等网络设备具备网络代理功能。一般认为代理服务有利于保障网络终端的隐私或安全，防止攻击

方案一

如果是通过 `vue-cli` 脚手架工具搭建项目，我们可以通过 `webpack` 为我们起一个本地服务器作为请求的代理对象

通过该服务器转发请求至目标服务器，得到结果再转发给前端，但是最终发布上线时如果web应用和接口服务器不在一起仍会跨域

在 `vue.config.js` 文件，新增以下代码

```
1  module.exports = {
2    devServer: {
3      host: '127.0.0.1',
4      port: 8084,
5      open: true, // vue项目启动时自动打开浏览器
6      proxy: {
7        '/api': { // '/api'是代理标识，用于告诉node，url前面是/api的就是使用代
理的
8          target: "http://xxx.xxx.xx.xx:8080", //目标地址，一般是指后台服务
器地址
9          changeOrigin: true, //是否跨域
10         pathRewrite: { // pathRewrite 的作用是把实际Request Url中
的'/api'用""代替
11           '^/api': ""
12         }
13       }
14     }
15   }
16 }
```

通过 `axios` 发送请求中，配置请求的根路径

```
1  axios.defaults.baseURL = '/api'
```

方案二

此外，还可通过服务端实现代理请求转发

以 `express` 框架为例

```
1 var express = require('express');
2 const proxy = require('http-proxy-middleware')
3 const app = express()
4 app.use(express.static(__dirname + '/'))
5 app.use('/api', proxy({ target: 'http://localhost:4000', changeOrigin: false
6                          }));
7 module.exports = app
```

方案三

通过配置 `nginx` 实现代理

```
1 server {
2     listen    80;
3     # server_name www.josephxia.com;
4     location / {
5         root   /var/www/html;
6         index  index.html index.htm;
7         try_files $uri $uri/ /index.html;
8     }
9     location /api {
10        proxy_pass http://127.0.0.1:3000;
11        proxy_redirect off;
12        proxy_set_header Host $host;
13        proxy_set_header X-Real-IP $remote_addr;
14        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
15    }
16 }
```

第28题：Vue怎么实现权限管理？控制到按钮级别的权限怎么做？

一、是什么

权限是对特定资源的访问许可，所谓权限控制，也就是确保用户只能访问到被分配的资源

而前端权限归根结底是请求的发起权，请求的发起可能有下面两种形式触发

- 页面加载触发
- 页面上的按钮点击触发

总的来说，所有的请求发起都触发自前端路由或视图

所以我们可以从这两方面入手，对触发权限的源头进行控制，最终要实现的目标是：

- 路由方面，用户登录后只能看到自己有权访问的导航菜单，也只能访问自己有权访问的路由地址，否则将跳转 `4xx` 提示页
- 视图方面，用户只能看到自己有权浏览的内容和有权操作的控件
- 最后再加上请求控制作为最后一道防线，路由可能配置失误，按钮可能忘了加权限，这种时候请求控制可以用来兜底，越权请求将在前端被拦截

二、如何做

前端权限控制可以分为四个方面：

- 接口权限
- 按钮权限
- 菜单权限
- 路由权限

接口权限

接口权限目前一般采用 `jwt` 的形式来验证，没有通过的话一般返回 `401`，跳转到登录页面重新进行登录

登录完拿到 `token`，将 `token` 存起来，通过 `axios` 请求拦截器进行拦截，每次请求的时候头部携带 `token`

```
1  axios.interceptors.request.use(config => {
2      config.headers['token'] = cookie.get('token')
3      return config
4  })
5  axios.interceptors.response.use(res=>{},{response}>={
6      if (response.data.code === 40099 || response.data.code === 40098) {
7          //token过期或者错误
8          router.push('/login')
9      }
10 })
```

路由权限控制

方案一

初始化即挂载全部路由，并且在路由上标记相应的权限信息，每次路由跳转前做校验

```
1  const routerMap = [
```

```

2    {
3      path: '/permission',
4      component: Layout,
5      redirect: '/permission/index',
6      alwaysShow: true, // will always show the root menu
7      meta: {
8        title: 'permission',
9        icon: 'lock',
10       roles: ['admin', 'editor'] // you can set roles in root nav
11     },
12     children: [{
13       path: 'page',
14       component: () => import('@views/permission/page'),
15       name: 'pagePermission',
16       meta: {
17         title: 'pagePermission',
18         roles: ['admin'] // or you can only set roles in sub nav
19       }
20     }, {
21       path: 'directive',
22       component: () => import('@views/permission/directive'),
23       name: 'directivePermission',
24       meta: {
25         title: 'directivePermission'
26         // if do not set roles, means: this page does not require permission
27       }
28     }]
29   }]

```

这种方式存在以下四种缺点：

- 加载所有的路由，如果路由很多，而用户并不是所有的路由都有权限访问，对性能会有影响。
- 全局路由守卫里，每次路由跳转都要做权限判断。
- 菜单信息写死在前端，要改个显示文字或权限信息，需要重新编译
- 菜单跟路由耦合在一起，定义路由的时候还有添加菜单显示标题，图标之类的信息，而且路由不一定作为菜单显示，还要多加字段进行标识

方案二

初始化的时候先挂载不需要权限控制的路由，比如登录页，404等错误页。如果用户通过URL进行强制访问，则会直接进入404，相当于从源头上做了控制

登录后，获取用户的权限信息，然后筛选有权限访问的路由，在全局路由守卫里进行调用

`addRoutes` 添加路由

```

1  import router from './router'
2  import store from './store'
3  import { Message } from 'element-ui'
4  import NProgress from 'nprogress' // progress bar
5  import 'nprogress/nprogress.css' // progress bar style
6  import { getToken } from '@utils/auth' // getToken from cookie
7
8  NProgress.configure({ showSpinner: false }) // NProgress Configuration
9
10 // permission judge function
11 function hasPermission(roles, permissionRoles) {
12   if (roles.indexOf('admin') >= 0) return true // admin permission passed
    directly
13   if (!permissionRoles) return true
14   return roles.some(role => permissionRoles.indexOf(role) >= 0)
15 }
16
17 const whiteList = ['/login', '/authredirect'] // no redirect whitelist
18
19 router.beforeEach((to, from, next) => {
20   NProgress.start() // start progress bar
21   if (getToken()) { // determine if there has token
22     /* has token*/
23     if (to.path === '/login') {
24       next({ path: '/' })
25       NProgress.done() // if current page is dashboard will not trigger
        afterEach hook, so manually handle it
26     } else {
27       if (store.getters.roles.length === 0) { // 判断当前用户是否已拉取完
        user_info信息
28         store.dispatch('GetUserInfo').then(res => { // 拉取user_info
29           const roles = res.data.roles // note: roles must be a array! such
            as: ['editor', 'develop']
30           store.dispatch('GenerateRoutes', { roles }).then(() => { // 根据
            roles权限生成可访问的路由表
31             router.addRoutes(store.getters.addRouters) // 动态添加可访问路由表
32             next({ ...to, replace: true }) // hack方法 确保addRoutes已完成 ,set
              the replace: true so the navigation will not leave a history record
33           })
34         }).catch((err) => {
35           store.dispatch('FedLogOut').then(() => {
36             Message.error(err || 'Verification failed, please login again')
37             next({ path: '/' })
38           })
39         })
40       } else {
41         // 没有动态改变权限的需求可直接next() 删除下方权限判断 ↓

```

```

42         if (hasPermission(store.getters.roles, to.meta.roles)) {
43             next()//
44         } else {
45             next({ path: '/401', replace: true, query: { noGoBack: true }})
46         }
47         // 可删 ↑
48     }
49 }
50 } else {
51     /* has no token*/
52     if (whiteList.indexOf(to.path) !== -1) { // 在免登录白名单, 直接进入
53         next()
54     } else {
55         next('/login') // 否则全部重定向到登录页
56         NProgress.done() // if current page is login will not trigger afterEach
57         hook, so manually handle it
58     }
59 })
60
61 router.afterEach(() => {
62     NProgress.done() // finish progress bar
63 })

```

按需挂载，路由就需要知道用户的路由权限，也就是在用户登录进来的时候就要知道当前用户拥有哪些路由权限

这种方式也存在了以下的缺点：

- 全局路由守卫里，每次路由跳转都要做判断
- 菜单信息写死在前端，要改个显示文字或权限信息，需要重新编译
- 菜单跟路由耦合在一起，定义路由的时候还有添加菜单显示标题，图标之类的信息，而且路由不一定作为菜单显示，还要多加字段进行标识

菜单权限

菜单权限可以理解成将页面与理由进行解耦

方案一

菜单与路由分离，菜单由后端返回

前端定义路由信息

```

1  {
2      name: "login",

```

```
3     path: "/login",
4     component: () => import("@/pages/Login.vue")
5 }
```

`name` 字段都不为空，需要根据此字段与后端返回菜单做关联，后端返回的菜单信息中必须要有 `name` 对应的字段，并且做唯一性校验

全局路由守卫里做判断

```
1  function hasPermission(router, accessMenu) {
2    if (whiteList.indexOf(router.path) !== -1) {
3      return true;
4    }
5    let menu = Util.getMenuByName(router.name, accessMenu);
6    if (menu.name) {
7      return true;
8    }
9    return false;
10 }
11
12
13 Router.beforeEach(async (to, from, next) => {
14   if (getToken()) {
15     let userInfo = store.state.user.userInfo;
16     if (!userInfo.name) {
17       try {
18         await store.dispatch("GetUserInfo")
19         await store.dispatch('updateAccessMenu')
20         if (to.path === '/login') {
21           next({ name: 'home_index' })
22         } else {
23           //Util.toDefaultPage([...routers], to.name, router, next);
24           next({ ...to, replace: true })//菜单权限更新完成,重新进一次当前路由
25         }
26       }
27       catch (e) {
28         if (whiteList.indexOf(to.path) !== -1) { // 在免登录白名单,直接进入
29           next()
30         } else {
31           next('/login')
32         }
33       }
34     } else {
35       if (to.path === '/login') {
36         next({ name: 'home_index' })
```

```

37     } else {
38         if (hasPermission(to, store.getters.accessMenu)) {
39             Util.toDefaultPage(store.getters.accessMenu,to, routes, next);
40         } else {
41             next({ path: '/403',replace:true })
42         }
43     }
44 }
45 } else {
46     if (whiteList.indexOf(to.path) !== -1) { // 在免登录白名单, 直接进入
47         next()
48     } else {
49         next('/login')
50     }
51 }
52 let menu = Util.getMenuByName(to.name, store.getters.accessMenu);
53 Util.title(menu.title);
54 });
55
56 Router.afterEach((to) => {
57     window.scrollTo(0, 0);
58 });

```

每次路由跳转的时候都要判断权限，这里的判断也很简单，因为菜单的 `name` 与路由的 `name` 是一一对应的，而后端返回的菜单就已经是经过权限过滤的

如果根据路由 `name` 找不到对应的菜单，就表示用户有没有权限访问

如果路由很多，可以在应用初始化的时候，只挂载不需要权限控制的路由。取得后端返回的菜单后，根据菜单与路由的对应关系，筛选出可访问的路由，通过 `addRoutes` 动态挂载

这种方式的缺点：

- 菜单需要与路由做一一对应，前端添加了新功能，需要通过菜单管理功能添加新的菜单，如果菜单配置的不对会导致应用不能正常使用
- 全局路由守卫里，每次路由跳转都要做判断

方案二

菜单和路由都由后端返回

前端统一定义路由组件

```

1  const Home = () => import("../pages/Home.vue");
2  const UserInfo = () => import("../pages/UserInfo.vue");
3  export default {
4      home: Home,

```

```
5     userInfo: UserInfo
6   };
```

后端路由组件返回以下格式

```
1  [
2    {
3      name: "home",
4      path: "/",
5      component: "home"
6    },
7    {
8      name: "home",
9      path: "/userinfo",
10     component: "userInfo"
11   }
12 ]
```

在将后端返回路由通过 `addRoutes` 动态挂载之间，需要将数据处理一下，将 `component` 字段换为真正的组件

如果有嵌套路由，后端功能设计的时候，要注意添加相应的字段，前端拿到数据也要做相应的处理

这种方法也会存在缺点：

- 全局路由守卫里，每次路由跳转都要做判断
- 前后端的配合要求更高

按钮权限

方案一

按钮权限也可以用 `v-if` 判断

但是如果页面过多，每个页面都要获取用户权限 `role` 和路由表里的 `meta.btnPermissions`，然后再做判断

这种方式就不展开举例了

方案二

通过自定义指令进行按钮权限的判断

首先配置路由

```
1  {
```

```

2     path: '/permission',
3     component: Layout,
4     name: '权限测试',
5     meta: {
6         btnPermissions: ['admin', 'supper', 'normal']
7     },
8     //页面需要的权限
9     children: [{
10         path: 'supper',
11         component: _import('system/supper'),
12         name: '权限测试页',
13         meta: {
14             btnPermissions: ['admin', 'supper']
15         } //页面需要的权限
16     },
17     {
18         path: 'normal',
19         component: _import('system/normal'),
20         name: '权限测试页',
21         meta: {
22             btnPermissions: ['admin']
23         } //页面需要的权限
24     }]
25 }

```

自定义权限鉴定指令

```

1  import Vue from 'vue'
2  /**权限指令**/
3  const has = Vue.directive('has', {
4      bind: function (el, binding, vnode) {
5          // 获取页面按钮权限
6          let btnPermissionsArr = [];
7          if(binding.value){
8              // 如果指令传值，获取指令参数，根据指令参数和当前登录人按钮权限做比较。
9              btnPermissionsArr = Array.of(binding.value);
10         }else{
11             // 否则获取路由中的参数，根据路由的btnPermissionsArr和当前登录人按钮权限
12             做比较。
13             btnPermissionsArr = vnode.context.$route.meta.btnPermissions;
14         }
15         if (!Vue.prototype._has(btnPermissionsArr)) {
16             el.parentNode.removeChild(el);
17         }
18     }
19 }

```

```

18  });
19  // 权限检查方法
20  Vue.prototype.$_has = function (value) {
21      let isExist = false;
22      // 获取用户按钮权限
23      let btnPermissionsStr = sessionStorage.getItem("btnPermissions");
24      if (btnPermissionsStr == undefined || btnPermissionsStr == null) {
25          return false;
26      }
27      if (value.indexOf(btnPermissionsStr) > -1) {
28          isExist = true;
29      }
30      return isExist;
31  };
32  export {has}

```

在使用的按钮中只需要引用 `v-has` 指令

```

1  <el-button @click='editClick' type="primary" v-has>编辑</el-button>

```

小结

关于权限如何选择哪种合适的方案，可以根据自己项目的方案项目，如考虑路由与菜单是否分离
权限需要前后端结合，前端尽可能的去控制，更多的需要后台判断

第29题：大型项目中，Vue项目怎么划分结构和划分组件比较合理呢？

一、为什么要划分

使用 `vue` 构建项目，项目结构清晰会提高开发效率，熟悉项目的各种配置同样会让开发效率更高
在划分项目结构的时候，需要遵循一些基本的原则：

- 文件夹和文件夹内部文件的语义一致性
- 单一入口/出口
- 就近原则，紧耦合的文件应该放到一起，且应以相对路径引用
- 公共的文件应该以绝对路径的方式从根目录引用
- `/src` 外的文件不应该被引入

文件夹和文件夹内部文件的语义一致性

我们的目录结构都会有一个文件夹是按照路由模块来划分的，如 `pages` 文件夹，这个文件夹里面应该包含我们项目所有的路由模块，并且仅应该包含路由模块，而不应该有别的其他的非路由模块的文件夹

这样做的好处在于一眼就从 `pages` 文件夹看出这个项目的路由有哪些

单一入口/出口

举个例子，在 `pages` 文件夹里面存在一个 `seller` 文件夹，这时候 `seller` 文件夹应该作为一个独立的模块由外部引入，并且 `seller/index.js` 应该作为外部引入 `seller` 模块的唯一入口

```
1 // 错误用法
2 import sellerReducer from 'src/pages/seller/reducer'
3
4 // 正确用法
5 import { reducer as sellerReducer } from 'src/pages/seller'
```

这样做的好处在于，无论你的模块文件夹内部有多乱，外部引用的时候，都是从一个入口文件引入，这样就很好的实现了隔离，如果后续有重构需求，你就会发现这种方式的优点

就近原则，紧耦合的文件应该放到一起，且应以相对路径引用

使用相对路径可以保证模块内部的独立性

```
1 // 正确用法
2 import styles from './index.module.scss'
3 // 错误用法
4 import styles from 'src/pages/seller/index.module.scss'
```

举个例子

假设我们现在的 `seller` 目录是在 `src/pages/seller`，如果我们后续发生了路由变更，需要加一个层级，变成 `src/pages/user/seller`。

如果我们采用第一种相对路径的方式，那就可以直接将整个文件夹拖过去就好，`seller` 文件夹内部不需要做任何变更。

但是如果我们采用第二种绝对路径的方式，移动文件夹的同时，还需要对每个 `import` 的路径做修改

公共的文件应该以绝对路径的方式从根目录引用

公共指的是多个路由模块共用，如一些公共的组件，我们可以放在 `src/components` 下
在使用到的页面中，采用绝对路径的形式引用

```
1 // 错误用法
2 import Input from '../../components/input'
3 // 正确用法
4 import Input from 'src/components/input'
```

同样的，如果我们需要对文件夹结构进行调整。将 `/src/components/input` 变成
`/src/components/new/input`，如果使用绝对路径，只需要全局搜索替换
再加上绝对路径有全局的语义，相对路径有独立模块的语义

/src 外的文件不应该被引入

`vue-cli` 脚手架已经帮我们做了相关的约束了，正常我们的前端项目都会有个 `src` 文件夹，里面放着所有的项目需要的资源，`js`，`css`，`png`，`svg` 等等。`src` 外会放一些项目配置，依赖，环境等文件

这样的好处是方便划分项目代码文件和配置文件

二、目录结构

单页面目录结构

```
1 project
2 |   .browserslistrc
3 |   .env.production
4 |   .eslintrc.js
5 |   .gitignore
6 |   babel.config.js
7 |   package-lock.json
8 |   package.json
9 |   README.md
10 |   vue.config.js
11 |   yarn-error.log
12 |   yarn.lock
13 |
14 |-- public
15 |     favicon.ico
16 |     index.html
17 |
18 |-- src
19     |-- components
```

```

20         |-- input
21         |-- index.js
22         |-- index.module.scss
23     |-- pages
24         |-- seller
25             |-- components
26                 |-- input
27                     |-- index.js
28                     |-- index.module.scss
29             |-- reducer.js
30             |-- saga.js
31             |-- index.js
32             |-- index.module.scss
33     |-- buyer
34         |-- index.js
35     |-- index.js

```

多页面目录结构

```

1  my-vue-test:
2  |   .browserslistrc
3  |   .env.production
4  |   .eslintrc.js
5  |   .gitignore
6  |   babel.config.js
7  |   package-lock.json
8  |   package.json
9  |   README.md
10 |   vue.config.js
11 |   yarn-error.log
12 |   yarn.lock
13 |
14 |─public
15 |   favicon.ico
16 |   index.html
17 |
18 |─src
19 |   |─apis //接口文件根据页面或实例模块化
20 |       index.js
21 |       login.js
22 |
23 |   |─components //全局公共组件
24 |       |─header
25 |           index.less
26 |           index.vue

```

```
27 |
28 |├─config //配置 (环境变量配置不同passid等)
29 |   env.js
30 |   index.js
31 |
32 |├─contant //常量
33 |   index.js
34 |
35 |├─images //图片
36 |   logo.png
37 |
38 |├─pages //多页面vue项目，不同的实例
39 |   │├─index //主实例
40 |   │   index.js
41 |   │   index.vue
42 |   │   main.js
43 |   │   router.js
44 |   │   store.js
45 |   │
46 |   │├─components //业务组件
47 |   │├─pages //此实例中的各个路由
48 |   │   │├─amenu
49 |   │   │   index.vue
50 |   │   │
51 |   │   │├─bmenu
52 |   │   │   index.vue
53 |   │
54 |   │├─login //另一个实例
55 |   │   index.js
56 |   │   index.vue
57 |   │   main.js
58 |
59 |├─scripts //包含各种常用配置，工具函数
60 |   │ map.js
61 |   │
62 |   │├─utils
63 |   │   helper.js
64 |
65 |├─store //vuex仓库
66 |   │ index.js
67 |   │
68 |   │├─index
69 |   │   actions.js
70 |   │   getters.js
71 |   │   index.js
72 |   │   mutation-types.js
73 |   │   mutations.js
```

```

74      | |      state.js
75      | |
76      | └─user
77      |      actions.js
78      |      getters.js
79      |      index.js
80      |      mutation-types.js
81      |      mutations.js
82      |      state.js
83      |
84      └─styles //样式统一配置
85          | components.less
86          |
87          └─animation
88              | index.less
89              | slide.less
90              |
91              └─base
92                  | index.less
93                  | style.less
94                  | var.less
95                  | widget.less
96                  |
97                  └─common
98                      | index.less
99                      | reset.less
100                     | style.less
101                     | transition.less

```

小结

项目的目录结构很重要，因为目录结构能体现很多东西，怎么规划目录结构可能每个人有自己的理解，但是按照一定的规范去进行目录的设计，能让项目整个架构看起来更为简洁，更加易用

第30题：Vue项目中有封装过axios吗？怎么封装的？

一、axios是什么

axios 是一个轻量的 HTTP 客户端

基于 XMLHttpRequest 服务来执行 HTTP 请求，支持丰富的配置，支持 Promise，支持浏览器端和 Node.js 端。自 Vue 2.0起，尤大宣布取消对 vue-resource 的官方推荐，转而推荐 axios。现在 axios 已经成为大部分 Vue 开发者的首选

特性

- 从浏览器中创建 `XMLHttpRequests`
- 从 `node.js` 创建 `http` 请求
- 支持 `Promise` API
- 拦截请求和响应
- 转换请求数据和响应数据
- 取消请求
- 自动转换 `JSON` 数据
- 客户端支持防御 `XSRF`

基本使用

安装

```
1 // 项目中安装
2 npm install axios --S
3 // cdn 引入
4 <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
```

导入

```
1 import axios from 'axios'
```

发送请求

```
1 axios({
2   url: 'xxx', // 设置请求的地址
3   method: 'GET', // 设置请求方法
4   params: { // get请求使用params进行参数凭借,如果是post请求用data
5     type: '',
6     page: 1
7   }
8 }).then(res => {
9   // res为后端返回的数据
10  console.log(res);
11 })
```

并发请求 `axios.all([])`

```

1  function getUserAccount() {
2      return axios.get('/user/12345');
3  }
4
5  function getUserPermissions() {
6      return axios.get('/user/12345/permissions');
7  }
8
9  axios.all([getUserAccount(), getUserPermissions()])
10     .then(axios.spread(function (res1, res2) {
11         // res1第一个请求的返回的内容, res2第二个请求返回的内容
12         // 两个请求都执行完成才会执行
13     })));

```

二、为什么要封装

`axios` 的 API 很友好，你完全可以很轻松地在项目中直接使用。

不过随着项目规模增大，如果每发起一次 `HTTP` 请求，就要把这些比如设置超时时间、设置请求头、根据项目环境判断使用哪个请求地址、错误处理等等操作，都需要写一遍

这种重复劳动不仅浪费时间，而且让代码变得冗余不堪，难以维护。为了提高我们的代码质量，我们应该在项目中二次封装一下 `axios` 再使用

举个例子：

```

1  axios('http://localhost:3000/data', {
2      // 配置代码
3      method: 'GET',
4      timeout: 1000,
5      withCredentials: true,
6      headers: {
7          'Content-Type': 'application/json',
8          Authorization: 'xxx',
9      },
10     transformRequest: [function (data, headers) {
11         return data;
12     }],
13     // 其他请求配置...
14 })
15 .then((data) => {
16     // todo: 真正业务逻辑代码
17     console.log(data);
18 }, (err) => {
19     // 错误处理代码

```

```

20     if (err.response.status === 401) {
21       // handle authorization error
22     }
23     if (err.response.status === 403) {
24       // handle server forbidden error
25     }
26     // 其他错误处理.....
27     console.log(err);
28   });

```

如果每个页面都发送类似的请求，都要写一堆的配置与错误处理，就显得过于繁琐了

这时候我们就需要对 `axios` 进行二次封装，让使用更为便利

三、如何封装

封装的同时，你需要和 后端协商好一些约定，请求头，状态码，请求超时时间.....

设置接口请求前缀：根据开发、测试、生产环境的不同，前缀需要加以区分

请求头：来实现一些具体的业务，必须携带一些参数才可以请求(例如：会员业务)

状态码：根据接口返回的不同 `status`，来执行不同的业务，这块需要和后端约定好

请求方法：根据 `get`、`post` 等方法进行一个再次封装，使用起来更为方便

请求拦截器：根据请求的请求头设定，来决定哪些请求可以访问

响应拦截器：这块就是根据 后端`返回来的状态码判定执行不同业务

设置接口请求前缀

利用 `node` 环境变量来作判断，用来区分开发、测试、生产环境

```

1   if (process.env.NODE_ENV === 'development') {
2     axios.defaults.baseURL = 'http://dev.xxx.com'
3   } else if (process.env.NODE_ENV === 'production') {
4     axios.defaults.baseURL = 'http://prod.xxx.com'
5   }

```

在本地调试的时候，还需要在 `vue.config.js` 文件中配置 `devServer` 实现代理转发，从而实现跨域

```

1   devServer: {
2     proxy: {
3       '/proxyApi': {
4         target: 'http://dev.xxx.com',

```

```

5      changeOrigin: true,
6      pathRewrite: {
7        '/proxyApi': ''
8      }
9    }
10  }
11  }

```

设置请求头与超时时间

大部分情况下，请求头都是固定的，只有少部分情况下，会需要一些特殊的请求头，这里将普适性的请求头作为基础配置。当需要特殊请求头时，将特殊请求头作为参数传入，覆盖基础配置

```

1  const service = axios.create({
2    ...
3    timeout: 30000, // 请求 30s 超时
4    headers: {
5      get: {
6        'Content-Type': 'application/x-www-form-urlencoded;charset=utf-8'
7        // 在开发中，一般还需要单点登录或者其他功能的通用请求头，可以一并配置进来
8      },
9      post: {
10        'Content-Type': 'application/json;charset=utf-8'
11        // 在开发中，一般还需要单点登录或者其他功能的通用请求头，可以一并配置进来
12      }
13    },
14  })

```

封装请求方法

先引入封装好的方法，在要调用的接口重新封装成一个方法暴露出去

```

1  // get 请求
2  export function httpGet({
3    url,
4    params = {}
5  }) {
6    return new Promise((resolve, reject) => {
7      axios.get(url, {
8        params
9      }).then((res) => {
10        resolve(res.data)
11      }).catch(err => {

```

```

12     reject(err)
13   })
14 })
15 }
16
17 // post
18 // post请求
19 export function httpPost({
20   url,
21   data = {},
22   params = {}
23 }) {
24   return new Promise((resolve, reject) => {
25     axios({
26       url,
27       method: 'post',
28       transformRequest: [function (data) {
29         let ret = ''
30         for (let it in data) {
31           ret += encodeURIComponent(it) + '=' + encodeURIComponent(data[it])
32         }
33         return ret
34       }],
35       // 发送的数据
36       data,
37       // url参数
38       params
39     })
40   }).then(res => {
41     resolve(res.data)
42   })
43 })
44 }

```

把封装的方法放在一个 `api.js` 文件中

```

1  import { httpGet, httpPost } from './http'
2  export const getorglist = (params = {}) => httpGet({ url:
  'apps/api/org/list', params })

```

页面中就能直接调用

```

1  // .vue
2  import { getorglist } from '@assets/js/api'
3
4  getorglist({ id: 200 }).then(res => {
5    console.log(res)
6  })

```

这样可以把 `api` 统一管理起来，以后维护修改只需要在 `api.js` 文件操作即可

请求拦截器

请求拦截器可以在每个请求里加上token，做了统一处理后维护起来也方便

```

1  // 请求拦截器
2  axios.interceptors.request.use(
3    config => {
4      // 每次发送请求之前判断是否存在token
5      // 如果存在，则统一在http请求的header都加上token，这样后台根据token判断你的登录情况，此处token一般是用户完成登录后储存到localStorage里的
6      token && (config.headers.Authorization = token)
7      return config
8    },
9    error => {
10     return Promise.error(error)
11   })

```

响应拦截器

响应拦截器可以在接收到响应后先做一层操作，如根据状态码判断登录状态、授权

```

1  // 响应拦截器
2  axios.interceptors.response.use(response => {
3    // 如果返回的状态码为200，说明接口请求成功，可以正常拿到数据
4    // 否则的话抛出错误
5    if (response.status === 200) {
6      if (response.data.code === 511) {
7        // 未授权调取授权接口
8      } else if (response.data.code === 510) {
9        // 未登录跳转登录页
10     } else {
11       return Promise.resolve(response)
12     }
13   } else {
14     return Promise.reject(response)

```

```
15     }
16   }, error => {
17     // 我们可以在这里对异常状态作统一处理
18     if (error.response.status) {
19       // 处理请求失败的情况
20       // 对不同返回码对相应处理
21       return Promise.reject(error.response)
22     }
23   })
```

小结

- 封装是编程中很有意义的手段，简单的 `axios` 封装，就可以让我们领略到它的魅力
- 封装 `axios` 没有一个绝对的标准，只要你的封装可以满足你的项目需求，并且用起来方便，那就是一个好的封装方案

第31题：说说vue中的diff算法

一、是什么

`diff` 算法是一种通过同层的树节点进行比较的高效算法

其有两个特点：

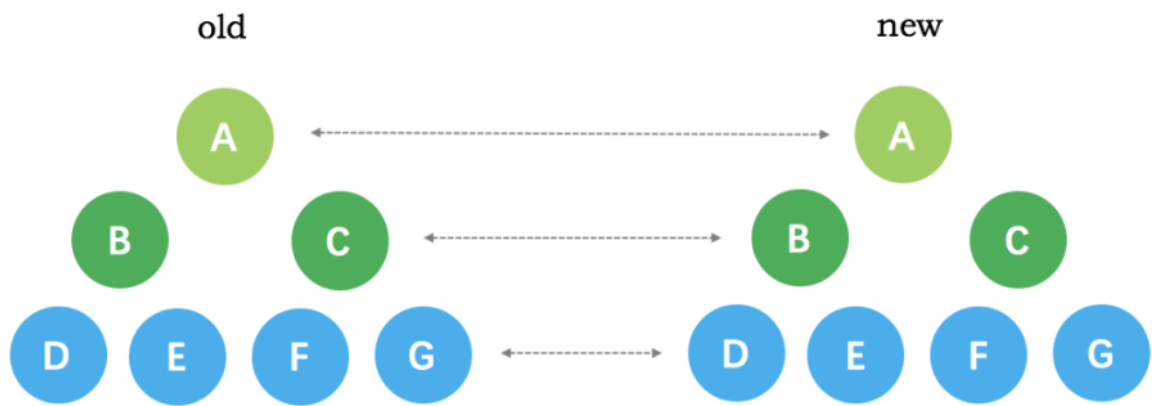
- 比较只会在同层级进行, 不会跨层级比较
- 在diff比较的过程中，循环从两边向中间比较

`diff` 算法的在很多场景下都有应用，在 `vue` 中，作用于虚拟 `dom` 渲染成真实 `dom` 的新旧 `VNode` 节点比较

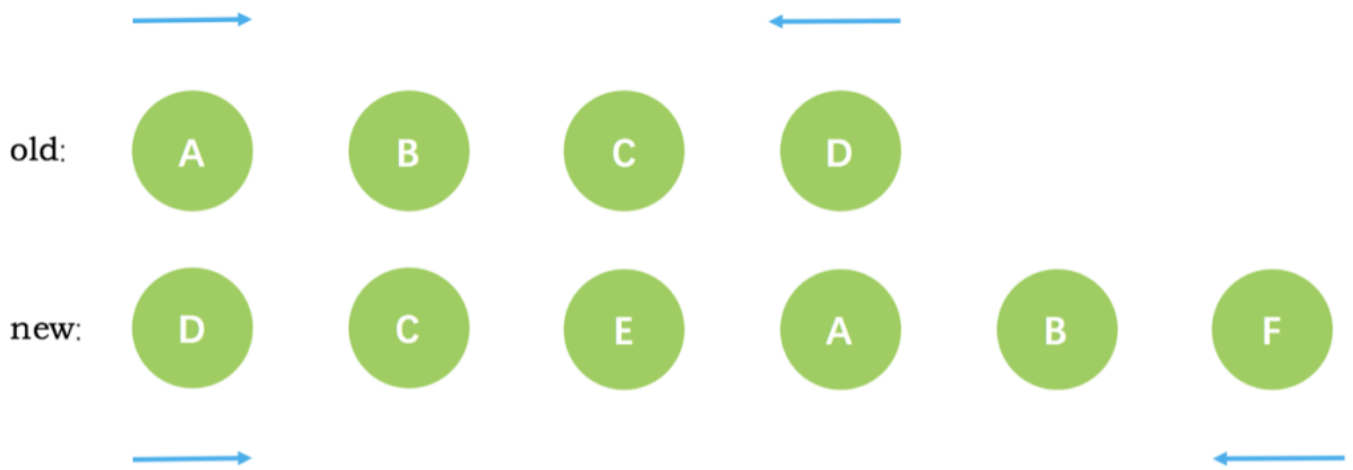
二、比较方式

`diff` 整体策略为：深度优先，同层比较

1. 比较只会在同层级进行, 不会跨层级比较

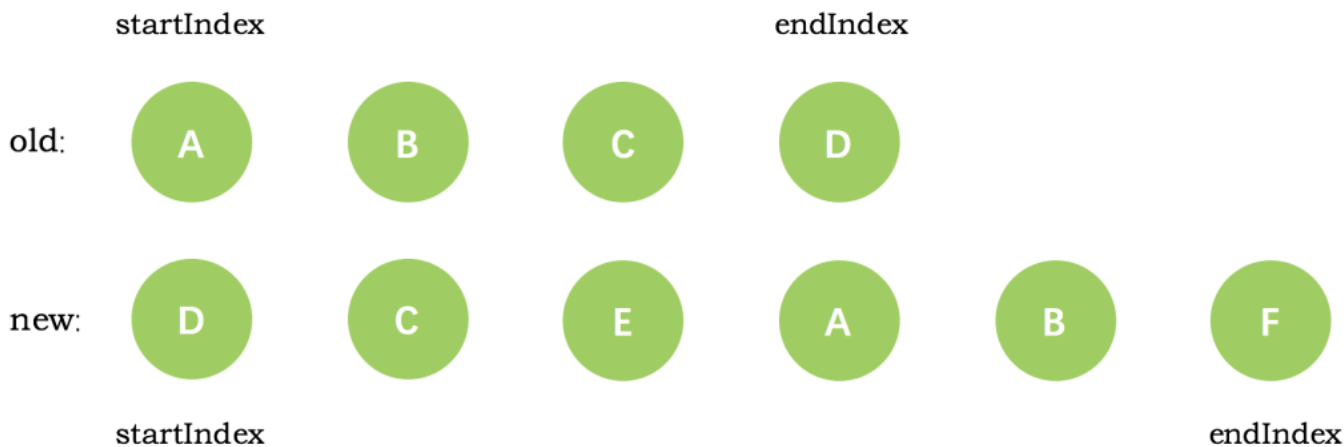


1. 比较的过程中，循环从两边向中间收拢



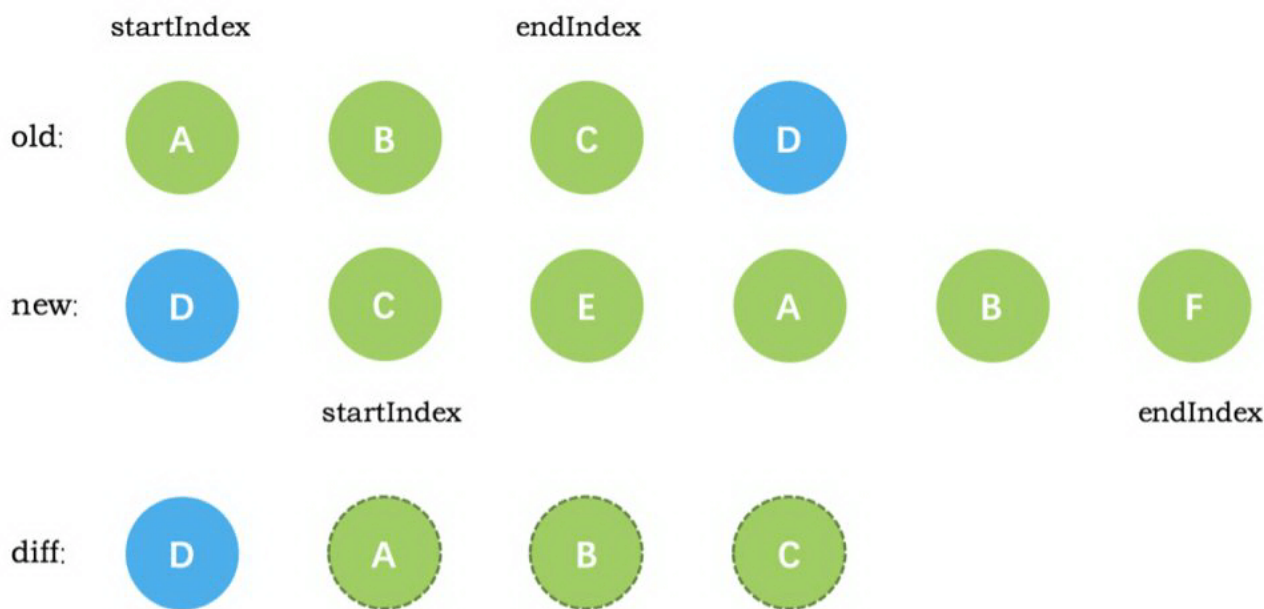
下面举个 `vue` 通过 `diff` 算法更新的例子：

新旧 `VNode` 节点如下图所示：



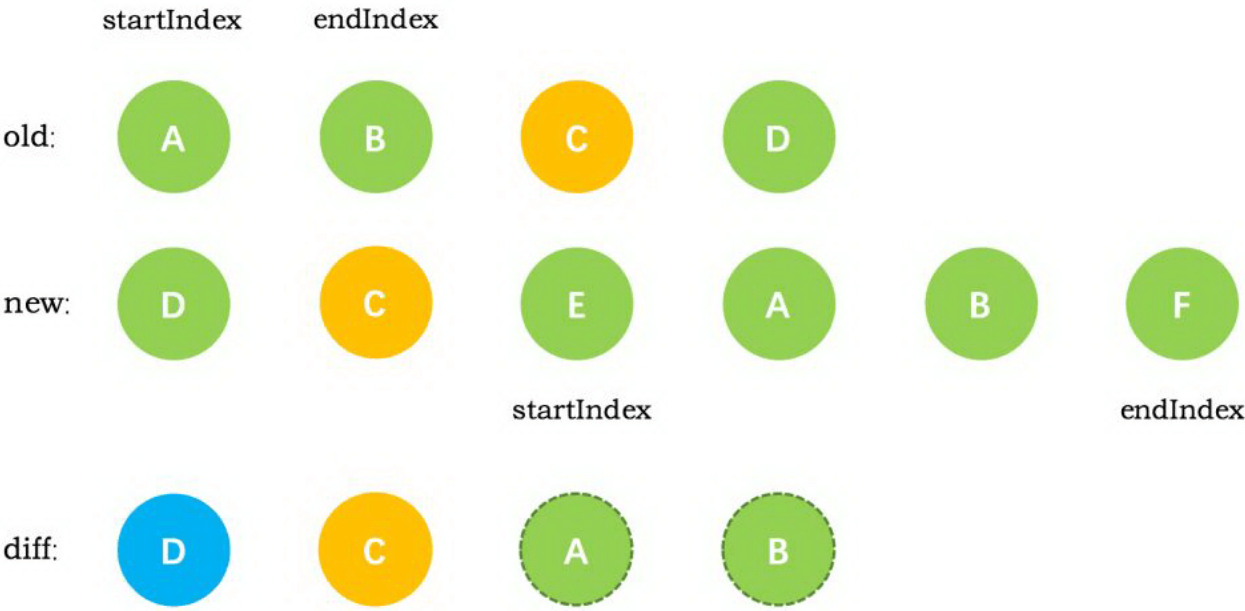
第一次循环后，发现旧节点D与新节点D相同，直接复用旧节点D作为 `diff` 后的第一个真实节点，同时旧节点 `endIndex` 移动到C，新节点的 `startIndex` 移动到了 C

第一次:



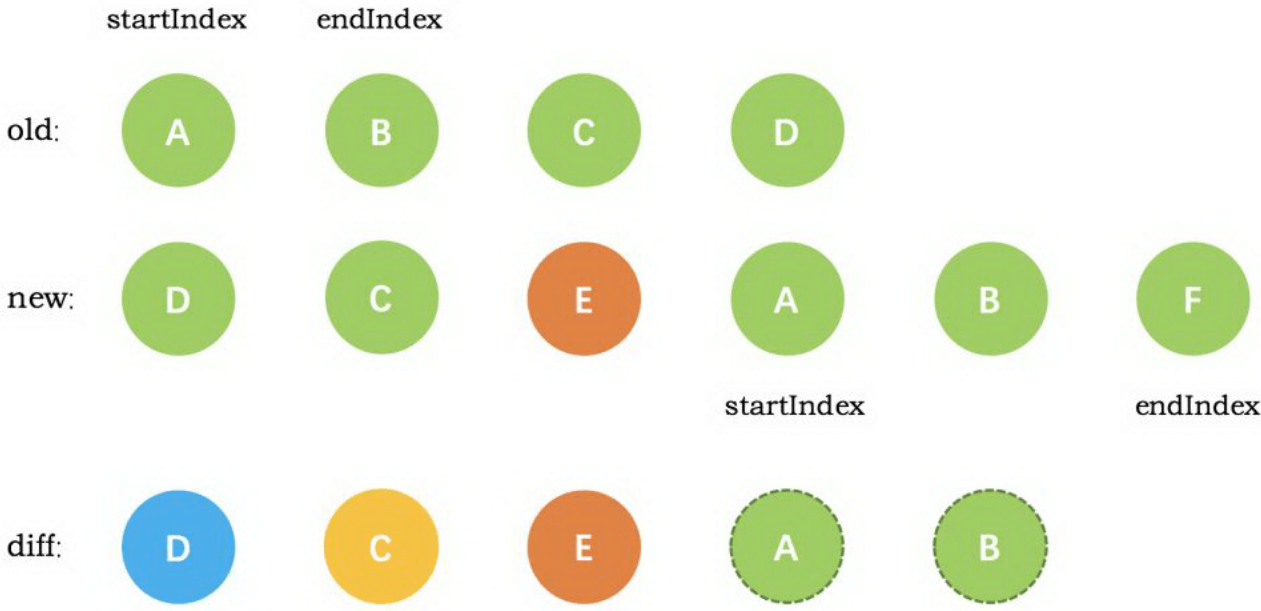
第二次循环后，同样是旧节点的末尾和新节点的开头(都是 C)相同，同理， `diff` 后创建了 C 的真实节点插入到第一次创建的 B 节点后面。同时旧节点的 `endIndex` 移动到了 B，新节点的 `startIndex` 移动到了 E

第二次:



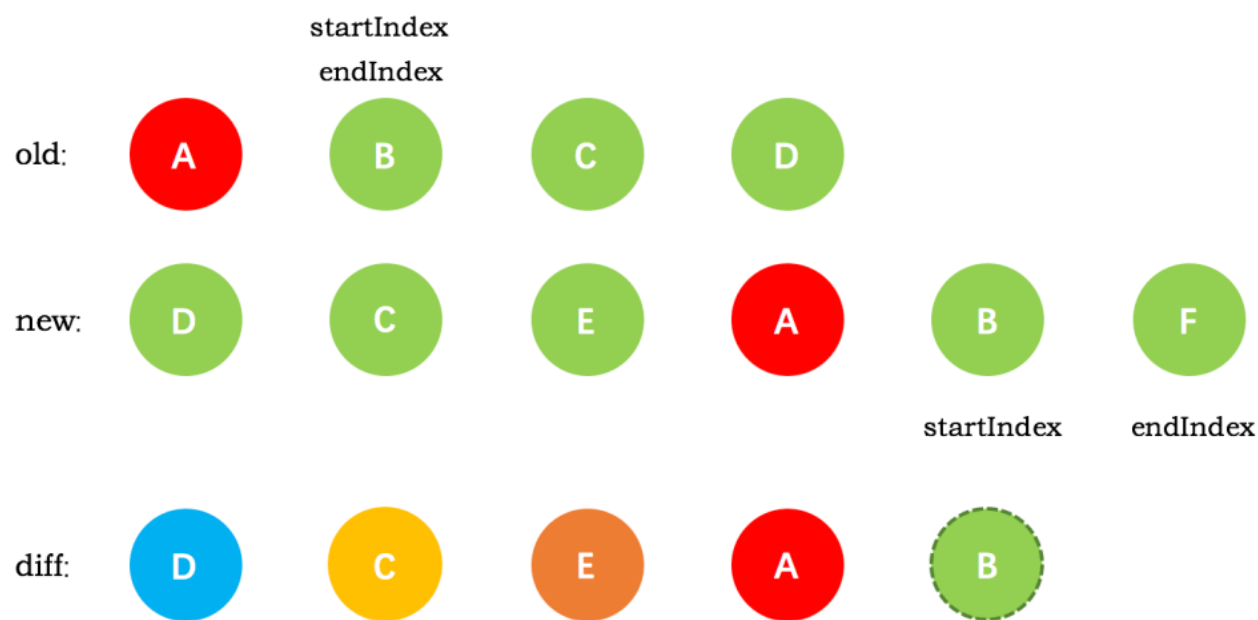
第三次循环中，发现E没有找到，这时候只能直接创建新的真实节点 E，插入到第二次创建的 C 节点之后。同时新节点的 `startIndex` 移动到了 A。旧节点的 `startIndex` 和 `endIndex` 都保持不动

第三次:



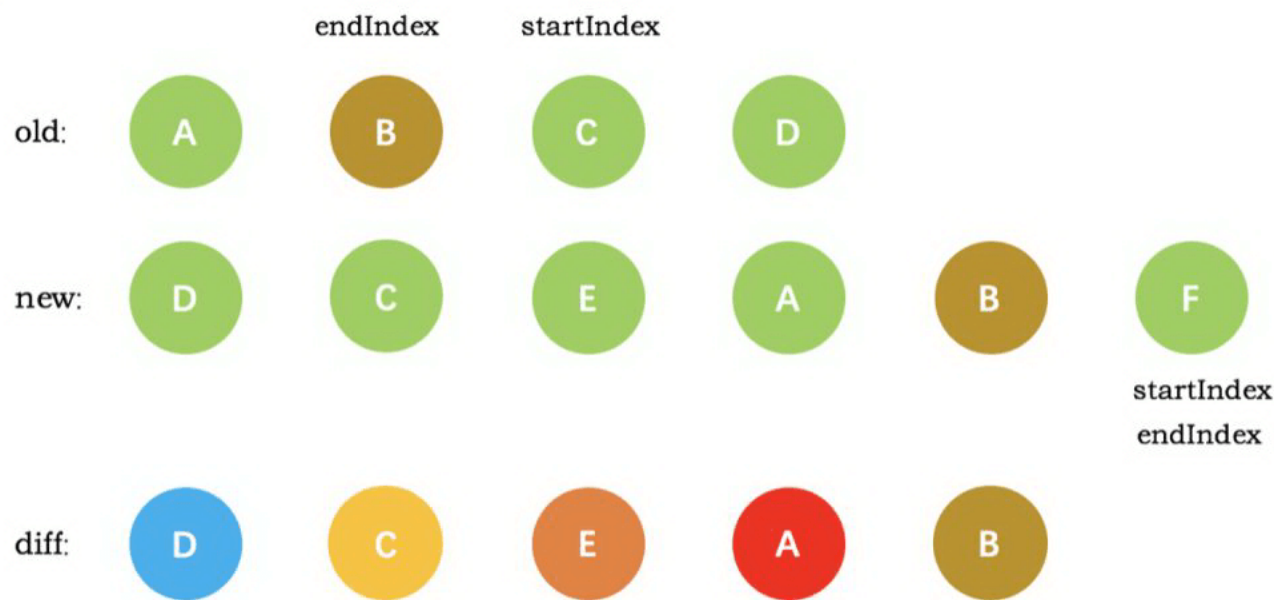
第四次循环中，发现了新旧节点的开头(都是 A)相同，于是 `diff` 后创建了 A 的真实节点，插入到前一次创建的 E 节点后面。同时旧节点的 `startIndex` 移动到了 B，新节点的 `startIndex` 移动到了 B

第四次:



第五次循环中，情形同第四次循环一样，因此 `diff` 后创建了 B 真实节点 插入到前一次创建的 A 节点后面。同时旧节点的 `startIndex` 移动到了 C，新节点的 `startIndex` 移动到了 F

第五次:



新节点的 `startIndex` 已经大于 `endIndex` 了，需要创建 `newStartIdx` 和 `newEndIdx` 之间的所有节点，也就是节点F，直接创建 F 节点对应的真实节点放到 B 节点后面

第六次:

因 $oldStartIndex > oldEndIndex$, 退出循环, 补充new增加的节点F



三、原理分析

当数据发生改变时, `set` 方法会调用 `Dep.notify` 通知所有订阅者 `Watcher`, 订阅者就会调用 `patch` 给真实的 `DOM` 打补丁, 更新相应的视图

源码位置: `src/core/vdom/patch.js`

```
1  function patch(oldVnode, vnode, hydrating, removeOnly) {
2    if (isUndef(vnode)) { // 没有新节点, 直接执行destory钩子函数
3      if (isDef(oldVnode)) invokeDestroyHook(oldVnode)
4      return
5    }
6
7    let isInitialPatch = false
8    const insertedVnodeQueue = []
9
10   if (isUndef(oldVnode)) {
11     isInitialPatch = true
12     createElm(vnode, insertedVnodeQueue) // 没有旧节点, 直接用新节点生成dom元
13   } else {
14     const isRealElement = isDef(oldVnode.nodeType)
15     if (!isRealElement && sameVnode(oldVnode, vnode)) {
16       // 判断旧节点和新节点自身一样, 一致执行patchVnode
17       patchVnode(oldVnode, vnode, insertedVnodeQueue, null, null,
18 removeOnly)
19     } else {
20       // 否则直接销毁及旧节点, 根据新节点生成dom元素
21       if (isRealElement) {
```

```

22         if (oldVnode.nodeType === 1 &&
oldVnode.hasAttribute(SSR_ATTR)) {
23             oldVnode.removeAttribute(SSR_ATTR)
24             hydrating = true
25         }
26         if (isTrue(hydrating)) {
27             if (hydrate(oldVnode, vnode, insertedVnodeQueue)) {
28                 invokeInsertHook(vnode, insertedVnodeQueue, true)
29                 return oldVnode
30             }
31         }
32         oldVnode = emptyNodeAt(oldVnode)
33     }
34     return vnode.elm
35 }
36 }
37 }

```

`patch` 函数前两个参数位为 `oldVnode` 和 `Vnode` ，分别代表新的节点和之前的旧节点，主要做了四个判断：

- 没有新节点，直接触发旧节点的 `destory` 钩子
- 没有旧节点，说明是页面刚开始初始化的时候，此时，根本不需要比较了，直接全是新建，所以只调用 `createElm`
- 旧节点和新节点自身一样，通过 `sameVnode` 判断节点是否一样，一样时，直接调用 `patchVnode` 去处理这两个节点
- 旧节点和新节点自身不一样，当两个节点不一样的时候，直接创建新节点，删除旧节点

下面主要讲的是 `patchVnode` 部分

```

1  function patchVnode (oldVnode, vnode, insertedVnodeQueue, removeOnly) {
2      // 如果新旧节点一致，什么都不做
3      if (oldVnode === vnode) {
4          return
5      }
6
7      // 让vnode.el引用到现在的真实dom，当el修改时，vnode.el会同步变化
8      const elm = vnode.elm = oldVnode.elm
9
10     // 异步占位符
11     if (isTrue(oldVnode.isAsyncPlaceholder)) {
12         if (isDef(vnode.asyncFactory.resolved)) {
13             hydrate(oldVnode.elm, vnode, insertedVnodeQueue)
14         } else {

```

```

15         vnode.isAsyncPlaceholder = true
16     }
17     return
18 }
19 // 如果新旧都是静态节点，并且具有相同的key
20 // 当vnode是克隆节点或是v-once指令控制的节点时，只需要把oldVnode.elm和
oldVnode.child都复制到vnode上
21 // 也不用再有其他操作
22 if (isTrue(vnode.isStatic) &&
23     isTrue(oldVnode.isStatic) &&
24     vnode.key === oldVnode.key &&
25     (isTrue(vnode.isCloned) || isTrue(vnode.isOnce)))
26 ) {
27     vnode.componentInstance = oldVnode.componentInstance
28     return
29 }
30
31 let i
32 const data = vnode.data
33 if (isDef(data) && isDef(i = data.hook) && isDef(i = i.prepatch)) {
34     i(oldVnode, vnode)
35 }
36
37 const oldCh = oldVnode.children
38 const ch = vnode.children
39 if (isDef(data) && isPatchable(vnode)) {
40     for (i = 0; i < cbs.update.length; ++i) cbs.update[i](oldVnode, vnode)
41     if (isDef(i = data.hook) && isDef(i = i.update)) i(oldVnode, vnode)
42 }
43 // 如果vnode不是文本节点或者注释节点
44 if (isUndef(vnode.text)) {
45     // 并且都有子节点
46     if (isDef(oldCh) && isDef(ch)) {
47         // 并且子节点不完全一致，则调用updateChildren
48         if (oldCh !== ch) updateChildren(elm, oldCh, ch, insertedVnodeQueue,
removeOnly)
49
50         // 如果只有新的vnode有子节点
51     } else if (isDef(ch)) {
52         if (isDef(oldVnode.text)) nodeOps.setTextContent(elm, '')
53         // elm已经引用了老的dom节点，在老的dom节点上添加子节点
54         addVnodes(elm, null, ch, 0, ch.length - 1, insertedVnodeQueue)
55
56         // 如果新vnode没有子节点，而vnode有子节点，直接删除老的oldCh
57     } else if (isDef(oldCh)) {
58         removeVnodes(elm, oldCh, 0, oldCh.length - 1)
59

```

```

60      // 如果老节点是文本节点
61    } else if (isDef(oldVnode.text)) {
62      nodeOps.setTextContent(elm, '')
63    }
64
65    // 如果新vnode和老vnode是文本节点或注释节点
66    // 但是vnode.text !== oldVnode.text时, 只需要更新vnode.elm的文本内容就可以
67  } else if (oldVnode.text !== vnode.text) {
68    nodeOps.setTextContent(elm, vnode.text)
69  }
70  if (isDef(data)) {
71    if (isDef(i = data.hook) && isDef(i = i.postpatch)) i(oldVnode, vnode)
72  }
73  }

```

`patchVnode` 主要做了几个判断：

- 新节点是否是文本节点，如果是，则直接更新 `dom` 的文本内容为新节点的文本内容
- 新节点和旧节点如果都有子节点，则处理比较更新子节点
- 只有新节点有子节点，旧节点没有，那么不用比较了，所有节点都是全新的，所以直接全部新建就好了，新建是指创建出所有新 `DOM`，并且添加进父节点
- 只有旧节点有子节点而新节点没有，说明更新后的页面，旧节点全部都不见了，那么要做的，就是把所有的旧节点删除，也就是直接把 `DOM` 删除

子节点不完全一致，则调用 `updateChildren`

```

1  function updateChildren (parentElm, oldCh, newCh, insertedVnodeQueue,
    removeOnly) {
2    let oldStartIdx = 0 // 旧头索引
3    let newStartIdx = 0 // 新头索引
4    let oldEndIdx = oldCh.length - 1 // 旧尾索引
5    let newEndIdx = newCh.length - 1 // 新尾索引
6    let oldStartVnode = oldCh[0] // oldVnode的第一个child
7    let oldEndVnode = oldCh[oldEndIdx] // oldVnode的最后一个child
8    let newStartVnode = newCh[0] // newVnode的第一个child
9    let newEndVnode = newCh[newEndIdx] // newVnode的最后一个child
10   let oldKeyToIdx, idxInOld, vnodeToMove, refElm
11
12   // removeOnly is a special flag used only by <transition-group>
13   // to ensure removed elements stay in correct relative positions
14   // during leaving transitions
15   const canMove = !removeOnly
16

```

```

17 // 如果oldStartVnode和oldEndVnode重合, 并且新的也都重合了, 证明diff完了, 循环结
束
18 while (oldStartIdx <= oldEndIdx && newStartIdx <= newEndIdx) {
19 // 如果oldVnode的第一个child不存在
20 if (isUndef(oldStartVnode)) {
21 // oldStart索引右移
22 oldStartVnode = oldCh[++oldStartIdx] // Vnode has been moved left
23
24 // 如果oldVnode的最后一个child不存在
25 } else if (isUndef(oldEndVnode)) {
26 // oldEnd索引左移
27 oldEndVnode = oldCh[--oldEndIdx]
28
29 // oldStartVnode和newStartVnode是同一个节点
30 } else if (sameVnode(oldStartVnode, newStartVnode)) {
31 // patch oldStartVnode和newStartVnode, 索引左移, 继续循环
32 patchVnode(oldStartVnode, newStartVnode, insertedVnodeQueue)
33 oldStartVnode = oldCh[++oldStartIdx]
34 newStartVnode = newCh[++newStartIdx]
35
36 // oldEndVnode和newEndVnode是同一个节点
37 } else if (sameVnode(oldEndVnode, newEndVnode)) {
38 // patch oldEndVnode和newEndVnode, 索引右移, 继续循环
39 patchVnode(oldEndVnode, newEndVnode, insertedVnodeQueue)
40 oldEndVnode = oldCh[--oldEndIdx]
41 newEndVnode = newCh[--newEndIdx]
42
43 // oldStartVnode和newEndVnode是同一个节点
44 } else if (sameVnode(oldStartVnode, newEndVnode)) { // Vnode moved right
45 // patch oldStartVnode和newEndVnode
46 patchVnode(oldStartVnode, newEndVnode, insertedVnodeQueue)
47 // 如果removeOnly是false, 则将oldStartVnode.elm移动到oldEndVnode.elm之后
48 canMove && nodeOps.insertBefore(parentElm, oldStartVnode.elm,
nodeOps.nextSibling(oldEndVnode.elm))
49 // oldStart索引右移, newEnd索引左移
50 oldStartVnode = oldCh[++oldStartIdx]
51 newEndVnode = newCh[--newEndIdx]
52
53 // 如果oldEndVnode和newStartVnode是同一个节点
54 } else if (sameVnode(oldEndVnode, newStartVnode)) { // Vnode moved left
55 // patch oldEndVnode和newStartVnode
56 patchVnode(oldEndVnode, newStartVnode, insertedVnodeQueue)
57 // 如果removeOnly是false, 则将oldEndVnode.elm移动到oldStartVnode.elm之前
58 canMove && nodeOps.insertBefore(parentElm, oldEndVnode.elm,
oldStartVnode.elm)
59 // oldEnd索引左移, newStart索引右移
60 oldEndVnode = oldCh[--oldEndIdx]

```

```

61     newStartVnode = newCh[++newStartIdx]
62
63     // 如果都不匹配
64     } else {
65         if (isUndef(oldKeyToIdx)) oldKeyToIdx = createKeyToOldIdx(oldCh,
oldStartIdx, oldEndIdx)
66
67         // 尝试在oldChildren中寻找和newStartVnode的具有相同的key的Vnode
68         idxInOld = isDef(newStartVnode.key)
69             ? oldKeyToIdx[newStartVnode.key]
70             : findIdxInOld(newStartVnode, oldCh, oldStartIdx, oldEndIdx)
71
72         // 如果未找到, 说明newStartVnode是一个新的节点
73         if (isUndef(idxInOld)) { // New element
74             // 创建一个新Vnode
75             createElm(newStartVnode, insertedVnodeQueue, parentElm,
oldStartVnode.elm)
76
77             // 如果找到了和newStartVnodej具有相同的key的Vnode, 叫vnodeToMove
78         } else {
79             vnodeToMove = oldCh[idxInOld]
80             /* istanbul ignore if */
81             if (process.env.NODE_ENV !== 'production' && !vnodeToMove) {
82                 warn(
83                     'It seems there are duplicate keys that is causing an update
error. ' +
84                     'Make sure each v-for item has a unique key.'
85                 )
86             }
87
88             // 比较两个具有相同的key的新节点是否是同一个节点
89             // 不设key, newCh和oldCh只会进行头尾两端的相互比较, 设key后, 除了头尾两端的
比较外, 还会从用key生成的对象oldKeyToIdx中查找匹配的节点, 所以为节点设置key可以更高效的
利用dom。
90             if (sameVnode(vnodeToMove, newStartVnode)) {
91                 // patch vnodeToMove和newStartVnode
92                 patchVnode(vnodeToMove, newStartVnode, insertedVnodeQueue)
93                 // 清除
94                 oldCh[idxInOld] = undefined
95                 // 如果removeOnly是false, 则将找到的和newStartVnodej具有相同的key的
Vnode, 叫vnodeToMove.elm
96                 // 移动到oldStartVnode.elm之前
97                 canMove && nodeOps.insertBefore(parentElm, vnodeToMove.elm,
oldStartVnode.elm)
98
99                 // 如果key相同, 但是节点不相同, 则创建一个新的节点
100             } else {

```

```

101         // same key but different element. treat as new element
102         createElm(newStartVnode, insertedVnodeQueue, parentElm,
            oldStartVnode.elm)
103     }
104 }
105
106     // 右移
107     newStartVnode = newCh[++newStartIdx]
108 }
109 }

```

`while` 循环主要处理了以下五种情景：

- 当新老 `VNode` 节点的 `start` 相同时，直接 `patchVnode`，同时新老 `VNode` 节点的开始索引都加 1
- 当新老 `VNode` 节点的 `end` 相同时，同样直接 `patchVnode`，同时新老 `VNode` 节点的结束索引都减 1
- 当老 `VNode` 节点的 `start` 和新 `VNode` 节点的 `end` 相同时，这时候在 `patchVnode` 后，还需要将当前真实 `dom` 节点移动到 `oldEndVnode` 的后面，同时老 `VNode` 节点开始索引加 1，新 `VNode` 节点的结束索引减 1
- 当老 `VNode` 节点的 `end` 和新 `VNode` 节点的 `start` 相同时，这时候在 `patchVnode` 后，还需要将当前真实 `dom` 节点移动到 `oldStartVnode` 的前面，同时老 `VNode` 节点结束索引减 1，新 `VNode` 节点的开始索引加 1
- 如果都不满足以上四种情形，那说明没有相同的节点可以复用，则会分为以下两种情况：
 - 从旧的 `VNode` 为 `key` 值，对应 `index` 序列为 `value` 值的哈希表中找到与 `newStartVnode` 一致 `key` 的旧的 `VNode` 节点，再进行 `patchVnode`，同时将这个真实 `dom` 移动到 `oldStartVnode` 对应的真实 `dom` 的前面
 - 调用 `createElm` 创建一个新的 `dom` 节点放到当前 `newStartIdx` 的位置

小结

- 当数据发生改变时，订阅者 `watcher` 就会调用 `patch` 给真实的 `DOM` 打补丁
- 通过 `isSameVnode` 进行判断，相同则调用 `patchVnode` 方法
- `patchVnode` 做了以下操作：
 - 找到对应的真实 `dom`，称为 `el`
 - 如果都有都有文本节点且不相等，将 `el` 文本节点设置为 `Vnode` 的文本节点
 - 如果 `oldVnode` 有子节点而 `VNode` 没有，则删除 `el` 子节点
 - 如果 `oldVnode` 没有子节点而 `VNode` 有，则将 `VNode` 的子节点真实化后添加到 `el`

- 如果两者都有子节点，则执行 `updateChildren` 函数比较子节点
- `updateChildren` 主要做了以下操作：
 - 设置新旧 `VNode` 的头尾指针
 - 新旧头尾指针进行比较，循环向中间靠拢，根据情况调用 `patchVnode` 进行 `patch` 重复流程、调用 `createElement` 创建一个新节点，从哈希表寻找 `key` 一致的 `VNode` 节点再分情况操作

第32题：什么是虚拟DOM？如何实现一个虚拟DOM？说说你的思路

一、什么是虚拟DOM

虚拟 DOM (`Virtual DOM`) 这个概念相信大家都不陌生，从 `React` 到 `Vue` ，虚拟 `DOM` 为这两个框架都带来了跨平台的能力 (`React-Native` 和 `Weex`)

实际上它只是一层对真实 `DOM` 的抽象，以 `JavaScript` 对象 (`VNode` 节点) 作为基础的树，用对象的属性来描述节点，最终可以通过一系列操作使这棵树映射到真实环境上

在 `Javascript` 对象中，虚拟 `DOM` 表现为一个 `Object` 对象。并且最少包含标签名 (`tag`)、属性 (`attrs`) 和子元素对象 (`children`) 三个属性，不同框架对这三个属性的命名可能会有差别

创建虚拟 `DOM` 就是为了更好将虚拟的节点渲染到页面视图中，所以虚拟 `DOM` 对象的节点与真实 `DOM` 的属性一一照应

在 `vue` 中同样使用到了虚拟 `DOM` 技术

定义真实 `DOM`

```
1 <div id="app">
2   <p class="p">节点内容</p>
3   <h3>{{ foo }}</h3>
4 </div>
```

实例化 `vue`

```
1 const app = new Vue({
2   el: "#app",
3   data: {
4     foo: "foo"
5   }
6 })
```

观察 `render` 的 `render` ，我们能得到虚拟 `DOM`

```
1  (function anonymous(  
2  ) {  
3      with(this){return _c('div',{attrs:{"id":"app"}},[_c('p',  
4      {staticClass:"p"},  
      [_v("节点内容")]),_v(" "),_c('h3',  
      [_v(_s(foo))])])])})
```

通过 `VNode` ， `vue` 可以对这颗抽象树进行创建节点,删除节点以及修改节点的操作， 经过 `diff` 算法得出一些需要修改的最小单位,再更新视图，减少了 `dom` 操作，提高了性能

二、为什么需要虚拟DOM

`DOM` 是很慢的，其元素非常庞大，页面的性能问题，大部分都是由 `DOM` 操作引起的

真实的 `DOM` 节点，哪怕一个最简单的 `div` 也包含着很多属性，可以打印出来直观感受一下：

```
> var div = document.createElement('div')  
var str = ""  
for (var key in div) {  
  str = str + key + " "  
}  
console.log(str)  
align title lang translate dir dataset hidden tabIndex accessKey draggable spellcheck contentEditable isContentEditable offsetParent offsetTop offsetLeft VM1696:  
offsetWidth offsetHeight style innerText outerText webkitdropzone onabort onblur oncancel oncanplay oncanplaythrough onchange onclick onclose oncontextmenu oncuechange  
ondblclick ondrag ondragend ondragenter ondragleave ondragover ondragstart ondrop ondurationchange onemptied onended onerror onfocus oninput oninvalid onkeydown  
onkeypress onkeyup onload onloadeddata onloadedmetadata onloadstart onmousedown onmouseenter onmouseleave onmousemove onmouseout onmouseover onmouseup onmousewheel  
onpause onplay onplaying onprogress onratechange onreset onresize onscroll onseeked onseeking onselect onshow onstalled onsubmit onsuspend ontimeupdate ontoggle  
onvolumechange onwaiting click focus blur onautocomplete onautocompleteerror namespaceURI prefix localName tagName id className classList attributes innerHTML outerHTML  
shadowRoot scrollTop scrollLeft scrollWidth scrollHeight clientTop clientLeft clientWidth clientHeight onbeforecopy onbeforecut onbeforepaste oncopy oncut onpaste  
onsearch onselectstart onwheel onwebkitfullscreenchange onwebkitfullscreenerror previousElementSibling nextElementSibling children firstElementChild lastElementChild  
childElementCount hasAttributes getAttribute getAttributeNS setAttribute setAttributeNS removeAttribute removeAttributeNS hasAttribute hasAttributeNS getAttributeNode  
getAttributeNodeNS setAttributeNode setAttributeNodeNS removeAttributeNode closest matches getElementsByTagName getElementsByTagNameNS getElementsByClassName  
insertAdjacentHTML createShadowRoot getDestinationInsertionPoints requestPointerLock getClientRects getBoundingClientRect scrollIntoView insertAdjacentElement  
insertAdjacentText scrollIntoViewIfNeeded webkitMatchesSelector animate remove webkitRequestFullscreen webkitRequestFullscreen querySelector querySelectorAll prepend  
append before after replaceWith nodeType nodeName baseURI ownerDocument parentNode parentElement childNodes firstChild lastChild previousSibling nextSibling nodeValue  
textContent hasChildNodes normalize cloneNode isEqualNode compareDocumentPosition contains lookupPrefix lookupNamespaceURI isDefaultNamespace insertBefore appendChild  
replaceChild removeChild isSameNode ELEMENT_NODE ATTRIBUTE_NODE TEXT_NODE CDATA_SECTION_NODE ENTITY_REFERENCE_NODE ENTITY_NODE PROCESSING_INSTRUCTION_NODE COMMENT_NODE  
DOCUMENT_NODE DOCUMENT_TYPE_NODE DOCUMENT_FRAGMENT_NODE NOTATION_NODE DOCUMENT_POSITION_DISCONNECTED DOCUMENT_POSITION_PRECEDING DOCUMENT_POSITION_FOLLOWING  
DOCUMENT_POSITION_CONTAINS DOCUMENT_POSITION_CONTAINED_BY DOCUMENT_POSITION_IMPLEMENTATION_SPECIFIC addEventListener removeEventListener dispatchEvent
```

由此可见，操作 `DOM` 的代价仍旧是昂贵的，频繁操作还是会出现页面卡顿，影响用户的体验

举个例子：

你用传统的原生 `api` 或 `jQuery` 去操作 `DOM` 时，浏览器会从构建 `DOM` 树开始从头到尾执行一遍流程

当你在一次操作时，需要更新10个 `DOM` 节点，浏览器没那么智能，收到第一个更新 `DOM` 请求后，并不知道后续还有9次更新操作，因此会马上执行流程，最终执行10次流程

而通过 `VNode` ，同样更新10个 `DOM` 节点，虚拟 `DOM` 不会立即操作 `DOM` ，而是将这10次更新的 `diff` 内容保存到本地的一个 `js` 对象中，最终将这个 `js` 对象一次性 `attach` 到 `DOM` 树上，避免大量的无谓计算

很多人认为虚拟 `DOM` 最大的优势是 `diff` 算法，减少 `JavaScript` 操作真实 `DOM` 的带来的性能消耗。虽然这一个虚拟 `DOM` 带来的一个优势，但并不是全部。虚拟 `DOM` 最大的优势在于抽象了原本的渲

染过程，实现了跨平台的能力，而不仅仅局限于浏览器的 DOM，可以是安卓和 IOS 的原生组件，可以是近期很火热的小程序，也可以是各种GUI

三、如何实现虚拟DOM

首先可以看看 `vue` 中 `VNode` 的结构

源码位置：src/core/vdom/vnode.js

```
1  export default class VNode {
2    tag: string | void;
3    data: VNodeData | void;
4    children: ?Array<VNode>;
5    text: string | void;
6    elm: Node | void;
7    ns: string | void;
8    context: Component | void; // rendered in this component's scope
9    functionalContext: Component | void; // only for functional component root
    nodes
10   key: string | number | void;
11   componentOptions: VNodeComponentOptions | void;
12   componentInstance: Component | void; // component instance
13   parent: VNode | void; // component placeholder node
14   raw: boolean; // contains raw HTML? (server only)
15   isStatic: boolean; // hoisted static node
16   isRootInsert: boolean; // necessary for enter transition check
17   isComment: boolean; // empty comment placeholder?
18   isCloned: boolean; // is a cloned node?
19   isOnce: boolean; // is a v-once node?
20
21   constructor (
22     tag?: string,
23     data?: VNodeData,
24     children?: ?Array<VNode>,
25     text?: string,
26     elm?: Node,
27     context?: Component,
28     componentOptions?: VNodeComponentOptions
29   ) {
30     /*当前节点的标签名*/
31     this.tag = tag
32     /*当前节点对应的对象，包含了具体的一些数据信息，是一个VNodeData类型，可以参考
    VNodeData类型中的数据信息*/
33     this.data = data
34     /*当前节点的子节点，是一个数组*/
35     this.children = children
```

```

36     /*当前节点的文本*/
37     this.text = text
38     /*当前虚拟节点对应的真实dom节点*/
39     this.elm = elm
40     /*当前节点的名字空间*/
41     this.ns = undefined
42     /*编译作用域*/
43     this.context = context
44     /*函数化组件作用域*/
45     this.functionalContext = undefined
46     /*节点的key属性，被当作节点的标志，用以优化*/
47     this.key = data && data.key
48     /*组件的option选项*/
49     this.componentOptions = componentOptions
50     /*当前节点对应的组件的实例*/
51     this.componentInstance = undefined
52     /*当前节点的父节点*/
53     this.parent = undefined
54     /*简而言之就是是否为原生HTML或只是普通文本，innerHTML的时候为true，textContent的
    时候为false*/
55     this.raw = false
56     /*静态节点标志*/
57     this.isStatic = false
58     /*是否作为跟节点插入*/
59     this.isRootInsert = true
60     /*是否为注释节点*/
61     this.isComment = false
62     /*是否为克隆节点*/
63     this.isCloned = false
64     /*是否有v-once指令*/
65     this.isOnce = false
66 }
67
68 // DEPRECATED: alias for componentInstance for backwards compat.
69 /* istanbul ignore next https://github.com/answershuto/learnVue*/
70 get child (): Component | void {
71     return this.componentInstance
72 }
73 }

```

这里对 `VNode` 进行稍微的说明：

- 所有对象的 `context` 选项都指向了 `Vue` 实例
- `elm` 属性则指向了其相对应的真实 `DOM` 节点

`vue` 是通过 `createElement` 生成 `VNode`

源码位置: src/core/vdom/create-element.js

```
1  export function createElement (
2    context: Component,
3    tag: any,
4    data: any,
5    children: any,
6    normalizationType: any,
7    alwaysNormalize: boolean
8  ): VNode | Array<VNode> {
9    if (Array.isArray(data) || isPrimitive(data)) {
10      normalizationType = children
11      children = data
12      data = undefined
13    }
14    if (isTrue(alwaysNormalize)) {
15      normalizationType = ALWAYS_NORMALIZE
16    }
17    return _createElement(context, tag, data, children, normalizationType)
18  }
```

上面可以看到 `createElement` 方法实际上是对 `_createElement` 方法的封装, 对参数的传入进行了判断

```
1  export function _createElement(
2    context: Component,
3    tag?: string | Class<Component> | Function | Object,
4    data?: VNodeData,
5    children?: any,
6    normalizationType?: number
7  ): VNode | Array<VNode> {
8    if (isDef(data) && isDef((data: any).__ob__)) {
9      process.env.NODE_ENV !== 'production' && warn(
10        `Avoid using observed data object as vnode data:
11        ${JSON.stringify(data)}\n` +
12        'Always create fresh vnode data objects in each render!',
13        context
14      )
15      return createEmptyVNode()
16    }
17    // object syntax in v-bind
18    if (isDef(data) && isDef(data.is)) {
19      tag = data.is
20    }
```

```

20     if (!tag) {
21         // in case of component :is set to falsy value
22         return createEmptyVNode()
23     }
24     ...
25     // support single function children as default scoped slot
26     if (Array.isArray(children) &&
27         typeof children[0] === 'function'
28     ) {
29         data = data || {}
30         data.scopedSlots = { default: children[0] }
31         children.length = 0
32     }
33     if (normalizationType === ALWAYS_NORMALIZE) {
34         children = normalizeChildren(children)
35     } else if ( === SIMPLE_NORMALIZE) {
36         children = simpleNormalizeChildren(children)
37     }
38     // 创建VNode
39     ...
40 }

```

可以看到 `_createElement` 接收5个参数：

- `context` 表示 `VNode` 的上下文环境，是 `Component` 类型
- `tag` 表示标签，它可以是一个字符串，也可以是一个 `Component`
- `data` 表示 `VNode` 的数据，它是一个 `VNodeData` 类型
- `children` 表示当前 `VNode` 的子节点，它是任意类型的
- `normalizationType` 表示子节点规范的类型，类型不同规范的方法也就不一样，主要是参考 `render` 函数是编译生成的还是用户手写的

根据 `normalizationType` 的类型，`children` 会有不同的定义

```

1  if (normalizationType === ALWAYS_NORMALIZE) {
2      children = normalizeChildren(children)
3  } else if ( === SIMPLE_NORMALIZE) {
4      children = simpleNormalizeChildren(children)
5  }

```

`simpleNormalizeChildren` 方法调用场景是 `render` 函数是编译生成的

`normalizeChildren` 方法调用场景分为下面两种：

- `render` 函数是用户手写的
- 编译 `slot`、`v-for` 的时候会产生嵌套数组

无论是 `simpleNormalizeChildren` 还是 `normalizeChildren` 都是对 `children` 进行规范（使 `children` 变成了一个类型为 `VNode` 的 `Array`），这里就不展开说了

规范化 `children` 的源码位置在：`src/core/vdom/helpers/normalize-children.js`

在规范化 `children` 后，就去创建 `VNode`

```
1  let vnode, ns
2  // 对tag进行判断
3  if (typeof tag === 'string') {
4    let Ctor
5    ns = (context.$vnode && context.$vnode.ns) || config.getTagNamespace(tag)
6    if (config.isReservedTag(tag)) {
7      // 如果是内置的节点，则直接创建一个普通VNode
8      vnode = new VNode(
9        config.parsePlatformTagName(tag), data, children,
10       undefined, undefined, context
11     )
12   } else if (isDef(Ctor = resolveAsset(context.$options, 'components', tag)))
13   {
14     // component
15     // 如果是component类型，则会通过createComponent创建VNode节点
16     vnode = createComponent(Ctor, data, context, children, tag)
17   } else {
18     vnode = new VNode(
19       tag, data, children,
20       undefined, undefined, context
21     )
22   } else {
23     // direct component options / constructor
24     vnode = createComponent(tag, data, context, children)
25   }
```

`createComponent` 同样是创建 `VNode`

源码位置：`src/core/vdom/create-component.js`

```
1  export function createComponent (
2    Ctor: Class<Component> | Function | Object | void,
3    data: ?VNodeData,
4    context: Component,
```

```

5     children: ?Array<VNode>,
6     tag?: string
7 ): VNode | Array<VNode> | void {
8     if (isUndef(Ctor)) {
9         return
10    }
11    // 构建子类构造函数
12    const baseCtor = context.$options._base
13
14    // plain options object: turn it into a constructor
15    if (isObject(Ctor)) {
16        Ctor = baseCtor.extend(Ctor)
17    }
18
19    // if at this stage it's not a constructor or an async component factory,
20    // reject.
21    if (typeof Ctor !== 'function') {
22        if (process.env.NODE_ENV !== 'production') {
23            warn(`Invalid Component definition: ${String(Ctor)}`, context)
24        }
25        return
26    }
27
28    // async component
29    let asyncFactory
30    if (isUndef(Ctor.cid)) {
31        asyncFactory = Ctor
32        Ctor = resolveAsyncComponent(asyncFactory, baseCtor, context)
33        if (Ctor === undefined) {
34            return createAsyncPlaceholder(
35                asyncFactory,
36                data,
37                context,
38                children,
39                tag
40            )
41        }
42    }
43
44    data = data || {}
45
46    // resolve constructor options in case global mixins are applied after
47    // component constructor creation
48    resolveConstructorOptions(Ctor)
49
50    // transform component v-model data into props & events
51    if (isDef(data.model)) {

```

```

52     transformModel(Ctor.options, data)
53   }
54
55   // extract props
56   const propsData = extractPropsFromVNodeData(data, Ctor, tag)
57
58   // functional component
59   if (isTrue(Ctor.options.functional)) {
60     return createFunctionalComponent(Ctor, propsData, data, context, children)
61   }
62
63   // extract listeners, since these needs to be treated as
64   // child component listeners instead of DOM listeners
65   const listeners = data.on
66   // replace with listeners with .native modifier
67   // so it gets processed during parent component patch.
68   data.on = data.nativeOn
69
70   if (isTrue(Ctor.options.abstract)) {
71     const slot = data.slot
72     data = {}
73     if (slot) {
74       data.slot = slot
75     }
76   }
77
78   // 安装组件钩子函数，把钩子函数合并到data.hook中
79   installComponentHooks(data)
80
81   //实例化一个VNode返回。组件的VNode是没有children的
82   const name = Ctor.options.name || tag
83   const vnode = new VNode(
84     `vue-component-${Ctor.cid}${name ? `-${name}` : ''}`,
85     data, undefined, undefined, undefined, context,
86     { Ctor, propsData, listeners, tag, children },
87     asyncFactory
88   )
89   if (__WEEX__ && isRecyclableComponent(vnode)) {
90     return renderRecyclableComponentTemplate(vnode)
91   }
92
93   return vnode
94 }

```

稍微提下 `createComponent` 生成 `VNode` 的三个关键流程：

- 构造子类构造函数 `Ctor`
- `installComponentHooks` 安装组件钩子函数
- 实例化 `vnode`

小结

`createElement` 创建 `VNode` 的过程，每个 `VNode` 有 `children`，`children` 每个元素也是一个 `VNode`，这样就形成了一个虚拟树结构，用于描述真实的 `DOM` 树结构

第33题：说说你对Vue中 keep-alive 的理解

一、Keep-alive 是什么

`keep-alive` 是 `vue` 中的内置组件，能在组件切换过程中将状态保留在内存中，防止重复渲染 `DOM`

`keep-alive` 包裹动态组件时，会缓存不活动的组件实例，而不是销毁它们

`keep-alive` 可以设置以下 `props` 属性：

- `include` 字符串或正则表达式。只有名称匹配的组件会被缓存
- `exclude` 字符串或正则表达式。任何名称匹配的组件都不会被缓存
- `max` 数字。最多可以缓存多少组件实例

关于 `keep-alive` 的基本用法：

```
1 <keep-alive>
2   <component :is="view"></component>
3 </keep-alive>
```

使用 `includes` 和 `exclude`：

```
1 <keep-alive include="a,b">
2   <component :is="view"></component>
3 </keep-alive>
4
5 <!-- 正则表达式（使用 `v-bind`） -->
6 <keep-alive :include="/a|b/">
7   <component :is="view"></component>
8 </keep-alive>
9
10 <!-- 数组（使用 `v-bind`） -->
11 <keep-alive :include="['a', 'b']">
```

```
12     <component :is="view"></component>
13 </keep-alive>
```

匹配首先检查组件自身的 `name` 选项, 如果 `name` 选项不可用, 则匹配它的局部注册名称 (父组件 `components` 选项的键值), 匿名组件不能被匹配

设置了 `keep-alive` 缓存的组件, 会多出两个生命周期钩子 (`activated` 与 `deactivated`) :

- 首次进入组件时: `beforeRouteEnter` > `beforeCreate` > `created` > `mounted` > `activated` ... > `beforeRouteLeave` > `deactivated`
- 再次进入组件时: `beforeRouteEnter` > `activated` ... > `beforeRouteLeave` > `deactivated`

二、使用场景

使用原则: 当我们在某些场景下不需要让页面重新加载时我们可以使用 `keep-alive`

举个栗子:

当我们从 首页 -> 列表页 -> 商详页 -> 再返回, 这时候列表页应该是需要 `keep-alive`

从 首页 -> 列表页 -> 商详页 -> 返回到列表页 (需要缓存) -> 返回到首页 (需要缓存) -> 再次进入列表页 (不需要缓存), 这时候可以按需来控制页面的 `keep-alive`

在路由中设置 `keepAlive` 属性判断是否需要缓存

```
1  {
2    path: 'list',
3    name: 'itemList', // 列表页
4    component (resolve) {
5      require(['@/pages/item/list'], resolve)
6    },
7    meta: {
8      keepAlive: true,
9      title: '列表页'
10   }
11 }
```

使用 `<keep-alive>`

```
1 <div id="app" class='wrapper'>
2   <keep-alive>
3     <!-- 需要缓存的视图组件 -->
4     <router-view v-if="$route.meta.keepAlive"></router-view>
5   </keep-alive>
```

```
6      <!-- 不需要缓存的视图组件 -->
7      <router-view v-if="!$route.meta.keepAlive"></router-view>
8  </div>
```

三、原理分析

`keep-alive` 是 `vue` 中内置的一个组件

源码位置: `src/core/components/keep-alive.js`

```
1  export default {
2    name: 'keep-alive',
3    abstract: true,
4
5    props: {
6      include: [String, RegExp, Array],
7      exclude: [String, RegExp, Array],
8      max: [String, Number]
9    },
10
11    created () {
12      this.cache = Object.create(null)
13      this.keys = []
14    },
15
16    destroyed () {
17      for (const key in this.cache) {
18        pruneCacheEntry(this.cache, key, this.keys)
19      }
20    },
21
22    mounted () {
23      this.$watch('include', val => {
24        pruneCache(this, name => matches(val, name))
25      })
26      this.$watch('exclude', val => {
27        pruneCache(this, name => !matches(val, name))
28      })
29    },
30
31    render() {
32      /* 获取默认插槽中的第一个组件节点 */
33      const slot = this.$slots.default
34      const vnode = getFirstComponentChild(slot)
35      /* 获取该组件节点的componentOptions */
36      const componentOptions = vnode && vnode.componentOptions
```

```

37
38     if (componentOptions) {
39         /* 获取该组件节点的名称，优先获取组件的name字段，如果name不存在则获取组件的tag
        */
40         const name = getComponentName(componentOptions)
41
42         const { include, exclude } = this
43         /* 如果name不在include中或者存在于exclude中则表示不缓存，直接返回vnode */
44         if (
45             (include && (!name || !matches(include, name))) ||
46             // excluded
47             (exclude && name && matches(exclude, name))
48         ) {
49             return vnode
50         }
51
52         const { cache, keys } = this
53         /* 获取组件的key值 */
54         const key = vnode.key == null
55             // same constructor may get registered as different local components
56             // so cid alone is not enough (#3269)
57             ? componentOptions.Ctor.cid + (componentOptions.tag ?
58 `:::${componentOptions.tag}` : '')
59             : vnode.key
60         /* 拿到key值后去this.cache对象中寻找是否有该值，如果有则表示该组件有缓存，即命
        中缓存 */
61         if (cache[key]) {
62             vnode.componentInstance = cache[key].componentInstance
63             // make current key freshest
64             remove(keys, key)
65             keys.push(key)
66         }
67         /* 如果没有命中缓存，则将其设置进缓存 */
68         else {
69             cache[key] = vnode
70             keys.push(key)
71             // prune oldest entry
72             /* 如果配置了max并且缓存的长度超过了this.max，则从缓存中删除第一个 */
73             if (this.max && keys.length > parseInt(this.max)) {
74                 pruneCacheEntry(cache, keys[0], keys, this._vnode)
75             }
76         }
77         vnode.data.keepAlive = true
78     }
79     return vnode || (slot && slot[0])
80 }

```

```
81 }
```

可以看到该组件没有 `template`，而是用了 `render`，在组件渲染的时候会自动执行 `render` 函数
`this.cache` 是一个对象，用来存储需要缓存的组件，它将以如下形式存储：

```
1  this.cache = {  
2    'key1': '组件1',  
3    'key2': '组件2',  
4    // ...  
5  }
```

在组件销毁的时候执行 `pruneCacheEntry` 函数

```
1  function pruneCacheEntry (  
2    cache: VNodeCache,  
3    key: string,  
4    keys: Array<string>,  
5    current?: VNode  
6  ) {  
7    const cached = cache[key]  
8    /* 判断当前没有处于被渲染状态的组件，将其销毁*/  
9    if (cached && (!current || cached.tag !== current.tag)) {  
10      cached.componentInstance.$destroy()  
11    }  
12    cache[key] = null  
13    remove(keys, key)  
14  }
```

在 `mounted` 钩子函数中观测 `include` 和 `exclude` 的变化，如下：

```
1  mounted () {  
2    this.$watch('include', val => {  
3      pruneCache(this, name => matches(val, name))  
4    })  
5    this.$watch('exclude', val => {  
6      pruneCache(this, name => !matches(val, name))  
7    })  
8  }
```

如果 `include` 或 `exclude` 发生了变化，即表示定义需要缓存的组件的规则或者不需要缓存的组件的规则发生了变化，那么就执行 `pruneCache` 函数，函数如下：

```
1 function pruneCache (keepAliveInstance, filter) {
2   const { cache, keys, _vnode } = keepAliveInstance
3   for (const key in cache) {
4     const cachedNode = cache[key]
5     if (cachedNode) {
6       const name = getComponentName(cachedNode.componentOptions)
7       if (name && !filter(name)) {
8         pruneCacheEntry(cache, key, keys, _vnode)
9       }
10    }
11  }
12 }
```

在该函数内对 `this.cache` 对象进行遍历，取出每一项的 `name` 值，用其与新的缓存规则进行匹配，如果匹配不上，则表示在新的缓存规则下该组件已经不需要被缓存，则调用 `pruneCacheEntry` 函数将其从 `this.cache` 对象剔除即可

关于 `keep-alive` 的最强大缓存功能是在 `render` 函数中实现

首先获取组件的 `key` 值：

```
1 const key = vnode.key == null?
2   componentOptions.Ctor.cid + (componentOptions.tag ?
3     `::${componentOptions.tag}` : '')
4   : vnode.key
```

拿到 `key` 值后去 `this.cache` 对象中去寻找是否有该值，如果有则表示该组件有缓存，即命中缓存，如下：

```
1  /* 如果命中缓存，则直接从缓存中拿 vnode 的组件实例 */
2  if (cache[key]) {
3    vnode.componentInstance = cache[key].componentInstance
4    /* 调整该组件key的顺序，将其从原来的地方删掉并重新放在最后一个 */
5    remove(keys, key)
6    keys.push(key)
7  }
```

直接从缓存中拿 `vnode` 的组件实例，此时重新调整该组件 `key` 的顺序，将其从原来的地方删掉并重新放在 `this.keys` 中最后一个

`this.cache` 对象中没有该 `key` 值的情况，如下：

```
1  /* 如果没有命中缓存，则将其设置进缓存 */
2  else {
3      cache[key] = vnode
4      keys.push(key)
5      /* 如果配置了max并且缓存的长度超过了this.max，则从缓存中删除第一个 */
6      if (this.max && keys.length > parseInt(this.max)) {
7          pruneCacheEntry(cache, keys[0], keys, this._vnode)
8      }
9  }
```

表明该组件还没有被缓存过，则以该组件的 `key` 为键，组件 `vnode` 为值，将其存入 `this.cache` 中，并且把 `key` 存入 `this.keys` 中

此时再判断 `this.keys` 中缓存组件的数量是否超过了设置的最大缓存数量值 `this.max`，如果超过了，则把第一个缓存组件删掉

四、思考题：缓存后如何获取数据

解决方案可以有以下两种：

- `beforeRouteEnter`
- `activated`

`beforeRouteEnter`

每次组件渲染的时候，都会执行 `beforeRouteEnter`

```
1  beforeRouteEnter(to, from, next){
2      next(vm=>{
3          console.log(vm)
4          // 每次进入路由执行
5          vm.getData() // 获取数据
6      })
7  },
```

`activated`

在 `keep-alive` 缓存的组件被激活的时候，都会执行 `activated` 钩子

```
1  activated(){
2      this.getData() // 获取数据
3  },
```

注意：服务器端渲染期间 `activated` 不被调用

第34题：Vue.observable是什么？

一、Observable 是什么

`Observable` 翻译过来我们可以理解成可观察的

我们先来看一下其在 `Vue` 中的定义

`Vue.observable`，让一个对象变成响应式数据。`Vue` 内部会用它来处理 `data` 函数返回的对象

返回的对象可以直接用于渲染函数和计算属性内，并且会在发生变更时触发相应的更新。也可以作为最小化的跨组件状态存储器

```
1  Vue.observable({ count : 1})
```

其作用等同于

```
1  new Vue({ count : 1})
```

在 `Vue 2.x` 中，被传入的对象会直接被 `Vue.observable` 变更，它和被返回的对象是同一个对象

在 `Vue 3.x` 中，则会返回一个可响应的代理，而对源对象直接进行变更仍然是不可响应的

二、使用场景

在非父子组件通信时，可以使用通常的 `bus` 或者使用 `vuex`，但是实现的功能不是太复杂，而使用上面两个又有点繁琐。这时，`observable` 就是一个很好的选择

创建一个 `js` 文件

```
1  // 引入vue
2  import Vue from 'vue'
3  // 创建state对象，使用observable让state对象可响应
```

```

4   export let state = Vue.observable({
5     name: '张三',
6     'age': 38
7   })
8   // 创建对应的方法
9   export let mutations = {
10    changeName(name) {
11      state.name = name
12    },
13    setAge(age) {
14      state.age = age
15    }
16  }

```

在 `.vue` 文件中直接使用即可

```

1   <template>
2     <div>
3       姓名: {{ name }}
4       年龄: {{ age }}
5       <button @click="changeName('李四')">改变姓名</button>
6       <button @click="setAge(18)">改变年龄</button>
7     </div>
8   </template>
9   import { state, mutations } from '@store'
10  export default {
11    // 在计算属性中拿到值
12    computed: {
13      name() {
14        return state.name
15      },
16      age() {
17        return state.age
18      }
19    },
20    // 调用mutations里面的方法，更新数据
21    methods: {
22      changeName: mutations.changeName,
23      setAge: mutations.setAge
24    }
25  }

```

三、原理分析

源码位置: src\core\observer\index.js

```
1  export function observe (value: any, asRootData: ?boolean): Observer | void {
2    if (!isObject(value) || value instanceof VNode) {
3      return
4    }
5    let ob: Observer | void
6    // 判断是否存在__ob__响应式属性
7    if (hasOwn(value, '__ob__') && value.__ob__ instanceof Observer) {
8      ob = value.__ob__
9    } else if (
10      shouldObserve &&
11      !isServerRendering() &&
12      (Array.isArray(value) || isPlainObject(value)) &&
13      Object.isExtensible(value) &&
14      !value._isVue
15    ) {
16      // 实例化Observer响应式对象
17      ob = new Observer(value)
18    }
19    if (asRootData && ob) {
20      ob.vmCount++
21    }
22    return ob
23  }
```

Observer 类

```
1  export class Observer {
2    value: any;
3    dep: Dep;
4    vmCount: number; // number of vms that have this object as root $data
5
6    constructor (value: any) {
7      this.value = value
8      this.dep = new Dep()
9      this.vmCount = 0
10     def(value, '__ob__', this)
11     if (Array.isArray(value)) {
12       if (hasProto) {
13         protoAugment(value, arrayMethods)
14       } else {
15         copyAugment(value, arrayMethods, arrayKeys)
16       }
17     }
18   }
```

```

17         this.observeArray(value)
18     } else {
19         // 实例化对象是一个对象，进入walk方法
20         this.walk(value)
21     }
22 }

```

walk 函数

```

1  walk (obj: Object) {
2      const keys = Object.keys(obj)
3      // 遍历key，通过defineReactive创建响应式对象
4      for (let i = 0; i < keys.length; i++) {
5          defineReactive(obj, keys[i])
6      }
7  }

```

defineReactive 方法

```

1  export function defineReactive (
2      obj: Object,
3      key: string,
4      val: any,
5      customSetter?: ?Function,
6      shallow?: boolean
7  ) {
8      const dep = new Dep()
9
10     const property = Object.getOwnPropertyDescriptor(obj, key)
11     if (property && property.configurable === false) {
12         return
13     }
14
15     // cater for pre-defined getter/setters
16     const getter = property && property.get
17     const setter = property && property.set
18     if ((!getter || setter) && arguments.length === 2) {
19         val = obj[key]
20     }
21
22     let childOb = !shallow && observe(val)
23     // 接下来调用Object.defineProperty()给对象定义响应式属性
24     Object.defineProperty(obj, key, {

```

```

25     enumerable: true,
26     configurable: true,
27     get: function reactiveGetter () {
28         const value = getter ? getter.call(obj) : val
29         if (Dep.target) {
30             dep.depend()
31             if (childOb) {
32                 childOb.dep.depend()
33                 if (Array.isArray(value)) {
34                     dependArray(value)
35                 }
36             }
37         }
38         return value
39     },
40     set: function reactiveSetter (newVal) {
41         const value = getter ? getter.call(obj) : val
42         /* eslint-disable no-self-compare */
43         if (newVal === value || (newVal !== newVal && value !== value)) {
44             return
45         }
46         /* eslint-enable no-self-compare */
47         if (process.env.NODE_ENV !== 'production' && customSetter) {
48             customSetter()
49         }
50         // #7981: for accessor properties without setter
51         if (getter && !setter) return
52         if (setter) {
53             setter.call(obj, newVal)
54         } else {
55             val = newVal
56         }
57         childOb = !shallow && observe(newVal)
58         // 对观察者watchers进行通知,state就成了全局响应式对象
59         dep.notify()
60     }
61 })
62 }

```

第35题：说说你对slot的理解？ slot使用场景有哪些？

一、slot是什么

在HTML中 `slot` 元素，作为 `Web Components` 技术套件的一部分，是Web组件内的一个占位符。该占位符可以在后期使用自己的标记语言填充。

举个栗子

```
1 <template id="element-details-template">
2   <slot name="element-name">Slot template</slot>
3 </template>
4 <element-details>
5   <span slot="element-name">1</span>
6 </element-details>
7 <element-details>
8   <span slot="element-name">2</span>
9 </element-details>
```

`template` 不会展示到页面中，需要用先获取它的引用，然后添加到 `DOM` 中，

```
1 customElements.define('element-details',
2   class extends HTMLElement {
3     constructor() {
4       super();
5       const template = document
6         .getElementById('element-details-template')
7         .content;
8       const shadowRoot = this.attachShadow({mode: 'open'})
9         .appendChild(template.cloneNode(true));
10    }
11  })
```

在 `Vue` 中的概念也是如此

`Slot` 又名插槽，花名“占坑”，我们可以理解为 `slot` 在组件模板中占好了位置，当使用该组件标签时候，组件标签里面的内容就会自动填坑（替换组件模板中 `slot` 位置），作为承载分发内容的出口

可以将其类比为插卡式的FC游戏机，游戏机暴露卡槽（插槽）让用户插入不同的游戏磁条（自定义内容）

放张图感受一下



二、使用场景

通过插槽可以让用户可以拓展组件，去更好地复用组件和对其做定制化处理

如果父组件在使用到一个复用组件的时候，获取这个组件在不同的地方有少量的更改，如果去重写组件是一件不明智的事情

通过 `slot` 插槽向组件内部指定位置传递内容，完成这个复用组件在不同场景的应用

比如布局组件、表格列、下拉选、弹框显示内容等

三、分类

`slot` 可以分来以下三种：

- 默认插槽
- 具名插槽
- 作用域插槽

默认插槽

子组件用 `<slot>` 标签来确定渲染的位置，标签里面可以放 `DOM` 结构，当父组件使用的时候没有往插槽传入内容，标签内 `DOM` 结构就会显示在页面

父组件在使用的时候，直接在子组件的标签内写入内容即可

子组件 `Child.vue`

```
1 <template>
2   <slot>
3     <p>插槽后备的内容</p>
4   </slot>
5 </template>
```

父组件

```
1 <Child>
2   <div>默认插槽</div>
3 </Child>
```

具名插槽

子组件用 `name` 属性来表示插槽的名字，不传为默认插槽

父组件中在使用时在默认插槽的基础上加上 `slot` 属性，值为子组件插槽 `name` 属性值

子组件 `Child.vue`

```
1 <template>
2   <slot>插槽后备的内容</slot>
3   <slot name="content">插槽后备的内容</slot>
4 </template>
```

父组件

```
1 <child>
2   <template v-slot:default>具名插槽</template>
3   <!-- 具名插槽用插槽名做参数 -->
4   <template v-slot:content>内容...</template>
5 </child>
```

作用域插槽

子组件在作用域上绑定属性来将子组件的信息传给父组件使用，这些属性会被挂在父组件 `v-slot` 接受的对象上

父组件中在使用时通过 `v-slot:`（简写：`#`）获取子组件的信息，在内容中使用子组件 `Child.vue`

```
1 <template>
2   <slot name="footer" testProps="子组件的值">
3     <h3>没传footer插槽</h3>
4   </slot>
5 </template>
```

父组件

```
1 <child>
2   <!-- 把v-slot的值指定为作用域上下文对象 -->
3   <template v-slot:default="slotProps">
4     来自子组件数据：{{slotProps.testProps}}
5   </template>
6   <template #default="slotProps">
7     来自子组件数据：{{slotProps.testProps}}
8   </template>
9 </child>
```

小结：

- `v-slot` 属性只能在 `<template>` 上使用，但在只有默认插槽时可以在组件标签上使用
- 默认插槽名为 `default`，可以省略`default`直接写 `v-slot`
- 缩写为 `#` 时不能不写参数，写成 `#default`
- 可以通过解构获取 `v-slot={user}`，还可以重命名 `v-slot="{user: newName}"` 和定义默认值 `v-slot="{user = '默认值'}"`

四、原理分析

`slot` 本质上是返回 `VNode` 的函数，一般情况下，`Vue` 中的组件要渲染到页面上需要经过 `template -> render function -> VNode -> DOM` 过程，这里看看 `slot` 如何实现：

编写一个 `buttonCounter` 组件，使用匿名插槽

```
1 Vue.component('button-counter', {
2   template: '<div> <slot>我是默认内容</slot></div>'
3 })
```

使用该组件

```
1  new Vue({
2    el: '#app',
3    template: '<button-counter><span>我是slot传入内容</span></button-counter>',
4    components: {buttonCounter}
5  })
```

获取 buttonCounter 组件渲染函数

```
1  (function anonymous(
2  ) {
3    with(this){return _c('div',[_t("default",[_v("我是默认内容")]),2])}
4  })
```

`_v` 表示穿件普通文本节点，`_t` 表示渲染插槽的函数

渲染插槽函数 `renderSlot` （做了简化）

```
1  function renderSlot (
2    name,
3    fallback,
4    props,
5    bindObject
6  ) {
7    // 得到渲染插槽内容的函数
8    var scopedSlotFn = this.$scopedSlots[name];
9    var nodes;
10   // 如果存在插槽渲染函数，则执行插槽渲染函数，生成nodes节点返回
11   // 否则使用默认值
12   nodes = scopedSlotFn(props) || fallback;
13   return nodes;
14 }
```

`name` 属性表示定义插槽的名字，默认值为 `default`，`fallback` 表示子组件中的 `slot` 节点的默认值

关于 `this.$scopedSlots` 是什么，我们可以先看看 `vm.slot`

```
1  function initRender (vm) {
2    ...
```

```

3    vm.$slots = resolveSlots(options._renderChildren, renderContext);
4    ...
5  }

```

`resolveSlots` 函数会对 `children` 节点做归类 and 过滤处理, 返回 `slots`

```

1  function resolveSlots (
2    children,
3    context
4  ) {
5    if (!children || !children.length) {
6      return {}
7    }
8    var slots = {};
9    for (var i = 0, l = children.length; i < l; i++) {
10     var child = children[i];
11     var data = child.data;
12     // remove slot attribute if the node is resolved as a Vue slot node
13     if (data && data.attrs && data.attrs.slot) {
14       delete data.attrs.slot;
15     }
16     // named slots should only be respected if the vnode was rendered in the
17     // same context.
18     if ((child.context === context || child.fnContext === context) &&
19       data && data.slot !== null
20     ) {
21       // 如果slot存在(slot="header") 则拿对应的值作为key
22       var name = data.slot;
23       var slot = (slots[name] || (slots[name] = []));
24       // 如果是tempalte元素 则把template的children添加进数组中, 这也就是为什么你写
25       // 的template标签并不会渲染成另一个标签到页面
26       if (child.tag === 'template') {
27         slot.push.apply(slot, child.children || []);
28       } else {
29         slot.push(child);
30       }
31     } else {
32       // 如果没有就默认是default
33       (slots.default || (slots.default = [])).push(child);
34     }
35     // ignore slots that contains only whitespace
36     for (var name$1 in slots) {
37       if (slots[name$1].every(isWhitespace)) {
38         delete slots[name$1];

```

```
39     }
40   }
41   return slots
42 }
```

`_render` 渲染函数通过 `normalizeScopedSlots` 得到 `vm.$scopedSlots`

```
1  vm.$scopedSlots = normalizeScopedSlots(
2    _parentVnode.data.scopedSlots,
3    vm.$slots,
4    vm.$scopedSlots
5  );
```

作用域插槽中父组件能够得到子组件的值是因为在 `renderSlot` 的时候执行会传入 `props`，也就是上述 `_t` 第三个参数，父组件则能够得到子组件传递过来的值

第36题：说说你对vue的mixin的理解，以及有哪些应用场景？

一、mixin是什么

`Mixin` 是面向对象程序设计语言中的类，提供了方法的实现。其他类可以访问 `mixin` 类的方法而不必成为其子类

`Mixin` 类通常作为功能模块使用，在需要该功能时“混入”，有利于代码复用又避免了多继承的复杂

Vue中的mixin

先来看一下官方定义

`mixin`（混入），提供了一种非常灵活的方式，来分发 `Vue` 组件中的可复用功能。

本质其实就是一个 `js` 对象，它可以包含我们组件中任意功能选项，如 `data`、`components`、`methods`、`created`、`computed` 等等

我们只要将共用的功能以对象的方式传入 `mixins` 选项中，当组件使用 `mixins` 对象时所有 `mixins` 对象的选项都将被混入该组件本身的选项中来

在 `Vue` 中我们可以局部混入跟全局混入

局部混入

定义一个 `mixin` 对象，有组件 `options` 的 `data`、`methods` 属性

```
1  var myMixin = {
2    created: function () {
```

```

3     this.hello()
4   },
5   methods: {
6     hello: function () {
7       console.log('hello from mixin!')
8     }
9   }
10  }

```

组件通过 `mixins` 属性调用 `mixin` 对象

```

1  Vue.component('componentA',{
2    mixins: [myMixin]
3  })

```

该组件在使用的时候，混合了 `mixin` 里面的方法，在自动执行 `create` 生命钩子，执行 `hello` 方法

全局混入

通过 `Vue.mixin()` 进行全局的混入

```

1  Vue.mixin({
2    created: function () {
3      console.log("全局混入")
4    }
5  })

```

使用全局混入需要特别注意，因为它会影响到每一个组件实例（包括第三方组件）

PS：全局混入常用于插件的编写

注意事项：

当组件存在与 `mixin` 对象相同的选项的时候，进行递归合并的时候组件的选项会覆盖 `mixin` 的选项
但是如果相同选项为生命周期钩子的时候，会合并成一个数组，先执行 `mixin` 的钩子，再执行组件的钩子

二、使用场景

在日常的开发中，我们经常会遇到在不同的组件中经常会需要用到一些相同或者相似的代码，这些代码的功能相对独立

这时，可以通过 `Vue` 的 `mixin` 功能将相同或者相似的代码提出来
举个例子

定义一个 `modal` 弹窗组件，内部通过 `isShowing` 来控制显示

```
1  const Modal = {
2    template: '#modal',
3    data() {
4      return {
5        isShowing: false
6      }
7    },
8    methods: {
9      toggleShow() {
10        this.isShowing = !this.isShowing;
11      }
12    }
13  }
```

定义一个 `tooltip` 提示框，内部通过 `isShowing` 来控制显示

```
1  const Tooltip = {
2    template: '#tooltip',
3    data() {
4      return {
5        isShowing: false
6      }
7    },
8    methods: {
9      toggleShow() {
10        this.isShowing = !this.isShowing;
11      }
12    }
13  }
```

通过观察上面两个组件，发现两者的逻辑是相同，代码控制显示也是相同的，这时候 `mixin` 就派上用场了

首先抽出共同代码，编写一个 `mixin`

```
1  const toggle = {
2    data() {
```

```

3     return {
4       isShowing: false
5     },
6   },
7   methods: {
8     toggleShow() {
9       this.isShowing = !this.isShowing;
10    }
11  }
12 }

```

两个组件在使用上，只需要引入 `mixin`

```

1  const Modal = {
2    template: '#modal',
3    mixins: [toggle]
4  };
5
6  const Tooltip = {
7    template: '#tooltip',
8    mixins: [toggle]
9  }

```

通过上面小小的例子，让我们知道了 `Mixin` 对于封装一些可复用的功能如此有趣、方便、实用

三、源码分析

首先从 `Vue.mixin` 入手

源码位置： `/src/core/global-api/mixin.js`

```

1  export function initMixin (Vue: GlobalAPI) {
2    Vue.mixin = function (mixin: Object) {
3      this.options = mergeOptions(this.options, mixin)
4      return this
5    }
6  }

```

主要是调用 `mergeOptions` 方法

源码位置： `/src/core/util/options.js`

```

1  export function mergeOptions (
2    parent: Object,
3    child: Object,
4    vm?: Component
5  ): Object {
6
7    if (child.mixins) { // 判断有没有mixin 也就是mixin里面挂mixin的情况 有的话递归进行
      合并
8      for (let i = 0, l = child.mixins.length; i < l; i++) {
9        parent = mergeOptions(parent, child.mixins[i], vm)
10      }
11    }
12
13    const options = {}
14    let key
15    for (key in parent) {
16      mergeField(key) // 先遍历parent的key 调对应的strats[XXX]方法进行合并
17    }
18    for (key in child) {
19      if (!hasOwn(parent, key)) { // 如果parent已经处理过某个key 就不处理了
20        mergeField(key) // 处理child中的key 也就parent中没有处理过的key
21      }
22    }
23    function mergeField (key) {
24      const strat = strats[key] || defaultStrat
25      options[key] = strat(parent[key], child[key], vm, key) // 根据不同类型的
      options调用strats中不同的方法进行合并
26    }
27    return options
28  }

```

从上面的源码，我们得到以下几点：

- 优先递归处理 `mixins`
- 先遍历合并 `parent` 中的 `key`，调用 `mergeField` 方法进行合并，然后保存在变量 `options`
- 再遍历 `child`，合并补上 `parent` 中没有的 `key`，调用 `mergeField` 方法进行合并，保存在变量 `options`
- 通过 `mergeField` 函数进行了合并

下面是关于 `Vue` 的几种类型的合并策略

- 替换型
- 合并型
- 队列型

- 叠加型

替换型

替换型合并有 `props`、`methods`、`inject`、`computed`

```
1  strats.props =
2  strats.methods =
3  strats.inject =
4  strats.computed = function (
5    parentVal: ?Object,
6    childVal: ?Object,
7    vm?: Component,
8    key: string
9  ): ?Object {
10    if (!parentVal) return childVal // 如果parentVal没有值, 直接返回childVal
11    const ret = Object.create(null) // 创建一个第三方对象 ret
12    extend(ret, parentVal) // extend方法实际是把parentVal的属性复制到ret中
13    if (childVal) extend(ret, childVal) // 把childVal的属性复制到ret中
14    return ret
15  }
16  strats.provide = mergeDataOrFn
```

同名的 `props`、`methods`、`inject`、`computed` 会被后来者代替

合并型

和并型合并有: `data`

```
1  strats.data = function(parentVal, childVal, vm) {
2    return mergeDataOrFn(
3      parentVal, childVal, vm
4    )
5  };
6
7  function mergeDataOrFn(parentVal, childVal, vm) {
8    return function mergedInstanceDataFn() {
9      var childData = childVal.call(vm, vm) // 执行data挂的函数得到对象
10     var parentData = parentVal.call(vm, vm)
11     if (childData) {
12       return mergeData(childData, parentData) // 将2个对象进行合并
13     } else {
14       return parentData // 如果没有childData 直接返回parentData
15     }
16   }
17 }
```

```

16     }
17 }
18
19 function mergeData(to, from) {
20     if (!from) return to
21     var key, toVal, fromVal;
22     var keys = Object.keys(from);
23     for (var i = 0; i < keys.length; i++) {
24         key = keys[i];
25         toVal = to[key];
26         fromVal = from[key];
27         // 如果不存在这个属性, 就重新设置
28         if (!to.hasOwnProperty(key)) {
29             set(to, key, fromVal);
30         }
31         // 存在相同属性, 合并对象
32         else if (typeof toVal == "object" && typeof fromVal == "object") {
33             mergeData(toVal, fromVal);
34         }
35     }
36     return to
37 }

```

`mergeData` 函数遍历了要合并的 data 的所有属性, 然后根据不同情况进行合并:

- 当目标 data 对象不包含当前属性时, 调用 `set` 方法进行合并 (set 方法其实就是一些合并重新赋值的方法)
- 当目标 data 对象包含当前属性并且当前值为纯对象时, 递归合并当前对象值, 这样做是为了防止对象存在新增属性

队列性

队列性合并有: 全部生命周期和 `watch`

```

1 function mergeHook (
2     parentVal: ?Array<Function>,
3     childVal: ?Function | ?Array<Function>
4 ): ?Array<Function> {
5     return childVal
6         ? parentVal
7           ? parentVal.concat(childVal)
8             : Array.isArray(childVal)
9               ? childVal
10              : [childVal]
11         : parentVal

```

```

12  }
13
14  LIFECYCLE_HOOKS.forEach(hook => {
15    strats[hook] = mergeHook
16  })
17
18  // watch
19  strats.watch = function (
20    parentVal,
21    childVal,
22    vm,
23    key
24  ) {
25    // work around Firefox's Object.prototype.watch...
26    if (parentVal === nativeWatch) { parentVal = undefined; }
27    if (childVal === nativeWatch) { childVal = undefined; }
28    /* istanbul ignore if */
29    if (!childVal) { return Object.create(parentVal || null) }
30    {
31      assertObjectType(key, childVal, vm);
32    }
33    if (!parentVal) { return childVal }
34    var ret = {};
35    extend(ret, parentVal);
36    for (var key$1 in childVal) {
37      var parent = ret[key$1];
38      var child = childVal[key$1];
39      if (parent && !Array.isArray(parent)) {
40        parent = [parent];
41      }
42      ret[key$1] = parent
43        ? parent.concat(child)
44        : Array.isArray(child) ? child : [child];
45    }
46    return ret
47  };

```

生命周期钩子和 `watch` 被合并为一个数组，然后正序遍历一次执行

叠加型

叠加型合并有： `component`、`directives`、`filters`

```

1  strats.components=
2  strats.directives=

```

```

3
4   strats.filters = function mergeAssets(
5     parentVal, childVal, vm, key
6   ) {
7     var res = Object.create(parentVal || null);
8     if (childVal) {
9       for (var key in childVal) {
10         res[key] = childVal[key];
11       }
12     }
13     return res
14   }

```

叠加型主要是通过原型链进行层层叠加

小结：

- 替换型策略有 `props`、`methods`、`inject`、`computed`，就是将新的同名参数替代旧的参数
- 合并型策略是 `data`，通过 `set` 方法进行合并和重新赋值
- 队列型策略有生命周期函数和 `watch`，原理是将函数存入一个数组，然后正序遍历依次执行
- 叠加型有 `component`、`directives`、`filters`，通过原型链进行层层叠加

第37题：Vue中的\$nextTick有什么作用？

一、NextTick是什么

官方对其的定义

在下次 DOM 更新循环结束之后执行延迟回调。在修改数据之后立即使用这个方法，获取更新后的 DOM

什么意思呢？

我们可以理解成，Vue 在更新 DOM 时是异步执行的。当数据发生变化，Vue 将开启一个异步更新队列，视图需要等队列中所有数据变化完成之后，再统一进行更新

举例一下

Html 结构

```

1   <div id="app"> {{ message }} </div>

```

构建一个 `vue` 实例

```
1  const vm = new Vue({
2    el: '#app',
3    data: {
4      message: '原始值'
5    }
6  })
```

修改 `message`

```
1  this.message = '修改后的值1'
2  this.message = '修改后的值2'
3  this.message = '修改后的值3'
```

这时候想获取页面最新的 `DOM` 节点，却发现获取到的是旧值

```
1  console.log(vm.$el.textContent) // 原始值
```

这是因为 `message` 数据在发现变化的时候，`vue` 并不会立刻去更新 `Dom`，而是将修改数据的操作放在了一个异步操作队列中

如果我们一直修改相同数据，异步操作队列还会进行去重

等待同一事件循环中的所有数据变化完成之后，会将队列中的事件拿来进行处理，进行 `DOM` 的更新

为什么要有nexttick

举个例子

```
1  {{num}}
2  for(let i=0; i<100000; i++){
3    num = i
4  }
```

如果没有 `nextTick` 更新机制，那么 `num` 每次更新值都会触发视图更新(上面这段代码也就是会更新10万次视图)，有了 `nextTick` 机制，只需要更新一次，所以 `nextTick` 本质是一种优化策略

二、使用场景

如果想要在修改数据后立刻得到更新后的 `DOM` 结构，可以使用 `Vue.nextTick()`

第一个参数为：回调函数（可以获取最近的 `DOM` 结构）

第二个参数为：执行函数上下文

```
1  // 修改数据
2  vm.message = '修改后的值'
3  // DOM 还没有更新
4  console.log(vm.$el.textContent) // 原始的值
5  Vue.nextTick(function () {
6    // DOM 更新了
7    console.log(vm.$el.textContent) // 修改后的值
8  })
```

组件内使用 `vm.$nextTick()` 实例方法只需要通过 `this.$nextTick()`，并且回调函数中的 `this` 将自动绑定到当前的 `Vue` 实例上

```
1  this.message = '修改后的值'
2  console.log(this.$el.textContent) // => '原始的值'
3  this.$nextTick(function () {
4    console.log(this.$el.textContent) // => '修改后的值'
5  })
```

`$nextTick()` 会返回一个 `Promise` 对象，可以用 `async/await` 完成相同作用的事情

```
1  this.message = '修改后的值'
2  console.log(this.$el.textContent) // => '原始的值'
3  await this.$nextTick()
4  console.log(this.$el.textContent) // => '修改后的值'
```

三、实现原理

源码位置： `/src/core/util/next-tick.js`

`callbacks` 也就是异步操作队列

`callbacks` 新增回调函数后又执行了 `timerFunc` 函数，`pending` 是用来标识同一个时间只能执行一次

```
1  export function nextTick(cb?: Function, ctx?: Object) {
2    let _resolve;
```

```

3
4 // cb 回调函数会经统一处理压入 callbacks 数组
5 callbacks.push(() => {
6   if (cb) {
7     // 给 cb 回调函数执行加上了 try-catch 错误处理
8     try {
9       cb.call(ctx);
10    } catch (e) {
11      handleError(e, ctx, 'nextTick');
12    }
13  } else if (_resolve) {
14    _resolve(ctx);
15  }
16 });
17
18 // 执行异步延迟函数 timerFunc
19 if (!pending) {
20   pending = true;
21   timerFunc();
22 }
23
24 // 当 nextTick 没有传入函数参数的时候, 返回一个 Promise 化的调用
25 if (!cb && typeof Promise !== 'undefined') {
26   return new Promise(resolve => {
27     _resolve = resolve;
28   });
29 }
30 }

```

`timerFunc` 函数定义, 这里是根据当前环境支持什么方法则确定调用哪个, 分别有:

`Promise.then`、`MutationObserver`、`setImmediate`、`setTimeout`

通过上面任意一种方法, 进行降级操作

```

1 export let isUsingMicroTask = false
2 if (typeof Promise !== 'undefined' && isNative(Promise)) {
3   //判断1: 是否原生支持Promise
4   const p = Promise.resolve()
5   timerFunc = () => {
6     p.then(flushCallbacks)
7     if (isIOS) setTimeout(noop)
8   }
9   isUsingMicroTask = true
10 } else if (!isIE && typeof MutationObserver !== 'undefined' && (
11   isNative(MutationObserver) ||

```

```

12     MutationObserver.toString() === '[object MutationObserverConstructor]'
13   )) {
14     //判断2: 是否原生支持MutationObserver
15     let counter = 1
16     const observer = new MutationObserver(flushCallbacks)
17     const textNode = document.createTextNode(String(counter))
18     observer.observe(textNode, {
19       characterData: true
20     })
21     timerFunc = () => {
22       counter = (counter + 1) % 2
23       textNode.data = String(counter)
24     }
25     isUsingMicroTask = true
26   } else if (typeof setImmediate !== 'undefined' && isNative(setImmediate)) {
27     //判断3: 是否原生支持setImmediate
28     timerFunc = () => {
29       setImmediate(flushCallbacks)
30     }
31   } else {
32     //判断4: 上面都不行, 直接用setTimeout
33     timerFunc = () => {
34       setTimeout(flushCallbacks, 0)
35     }
36   }

```

无论是微任务还是宏任务，都会放到 `flushCallbacks` 使用

这里将 `callbacks` 里面的函数复制一份，同时 `callbacks` 置空

依次执行 `callbacks` 里面的函数

```

1  function flushCallbacks () {
2    pending = false
3    const copies = callbacks.slice(0)
4    callbacks.length = 0
5    for (let i = 0; i < copies.length; i++) {
6      copies[i]()
7    }
8  }

```

小结：

1. 把回调函数放入callbacks等待执行
2. 将执行函数放到微任务或者宏任务中

3. 事件循环到了微任务或者宏任务，执行函数依次执行callbacks中的回调

第38题：Vue组件间通信方式都有哪些？

一、组件间通信的概念

开始之前，我们把组件间通信这个词进行拆分

- 组件
- 通信

都知道组件是 `vue` 最强大的功能之一，`vue` 中每一个 `.vue` 我们都可以视之为一个组件。

通信指的是发送者通过某种媒体以某种格式来传递信息到受信者以达到某个目的。

广义上，任何信息的交通都是通信。

组件间通信，即指组件(.vue)通过某种方式来传递信息以达到某个目的。

举个栗子我们在使用 `UI` 框架中的 `table` 组件，可能会往 `table` 组件中传入某些数据，这个本质就形成了组件之间的通信。

二、组件间通信解决了什么

在古代，人们通过驿站、飞鸽传书、烽火报警、符号、语言、眼神、触碰等方式进行信息传递，到了今天，随着科技水平的飞速发展，通信基本完全利用有线或无线电完成，相继出现了有线电话、固定电话、无线电话、手机、互联网甚至视频电话等各种通信方式从上面这段话，我们可以看到通信的本质是信息同步，共享。

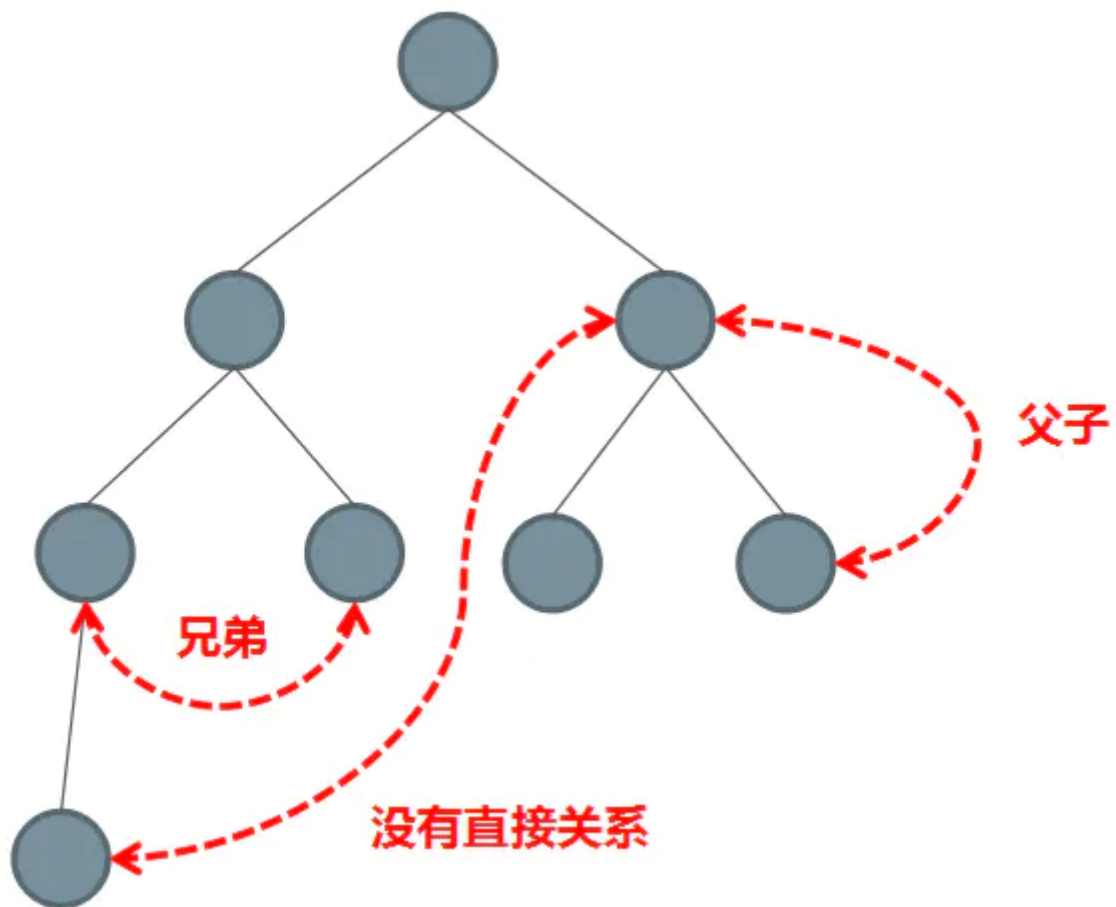
回到 `vue` 中，每个组件之间的都有独自的作用域，组件间的数据是无法共享的但实际开发工作中我们常常需要让组件之间共享数据，这也是组件通信的目的要让他们互相之间能进行通讯，这样才能构成一个有机的完整系统

二、组件间通信的分类

组件间通信的分类可以分成以下

- 父子组件之间的通信
- 兄弟组件之间的通信
- 祖孙与后代组件之间的通信
- 非关系组件间之间的通信

关系图:



三、组件间通信的方案

整理 `vue` 中8种常规的通信方案

1. 通过 props 传递
2. 通过 `$emit` 触发自定义事件
3. 使用 `ref`
4. EventBus
5. `parent` 或 `root`
6. `attrs` 与 `listeners`
7. Provide 与 Inject
8. Vuex

props传递数据



- 适用场景：父组件传递数据给子组件
- 子组件设置 `props` 属性，定义接收父组件传递过来的参数
- 父组件在使用子组件标签中通过字面量来传递值

`Children.vue`

```
1  props:{
2    // 字符串形式
3    name:String // 接收的类型参数
4    // 对象形式
5    age:{
6      type:Number, // 接收的类型为数值
7      defaule:18, // 默认值为18
8      require:true // age属性必须传递
9    }
10 }
```

`Father.vue` 组件

```
1  <Children name="jack" age=18 />
```

\$emit 触发自定义事件

- 适用场景：子组件传递数据给父组件

- 子组件通过 `$emit` 触发 自定义事件， `$emit` 第二个参数为传递的数值
- 父组件绑定监听器获取到子组件传递过来的参数

Chilfen.vue

```
1 this.$emit('add', good)
```

Father.vue

```
1 <Children @add="cartAdd($event)" />
```

ref

- 父组件在使用子组件的时候设置 `ref`
- 父组件通过设置子组件 `ref` 来获取数据

父组件

```
1 <Children ref="foo" />
2
3 this.$refs.foo // 获取子组件实例，通过子组件实例我们就能拿到对应的数据
```

EventBus

- 使用场景：兄弟组件传值
- 创建一个中央事件总线 `EventBus`
- 兄弟组件通过 `$emit` 触发自定义事件， `$emit` 第二个参数为传递的数值
- 另一个兄弟组件通过 `$on` 监听自定义事件

Bus.js

```
1 // 创建一个中央时间总线类
2 class Bus {
3   constructor() {
4     this.callbacks = {}; // 存放事件的名字
5   }
6   $on(name, fn) {
7     this.callbacks[name] = this.callbacks[name] || [];
```

```

8      this.callbacks[name].push(fn);
9    }
10    $emit(name, args) {
11      if (this.callbacks[name]) {
12        this.callbacks[name].forEach((cb) => cb(args));
13      }
14    }
15  }
16
17  // main.js
18  Vue.prototype.$bus = new Bus() // 将$bus挂载到vue实例的原型上
19  // 另一种方式
20  Vue.prototype.$bus = new Vue() // Vue已经实现了Bus的功能

```

Children1.vue

```

1  this.$bus.$emit('foo')

```

Children2.vue

```

1  this.$bus.$on('foo', this.handle)

```

parent 或 **root**

- 通过共同祖辈 `$parent` 或者 `$root` 搭建通信桥梁

兄弟组件

```

this.$parent.$on('add', this.add)

```

另一个兄弟组件

```

this.$parent.$emit('add')

```

attrs 与 **listeners**

- 适用场景：祖先传递数据给子孙
- 设置批量向下传属性 `$attrs` 和 `$listeners`
- 包含了父级作用域中不作为 `prop` 被识别 (且获取) 的特性绑定 (class 和 style 除外)。
- 可以通过 `v-bind="$attrs"` 传入内部组件

```

1 // child: 并未在props中声明foo
2 <p>{{attrs.foo}}</p>
3
4 // parent
5 <HelloWorld foo="foo"/>

```

```

1 // 给Grandson隔代传值, communication/index.vue
2 <Child2 msg="lalala" @some-event="onSomeEvent"></Child2>
3
4 // Child2做展开
5 <Grandson v-bind="$attrs" v-on="$listeners"></Grandson>
6
7 // Grandson使用
8 <div @click="$emit('some-event', 'msg from grandson')">
9   {{msg}}
10 </div>

```

provide 与 inject

- 在祖先组件定义 `provide` 属性，返回传递的值
- 在后代组件通过 `inject` 接收组件传递过来的值

祖先组件

```

1 provide(){
2   return {
3     foo:'foo'
4   }
5 }

```

后代组件

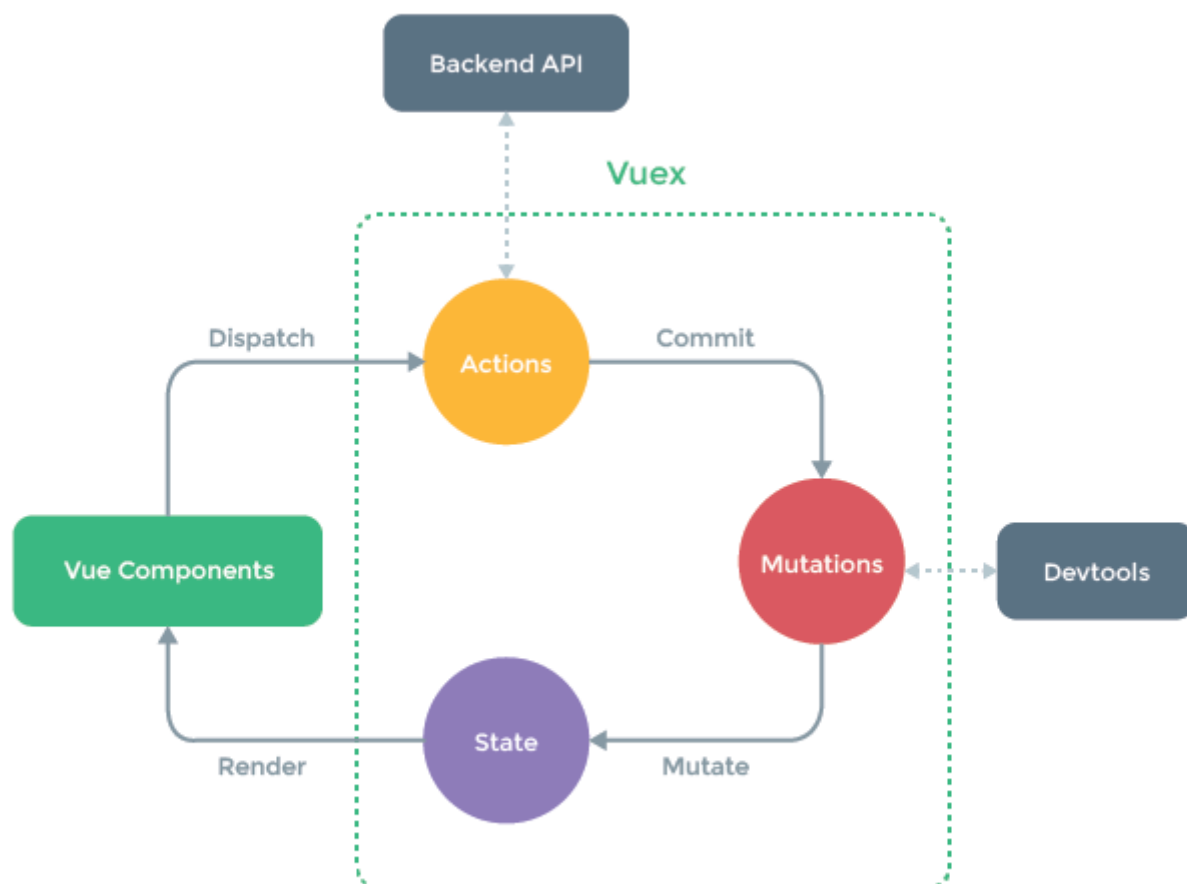
```

1 inject:['foo'] // 获取到祖先组件传递过来的值

```

vuex

- 适用场景: 复杂关系的组件数据传递
- `Vuex` 作用相当于一个用来存储共享变量的容器



- `state` 用来存放共享变量的地方
- `getter`，可以增加一个 `getter` 派生状态，(相当于 `store` 中的计算属性)，用来获得共享变量的值
- `mutations` 用来存放修改 `state` 的方法。
- `actions` 也是用来存放修改 `state` 的方法，不过 `action` 是在 `mutations` 的基础上进行。常用来做一些异步操作

小结

- 父子关系的组件数据传递选择 `props` 与 `$emit` 进行传递，也可选择 `ref`
- 兄弟关系的组件数据传递可选择 `$bus`，其次可以选择 `$parent` 进行传递
- 祖先与后代组件数据传递可选择 `attrs` 与 `listeners` 或者 `Provide` 与 `Inject`
- 复杂关系的组件数据传递可以通过 `vuex` 存放共享的变量

第39题：Vue中组件和插件有什么区别？

一、组件是什么

回顾以前对组件的定义：

组件就是把图形、非图形的各种逻辑均抽象为一个统一的概念（组件）来实现开发的模式，在 `Vue` 中每一个 `.vue` 文件都可以视为一个组件

组件的优势

- 降低整个系统的耦合度，在保持接口不变的情况下，我们可以替换不同的组件快速完成需求，例如输入框，可以替换为日历、时间、范围等组件作具体的实现
- 调试方便，由于整个系统是通过组件组合起来的，在出现问题的时候，可以用排除法直接移除组件，或者根据报错的组件快速定位问题，之所以能够快速定位，是因为每个组件之间低耦合，职责单一，所以逻辑会比分析整个系统要简单
- 提高可维护性，由于每个组件的职责单一，并且组件在系统中是被复用的，所以对代码进行优化可获得系统的整体升级

二、插件是什么

插件通常用来为 `Vue` 添加全局功能。插件的功能范围没有严格的限制——一般有下面几种：

- 添加全局方法或者属性。如： `vue-custom-element`
- 添加全局资源：指令/过滤器/过渡等。如 `vue-touch`
- 通过全局混入来添加一些组件选项。如 `vue-router`
- 添加 `Vue` 实例方法，通过把它们添加到 `Vue.prototype` 上实现。
- 一个库，提供自己的 `API`，同时提供上面提到的一个或多个功能。如 `vue-router`

三、两者的区别

两者的区别主要表现在以下几个方面：

- 编写形式
- 注册形式
- 使用场景

编写形式

编写组件

编写一个组件，可以有很多方式，我们最常见的就是 `vue` 单文件的这种格式，每一个 `.vue` 文件我们都可以看成是一个组件

`vue` 文件标准格式

```

1  <template>
2  </template>
3  <script>
4  export default{
5      ...
6  }
7  </script>
8  <style>
9  </style>

```

我们还可以通过 `template` 属性来编写一个组件，如果组件内容多，我们可以在外部定义 `template` 组件内容，如果组件内容并不多，我们可直接写在 `template` 属性上

```

1  <template id="testComponent">      // 组件显示的内容
2      <div>component!</div>
3  </template>
4
5  Vue.component('componentA',{
6      template: '#testComponent'
7      template: '<div>component</div>' // 组件内容少可以通过这种形式
8  })

```

编写插件

`vue` 插件的实现应该暴露一个 `install` 方法。这个方法的第一个参数是 `Vue` 构造器，第二个参数是一个可选的选项对象

```

1  MyPlugin.install = function (Vue, options) {
2      // 1. 添加全局方法或 property
3      Vue.myGlobalMethod = function () {
4          // 逻辑...
5      }
6
7      // 2. 添加全局资源
8      Vue.directive('my-directive', {
9          bind (el, binding, vnode, oldVnode) {
10             // 逻辑...
11         }
12         ...
13     })
14
15     // 3. 注入组件选项
16     Vue.mixin({

```

```

17     created: function () {
18         // 逻辑...
19     }
20     ...
21 })
22
23 // 4. 添加实例方法
24 Vue.prototype.$myMethod = function (methodOptions) {
25     // 逻辑...
26 }
27 }

```

注册形式

组件注册

`vue` 组件注册主要分为全局注册与局部注册

全局注册通过 `Vue.component` 方法，第一个参数为组件的名称，第二个参数为传入的配置项

```

1  Vue.component('my-component-name', { /* ... */ })

```

局部注册只需在用到的地方通过 `components` 属性注册一个组件

```

1  const component1 = {...} // 定义一个组件
2  export default {
3      components:{
4          component1    // 局部注册
5      }
6  }

```

插件注册

插件的注册通过 `Vue.use()` 的方式进行注册（安装），第一个参数为插件的名字，第二个参数是可选的配置项

```

1  Vue.use(插件名字,{ /* ... */ } )

```

注意的是：

注册插件的时候，需要在调用 `new Vue()` 启动应用之前完成

`Vue.use` 会自动阻止多次注册相同插件，只会注册一次

使用场景

具体的其实在插件是什么章节已经表述了，这里在总结一下

组件 (Component) 是用来构成你的 `App` 的业务模块，它的目标是 `App.vue`

插件 (Plugin) 是用来增强你的技术栈的功能模块，它的目标是 `Vue` 本身

简单来说，插件就是指对 `Vue` 的功能的增强或补充

第40题：为什么Vue中的data属性是一个函数而不是一个对象？

一、实例和组件定义data的区别

`vue` 实例的时候定义 `data` 属性既可以是一个对象，也可以是一个函数

```
1  const app = new Vue({
2    el:"#app",
3    // 对象格式
4    data:{
5      foo:"foo"
6    },
7    // 函数格式
8    data(){
9      return {
10        foo:"foo"
11      }
12    }
13  })
```

组件中定义 `data` 属性，只能是一个函数

如果为组件 `data` 直接定义为一个对象

```
1  Vue.component('component1',{
2    template:`<div>组件</div>`,
3    data:{
4      foo:"foo"
5    }
6  })
```

则会得到警告信息

```
✖ [Vue warn]: The "data" option should be a function that returns a per-instance value in component definitions. vue.js:634
```

警告说明：返回的 `data` 应该是一个函数在每一个组件实例中

二、组件data定义函数与对象的区别

上面讲到组件 `data` 必须是一个函数，不知道大家有没有思考过这是为什么呢？

在我们定义好一个组件的时候，`vue` 最终都会通过 `Vue.extend()` 构成组件实例

这里我们模仿组件构造函数，定义 `data` 属性，采用对象的形式

```
1 function Component(){
2
3 }
4 Component.prototype.data = {
5     count : 0
6 }
```

创建两个组件实例

```
1 const componentA = new Component()
2 const componentB = new Component()
```

修改 `componentA` 组件 `data` 属性的值，`componentB` 中的值也发生了改变

```
1 console.log(componentB.data.count) // 0
2 componentA.data.count = 1
3 console.log(componentB.data.count) // 1
```

产生这样的原因这是两者共用了同一个内存地址，`componentA` 修改的内容，同样对 `componentB` 产生了影响

如果我们采用函数的形式，则不会出现这种情况（函数返回的对象内存地址并不相同）

```
1 function Component(){
2     this.data = this.data()
```

```

3   }
4   Component.prototype.data = function () {
5       return {
6           count : 0
7       }
8   }

```

修改 `componentA` 组件 `data` 属性的值，`componentB` 中的值不受影响

```

1   console.log(componentB.data.count) // 0
2   componentA.data.count = 1
3   console.log(componentB.data.count) // 0

```

`vue` 组件可能会有很多个实例，采用函数返回一个全新 `data` 形式，使每个实例对象的数据不会受到其他实例对象数据的污染

三、原理分析

首先可以看看 `vue` 初始化 `data` 的代码，`data` 的定义可以是函数也可以是对象

源码位置： `/vue-dev/src/core/instance/state.js`

```

1   function initData (vm: Component) {
2       let data = vm.$options.data
3       data = vm._data = typeof data === 'function'
4           ? getData(data, vm)
5           : data || {}
6       ...
7   }

```

`data` 既能是 `object` 也能是 `function`，那为什么还会出现上文警告呢？

别急，继续看下文

组件在创建的时候，会进行选项的合并

源码位置： `/vue-dev/src/core/util/options.js`

自定义组件会进入 `mergeOptions` 进行选项合并

```

1   Vue.prototype._init = function (options?: Object) {
2       ...
3       // merge options

```

```

4     if (options && options._isComponent) {
5         // optimize internal component instantiation
6         // since dynamic options merging is pretty slow, and none of the
7         // internal component options needs special treatment.
8         initInternalComponent(vm, options)
9     } else {
10        vm.$options = mergeOptions(
11            resolveConstructorOptions(vm.constructor),
12            options || {},
13            vm
14        )
15    }
16    ...
17 }

```

定义 `data` 会进行数据校验

源码位置: `/vue-dev/src/core/instance/init.js`

这时候 `vm` 实例为 `undefined`，进入 `if` 判断，若 `data` 类型不是 `function`，则出现警告提示

```

1  strats.data = function (
2    parentVal: any,
3    childVal: any,
4    vm?: Component
5  ): ?Function {
6    if (!vm) {
7      if (childVal && typeof childVal !== "function") {
8        process.env.NODE_ENV !== "production" &&
9          warn(
10            'The "data" option should be a function ' +
11              "that returns a per-instance value in component " +
12              "definitions.",
13            vm
14          );
15
16        return parentVal;
17      }
18      return mergeDataOrFn(parentVal, childVal);
19    }
20    return mergeDataOrFn(parentVal, childVal, vm);
21  };

```

四、结论

- 根实例对象 `data` 可以是对象也可以是函数（根实例是单例），不会产生数据污染情况
- 组件实例对象 `data` 必须为函数，目的是为了防止多个组件实例对象之间共用一个 `data`，产生数据污染。采用函数的形式，`initData` 时会将其作为工厂函数都会返回全新 `data` 对象

第41题：说说你对Vue生命周期的理解

一、生命周期是什么

生命周期（Life Cycle）的概念应用很广泛，特别是在政治、经济、环境、技术、社会等诸多领域经常出现，其基本涵义可以通俗地理解为“从摇篮到坟墓”（Cradle-to-Grave）的整个过程。在 `Vue` 中实例从创建到销毁的过程就是生命周期，即指从创建、初始化数据、编译模板、挂载Dom→渲染、更新→渲染、卸载等一系列过程。我们可以把组件比喻成工厂里面的一条流水线，每个工人（生命周期）站在各自的岗位，当任务流转到工人身边的时候，工人就开始工作。PS：在 `Vue` 生命周期钩子会自动绑定 `this` 上下文到实例中，因此你可以访问数据，对 `property` 和方法进行运算。这意味着你不能使用箭头函数来定义一个生命周期方法（例如 `created: () => this.fetchTodos()`）。

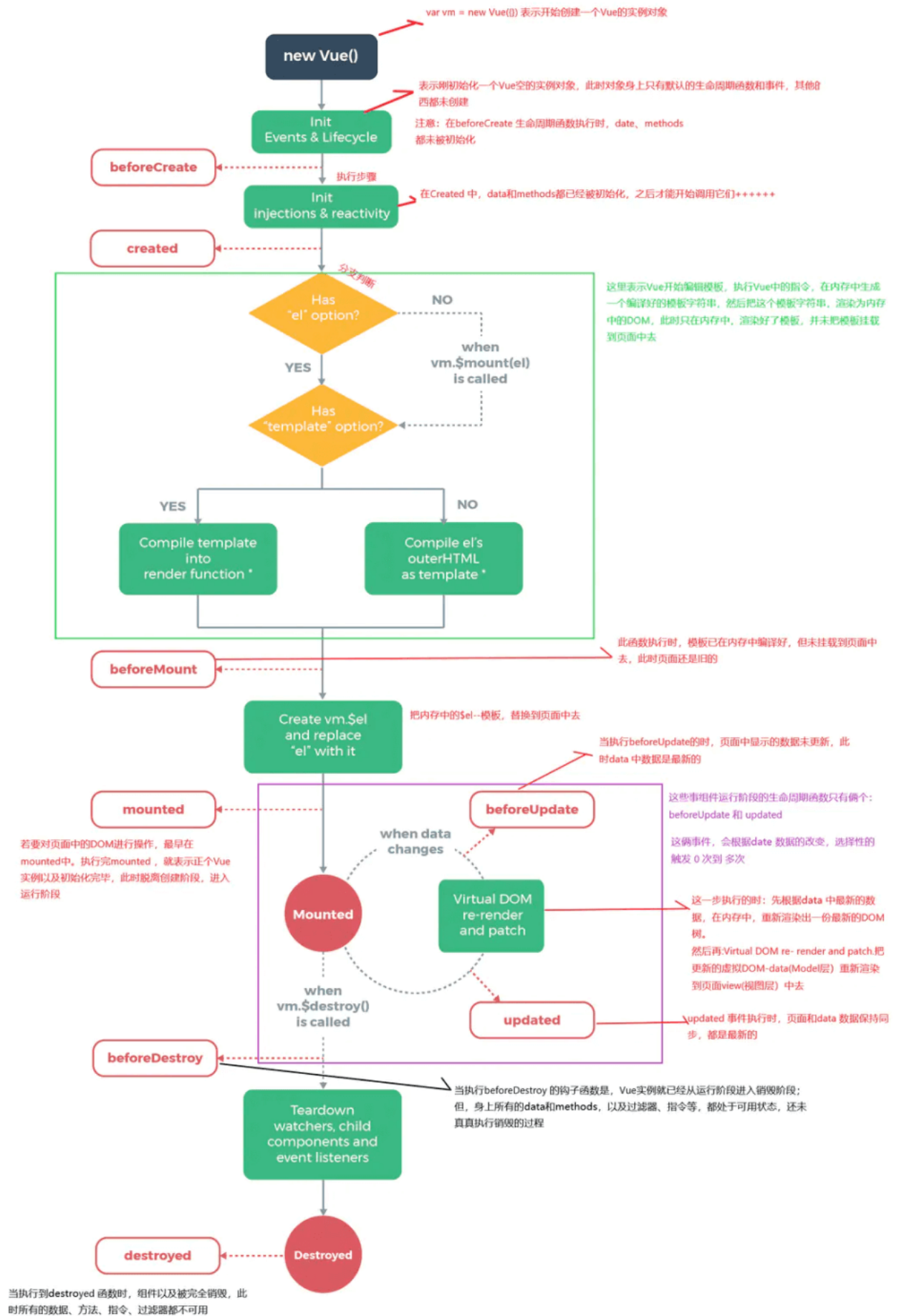
二、生命周期有哪些

`Vue` 生命周期总共可以分为8个阶段：创建前后, 载入前后, 更新前后, 销毁前销毁后，以及一些特殊场景的生命周期。

生命周期	描述
beforeCreate	组件实例被创建之初
created	组件实例已经完全创建
beforeMount	组件挂载之前
mounted	组件挂载到实例上去之后
beforeUpdate	组件数据发生变化，更新之前
updated	数据数据更新之后
beforeDestroy	组件实例销毁之前
destroyed	组件实例销毁之后
activated	keep-alive 缓存的组件激活时
deactivated	keep-alive 缓存的组件停用时调用
errorCaptured	捕获一个来自子孙组件的错误时被调用

三、生命周期整体流程

Vue 生命周期流程图



* template compilation is performed ahead-of-time if using a build step, e.g. single-file components

具体分析

beforeCreate -> created

- 初始化 `vue` 实例，进行数据观测

created

- 完成数据观测，属性与方法的运算，`watch`、`event` 事件回调的配置
- 可调用 `methods` 中的方法，访问和修改data数据触发响应式渲染 `dom`，可通过 `computed` 和 `watch` 完成数据计算
- 此时 `vm.$el` 并没有被创建

created -> beforeMount

- 判断是否存在 `el` 选项，若不存在则停止编译，直到调用 `vm.$mount(el)` 才会继续编译
- 优先级：`render` > `template` > `outerHTML`
- `vm.el` 获取到的是挂载 `DOM` 的

beforeMount

- 在此阶段可获取到 `vm.el`
- 此阶段 `vm.el` 虽已完成DOM初始化，但并未挂载在 `el` 选项上

beforeMount -> mounted

- 此阶段 `vm.el` 完成挂载，`vm.$el` 生成的 `DOM` 替换了 `el` 选项所对应的 `DOM`

mounted

- `vm.el` 已完成 `DOM` 的挂载与渲染，此刻打印 `vm.$el`，发现之前的挂载点及内容已被替换成新的 `DOM`

beforeUpdate

- 更新的数据必须是被渲染在模板上的（`el`、`template`、`render`之一）
- 此时 `view` 层还未更新
- 若在 `beforeUpdate` 中再次修改数据，不会再次触发更新方法

updated

- 完成 `view` 层的更新
- 若在 `updated` 中再次修改数据，会再次触发更新方法（`beforeUpdate`、`updated`）

beforeDestroy

- 实例被销毁前调用，此时实例属性与方法仍可访问

destroyed

- 完全销毁一个实例。可清理它与其它实例的连接，解绑它的全部指令及事件监听器
- 并不能清除DOM，仅仅销毁实例

使用场景分析

生命周期	描述
beforeCreate	执行时组件实例还未创建，通常用于插件开发中执行一些初始化任务
created	组件初始化完毕，各种数据可以使用，常用于异步数据获取
beforeMount	未执行渲染、更新，dom未创建
mounted	初始化结束，dom已创建，可用于获取访问数据和dom元素
beforeUpdate	更新前，可用于获取更新前各种状态
updated	更新后，所有状态已是最新
beforeDestroy	销毁前，可用于一些定时器或订阅的取消
destroyed	组件已销毁，作用同上

四、题外话：数据请求在created和mounted的区别

`created` 是在组件实例一旦创建完成的时候立刻调用，这时候页面 `dom` 节点并未生成 `mounted` 是在页面 `dom` 节点渲染完毕之后就立刻执行的触发时机上 `created` 是比 `mounted` 要更早的两者相同点：都能拿到实例对象的属性和方法讨论这个问题本质就是触发的时机，放在 `mounted` 请求有可能导致页面闪动（页面 `dom` 结构已经生成），但如果在页面加载前完成则不会出现此情况建议：放在 `create` 生命周期当中

第42题：Vue实例挂载的过程中发生了什么？

一、思考

我们都听过知其然知其所以然这句话

那么不知道大家是否思考过 `new Vue()` 这个过程中究竟做了些什么？

过程中是如何完成数据的绑定，又是如何将数据渲染到视图的等等

一、分析

首先找到 `vue` 的构造函数

源码位置：src\core\instance\index.js

```
1 function Vue (options) {
2   if (process.env.NODE_ENV !== 'production' &&
3     !(this instanceof Vue)
4   ) {
5     warn('Vue is a constructor and should be called with the `new` keyword')
6   }
7   this._init(options)
8 }
```

`options` 是用户传递过来的配置项，如 `data`、`methods` 等常用的方法

`vue` 构造函数调用 `_init` 方法，但我们发现本文件中并没有此方法，但仔细可以看到文件下方定义了很多初始化方法

```
1 initMixin(Vue);    // 定义 _init
2 stateMixin(Vue);   // 定义 $set $get $delete $watch 等
3 eventsMixin(Vue);  // 定义事件 $on $once $off $emit
4 lifecycleMixin(Vue); // 定义 _update $forceUpdate $destroy
5 renderMixin(Vue);  // 定义 _render 返回虚拟dom
```

首先可以看 `initMixin` 方法，发现该方法在 `Vue` 原型上定义了 `_init` 方法

源码位置：src\core\instance\init.js

```
1  Vue.prototype._init = function (options?: Object) {
2    const vm: Component = this
3    // a uid
4    vm._uid = uid++
5    let startTag, endTag
6    /* istanbul ignore if */
7    if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
8      startTag = `vue-perf-start:${vm._uid}`
9      endTag = `vue-perf-end:${vm._uid}`
10     mark(startTag)
11   }
12
13   // a flag to avoid this being observed
14   vm._isVue = true
15   // merge options
16   // 合并属性，判断初始化的是否是组件，这里合并主要是 mixins 或 extends 的方法
17   if (options && options._isComponent) {
18     // optimize internal component instantiation
19     // since dynamic options merging is pretty slow, and none of the
20     // internal component options needs special treatment.
21     initInternalComponent(vm, options)
22   } else { // 合并vue属性
23     vm.$options = mergeOptions(
24       resolveConstructorOptions(vm.constructor),
25       options || {},
26       vm
27     )
28   }
29   /* istanbul ignore else */
30   if (process.env.NODE_ENV !== 'production') {
31     // 初始化proxy拦截器
32     initProxy(vm)
33   } else {
34     vm._renderProxy = vm
35   }
36   // expose real self
37   vm._self = vm
38   // 初始化组件生命周期标志位
39   initLifecycle(vm)
40   // 初始化组件事件侦听
41   initEvents(vm)
42   // 初始化渲染方法
43   initRender(vm)
```

```

44     callHook(vm, 'beforeCreate')
45     // 初始化依赖注入内容, 在初始化data、props之前
46     initInjections(vm) // resolve injections before data/props
47     // 初始化props/data/method/watch/methods
48     initState(vm)
49     initProvide(vm) // resolve provide after data/props
50     callHook(vm, 'created')
51
52     /* istanbul ignore if */
53     if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
54       vm._name = formatComponentName(vm, false)
55       mark(endTag)
56       measure(`vue ${vm._name} init`, startTag, endTag)
57     }
58     // 挂载元素
59     if (vm.$options.el) {
60       vm.$mount(vm.$options.el)
61     }
62   }

```

仔细阅读上面的代码，我们得到以下结论：

- 在调用 `beforeCreate` 之前，数据初始化并未完成，像 `data`、`props` 这些属性无法访问到
- 到了 `created` 的时候，数据已经初始化完成，能够访问 `data`、`props` 这些属性，但这时候并未完成 `dom` 的挂载，因此无法访问到 `dom` 元素
- 挂载方法是调用 `vm.$mount` 方法

`initState` 方法是完成 `props/data/method/watch/methods` 的初始化

源码位置：src\core\instance\state.js

```

1  export function initState (vm: Component) {
2    // 初始化组件的watcher列表
3    vm._watchers = []
4    const opts = vm.$options
5    // 初始化props
6    if (opts.props) initProps(vm, opts.props)
7    // 初始化methods方法
8    if (opts.methods) initMethods(vm, opts.methods)
9    if (opts.data) {
10     // 初始化data
11     initData(vm)
12   } else {
13     observe(vm._data = {}, true /* asRootData */)
14   }

```

```

15     if (opts.computed) initComputed(vm, opts.computed)
16     if (opts.watch && opts.watch !== nativeWatch) {
17       initWatch(vm, opts.watch)
18     }
19   }

```

我们和这里主要看初始化 `data` 的方法为 `initData`，它与 `initState` 在同一文件上

```

1  function initData (vm: Component) {
2    let data = vm.$options.data
3    // 获取到组件上的data
4    data = vm._data = typeof data === 'function'
5      ? getData(data, vm)
6      : data || {}
7    if (!isPlainObject(data)) {
8      data = {}
9      process.env.NODE_ENV !== 'production' && warn(
10        'data functions should return an object:\n' +
11        'https://vuejs.org/v2/guide/components.html#data-Must-Be-a-Function',
12        vm
13      )
14    }
15    // proxy data on instance
16    const keys = Object.keys(data)
17    const props = vm.$options.props
18    const methods = vm.$options.methods
19    let i = keys.length
20    while (i--) {
21      const key = keys[i]
22      if (process.env.NODE_ENV !== 'production') {
23        // 属性名不能与方法名重复
24        if (methods && hasOwn(methods, key)) {
25          warn(
26            `Method "${key}" has already been defined as a data property.`,
27            vm
28          )
29        }
30      }
31      // 属性名不能与state名称重复
32      if (props && hasOwn(props, key)) {
33        process.env.NODE_ENV !== 'production' && warn(
34          `The data property "${key}" is already declared as a prop. ` +
35          `Use prop default value instead.`,
36          vm
37        )

```

```

38     } else if (!isReserved(key)) { // 验证key值的合法性
39         // 将_data中的数据挂载到组件vm上,这样就可以通过this.xxx访问到组件上的数据
40         proxy(vm, `_data`, key)
41     }
42 }
43 // observe data
44 // 响应式监听data是数据的变化
45 observe(data, true /* asRootData */)
46 }

```

仔细阅读上面的代码，我们可以得到以下结论：

- 初始化顺序： `props`、`methods`、`data`
- `data` 定义的时候可选择函数形式或者对象形式（组件只能为函数形式）

关于数据响应式在这就不展开详细说明

上文提到挂载方法是调用 `vm.$mount` 方法

源码位置：

```

1  Vue.prototype.$mount = function (
2    el?: string | Element,
3    hydrating?: boolean
4  ): Component {
5    // 获取或查询元素
6    el = el && query(el)
7
8    /* istanbul ignore if */
9    // vue 不允许直接挂载到body或页面文档上
10   if (el === document.body || el === document.documentElement) {
11     process.env.NODE_ENV !== 'production' && warn(
12       `Do not mount Vue to <html> or <body> - mount to normal elements
13       instead.`
14     )
15     return this
16   }
17   const options = this.$options
18   // resolve template/el and convert to render function
19   if (!options.render) {
20     let template = options.template
21     // 存在template模板, 解析vue模板文件
22     if (template) {
23       if (typeof template === 'string') {
24         if (template.charAt(0) === '#') {

```

```

25     template = idToTemplate(template)
26     /* istanbul ignore if */
27     if (process.env.NODE_ENV !== 'production' && !template) {
28         warn(
29             `Template element not found or is empty: ${options.template}`,
30             this
31         )
32     }
33 }
34 } else if (template.nodeType) {
35     template = template.innerHTML
36 } else {
37     if (process.env.NODE_ENV !== 'production') {
38         warn('invalid template option:' + template, this)
39     }
40     return this
41 }
42 } else if (el) {
43     // 通过选择器获取元素内容
44     template = getOuterHTML(el)
45 }
46 if (template) {
47     /* istanbul ignore if */
48     if (process.env.NODE_ENV !== 'production' && config.performance &&
mark) {
49         mark('compile')
50     }
51     /**
52      * 1. 将temmplate解析ast tree
53      * 2. 将ast tree转换成render语法字符串
54      * 3. 生成render方法
55      */
56     const { render, staticRenderFns } = compileToFunctions(template, {
57         outputSourceRange: process.env.NODE_ENV !== 'production',
58         shouldDecodeNewlines,
59         shouldDecodeNewlinesForHref,
60         delimiters: options.delimiters,
61         comments: options.comments
62     }, this)
63     options.render = render
64     options.staticRenderFns = staticRenderFns
65
66     /* istanbul ignore if */
67     if (process.env.NODE_ENV !== 'production' && config.performance &&
mark) {
68         mark('compile end')
69         measure(`vue ${this._name} compile`, 'compile', 'compile end')

```

```

70     }
71     }
72   }
73   return mount.call(this, el, hydrating)
74 }

```

阅读上面代码，我们能得到以下结论：

- 不要将根元素放到 `body` 或者 `html` 上
- 可以在对象中定义 `template/render` 或者直接使用 `template`、`el` 表示元素选择器
- 最终都会解析成 `render` 函数，调用 `compileToFunctions`，会将 `template` 解析成 `render` 函数

对 `template` 的解析步骤大致分为以下几步：

- 将 `html` 文档片段解析成 `ast` 描述符
- 将 `ast` 描述符解析成字符串
- 生成 `render` 函数

生成 `render` 函数，挂载到 `vm` 上后，会再次调用 `mount` 方法

源码位置：src\platforms\web\runtime\index.js

```

1  // public mount method
2  Vue.prototype.$mount = function (
3    el?: string | Element,
4    hydrating?: boolean
5  ): Component {
6    el = el && inBrowser ? query(el) : undefined
7    // 渲染组件
8    return mountComponent(this, el, hydrating)
9  }

```

调用 `mountComponent` 渲染组件

```

1  export function mountComponent (
2    vm: Component,
3    el: ?Element,
4    hydrating?: boolean
5  ): Component {
6    vm.$el = el
7    // 如果没有获取解析的render函数，则会抛出警告
8    // render是解析模板文件生成的

```

```

9     if (!vm.$options.render) {
10       vm.$options.render = createEmptyVNode
11       if (process.env.NODE_ENV !== 'production') {
12         /* istanbul ignore if */
13         if ((vm.$options.template && vm.$options.template.charAt(0) !== '#') ||
14           vm.$options.el || el) {
15           warn(
16             'You are using the runtime-only build of Vue where the template ' +
17             'compiler is not available. Either pre-compile the templates into '
18             +
19             'render functions, or use the compiler-included build.',
20             vm
21           )
22         } else {
23           // 没有获取到vue的模板文件
24           warn(
25             'Failed to mount component: template or render function not
26             defined.',
27             vm
28           )
29         }
30         // 执行beforeMount钩子
31         callHook(vm, 'beforeMount')
32
33         let updateComponent
34         /* istanbul ignore if */
35         if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
36           updateComponent = () => {
37             const name = vm._name
38             const id = vm._uid
39             const startTag = `vue-perf-start:${id}`
40             const endTag = `vue-perf-end:${id}`
41
42             mark(startTag)
43             const vnode = vm._render()
44             mark(endTag)
45             measure(`vue ${name} render`, startTag, endTag)
46
47             mark(startTag)
48             vm._update(vnode, hydrating)
49             mark(endTag)
50             measure(`vue ${name} patch`, startTag, endTag)
51           }
52         } else {
53           // 定义更新函数

```

```

54     updateComponent = () => {
55         // 实际调用是在lifeCycleMixin中定义的_update和renderMixin中定义的_render
56         vm._update(vm._render(), hydrating)
57     }
58 }
59 // we set this to vm._watcher inside the watcher's constructor
60 // since the watcher's initial patch may call $forceUpdate (e.g. inside
    child
61 // component's mounted hook), which relies on vm._watcher being already
    defined
62 // 监听当前组件状态，当有数据变化时，更新组件
63 new Watcher(vm, updateComponent, noop, {
64     before () {
65         if (vm._isMounted && !vm._isDestroyed) {
66             // 数据更新引发的组件更新
67             callHook(vm, 'beforeUpdate')
68         }
69     }
70 }, true /* isRenderWatcher */)
71 hydrating = false
72
73 // manually mounted instance, call mounted on self
74 // mounted is called for render-created child components in its inserted
    hook
75 if (vm.$vnode == null) {
76     vm._isMounted = true
77     callHook(vm, 'mounted')
78 }
79 return vm
80 }

```

阅读上面代码，我们得到以下结论：

- 会触发 `beforeCreate` 钩子
- 定义 `updateComponent` 渲染页面视图的方法
- 监听组件数据，一旦发生变化，触发 `beforeUpdate` 生命钩子

`updateComponent` 方法主要执行在 `vue` 初始化时声明的 `render`，`update` 方法

`render` 的作用主要是生成 `vnode`

源码位置：src\core\instance\render.js

```

1 // 定义vue 原型上的render方法
2 Vue.prototype._render = function (): VNode {
3     const vm: Component = this

```

```

4      // render函数来自于组件的option
5      const { render, _parentVnode } = vm.$options
6
7      if (_parentVnode) {
8          vm.$scopedSlots = normalizeScopedSlots(
9              _parentVnode.data.scopedSlots,
10             vm.$slots,
11             vm.$scopedSlots
12         )
13     }
14
15     // set parent vnode. this allows render functions to have access
16     // to the data on the placeholder node.
17     vm.$vnode = _parentVnode
18     // render self
19     let vnode
20     try {
21         // There's no need to maintain a stack because all render fns are
22         // called separately from one another. Nested component's render fns are
23         // called when parent component is patched.
24         currentRenderingInstance = vm
25         // 调用render方法，自己的独特的render方法， 传入createElement参数，生成
26         // vnode
27         vnode = render.call(vm._renderProxy, vm.$createElement)
28     } catch (e) {
29         handleError(e, vm, `render`)
30         // return error render result,
31         // or previous vnode to prevent render error causing blank component
32         /* istanbul ignore else */
33         if (process.env.NODE_ENV !== 'production' && vm.$options.renderError) {
34             {
35                 try {
36                     vnode = vm.$options.renderError.call(vm._renderProxy,
37                         vm.$createElement, e)
38                 } catch (e) {
39                     handleError(e, vm, `renderError`)
40                     vnode = vm._vnode
41                 }
42             } else {
43                 vnode = vm._vnode
44             }
45         } finally {
46             currentRenderingInstance = null
47         }
48         // if the returned array contains only a single node, allow it

```

```

46     if (Array.isArray(vnode) && vnode.length === 1) {
47         vnode = vnode[0]
48     }
49     // return empty vnode in case the render function errored out
50     if (!(vnode instanceof VNode)) {
51         if (process.env.NODE_ENV !== 'production' && Array.isArray(vnode)) {
52             warn(
53                 'Multiple root nodes returned from render function. Render
function ' +
54                 'should return a single root node.',
55                 vm
56             )
57         }
58         vnode = createEmptyVNode()
59     }
60     // set parent
61     vnode.parent = _parentVnode
62     return vnode
63 }

```

`_update` 主要功能是调用 `patch`，将 `vnode` 转换为真实 DOM，并且更新到页面中

源码位置：src\core\instance\lifecycle.js

```

1  Vue.prototype._update = function (vnode: VNode, hydrating?: boolean) {
2      const vm: Component = this
3      const prevEl = vm.$el
4      const prevVnode = vm._vnode
5      // 设置当前激活的作用域
6      const restoreActiveInstance = setActiveInstance(vm)
7      vm._vnode = vnode
8      // Vue.prototype.__patch__ is injected in entry points
9      // based on the rendering backend used.
10     if (!prevVnode) {
11         // initial render
12         // 执行具体的挂载逻辑
13         vm.$el = vm.__patch__(vm.$el, vnode, hydrating, false /* removeOnly */)
14     } else {
15         // updates
16         vm.$el = vm.__patch__(prevVnode, vnode)
17     }
18     restoreActiveInstance()
19     // update __vue__ reference
20     if (prevEl) {
21         prevEl.__vue__ = null
22     }

```

```
23     if (vm.$el) {
24         vm.$el.__vue__ = vm
25     }
26     // if parent is an HOC, update its $el as well
27     if (vm.$vnode && vm.$parent && vm.$vnode === vm.$parent._vnode) {
28         vm.$parent.$el = vm.$el
29     }
30     // updated hook is called by the scheduler to ensure that children are
31     // updated in a parent's updated hook.
32 }
```

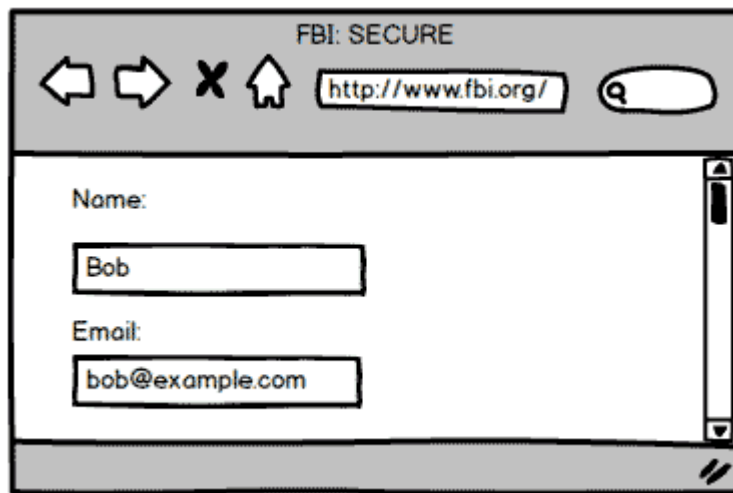
三、结论

- new Vue 的时候调用会调用 _init 方法
 - 定义 \$set、\$get、\$delete、\$watch 等方法
 - 定义 \$on、\$off、\$emit、\$off 等事件
 - 定义 _update、\$forceUpdate、\$destroy 生命周期
- 调用 \$mount 进行页面的挂载
- 挂载的时候主要是通过 mountComponent 方法
- 定义 updateComponent 更新函数
- 执行 render 生成虚拟 DOM
- _update 将虚拟 DOM 生成真实 DOM 结构，并且渲染到页面中

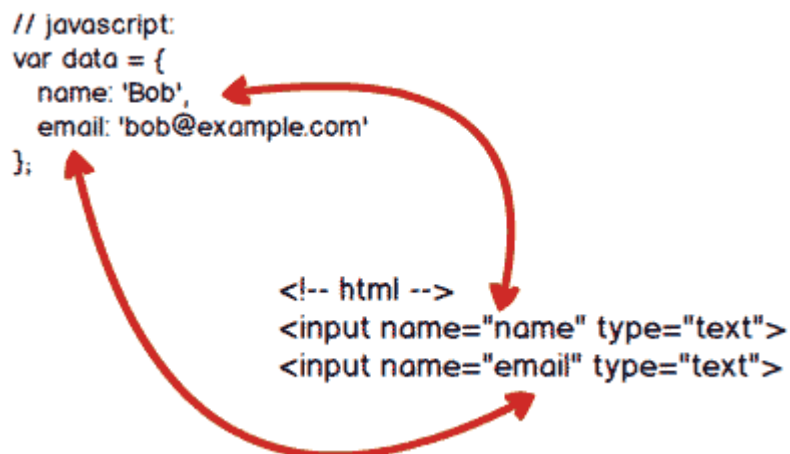
第43题：谈谈对Vue中双向绑定的理解

一、什么是双向绑定

我们先从单向绑定切入单向绑定非常简单，就是把 Model 绑定到 View，当我们用 JavaScript 代码更新 Model 时，View 就会自动更新双向绑定就很容易联想到了，在单向绑定的基础上，用户更新了 View，Model 的数据也自动被更新了，这种情况就是双向绑定举个栗子



当用户填写表单时，View 的状态就被更新了，如果此时可以自动更新 Model 的状态，那就相当于我们把 Model 和 View 做了双向绑定关系图如下



二、双向绑定的原理是什么

我们都知道 Vue 是数据双向绑定的框架，双向绑定由三个重要部分构成

- 数据层（Model）：应用的数据及业务逻辑
- 视图层（View）：应用的展示效果，各类UI组件
- 业务逻辑层（ViewModel）：框架封装的核心，它负责将数据与视图关联起来

而上面的这个分层的架构方案，可以用一个专业术语进行称呼：MVVM 这里的控制层的核心功能便是“数据双向绑定”。自然，我们只需弄懂它是什么，便可以进一步了解数据绑定的原理

理解ViewModel

它的主要职责就是：

- 数据变化后更新视图
- 视图变化后更新数据

当然，它还有两个主要部分组成

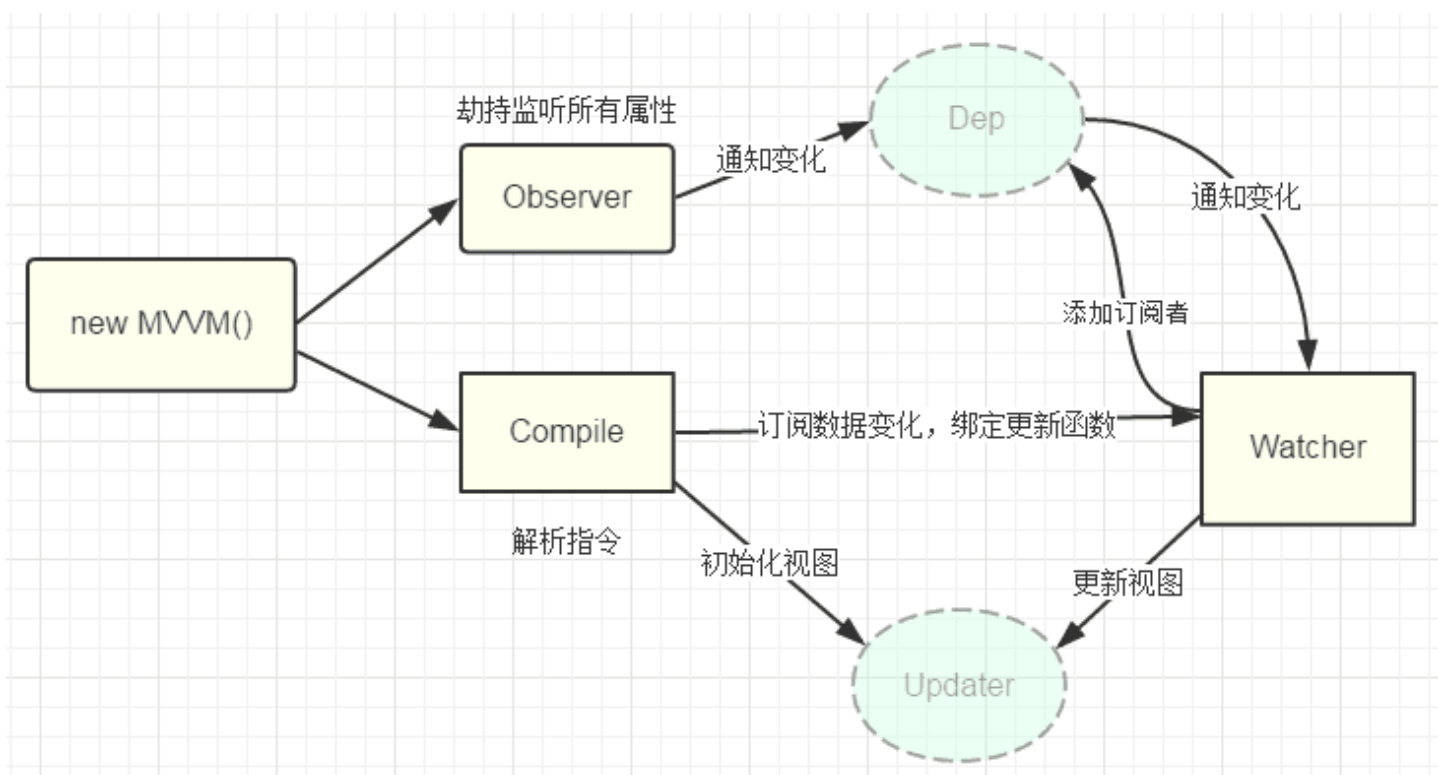
- 监听器（Observer）：对所有数据的属性进行监听
- 解析器（Compiler）：对每个元素节点的指令进行扫描跟解析,根据指令模板替换数据,以及绑定相应的更新函数

三、实现双向绑定

我们还是以 `Vue` 为例，先来看看 `Vue` 中的双向绑定流程是什么的

1. `new Vue()` 首先执行初始化，对 `data` 执行响应化处理，这个过程发生在 `Observe` 中
2. 同时对模板执行编译，找到其中动态绑定的数据，从 `data` 中获取并初始化视图，这个过程发生在 `Compile` 中
3. 同时定义一个更新函数和 `Watcher`，将来对应数据变化时 `Watcher` 会调用更新函数
4. 由于 `data` 的某个 `key` 在一个视图中可能出现多次，所以每个 `key` 都需要一个管家 `Dep` 来管理多个 `Watcher`
5. 将来`data`中数据一旦发生变化，会首先找到对应的 `Dep`，通知所有 `Watcher` 执行更新函数

流程图如下：



实现

先来一个构造函数：执行初始化，对 `data` 执行响应化处理

```

1  class Vue {
2    constructor(options) {
3      this.$options = options;
4      this.$data = options.data;
5
6      // 对data选项做响应式处理
7      observe(this.$data);
8
9      // 代理data到vm上
10     proxy(this);
11
12     // 执行编译
13     new Compile(options.el, this);
14   }
15 }

```

对 `data` 选项执行响应化具体操作

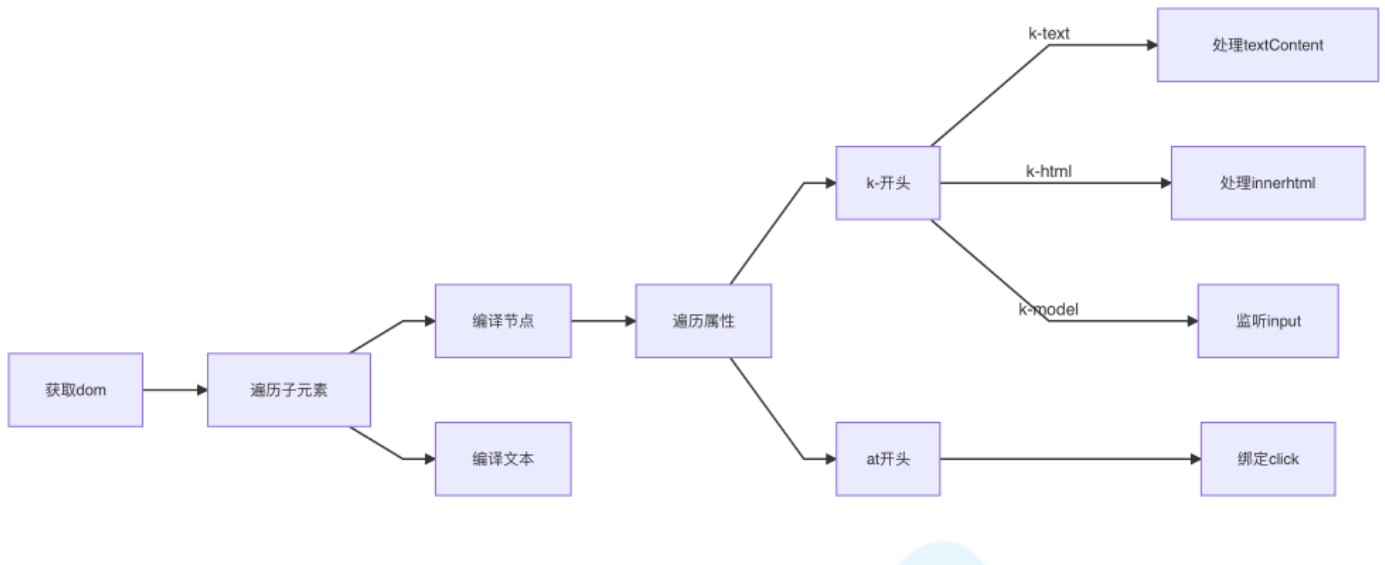
```

1  function observe(obj) {
2    if (typeof obj !== "object" || obj == null) {
3      return;
4    }
5    new Observer(obj);
6  }
7
8  class Observer {
9    constructor(value) {
10     this.value = value;
11     this.walk(value);
12   }
13   walk(obj) {
14     Object.keys(obj).forEach((key) => {
15       defineReactive(obj, key, obj[key]);
16     });
17   }
18 }

```

编译 `Compile`

对每个元素节点的指令进行扫描跟解析,根据指令模板替换数据,以及绑定相应的更新函数



```

1  class Compile {
2
3    constructor(el, vm) {
4
5      this.
6    $vm = vm;
7      this.$
8    el = document.querySelector(el); // 获取dom
9
10     if (this.
11    $el) {
12       this.compile(this.$
13    el);
14
15     }
16
17   }
18
19   compile(el) {
20
21     const childNodes = el.childNodes;
22
23     Array.from(childNodes).forEach((node) => { // 遍历子元素
24
25       if (this.isElement(node)) { // 判断是否为节点
26
27         console.log("编译元素" + node.nodeName);
28
29       } else if (this.isInterpolation(node)) {

```

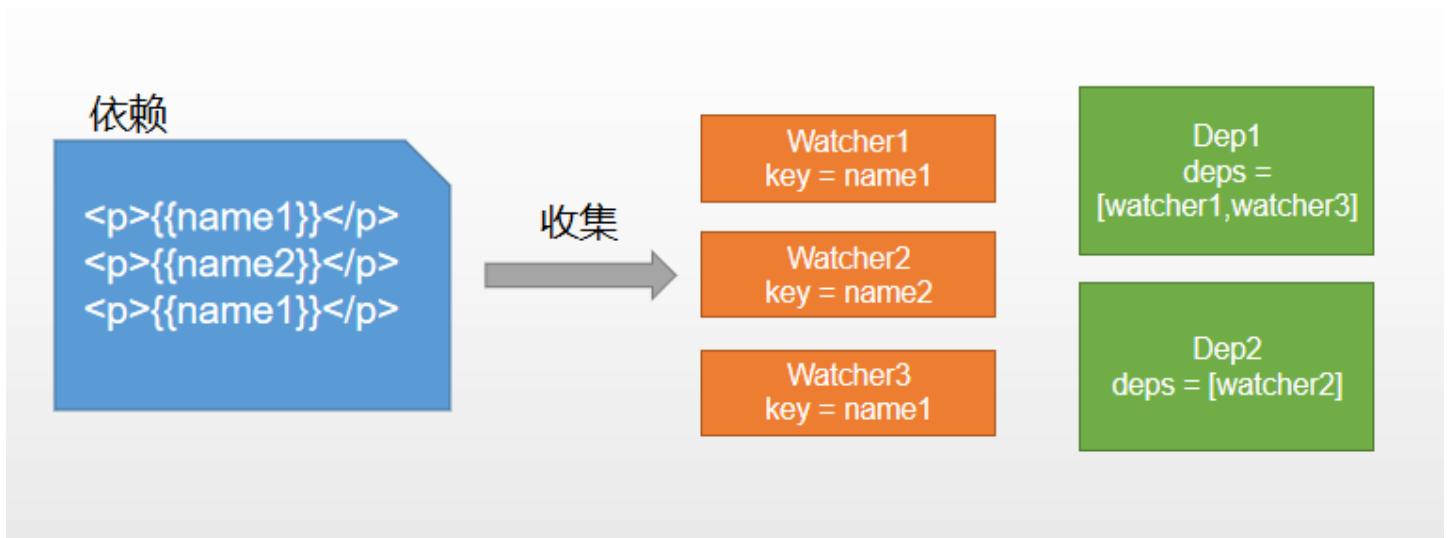
```

30
31         console.log("编译插值文本" + node.textContent); // 判断是否为插值文本
32     {{{
33     }}
34
35     if (node.childNodes && node.childNodes.length > 0) { // 判断是否有子元素
36
37         this.compile(node); // 对子元素进行递归遍历
38
39     }
40
41     });
42
43     }
44
45     isElement(node) {
46
47         return node.nodeType == 1;
48
49     }
50
51     isInterpolation(node) {
52
53         return node.nodeType == 3 && /\{\{(.*)\}\}/.test(node.textContent);
54
55     }
56 }
57

```

依赖收集

视图中会用到 `data` 中某 `key`，这称为依赖。同一个 `key` 可能出现多次，每次都需要收集出来用一个 `Watcher` 来维护它们，此过程称为依赖收集多个 `Watcher` 需要一个 `Dep` 来管理，需要更新时由 `Dep` 统一通知



实现思路

1. `defineReactive` 时为每一个 `key` 创建一个 `Dep` 实例
2. 初始化视图时读取某个 `key`，例如 `name1`，创建一个 `watcher1`
3. 由于触发 `name1` 的 `getter` 方法，便将 `watcher1` 添加到 `name1` 对应的 `Dep` 中
4. 当 `name1` 更新，`setter` 触发时，便可通过对应该 `Dep` 通知其管理所有 `Watcher` 更新

```
1  // 负责更新视图
2  class Watcher {
3      constructor(vm, key, updater) {
4          this.vm = vm
5          this.key = key
6          this.updaterFn = updater
7
8          // 创建实例时，把当前实例指定到Dep.target静态属性上
9          Dep.target = this
10         // 读一下key，触发get
11         vm[key]
12         // 置空
13         Dep.target = null
14     }
15
16     // 未来执行dom更新函数，由dep调用的
17     update() {
18         this.updaterFn.call(this.vm, this.vm[this.key])
19     }
20 }
```

声明 `Dep`

```

1  class Dep {
2    constructor() {
3      this.deps = []; // 依赖管理
4    }
5    addDep(dep) {
6      this.deps.push(dep);
7    }
8    notify() {
9      this.deps.forEach((dep) => dep.update());
10   }
11 }

```

创建 `watcher` 时触发 `getter`

```

1  class Watcher {
2    constructor(vm, key, updateFn) {
3      Dep.target = this;
4      this.vm[this.key];
5      Dep.target = null;
6    }
7  }
8

```

依赖收集，创建 `Dep` 实例

```

1  function defineReactive(obj, key, val) {
2    this.observe(val);
3    const dep = new Dep();
4    Object.defineProperty(obj, key, {
5      get() {
6        Dep.target && dep.addDep(Dep.target); // Dep.target 也就是 Watcher 实例
7        return val;
8      },
9      set(newVal) {
10       if (newVal === val) return;
11       dep.notify(); // 通知 dep 执行更新方法
12     },
13   });
14 }

```

第44题：Vue 模板是如何编译的

```
1 new Vue({
2   render: h => h(App)
3 })
```

这个大家都熟悉，调用 render 就会得到传入的模板(`.vue` 文件)对应的虚拟 DOM，那么这个 render 是哪来的呢？它是怎么把 `.vue` 文件转成浏览器可识别的代码的呢？

render 函数是怎么来的有两种方式

- 第一种就是经过模板编译生成 render 函数
- 第二种是我们自己在组件里定义了 render 函数，这种会跳过模板编译的过程

本文将为大家分别介绍这两种，以及详细的编译过程原理

认识模板编译

我们知道 `<template></template>` 这个是模板，不是真实的 HTML，浏览器是不认识模板的，所以我们需要把它编译成浏览器认识的原生的 HTML

这一块的主要流程就是

1. 提取出模板中的原生 HTML 和非原生 HTML，比如绑定的属性、事件、指令等等
2. 经过一些处理生成 render 函数
3. render 函数再将模板内容生成对应的 vnode
4. 再经过 patch 过程(Diff)得到要渲染到视图中的 vnode
5. 最后根据 vnode 创建真实的 DOM 节点，也就是原生 HTML 插入到视图中，完成渲染

上面的 1、2、3 条就是模板编译的过程了

那它是怎么编译，最终生成 render 函数的呢？

模板编译详解——源码

baseCompile()

这就是模板编译的入口函数，它接收两个参数

- `template`：就是要转换的模板字符串
- `options`：就是转换时需要的参数

编译的流程，主要有三步：

1. 模板解析：通过正则等方式提取出 `<template></template>` 模板里的标签元素、属性、变量等信息，并解析成抽象语法树 `AST`
2. 优化：遍历 `AST` 找出其中的静态节点和静态根节点，并添加标记

3. 代码生成：根据 AST 生成渲染函数 render

这三步分别对应三个函数，后面会一一介绍，先看一下 `baseCompile` 源码中是在哪里调用的
源码地址：`src/complier/index.js - 11行`

```
1  export const createCompiler = createCompilerCreator(function baseCompile (  
2    template: string, // 就是要转换的模板字符串  
3    options: CompilerOptions //就是转换时需要的参数  
4  ): CompiledResult {  
5    // 1. 进行模板解析，并将结果保存为 AST  
6    const ast = parse(template.trim(), options)  
7  
8    // 没有禁用静态优化的话  
9    if (options.optimize !== false) {  
10     // 2. 就遍历 AST，并找出静态节点并标记  
11     optimize(ast, options)  
12   }  
13   // 3. 生成渲染函数  
14   const code = generate(ast, options)  
15   return {  
16     ast,  
17     render: code.render, // 返回渲染函数 render  
18     staticRenderFns: code.staticRenderFns  
19   }  
20 })
```

就这么几行代码，三步，调用了三个方法很清晰

我们先看一下最后 return 出去的是个啥，再来深入上面这三步分别调用的方法源码，也好更清楚的知道这三步分别是要做哪些处理

编译结果

比如有这样的模板

```
1  <template>  
2    <div id="app">{{name}}</div>  
3  </template>
```

打印一下编译后的结果，也就是上面源码 return 出去的结果，看看是啥

```
1  {  
2    ast: {
```

```

3      type: 1,
4      tag: 'div',
5      attrsList: [ { name: 'id', value: 'app' } ],
6      attrsMap: { id: 'app' },
7      rawAttrsMap: {},
8      parent: undefined,
9      children: [
10     {
11       type: 2,
12       expression: '_s(name)',
13       tokens: [ { '@binding': 'name' } ],
14       text: '{{name}}',
15       static: false
16     }
17   ],
18   plain: false,
19   attrs: [ { name: 'id', value: '"app"', dynamic: undefined } ],
20   static: false,
21   staticRoot: false
22 },
23 render: `with(this){return _c('div',{attrs:{"id":"app"}},[_v(_s(name))])}`,
24 staticRenderFns: [],
25 errors: [],
26 tips: []
27 }

```

看不明白也没有关系，注意看上面提到的三步都干了啥

- `ast` 字段，就是第一步生成的
- `static` 字段，就是标记，是在第二步中根据 `ast` 里的 `type` 加上去的
- `render` 字段，就是第三步生成的

有个大概的印象了，然后再来看源码

4. parse()

源码地址： `src/complier/parser/index.js - 79行`

就是这个方法就是解析器的主函数，就是它通过正则等方法提取出 `<template></template>` 模板字符串里所有的 `tag`、`props`、`children` 信息，生成一个对应结构的 `ast` 对象

`parse` 接收两个参数

- `template`：就是要转换的模板字符串
- `options`：就是转换时需要的参数。它包含有四个钩子函数，就是用来把 `parseHTML` 解析出来的字符串提取出来，并生成对应的 `AST`

核心步骤是这样的：

调用 `parseHTML` 函数对模板字符串进行解析

- 解析到开始标签、结束标签、文本、注释分别进行不同的处理
- 解析过程中遇到文本信息就调用文本解析器 `parseText` 函数进行文本解析
- 解析过程中遇到包含过滤器，就调用过滤器解析器 `parseFilters` 函数进行解析

每一步解析的结果都合并到一个对象上(就是最后的 AST)

这个地方的源码实在是太长了，有大几百行代码，我就只贴个大概吧，有兴趣的自己去看一下

```
1  export function parse (  
2    template: string, // 要转换的模板字符串  
3    options: CompilerOptions // 转换时需要的参数  
4  ): ASTElement | void {  
5    parseHTML(template, {  
6      warn,  
7      expectHTML: options.expectHTML,  
8      isUnaryTag: options.isUnaryTag,  
9      canBeLeftOpenTag: options.canBeLeftOpenTag,  
10     shouldDecodeNewlines: options.shouldDecodeNewlines,  
11     shouldDecodeNewlinesForHref: options.shouldDecodeNewlinesForHref,  
12     shouldKeepComment: options.comments,  
13     outputSourceRange: options.outputSourceRange,  
14     // 解析到开始标签时调用，如 <div>  
15     start (tag, attrs, unary, start, end) {  
16       // unary 是否是自闭合标签，如 <img />  
17       ...  
18     },  
19     // 解析到结束标签时调用，如 </div>  
20     end (tag, start, end) {  
21       ...  
22     },  
23     // 解析到文本时调用  
24     chars (text: string, start: number, end: number) {  
25       // 这里会判断判断很多东西，来看它是不是带变量的动态文本  
26       // 然后创建动态文本或静态文本对应的 AST 节点  
27       ...  
28     },  
29     // 解析到注释时调用  
30     comment (text: string, start, end) {  
31       // 注释是这么找的  
32       const comment = /^<!\-\-/  
33       if (comment.test(html)) {  
34         // 如果是注释，就继续找 '-->  
35         const commentEnd = html.indexOf('-->')  
36         ...
```

```

37     }
38   })
39   // 返回的这个就是 AST
40   return root
41 }

```

上面解析文本时调用的 `chars()` 会根据不同类型节点加上不同 `type`，来标记 `AST` 节点类型，这个属性在下一步标记的时候会用到

type	AST 节点类型
1	元素节点
2	包含变量的动态文本节点
3	没有变量的纯文本节点

5. optimize()

这个函数就是在 `AST` 里找出静态节点和静态根节点，并添加标记，为了后面 `patch` 过程中就会跳过静态节点的对比，直接克隆一份过去，从而优化了 `patch` 的性能

函数里面调用的外部函数就不贴代码了，大致过程是这样的

- 标记静态节点(markStatic)。就是判断 `type`，上面介绍了值为 1、2、3 的三种类型
 - `type` 值为 1：就是包含子元素的节点，设置 `static` 为 `false` 并递归标记子节点，直到标记完所有子节点
 - `type` 值为 2：设置 `static` 为 `false`
 - `type` 值为 3：就是不包含子节点和动态属性的纯文本节点，把它的 `static = true`，`patch` 的时候就会跳过这个，直接克隆一份去
- 标记静态根节点(markStaticRoots)，这里的原理和标记静态节点基本相同，只是需要满足下面条件的节点才能算作是静态根节点
 - 节点本身必须是静态节点
 - 必须有子节点
 - 子节点不能只有一个文本节点

源码地址：`src/complier/optimizer.js - 21行`

```

1  export function optimize (root: ?ASTElement, options: CompilerOptions) {

```

```

2     if (!root) return
3     isStaticKey = genStaticKeysCached(options.staticKeys || '')
4     isPlatformReservedTag = options.isReservedTag || no
5     // 标记静态节点
6     markStatic(root)
7     // 标记静态根节点
8     markStaticRoots(root, false)
9 }

```

6. generate()

这个就是生成 render 的函数，就是说最终会返回下面这样的东西

```

1  // 比如有这么个模板
2  <template>
3      <div id="app">{{name}}</div>
4  </template>
5
6  // 上面模板编译后返回的 render 字段 就是这样的
7  render: `with(this){return _c('div',{attrs:{"id":"app"}},[_v(_s(name))])}`
8
9  // 把内容格式化一下，容易理解一点
10 with(this){
11     return _c(
12         'div',
13         { attrs:{"id":"app"} },
14         [ _v(_s(name)) ]
15     )
16 }

```

这个结构是不是有点熟悉？

了解虚拟 DOM 就可以看出来，上面的 render 正是虚拟 DOM 的结构，就是把一个标签分为 tag、props、children，没有错

在看 generate 源码之前，我们要先了解一下上面这最后返回的 render 字段是什么意思，再来看 generate 源码，就会轻松得多，不然连函数返回的东西是干嘛的都不知道怎么可能看得懂这个函数呢

render

我们来翻译一下上面编译出来的 render

这个 with 在《你不知道的JavaScript上卷里介绍的是，用来欺骗词法作用域的关键字，它可以让我们更快的引用一个对象上的多个属性

看个例子

```
1  const name = '掘金'
2  const obj = { name:'沐华', age: 18 }
3  with(obj){
4      console.log(name) // 沐华 不需要写 obj.name 了
5      console.log(age) // 18 不需要写 obj.age 了
6  }
```

上面的 `with(this){}` 里的 `this` 就是当前组件实例。因为通过 `with` 改变了词法作用域中属性的指向，所以标签里使用 `name` 直接用就是了，而不需要 `this.name` 这样

那 `_c`、`_v` 和 `_s` 是什么呢？

在源码里是这样定义的，格式是：`_c` (缩写) = `createElement` (函数名)

源码地址：`src/core/instance/render-helpers/index.js - 15行`

```
1  // 其实不止这几个，由于本文例子中没有用到就没都复制过来占位了
2  export function installRenderHelpers (target: any) {
3      target._s = toString // 转字符串函数
4      target._l = renderList // 生成列表函数
5      target._v = createTextVNode // 创建文本节点函数
6      target._e = createEmptyVNode // 创建空节点函数
7  }
8  // 补充
9  _c = createElement // 创建虚拟节点函数
```

再来看是不是就清楚多了呢

```
1  with(this){ // 欺骗词法作用域，将该作用域里所有属性和方法都指向当前组件
2      return _c( // 创建一个虚拟节点
3          'div', // 标签为 div
4          { attrs:{"id":"app"} }, // 有一个属性 id 为 'app'
5          [ _v(_s(name)) ] // 是一个文本节点，所以把获取到的动态属性 name 转成字符串
6      )
7  }
```

接下来我们再来看 `generate()` 源码

generate

源码地址：`src/compiler/codegen/index.js - 43行`

这个流程很简单，只有几行代码，就是先判断 `AST` 是不是为空，不为空就根据 `AST` 创建 `vnode`，否则就创建一个空 `div` 的 `vnode`

```
1  export function generate (  
2    ast: ASTElement | void,  
3    options: CompilerOptions  
4  ): CodegenResult {  
5    const state = new CodegenState(options)  
6    // 就是先判断 AST 是不是为空，不为空就根据 AST 创建 vnode，否则就创建一个空div的  
    vnode  
7    const code = ast ? (ast.tag === 'script' ? 'null' : genElement(ast, state))  
      : '_c("div")'  
8  
9    return {  
10      render: `with(this){return ${code}}`,  
11      staticRenderFns: state.staticRenderFns  
12    }  
13  }
```

可以看出这里面主要就是通过 `genElement()` 方法来创建 `vnode` 的，所以我们来看一下它的源码，看是怎么创建的

genElement()

源码地址: `src/complier/codegen/index.js - 56行`

这里的逻辑还是很清晰的，就是一堆 `if/else` 判断传进来的 `AST` 元素节点的属性来执行不同的生成函数

这里还可以发现另一个知识点 `v-for` 的优先级要高于 `v-if`，因为先判断 `for` 的

```
1  export function genElement (el: ASTElement, state: CodegenState): string {  
2    if (el.parent) {  
3      el.pre = el.pre || el.parent.pre  
4    }  
5  
6    if (el.staticRoot && !el.staticProcessed) {  
7      return genStatic(el, state)  
8    } else if (el.once && !el.onceProcessed) { // v-once  
9      return genOnce(el, state)  
10   } else if (el.for && !el.forProcessed) { // v-for  
11     return genFor(el, state)  
12   } else if (el.if && !el.ifProcessed) { // v-if  
13     return genIf(el, state)  
14   }
```

```

15    // template 节点 && 没有插槽 && 没有 pre 标签
16  } else if (el.tag === 'template' && !el.slotTarget && !state.pre) {
17    return genChildren(el, state) || 'void 0'
18  } else if (el.tag === 'slot') { // v-slot
19    return genSlot(el, state)
20  } else {
21    // component or element
22    let code
23    // 如果有子组件
24    if (el.component) {
25      code = genComponent(el.component, el, state)
26    } else {
27      let data
28      // 获取元素属性 props
29      if (!el.plain || (el.pre && state.maybeComponent(el))) {
30        data = genData(el, state)
31      }
32      // 获取元素子节点
33      const children = el.inlineTemplate ? null : genChildren(el, state, true)
34      code = `_c('${el.tag}'${
35        data ? ` ,${data}` : '' // data
36      }${
37        children ? ` ,${children}` : '' // children
38      })`
39    }
40    // module transforms
41    for (let i = 0; i < state.transforms.length; i++) {
42      code = state.transforms[i](el, code)
43    }
44    // 返回上面作为 with 作用域执行的内容
45    return code
46  }
47 }

```

每一种类型调用的生成函数就不一一列举了，总的来说最后创建出来的 vnode 节点类型无非就三种，元素节点、文本节点、注释节点

自定义的 render

先举个例子吧，三种情况如下

```

1  // 1. test.vue
2  <template>
3    <h1>我是沐华</h1>
4  </template>

```

```
5 <script>
6   export default {}
7 </script>
```

```
1 // 2. test.vue
2 <script>
3   export default {
4     render(h){
5       return h('h1',{},{},'我是沐华')
6     }
7   }
8 </script>
```

```
1 // 3. test.js
2 export default {
3   render(h){
4     return h('h1',{},{},'我是沐华')
5   }
6 }
```

上面三种，最后渲染的出来的就是完全一模一样的，因为这个 `h` 就是上面模板编译后的那个 `_c`

这时有人可能就会问，为什么要自己写呢，不是有模板编译自动生成吗？

这个问题问得好！自己写肯定是有好处的

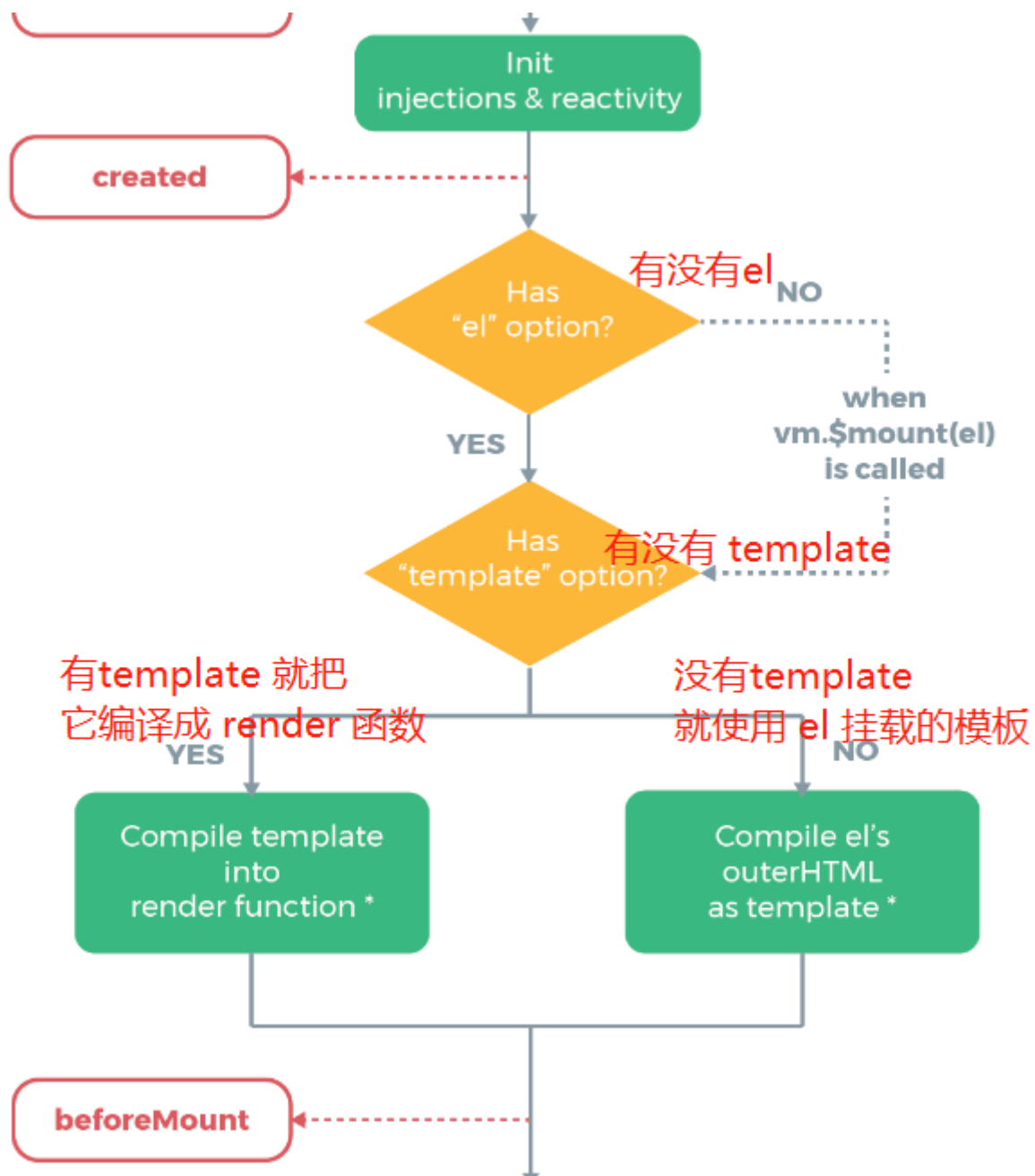
1. 自己把 `vnode` 给写了，就会直接跳过了模板编译，不用去解析模板里的动态属性、事件、指令等等了，所以性能上会有那么一丢丢提升。这一点在下面的渲染的优先级上就有体现
2. 还有一些情况，能让我们代码写法的更加灵活，更加方便简洁，不会冗余

比如 `Element-UI` 里面的组件源码里就有大量直接写 `render` 函数

接下来分别看下这两点是如何体现的

3. 渲染优先级

先看一下在官网的生命周期里，关于模板编译的部分



如图可以知道，如果有 `template`，就不会管 `el` 了，所以 `template` 比 `el` 的优先级更高，比如那我们自己写了 `render` 呢？

```
1 <div id='app'>
2   <p>{{ name }}</p>
3 </div>
4 <script>
5   new Vue({
6     el: '#app',
7     data: { name: '沐华' },
8     template: '<div>掘金</div>',
9     render(h) {
10       return h('div', {}, '好好学习，天天向上')
11     }
12   })
13 </script>
```

```
12     })
13   </script>
```

这个代码执行后页面渲染出来只有 `<div>好好学习，天天向上</div>`

可以得出 `render` 函数的优先级更高

因为不管是 `el` 挂载的，还是 `template` 最后都会被编译成 `render` 函数，而如果已经有了 `render` 函数了，就跳过前面的编译了

这一点在源码里也有体现

在源码中找到答案：`dist/vue.js - 11927行`

```
1   Vue.prototype.$mount = function ( el, hydrating ) {
2     el = el && query(el);
3     var options = this.$options;
4     // 如果没有 render
5     if (!options.render) {
6       var template = options.template;
7       // 再判断，如果有 template
8       if (template) {
9         if (typeof template === 'string') {
10           if (template.charAt(0) === '#') {
11             template = idToTemplate(template);
12           }
13           } else if (template.nodeType) {
14             template = template.innerHTML;
15           } else {
16             return this
17           }
18           // 再判断，如果有 el
19           } else if (el) {
20             template = getOuterHTML(el);
21           }
22         }
23         return mount.call(this, el, hydrating)
24       };

```

4. 更灵活的写法

比如说我们需要写很多 `if` 判断的时候

```
1   <template>
2     <h1 v-if="level === 1">
```

```

3      <a href="xxx">
4        <slot></slot>
5      </a>
6    </h1>
7    <h2 v-else-if="level === 2">
8      <a href="xxx">
9        <slot></slot>
10     </a>
11   </h2>
12   <h3 v-else-if="level === 3">
13     <a href="xxx">
14       <slot></slot>
15     </a>
16   </h3>
17 </template>
18 <script>
19   export default {
20     props: ['level']
21   }
22 </script>

```

不知道你有没有写过类似上面这样的代码呢？

我们换一种方式来写出和上面一模一样的代码看看，直接写 render

```

1  <script>
2    export default {
3      props: ['level'],
4      render(h){
5        return h('h' + this.level, this.$slots.default())
6      }
7    }
8  </script>

```

搞定！就这！就这？

没错，就这！

或者下面这样，多次调用的时候就很方便

```

1  <script>
2    export default {
3      props: ['level'],
4      render(h){
5        const tag = 'h' + this.level

```

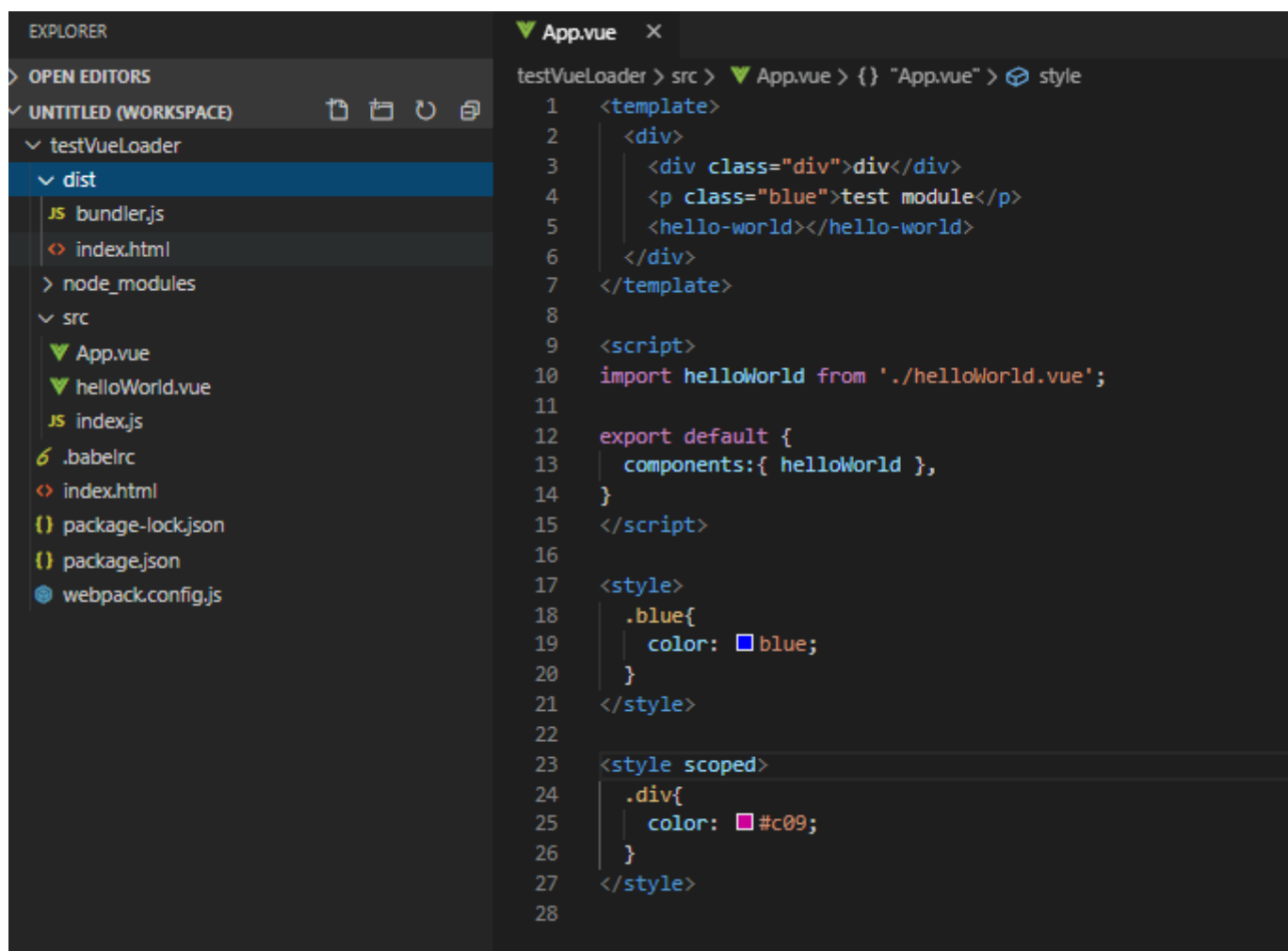
```
6         return (<tag>{this.$slots.default()}</tag>)  
7     }  
8 }  
9 </script>
```

第45题：说说 Vue 中 CSS scoped 的原理

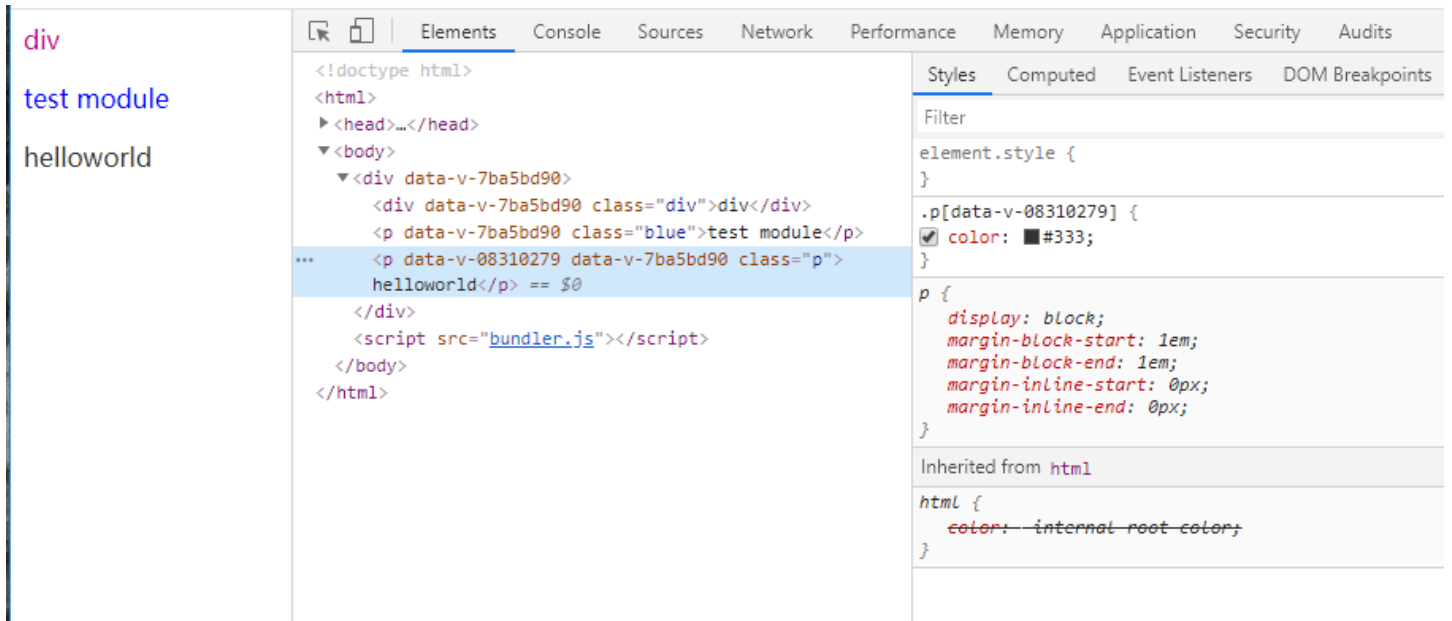
在日常的Vue项目开发过程中，为了让项目更好的维护一般都会使用模块化开发的方式进行。也就是每个组件维护独立的 `template`，`script`，`style`。今天主要介绍一下使用 `<style scoped>` 为什么在页面渲染完后样式之间并不会造成污染。

示例

搭建一个简单的Vue项目测试一下：



给个目录结构吧，代码并不是我们讲解的重点，如果需要源码测试的话后续我放到github上去。
终端执行 `npx webpack` 输出dist目录，我们在浏览器打开index.html调试一下看看现象：



1. 每个组件都会拥有一个 `[data-v-hash:8]` 插入HTML标签，子组件标签上也具体父组件 `[data-v-hash:8]`；
2. 如果style标签加了 `scoped`属性，里面的选择器都会变成 (Attribute Selector) `[data-v-hash:8]`；
3. 如果子组件选择器跟父组件选择器完全一样，那么就会出现子组件样式被父组件覆盖，因为子组件会优先于父组件mounted，有兴趣可以测试一下哦。

webpack.config.js配置

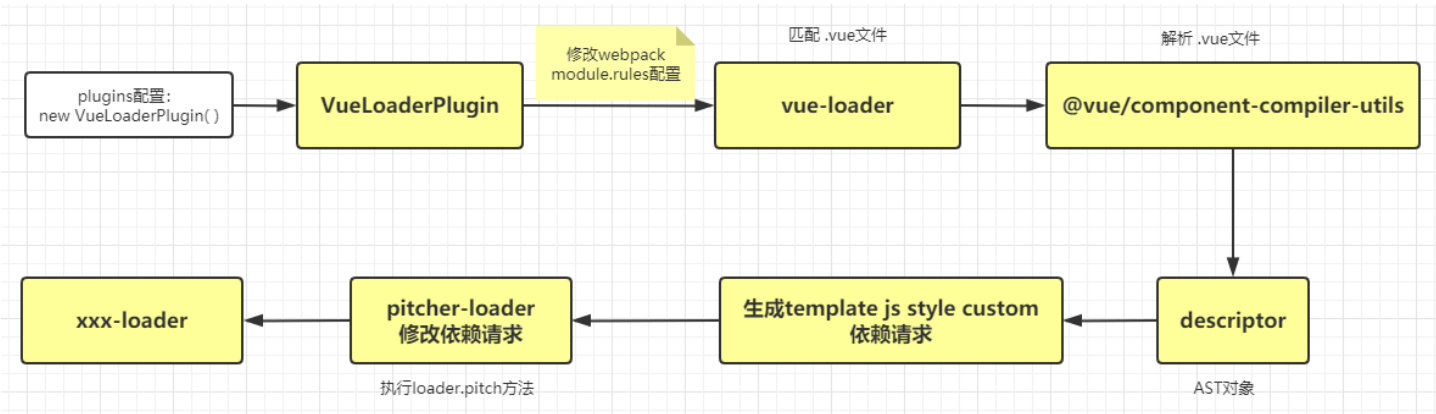
我们先看看在 `webpack.config.js` 中的配置：

```
▼ App.vue    ▼ helloWorld.vue    JS plugin-webpack4.js    webpack.config.js ×

testVueLoader > webpack.config.js > [?] <unknown>
1  const path = require('path');
2  const VueLoaderPlugin = require('vue-loader/lib/plugin.js');
3  const HtmlWebpackPlugin = require('html-webpack-plugin');
4
5  function resolve (dir) {
6    return path.join(__dirname, '..', dir)
7  }
8
9  module.exports = {
10   mode: 'development',
11   devtool: 'source-map',
12   entry: './src/index.js',
13   output: { ...
16   },
17   module: {
18     rules: [
19       {
20         test: /\.vue$/,
21         loader: 'vue-loader'
22       },
23       { ...
27     },
28     { ...
31   }
32   ],
33   },
34   plugins: [
35     new VueLoaderPlugin(),
36     new HtmlWebpackPlugin({ ...
40   })),
41   ]
42 }
```

vue-loader工作流

以下就是vue-loader工作大致的处理流程：



开启 `node调试模式` 进行查看阅读，package.json中配置如下：

```
1  "scripts": {
2
3    "debug": "node --inspect-brk ./node_modules/webpack/bin/webpack.js"
4
5  },
```

VueLoaderPlugin

先从入口文件 `lib/index.js` 开始分析，因为我的Webpack是4.x版本，所以

`VueLoaderPlugin = require('./plugin-webpack4')`，重点来看看这个 `lib/plugin-webpack4.js` 文件：

```
1  const qs = require('querystring')
2  const RuleSet = require('webpack/lib/RuleSet')
3  const id = 'vue-loader-plugin'
4  const NS = 'vue-loader'
5  // 很明显这就是一个webpack插件写法
6  class VueLoaderPlugin {
7    apply (compiler) {
8      if (compiler.hooks) {
9        // 编译创建之后，执行插件
10       compiler.hooks.compilation.tap(id, compilation => {
11         const normalModuleLoader = compilation.hooks.normalModuleLoader
12         normalModuleLoader.tap(id, loaderContext => {
13           loaderContext[NS] = true
14         })
15       })
16     } else {
17       // webpack < 4
18       compiler.plugin('compilation', compilation => {
19         compilation.plugin('normal-module-loader', loaderContext => {
20           loaderContext[NS] = true
21         })
22       })
23     }
24     // webpack.config.js 中配置好的 module.rules
25     const rawRules = compiler.options.module.rules
26     // 对 rawRules 做 normlized
27     const { rules } = new RuleSet(rawRules)
28     // 从 rawRules 中检查是否有规则去匹配 .vue 或 .vue.html
29     let vueRuleIndex = rawRules.findIndex(createMatcher(
30       foo.vue
31     ))
32     if (vueRuleIndex < 0) {
33       vueRuleIndex = rawRules.findIndex(createMatcher(
```

```

34   foo.vue.html
35   ))
36   }
37   const vueRule = rules[vueRuleIndex]
38   if (!vueRule) {
39     throw new Error(
40
41     [VueLoaderPlugin Error] No matching rule for .vue files found.\n
42     +
43
44     Make sure there is at least one root-level rule that matches .vue or
     .vue.html files.
45     )
46   }
47   if (vueRule.oneOf) {
48     throw new Error(
49
50     [VueLoaderPlugin Error] vue-loader 15 currently does not support vue rules
     with oneOf.
51     )
52   }
53   // 检查 normlized rawRules 中 .vue 规则中是否具有 vue-loader
54   const vueUse = vueRule.use
55   const vueLoaderUseIndex = vueUse.findIndex(u => {
56     return /^vue-loader|(\\/|\\|@)vue-loader/.test(u.loader)
57   })
58   if (vueLoaderUseIndex < 0) {
59     throw new Error(
60
61     [VueLoaderPlugin Error] No matching use for vue-loader is found.\n
62     +
63
64     Make sure the rule matching .vue files include vue-loader in its use.
65     )
66   }
67   // make sure vue-loader options has a known ident so that we can share
68   // options by reference in the template-loader by using a ref query like
69   // template-loader??vue-loader-options
70   const vueLoaderUse = vueUse[vueLoaderUseIndex]
71   vueLoaderUse.ident = 'vue-loader-options'
72   vueLoaderUse.options = vueLoaderUse.options || {}
73   // 过滤出 .vue 规则, 其他规则调用 cloneRule 方法重写了 resource 和
     resourceQuery 配置
74   // 用于编译vue文件后匹配依赖路径 query 中需要的loader
75   const clonedRules = rules
76     .filter(r => r !== vueRule)
77     .map(cloneRule)

```

```

78 // 加入全局 pitcher-loader, 路径query有vue字段就给loader添加pitch方法
79 const pitcher = {
80   loader: require.resolve('./loaders/pitcher'),
81   resourceQuery: query => {
82     const parsed = qs.parse(query.slice(1))
83     return parsed.vue !== null
84   },
85   options: {
86     cacheDirectory: vueLoaderUse.options.cacheDirectory,
87     cacheIdentifier: vueLoaderUse.options.cacheIdentifier
88   }
89 }
90 // 修改原始的 module.rules 配置
91 compiler.options.module.rules = [
92   pitcher,
93   ...clonedRules,
94   ...rules
95 ]
96 }
97 }

```

以上大概就是 `VueLoaderPlugin` 所做的事情。也就是说 `VueLoaderPlugin` 主要就是修改 `module.rules` 的配置。总的来说就是对vue单文件编写做的一个扩展(比如我们可以写less文件, 在vue style中也可以写less)

vue-loader

继续来看看 `vue-loader` 是如何操作.vue文件的, 目前只关心 `style` 部分, 逻辑在 `lib/index.js`:

vue文件解析

```

1 // 很明显这就是一个loader写法
2 module.exports = function (source) {
3   const loaderContext = this
4   // ...
5   const {
6     target,
7     request, // 请求资源路径
8     minimize,
9     sourceMap,
10    rootContext, // 根路径
11    resourcePath, // vue文件的路径
12    resourceQuery // vue文件的路径 query 参数
13  } = loaderContext

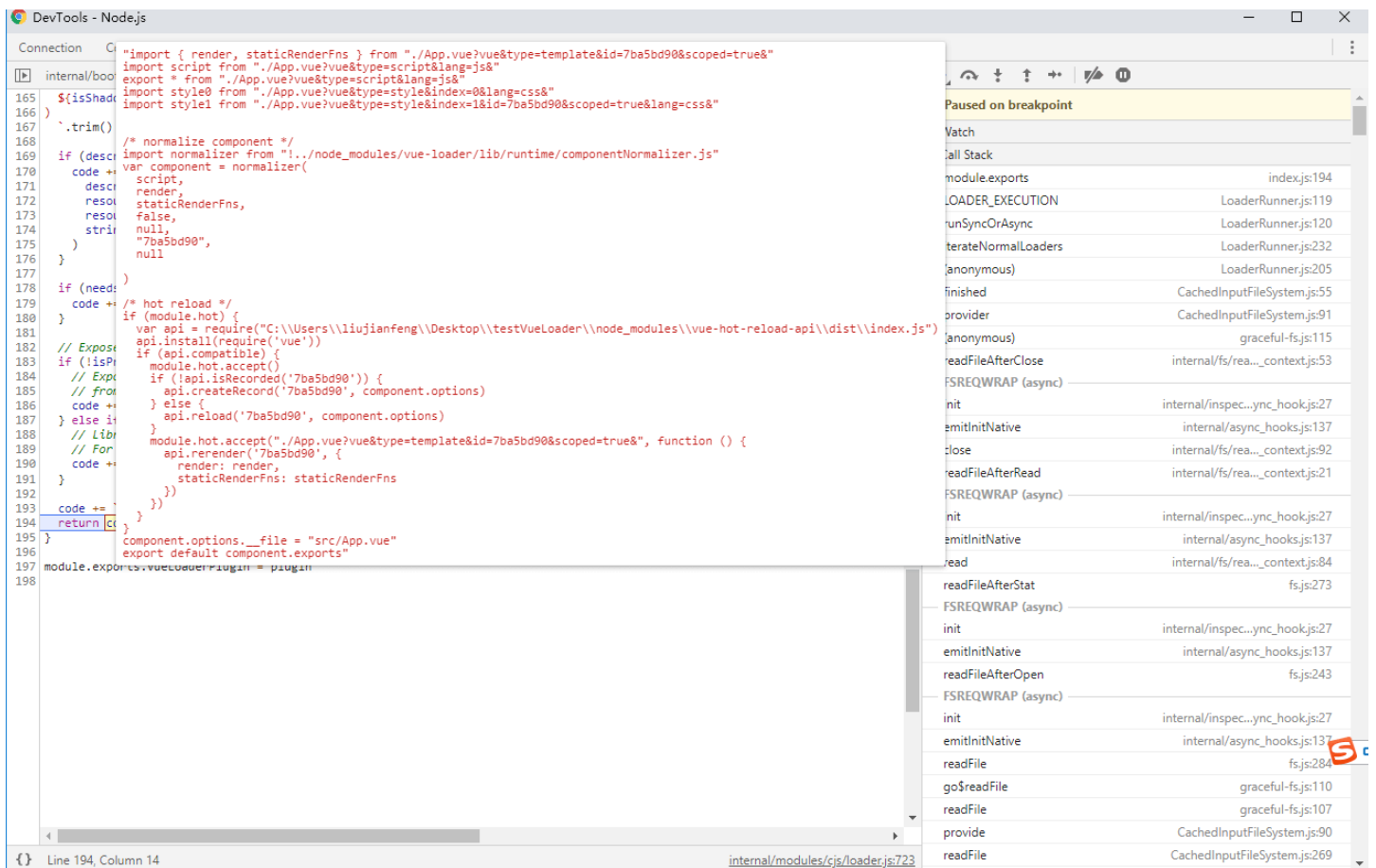
```

```

14      // ...
15
16      // 解析 vue 文件, descriptor 是AST抽象语法树的描述
17      const descriptor = parse({
18          source,
19          compiler: options.compiler || loadTemplateCompiler(loaderContext),
20          filename,
21          sourceRoot,
22          needMap: sourceMap
23      })
24      /**
25       *
26
27       /
28       // hash(文件路径 + 开发环境 ? 文件内容 : "")生成 id
29       const id = hash(
30           isProduction
31             ? (shortFilePath + '\n' + source)
32             : shortFilePath
33       )
34       // descriptor.styles 解析后是否具有 attrs: {scoped: true}
35       const hasScoped = descriptor.styles.some(s => s.scoped)
36       *   /*
37       *
38       *
39       */
40       let stylesCode = ``
41       if (descriptor.styles.length) {
42           // 最终生成一个import依赖请求
43           stylesCode = genStylesCode(
44               loaderContext,
45               descriptor.styles,
46               id,
47               resourcePath,
48               stringifyRequest,
49               needsHotReload,
50               isServer || isShadow // needs explicit injection?
51           )
52       }
53   }

```

可以看到解析完vue文件的结果大概就是这样的：



依赖解析

vue文件解析完之后template, script, style等都有个依赖的路径, 后续可以通过配置的loader进行解析了, 因为我们已经在 `VuePluginLoader` 中修改了module.rules的配置, 而且依赖的路径中query中都拥有vue字段, 所以会先走到pitcher-loader,现在来分析 `lib/loaders/pitcher.js` 中的逻辑:

```
1  /**
2   *
3   */
4  module.exports = code => code
5  module.exports.pitch = function (remainingRequest) {
6    const options = loaderUtils.getOptions(this)
7    const { cacheDirectory, cacheIdentifier } = options
8    const query = qs.parse(this.resourceQuery.slice(1))
9    let loaders = this.loaders
10   if (query.type) {
11     if (/\.vue$/i.test(this.resourcePath)) {
12       // 过滤eslint-loader
13       loaders = loaders.filter(l => !isESLintLoader(l))
14     } else {
15       loaders = dedupeESLintLoader(loaders)
```

```

16     }
17 }
18 // 过滤pitcher-loader
19 loaders = loaders.filter(isPitcher)
20
21 const genRequest = loaders => {
22     const seen = new Map()
23     const loaderStrings = []
24     loaders.forEach(loader => {
25         const identifier = typeof loader === 'string'
26             ? loader
27             : (loader.path + loader.query)
28         const request = typeof loader === 'string' ? loader : loader.request
29         if (!seen.has(identifier)) {
30             seen.set(identifier, true)
31             // loader.request contains both the resolved loader path and its
options
32             // query (e.g. ??ref-0)
33             loaderStrings.push(request)
34         }
35     })
36     return loaderUtils.stringifyRequest(this, '!' + [
37         ...loaderStrings,
38         this.resourcePath + this.resourceQuery
39     ].join('!'))
40 }
41
42
43 if (query.type ===
44 style
45 ) {
46     const cssLoaderIndex = loaders.findIndex(isCSSLoader)
47     // 调整loader执行顺序
48     if (cssLoaderIndex > -1) {
49         const afterLoaders = loaders.slice(0, cssLoaderIndex + 1)
50         const beforeLoaders = loaders.slice(cssLoaderIndex + 1)
51         const request = genRequest([
52             ...afterLoaders, // [style-loader,css-loader]
53             stylePostLoaderPath, // style-post-loader
54             ...beforeLoaders // [vue-loader]
55         ])
56         return
57 import mod from ${request}; export default mod; export * from ${request}
58     }
59 }
60 /**
61 *

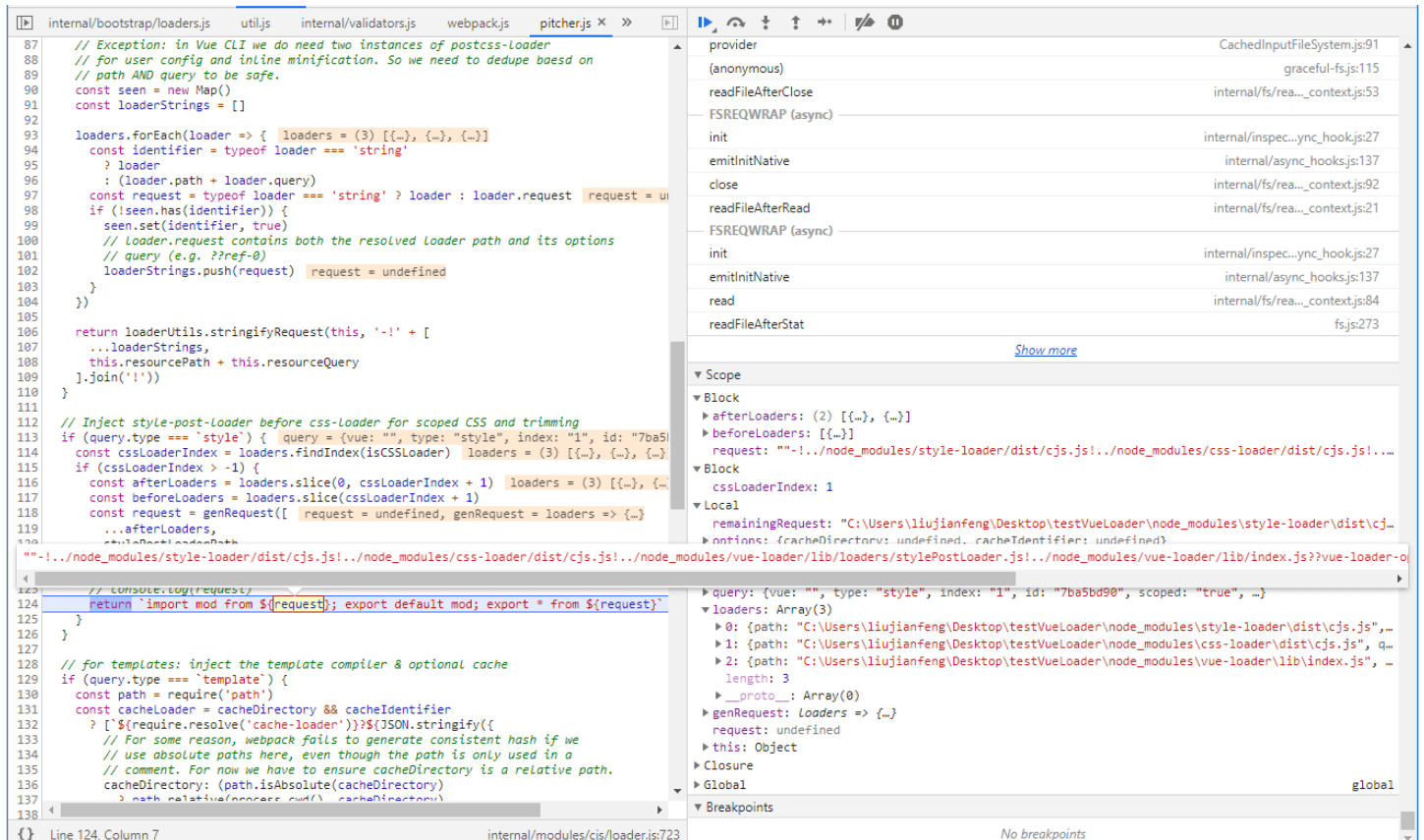
```

```

62    */
63    const request = genRequest(loaders)
64    return
65    import mod from ${request}; export default mod; export * from ${request}
66  }

```

可以看到解析带scoped属性的style的结果大概是这样的：



新的依赖解析

分析 `{type: style}` 的处理流程顺序：

- vue-loader、style-post-loader、css-loader、style-loader。

处理资源的时候先走的是 `vue-loader`，这时vue-loader中的处理逻辑与第一次解析vue文件不一样了：

```

1  const incomingQuery = qs.parse(rawQuery)
2  // 拥有{type:style}
3  if (incomingQuery.type) {
4    return selectBlock(
5      descriptor,
6      loaderContext,
7      incomingQuery,

```

```

8      !!options.appendExtension
9    )
10  }
11
12  // lib/select.js
13  module.exports = function selectBlock (
14    descriptor,
15    loaderContext,
16    query,
17    appendExtension
18  ) {
19    // ...
20    if (query.type ===
21  style
22    && query.index !== null) {
23      const style = descriptor.styles[query.index]
24      if (appendExtension) {
25        loaderContext.resourcePath += '.' + (style.lang || 'css')
26      }
27      loaderContext.callback(
28        null,
29        style.content,
30        style.map
31      )
32      return
33    }

```

可以看到vue-loader处理完后返回的就是style.content，也就是style标签下的内容，然后交给后续的loader继续处理

再来看一下 style-post-loader 是如何生成 data-v-hash:8 的,逻辑主要在 lib/loaders/stylePostLoaders.js 中:

```

1  const qs = require('querystring')
2  const { compileStyle } = require('@vue/component-compiler-utils')
3  module.exports = function (source, inMap) {
4    const query = qs.parse(this.resourceQuery.slice(1))
5    const { code, map, errors } = compileStyle({
6      source,
7      filename: this.resourcePath,
8      id:
9  data-v-${query.id}
10 ,
11   map: inMap,
12   scoped: !!query.scoped,

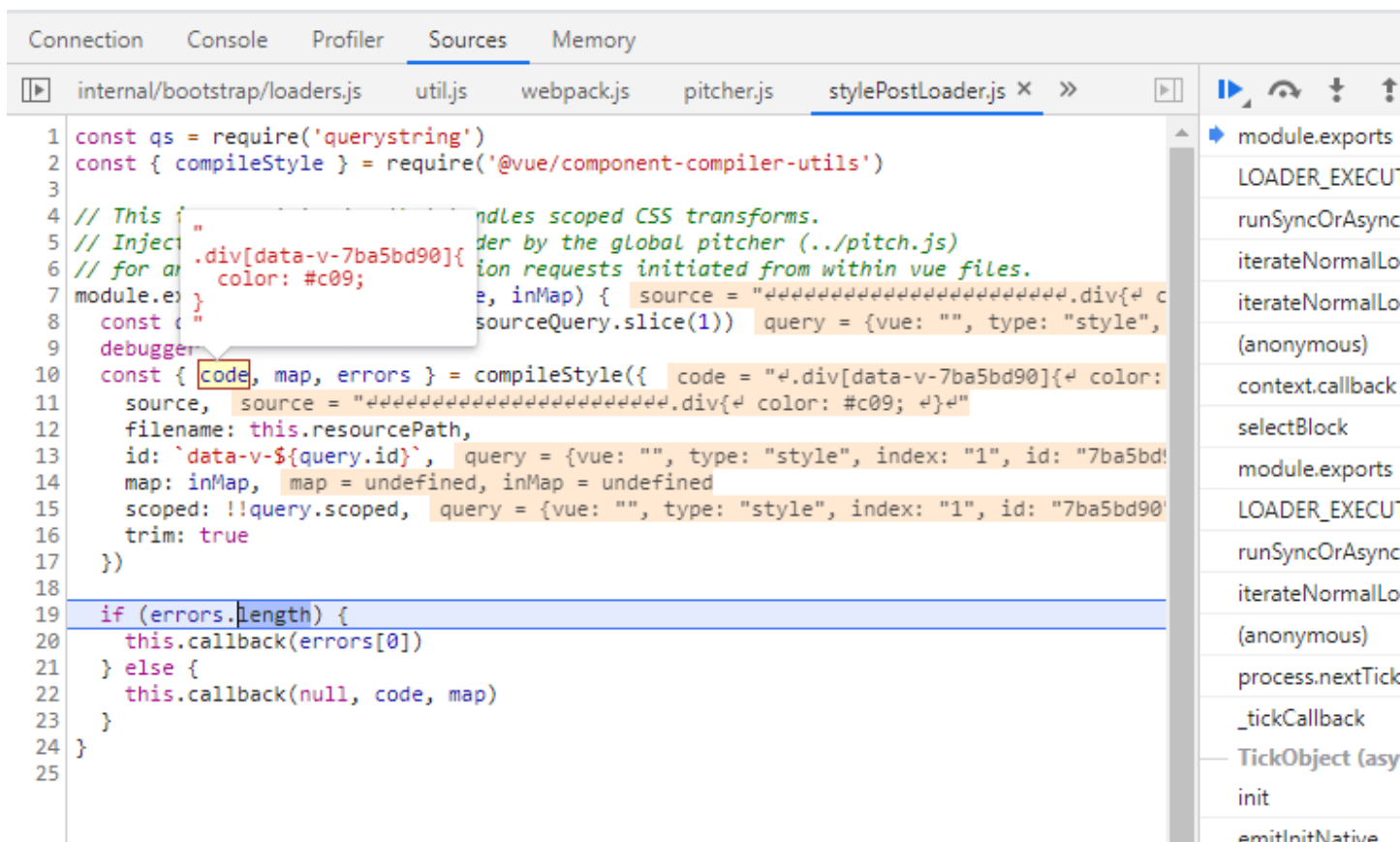
```

```

13     trim: true
14   })
15   if (errors.length) {
16     this.callback(errors[0])
17   } else {
18     this.callback(null, code, map)
19   }
20 }

```

处理最终返回的code是这样的：



第46题：说说你对渐进式框架的理解

渐进式的含义：没有多做职责之外的事，只做了自己该做的事，没有做不该做的事，仅此而已。

更直白一点就是，用你想用或者能用的功能特性，你不想用的部分功能可以先不用。VUE不强求你一次性接受并使用它的全部功能特性。

比如以下两种场景，Vue 发挥了很大的优点：

- 场景一：公司刚开始一个项目，技术人员对Vue的掌握也不足够。那么我们就不能使用VUE了么？当然不是，如果你只是使用VUE做些基础操作，如：页面渲染、表单处理提交功能，那还是非常简单的，成熟技术人员上手也就一两天。完全可以用它去代替jquery。并不需要你去引入其他复杂特性功能。

- 场景二：我们的项目规模逐渐的变大了，我们可能会逐渐用到前端路由、状态集中管理、并最终实现一个高度工程化的前端项目。这些功能特性我们可以逐步引入，当然不用也可以。

Vue 的适用面很广，你可以用它代替老项目中的JQuery。也可以在新项目启动初期，有限的使用VUE的功能特性，从而降低上手的成本。

第47题：Vue中的 v-show 和 v-if 有什么区别

`v-if` 和 `v-show` 是 Vue.js 中用于条件渲染的指令，它们的作用是根据条件来控制元素的显示和隐藏。它们之间有一些重要的区别：

1. 编译时刻 vs 运行时刻：

- `v-if` 是一个“惰性”指令，在编译时刻，Vue.js 会根据条件决定是否编译或挂载元素到 DOM 中。如果条件为 `false`，元素根本不会被编译和渲染到 DOM 中。
- `v-show` 是一个“非惰性”指令，在编译时刻，元素总是会被编译和渲染到 DOM 中。但是，根据条件的值，`v-show` 会通过 CSS 控制元素的显示和隐藏，不会从 DOM 中移除元素。

2. 显示隐藏方式：

- `v-if` 在条件为 `true` 时会渲染元素到 DOM，而在条件为 `false` 时会从 DOM 中移除元素。`v-if` 也可以触发组件创建和销毁的生命钩子。
- `v-show` 在条件为 `true` 时会通过 CSS 设置元素的 `display` 属性为可见（通常是 `display: block`），在条件为 `false` 时设置为隐藏（`display: none`）。元素始终存在于 DOM 中，只是通过 CSS 控制其显示状态。

3. 切换开销：

- `v-if` 在条件切换时，如果条件从 `true` 切换为 `false`，会销毁并重新创建元素，这涉及到 DOM 的删除和重新插入，可能会有一定的性能开销。
- `v-show` 在条件切换时，只是简单地通过 CSS 控制元素的显示和隐藏，不会销毁和重新创建元素，因此切换的开销较小。

4. 初始渲染开销：

- `v-if` 在初始渲染时，如果条件为 `false`，元素不会被渲染到 DOM 中，因此在初始渲染时可能会有一定的性能优势。
- `v-show` 在初始渲染时，元素总是会被渲染到 DOM 中，因此在初始渲染时可能会有一些额外的开销。

综上所述，当需要频繁切换元素的显示状态时，且元素可能处于不同的状态，推荐使用 `v-show`。而当条件不会频繁改变，且希望在条件为 `false` 时不渲染元素到 DOM 中，推荐使用 `v-if`。在实际使用中，根据具体的场景和性能需求来选择合适的指令。

第47题：Vue中的 v-show 和 v-if 有什么区别

`v-if` 和 `v-show` 是 Vue.js 中用于条件渲染的指令，它们的作用是根据条件来控制元素的显示和隐藏。它们之间有一些重要的区别：

1. 编译时刻 vs 运行时刻：

- `v-if` 是一个“惰性”指令，在编译时刻，Vue.js 会根据条件决定是否编译或挂载元素到 DOM 中。如果条件为 `false`，元素根本不会被编译和渲染到 DOM 中。
- `v-show` 是一个“非惰性”指令，在编译时刻，元素总是会被编译和渲染到 DOM 中。但是，根据条件的值，`v-show` 会通过 CSS 控制元素的显示和隐藏，不会从 DOM 中移除元素。

2. 显示隐藏方式：

- `v-if` 在条件为 `true` 时会渲染元素到 DOM，而在条件为 `false` 时会从 DOM 中移除元素。`v-if` 也可以触发组件创建和销毁的生命钩子。
- `v-show` 在条件为 `true` 时会通过 CSS 设置元素的 `display` 属性为可见（通常是 `display: block`），在条件为 `false` 时设置为隐藏（`display: none`）。元素始终存在于 DOM 中，只是通过 CSS 控制其显示状态。

3. 切换开销：

- `v-if` 在条件切换时，如果条件从 `true` 切换为 `false`，会销毁并重新创建元素，这涉及到 DOM 的删除和重新插入，可能会有一定的性能开销。
- `v-show` 在条件切换时，只是简单地通过 CSS 控制元素的显示和隐藏，不会销毁和重新创建元素，因此切换的开销较小。

4. 初始渲染开销：

- `v-if` 在初始渲染时，如果条件为 `false`，元素不会被渲染到 DOM 中，因此在初始渲染时可能会有一定的性能优势。
- `v-show` 在初始渲染时，元素总是会被渲染到 DOM 中，因此在初始渲染时可能会有一些额外的开销。

综上所述，当需要频繁切换元素的显示状态时，且元素可能处于不同的状态，推荐使用 `v-show`。而当条件不会频繁改变，且希望在条件为 `false` 时不渲染元素到 DOM 中，推荐使用 `v-if`。在实际使用中，根据具体的场景和性能需求来选择合适的指令。

第48题：Vue3.0里为什么要用 Proxy API 替代 defineProperty API？

在 Vue 3.0 中，使用 Proxy API 替代 defineProperty API 是为了改进响应式系统的性能和功能：

1. 性能提升：Proxy API 比 defineProperty API 在许多情况下具有更好的性能。defineProperty 使用 Object.defineProperty 方法来拦截对象属性的访问和修改，但它需要遍历每个属性进行拦截。而 Proxy API 允许拦截整个对象，可以更高效地捕获对对象的访问和修改。

2. 更全面的拦截能力：Proxy API 提供了更多的拦截方法，比 defineProperty API 更灵活、丰富。它支持拦截目标的各种操作，包括读取、设置、删除、枚举等，甚至还可以拦截函数调用和构造函数实例化。
3. 更好的数组变化检测：Vue 3.0 使用 Proxy API 改善了数组的变化检测机制。Proxy 可以直接拦截数组的索引访问和修改，使得对数组的变化更容易被监听到，从而提供了更可靠的响应式行为。
4. 更易于处理嵌套对象：Proxy API 能够递归地拦截对象的嵌套属性，而 defineProperty 无法自动递归处理嵌套对象。这使得在 Vue 3.0 中处理嵌套对象更加简单和方便。
5. 更好的错误提示：相比于 defineProperty，Proxy API 提供了更好的错误追踪和调试信息。当使用 Proxy API 时，如果访问或修改了一个不存在的属性，会直接抛出错误，从而更容易发现和修复问题。

使用 Proxy API 取代 defineProperty API 是为了提升性能、增强功能，并提供更好的开发体验和错误提示。这些改进使得 Vue 3.0 的响应式系统更加高效、灵活和可靠。

第49题：SSR是什么？Vue中怎么实现？

什么是 SSR

SSR（Server-Side Rendering，服务器端渲染）是一种将应用程序的界面在服务器上进行预先渲染并以 HTML 形式发送到客户端的技术。与传统的客户端渲染（CSR）相比，SSR 在服务器端生成完整的 HTML 页面，然后将其发送到浏览器，以提供更好的性能和搜索引擎优化。

在传统的客户端渲染中，浏览器会下载一个包含 JavaScript 代码的文件，并在客户端执行该代码来构建和呈现页面。这意味着页面初始加载时只是一个空壳，页面内容需要在浏览器中通过 JavaScript 进行渲染。

而在 SSR 中，服务器接收到请求后，会根据请求的路由和数据，预先生成完整的 HTML 页面，其中包含了初始状态下的页面内容。服务器将这个完整的 HTML 页面发送给浏览器，浏览器无需再执行额外的 JavaScript，即可直接展示出页面内容。

SSR 的优势包括：

- 更快的首次渲染：由于服务器在响应请求时已经生成了完整的 HTML 页面，所以用户打开页面时可以立即看到内容，无需等待 JavaScript 下载和执行。
- 更好的搜索引擎优化（SEO）：搜索引擎爬虫能够抓取到完整的 HTML 页面，并且页面内容可直接被搜索引擎索引。
- 更好的用户体验：页面内容在服务器端渲染完成后即可展示，减少了白屏时间和加载等待。

需要注意的是，SSR 可能会增加服务器负载和响应时间，并且涉及到一些复杂性，例如处理路由、状态管理等。因此，在选择是否使用 SSR 时，需要根据项目需求和复杂性来权衡利弊。

怎么使用 Vue 框架实现 SSR

可以按照以下步骤进行操作：

1. 安装相关依赖：

- 首先，确保你的项目中已经安装了 Vue 相关的依赖和构建工具，如 Vue、Vue Router、Vue Server Renderer 等。

2. 创建服务器入口文件：

- 在项目中创建一个服务器入口文件，通常命名为 `server.js` 或类似名称。
- 在该文件中，引入必要的模块，包括 Vue、Vue Server Renderer、Express（或其他后端框架）等。
- 创建一个 Express 应用实例，并设置路由处理器来处理不同请求。

3. 编写服务器端渲染逻辑：

- 在服务器入口文件中，编写服务器端渲染的逻辑。
- 创建一个 Vue 实例，并配置路由、数据等相关内容。
- 使用 Vue Server Renderer 的 `createRenderer` 方法创建一个 renderer 实例。
- 在路由处理器中调用 renderer 实例的 `renderToString` 方法来将 Vue 实例渲染为字符串。

4. 处理静态资源：

- 在服务器端渲染时，需要处理静态资源（如样式表、图片等）的加载和引用。
- 可以使用 Webpack 进行服务器端渲染的配置，以处理静态资源的导出和加载。

5. 客户端激活：

- 在服务器端渲染后，需要在客户端激活 Vue 实例，以便能够响应交互事件和更新页面。
- 可以通过在 HTML 中插入一个 JavaScript 脚本，并在脚本中使用 `createApp` 方法来创建客户端应用程序实例。

以上步骤是一个简单的 SSR 实现流程，可以参考 Vue 官方文档中提供的 [SSR 指南](#) 获取更详细的信息和示例代码。

第50题：vue的祖孙组件的通信方案有哪些？

在 Vue 中，祖孙组件之间的通信可以通过以下几种方式来实现：

1. Props / \$emit:

- 祖组件通过 `props` 将数据传递给子组件，并且子组件通过 `$emit` 触发事件将数据传递回祖组件。
- 这是一种常见的父子组件通信方式，通过属性(props)和自定义事件(\$emit)进行数据交流。

2. Provide / Inject:

- 使用 `provide` 在祖组件中提供数据，然后使用 `inject` 在孙组件中注入这些数据。

- 这种方式允许祖组件向下级组件共享数据，无需显式地将数据逐层传递。但要注意潜在的耦合性。

3. Event Bus:

- 创建一个全局的事件总线(Event Bus)，用于在祖孙组件之间发送和接收事件。
- 通过在事件总线上注册事件监听器和触发器，组件可以相互通信，传递数据和触发特定操作。

4. Vuex:

- 使用 Vuex 进行状态管理，可以在祖孙组件之间共享和更新数据。
- Vuex 是 Vue 的官方状态管理库，提供了集中式存储管理和响应式更新，使得不同组件之间的通信更加简单和可预测。

这些通信方式各有特点，可以根据具体情况选择合适的方式来实现祖孙组件之间的通信。对于简单的父子组件通信，Props / \$emit 是常用的方式；而对于更复杂的应用程序状态管理和跨层级通信，使用 Vuex 或 Event Bus 可能更适合。